
УНИВЕРСАЛЬНОЕ ПОСОБИЕ ПО АЛГОРИТМАМ И СТРУКТУРАМ
ПОЛНЫЙ ИСХОДНЫЙ КОД ПРОЕКТА (ВСЕ ФАЙЛЫ, ВСЕ

Дата генерации: 11.09.2026, 16:32:07
Файлов исходного кода: 86
Суммарный объём кода: 1.16 МВ
Глав/билетов в пособии: 30

ОГЛАВЛЕНИЕ: ГЛАВЫ И БИЛЕТЫ

1. 1. Дерево отрезков с операциями на отрезках [id: segment-tree]
2. 2. Двумерная разреженная матрица (Sparse Table)
3. 3. Декартово дерево (Treap) [id: treap]
4. 4. Splay-дерево [id: splay-tree]
5. 5. Двумерная матрица префиксных сумм [id: prefix-sums]
6. Планарность: K5, K3,3 и формула Эйлера [id: planarity]
7. 6. Графы. Представление, DFS, BFS [id: graph-basics]
8. 7. Графы. Планарные. Покраска [id: graph-planarity]
9. 8. Графы. Компоненты связности [id: graph-components]
10. 9. Графы. Топологическая сортировка [id: topological-sort]
11. 10. Графы. Компоненты сильной связности (SCC)
12. 11. Графы. Мосты [id: graph-bridges]
13. 12. Графы. Точки сочленения [id: graph-articulation-points]
14. 13. Графы. Эйлеров цикл [id: graph-euler]
15. Билет 14. Дейкстра [id: dijkstra]
16. Билет 15. Форд-Беллман [id: bellman-ford]
17. Билет 16. Флойд-Уоршелл [id: floyd]
18. 17. Графы. Ускорения поиска [id: pathfinding-optimizations]
19. 17. Графы. Алгоритм Джонсона (Фокус с перевзвешиванием)
20. 18. Графы. Остовное дерево. Краскал [id: mst-kruskal]
21. 19. Графы. Остовное дерево. Прима [id: mst-prims]
22. 20. Графы. Остовное дерево. Борувка [id: mst-boruvka]
23. 21. Алгоритм Кнута-Морриса-Пракса (KMP) [id: string-kmp]
24. 22. Z-функция [id: string-z-func]
25. Билет 23. Алгоритм Ахо-Корасик [id: aho-corasick]

24. src/data/vizSync.ts

Билеты (учебный контент)

25. src/data/tickets/ticket1_5.ts

26. src/data/tickets/ticket14_20.ts

27. src/data/tickets/ticket21_24.ts

28. src/data/tickets/ticket6_13.ts

Интерактивные компоненты и симуляторы

29. src/components/ArticulationPointsViz.tsx

30. src/components/BellmanFordViz.tsx

31. src/components/BfsViz.tsx

32. src/components/ChapterImage.tsx

33. src/components/ChapterNav.tsx

34. src/components/ChapterView.tsx

35. src/components/DfsBridgesSimulator.tsx

36. src/components/DijkstraViz.tsx

37. src/components/DPVisualizer.tsx

38. src/components/EulerSimulator.tsx

39. src/components/ExpressQuiz.tsx

40. src/components/FloydViz.tsx

41. src/components/GraphTraversalViz.tsx

42. src/components/HeapViz.tsx

43. src/components/hints/demoKit.tsx

44. src/components/hints/demosExtra.tsx

45. src/components/hints/demosGraph.tsx

46. src/components/hints/demosStruct.tsx

47. src/components/hints/MiniDemo.tsx

48. src/components/hints/SimulatorModal.tsx

49. src/components/hints/TermPopover.tsx

50. src/components/JohnsonViz.tsx

51. src/components/KosarajuViz.tsx

52. src/components/KruskalSimulator.tsx

53. src/components/MnemonicCards.tsx

54. src/components/MobileToc.tsx

55. src/components/Navbar.tsx

56. src/components/PlanarityDemo.tsx

57. src/components/PythonCompiler.tsx

86. `.github/workflows/rebuild-pdf.yml`

РАЗДЕЛ: КОНФИГУРАЦИЯ ПРОЕКТА

ФАЙЛ 1/86: `add.md`

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 1224 | РАЗМЕР:

Изменённые файлы

```text

```
A .github/workflows/deploy-pages.yml
M .github/workflows/rebuild-pdf.yml
M index.html
M public/export/full_code_all_pages.pdf
M public/export/full_code_all_pages.txt
A public/favicon.svg
M src/App.tsx
M src/components/Navbar.tsx
A src/components/PythonCompiler.tsx
M vite.config.ts
```

```

`.github/workflows/deploy-pages.yml`

Полное содержимое после изменений

```yaml

name: Deploy app to GitHub Pages

on:

push:

branches:

- main

```
- name: Upload Pages artifact
 uses: actions/upload-pages-artifact@v5
 with:
 path: ./dist

deploy:
 name: Publish to GitHub Pages
 needs: build
 runs-on: ubuntu-latest
 environment:
 name: github-pages
 url: ${ steps.deployment.outputs.page_url }
 steps:
 - name: Deploy
 id: deployment
 uses: actions/deploy-pages@v5

Patch относительно `main`

```diff
diff --git a/.github/workflows/deploy-pages.yml b/.github/workflows/deploy-pages.yml
new file mode 100644
index 00000000..17418e3
--- /dev/null
+++ b/.github/workflows/deploy-pages.yml
@@ -0,0 +1,60 @@
+name: Deploy app to GitHub Pages
+
+on:
+  push:
+    branches:
+      - main
+      # Позволяет проверить Pages прямо из рабочей ветки
+      - "arena/**"
+  workflow_dispatch:
+
+permissions:
```

```
+   deploy:
+     name: Publish to GitHub Pages
+     needs: build
+     runs-on: ubuntu-latest
+     environment:
+       name: github-pages
+       url: ${ steps.deployment.outputs.page_url }
+     steps:
+       - name: Deploy
+         id: deployment
+         uses: actions/deploy-pages@v5
+     ....
```

`github/workflows/rebuild-pdf.yml`

Полное содержимое после изменений

```yaml

name: Rebuild code PDF

# Пайплайн пересборки PDF при каждом коммите:

```
#
исходный код → full_code_all_pages.txt (script)
#
|
|→ page-0001.png ... page-NNNN.png (
#
|
|→ full_code_all_page
```

# Готовые TXT и PDF коммитаются обратно в ветку (их отда  
# промежуточные PNG сохраняются как artifacts запуска.

on:

push:

branches:

- "\*\*"

paths-ignore:

# чтобы коммит самого workflow не запускал бескон

- "public/export/\*\*"

```
sudo apt-get install -y --no-install-recommen
На Ubuntu политика ImageMagick иногда запре
разрешаем то, что нужно пайплайну (text -> p
for policy in /etc/ImageMagick-6/policy.xml /
 if [-f "$policy"]; then
 sudo sed -i 's/rights="none" pattern="\{P
 fi
done
convert -version
```

- name: Install dependencies  
run: npm ci --no-audit --no-fund
- name: Step 1 – код → txt  
run: npm run export:txt
- name: Step 2 – txt → png  
env:  
EXPORT\_DENSITY: \${github.event.inputs.densi  
run: npm run export:png
- name: Step 3 – png → pdf  
run: npm run export:pdf
- name: Type-check приложения  
run: npx tsc --noEmit  
continue-on-error: true
- name: Summary  
run: |  
 {  
 echo "### Экспорт пересобран"  
 echo ""  
 echo "| Артефакт | Размер |"  
 echo "| --- | --- |"  
 echo "| \`\${public}/export/full\_code\_all\_pages  
 echo "| \`\${public}/export/full\_code\_all\_pages  
 echo "| PNG-страниц | \$(ls tmp/export-pages

--- a/.github/workflows/rebuild-pdf.yml

+++ b/.github/workflows/rebuild-pdf.yml

@@ -42,15 +42,15 @@ jobs:

steps:

- name: Checkout

- uses: actions/checkout@v4

+ uses: actions/checkout@v6

with:

fetch-depth: 0

persist-credentials: true

- name: Setup Node.js

- uses: actions/setup-node@v4

+ uses: actions/setup-node@v7

with:

- node-version: "22"

+ node-version: "24"

cache: npm

- name: Install ImageMagick + DejaVu fonts

@@ -97,7 +97,7 @@ jobs:

} >> "\$GITHUB\_STEP\_SUMMARY"

- name: Upload artifacts (PDF + TXT)

- uses: actions/upload-artifact@v4

+ uses: actions/upload-artifact@v7

with:

name: full-code-export

path: |

@@ -107,7 +107,7 @@ jobs:

retention-days: 30

- name: Upload artifacts (PNG-страницы)

- uses: actions/upload-artifact@v4

+ uses: actions/upload-artifact@v7

with:

name: full-code-pages-png

Универсальное пособие • полный исходный код

-----  
`````  
`public/favicon.svg`

Полное содержимое после изменений

`````xml

```
<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 64 64">
 <rect width="64" height="64" rx="16" fill="#4f46e5"/>
 <path d="M18 17h28v8H26v8h16v8H26v8h20v8H18z" fill="#4f46e5"/>
 <circle cx="47" cy="21" r="4" fill="#34d399"/>
</svg>
```

### Patch относительно `main`

`````diff

```
diff --git a/public/favicon.svg b/public/favicon.svg  
new file mode 100644  
index 00000000..3bbbbba0  
--- /dev/null  
+++ b/public/favicon.svg  
@@ -0,0 +1,5 @@  
+<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 64 64">  
+  <rect width="64" height="64" rx="16" fill="#4f46e5"/>  
+  <path d="M18 17h28v8H26v8h16v8H26v8h20v8H18z" fill="#4f46e5"/>  
+  <circle cx="47" cy="21" r="4" fill="#34d399"/>  
+</svg>
```

`src/App.tsx`

Полное содержимое после изменений

`````tsx

```
import { useCallback, useEffect, useMemo, useState } from "react";
import { chapters } from "../data/content";
import { Navbar } from "../components/Navbar";
```



```

const progress = useMemo(() => ((index + 1) / chapter

return (
 <div className="min-h-screen bg-slate-950 text-slate-50">
 <Navbar activeTab={activeTab} setActiveTab={setActiveTab}>
 {activeTab === "guide" ? (
 <>
 <MobileToc
 selectedChapterId={activeChapterId}
 setSelectedChapterId={goTo}
 currentTitle={activeChapter.title}
 index={index}
 total={chapters.length}
 />

 <div className="flex flex-col lg:flex-row max-w-7xl mx-auto" >
 {/* Сайдбар только на десктопе – на мобильном скрывается */}
 <div className="hidden lg:block lg:w-80 shrink-0" >
 <Sidebar selectedChapterId={activeChapterId} />
 </div>

 <main id="main-content" className="flex-1 m-0 p-4" >
 {/* Шапка главы с быстрыми переходами */}
 <div className="flex items-center justify-between" >
 <div className="min-w-0" >
 <div className="text-[10px] uppercase" >
 {activeChapter.category || "Универсальное"}
 </div>
 <h2 className="text-base sm:text-xl font-medium" >
 {activeChapter.title}
 </h2>
 </div>
 <ChapterNav prev={prev} next={next} index={index} />
 </div>

 <div className="h-1 w-full bg-slate-800 rounded-t" />
 </main>
 </div>
 </>
) : null}
 </Navbar>
 </div>
)

```

```

 </div>
);
}
....

Patch относительно `main`

```diff
diff --git a/src/App.tsx b/src/App.tsx
index 2091f33..4583756 100644
--- a/src/App.tsx
+++ b/src/App.tsx
@@ -7,9 +7,10 @@ import { ChapterView } from "../components/ChapterView";
import { ChapterNav } from "../components/ChapterNav";
import { SimulatorModal } from "../components/hints/SimulatorModal";
import { SimulatorHub } from "../components/SimulatorHub";
+import { PythonCompiler } from "../components/PythonCompiler";

export default function App() {
-   const [activeTab, setActiveTab] = useState<"guide" | "simulator">();
+   const [activeTab, setActiveTab] = useState<"guide" | "simulator">();
   const [activeChapterId, setActiveChapterId] = useState<string>();
   const [modalVizId, setModalVizId] = useState<string>();

@@ -95,7 +96,7 @@ export default function App() {
    </main>
    </div>
  </>
-   ) : (
+   ) : activeTab === "simulator" ? (
    <main className="px-4 sm:px-6 lg:px-10 py-6 lg:py-8">
      <SimulatorHub
        onOpen={setModalVizId}
@@ -105,6 +106,8 @@ export default function App() {
    </main>
  ) : (

```

```

    <div className="bg-gradient-to-tr from-indigo
      k-0">
      <Sparkles className="w-6 h-6" />
    </div>
    <div className="min-w-0 flex-1">
      <h1 className="text-xl font-extrabold text-
        Универсальное пособие
      </h1>
      <p className="text-xs text-indigo-400 font-
        Подготовка к экзаменам: Алгоритмы, Структ
      </p>
    </div>
  </div>

  <div className="flex items-center w-full lg:w-ar
    >
    <button
      id="tab-guide-btn"
      onClick={() => setActiveTab('guide')}
      className={`flex-1 lg:flex-none flex items-
        uration-200 whitespace-nowrap ${
          activeTab === 'guide'
            ? 'bg-indigo-600 text-white shadow-lg s
            : 'text-slate-400 hover:text-white'
        }`
    >
      <BookOpen className="w-4 h-4" /> Учебное по
    </button>
    <button
      id="tab-simulator-btn"
      onClick={() => setActiveTab('simulator')}
      className={`flex-1 lg:flex-none flex items-
        uration-200 whitespace-nowrap ${
          activeTab === 'simulator'
            ? 'bg-indigo-600 text-white shadow-lg s
            : 'text-slate-400 hover:text-white'
        }`
    >

```

```

        <button
          id="download-html-btn"
          onClick={saveBook}
          disabled={busy}
          className="flex items-center justify-center
            er:bg-amber-600/20 border border-amber-500/
          title="Скачать всё пособие одним автономным
        >
          {busy ? <Loader2 className="w-4 h-4 animate
        </button>
      </div>
    </div>
  </header>
);
};

```

Patch относительно `main`

```

````diff
diff --git a/src/components/Navbar.tsx b/src/components
index a528778..6f8cd36 100644
--- a/src/components/Navbar.tsx
+++ b/src/components/Navbar.tsx
@@ -1,11 +1,11 @@
 import React, { useState } from 'react';
- import { BookOpen, Cpu, Sparkles, FileDown, FileText,
+ import { BookOpen, Cpu, Sparkles, FileDown, FileText,
 import { chapters } from '../data/content';
 import { downloadBookHtml } from '../utils/exportHtml'

 interface NavbarProps {
- activeTab: 'guide' | 'simulator';
- setActiveTab: (tab: 'guide' | 'simulator') => void;
+ activeTab: 'guide' | 'simulator' | 'compiler';
+ setActiveTab: (tab: 'guide' | 'simulator' | 'compile
 }

```

```
```tsx
import { useCallback, useState } from "react";
import { CheckCircle2, ExternalLink, Info, Loader2, Play } from "lucide-react";

const PYODIDE_VERSION = "0.27.7";
const PYODIDE_MODULE_URL = `https://cdn.jsdelivr.net/pyodide/v${PYODIDE_VERSION}/bin/pyodide.js`;
const PYODIDE_INDEX_URL = `https://cdn.jsdelivr.net/pyodide/v${PYODIDE_VERSION}/index.html`;

const STARTER_CODE = `# Первый запуск загрузит Python в браузер
name = "алгоритмы"

for number in range(1, 4):
    print(f"{number}. Учим {name}!")
`;

type PyodideRuntime = {
  runPythonAsync: (source: string) => Promise<unknown>;
  setStdout: (options: { batched: (message: string) => void }) => void;
  setStderr: (options: { batched: (message: string) => void }) => void;
  setStdin: (options: { stdin: () => string | number | null }) => void;
};

type PyodideModule = {
  loadPyodide: (options: { indexURL: string }) => Promise<PyodideRuntime>;
};

let runtimePromise: Promise<PyodideRuntime> | null = null;

function getPythonRuntime() {
  if (!runtimePromise) {
    runtimePromise = import(/* @vite-ignore */ PYODIDE_INDEX_URL)
      .then((module as PyodideModule).loadPyodide({ indexURL: PYODIDE_INDEX_URL }));
  }
  return runtimePromise;
}
```

```

        setOutput(`${captured}${captured && !captured.end
        setRuntimeMessage("Не удалось выполнить код – про
    } finally {
        setRunning(false);
    }
}, [code, stdin, running, runtimeReady]);

```

```

const reset = () => {
    setCode(STARTER_CODE);
    setStdin("");
    setOutput("Нажмите «Запустить», чтобы увидеть резул
};

```

```

return (
    <main className="max-w-screen-2xl mx-auto px-4 sm:px-6">
        <div className="max-w-6xl mx-auto">
            <div className="flex flex-col sm:flex-row sm:items-center">
                <div>
                    <div className="inline-flex items-center gap-2">
                        <Terminal className="w-4 h-4" /> Боковая панель
                    </div>
                    <h2 className="text-2xl sm:text-3xl font-extrabold">
                        <p className="text-slate-400 mt-2 max-w-2xl">
                            Пишите небольшой код и запускайте его прямо в браузере,
                            который переводит CPython в WebAssembly.
                        </p>
                    </div>
                    <a
                        href="https://pyodide.org/"
                        target="_blank"
                        rel="noreferrer"
                        className="inline-flex items-center gap-1.5"
                    >
                        Как это работает <ExternalLink className="w-4 h-4" />
                    </a>
                </div>
            </div>
        </div>
    </div>

```

```

<div className="grid xl:grid-cols-[minmax(0,1.3333333333333333fr),minmax(0,1.3333333333333333fr)]"

```

```

        spellCheck={false}
        aria-label="Код Python"
        className="block w-full min-h-[360px] res
        none focus:ring-2 focus:ring-inset focus:r
        placeholder="Напишите Python-код..."
    />

<div className="border-t border-slate-800 b
    <label className="block px-4 pt-3 text-[1
        Ввод для input() • по одной строке на к
    </label>
    <textarea
        id="python-stdin"
        value={stdin}
        onChange={(event) => setStdin(event.tar
        spellCheck={false}
        rows={2}
        placeholder={'Например:\nАлиса'}
        className="block w-full resize-y bg-tra
        eholder:text-slate-700"
    />
</div>
</section>

<section className="rounded-2xl border border
    <div className="flex items-center justify-b
        <div className="flex items-center gap-2 t
            <Terminal className="w-4 h-4 text-emera
        </div>
        <span className={`inline-flex items-cente
            {runtimeReady && <CheckCircle2 classNam
            {runtimeReady ? "готов" : "не загружен"}
        </span>
    </div>
    <pre className="min-h-[360px] max-h-[560px]
    x] leading-6 text-slate-300">
        {output}
    </pre>

```

Универсальное пособие • полный исходный код

```
-----  
+const PYODIDE_INDEX_URL = `https://cdn.jsdelivr.net/py  
+  
+const STARTER_CODE = `# Первый запуск загрузит Python  
+name = "алгоритмы"  
+  
+for number in range(1, 4):  
+  print(f"{number}. Учим {name}!")  
+`;  
+  
+type PyodideRuntime = {  
+  runPythonAsync: (source: string) => Promise<unknown>  
+  setStdout: (options: { batched: (message: string) =>  
+  setStderr: (options: { batched: (message: string) =>  
+  setStdin: (options: { stdin: () => string | number |  
+});  
+  
+type PyodideModule = {  
+  loadPyodide: (options: { indexURL: string }) => Prom  
+});  
+  
+let runtimePromise: Promise<PyodideRuntime> | null = n  
+  
+function getPythonRuntime() {  
+  if (!runtimePromise) {  
+    runtimePromise = import(/* @vite-ignore */ PYODIDE  
+      (module as PyodideModule).loadPyodide({ indexURL  
+    });  
+  }  
+  return runtimePromise;  
+}  
+  
+function errorText(error: unknown) {  
+  if (error instanceof Error) return error.message;  
+  return String(error);  
+}  
+  
+export function PythonCompiler() {  
+  const [code, setCode] = useState(STARTER_CODE);
```



```

+   const reset = () => {
+     setCode(STARTER_CODE);
+     setStdin("");
+     setOutput("Нажмите «Запустить», чтобы увидеть резу.
+   };
+
+   return (
+     <main className="max-w-screen-2xl mx-auto px-4 sm:p-8">
+       <div className="max-w-6xl mx-auto">
+         <div className="flex flex-col sm:flex-row sm:items-center">
+           <div>
+             <div className="inline-flex items-center gap-2">
+               <Terminal className="w-4 h-4" /> Боковая панель
+             </div>
+             <h2 className="text-2xl sm:text-3xl font-extrabold">
+               <p className="text-slate-400 mt-2 max-w-2xl">
+                 Пишите небольшой код и запускайте его прямо в браузере,
+                 который переводит CPython в WebAssembly.
+               </p>
+             </div>
+             <a
+               href="https://pyodide.org/"
+               target="_blank"
+               rel="noreferrer"
+               className="inline-flex items-center gap-1.5"
+             >
+               Как это работает <ExternalLink className="text-slate-400" />
+             </a>
+           </div>
+
+           <div className="grid xl:grid-cols-[minmax(0,1fr)minmax(0,1fr)] gap-2">
+             <section className="rounded-2xl border border-slate-200 p-4">
+               <div className="flex flex-wrap items-center gap-2">
+                 <div className="flex items-center gap-2">
+                   <span className="w-2.5 h-2.5 rounded-full bg-slate-200">
+                     <span className="text-sm font-bold text-slate-700">
+                       <span className="text-[11px] text-slate-400">
+                         </div>
+                   </div>
+                 </div>
+               </div>
+             </div>
+           </div>
+         </div>
+       </div>
+     </main>
+   );
+ }
+
+ export default App;

```

```

+         <div className="border-t border-slate-800 p-4">
+             <label className="block px-4 pt-3 text-slate-600">
+                 Ввод для input() • по одной строке на строку
+             </label>
+             <textarea
+                 id="python-stdin"
+                 value={stdin}
+                 onChange={(event) => setStdin(event.target.value)}
+                 spellCheck={false}
+                 rows={2}
+                 placeholder={'Например: \nАлиса'}
+                 className="block w-full resize-y bg-tran
+ ceholder:text-slate-700"
+             />
+         </div>
+     </section>
+
+     <section className="rounded-2xl border border-slate-800 p-4">
+         <div className="flex items-center justify-between">
+             <div className="flex items-center gap-2">
+                 <Terminal className="w-4 h-4 text-emerald-500">
+                     {output}
+                 </div>
+                 <span className={`inline-flex items-center gap-2`}
+                     {runtimeReady && <CheckCircle2 className="text-emerald-500"/>
+                     {runtimeReady ? "готов" : "не загружен"}}
+                 </span>
+             </div>
+             <pre className="min-h-[360px] max-h-[560px] leading-6 text-slate-300">
+                 {output}
+             </pre>
+             <div className="border-t border-slate-800 p-4">
+                 <div className="border-slate-800 p-4">
+                     <div className="border-t border-slate-800 p-4">
+                         <div className="flex gap-2 rounded-xl border border-slate-800 p-4">
+                             <Info className="w-4 h-4 shrink-0 text-indigo-500">

```

```
    resolve: {
      alias: {
        "@": path.resolve(__dirname, "src"),
      },
    },
  },
  server: {
    host: "0.0.0.0",
    port: 5173,
    strictPort: true,
    // предпросмотр может открываться через внешний про
    allowedHosts: true,
  },
  preview: {
    host: "0.0.0.0",
    port: 4173,
    strictPort: true,
    allowedHosts: true,
  },
});
````
```

### Patch относительно `main`

```
````diff
diff --git a/vite.config.ts b/vite.config.ts
index c0456e7..e19d945 100644
--- a/vite.config.ts
+++ b/vite.config.ts
@@ -10,6 +10,9 @@ const __dirname = path.dirname(__file)

// https://vite.dev/config/
export default defineConfig({
+ // На GitHub Pages приложение живёт в /algv0/, локал
+ // VITE_BASE можно переопределить, если репозиторий
+ base: process.env.VITE_BASE || "/",
  plugins: [react(), tailwindcss(), viteSingleFile()],
  resolve: {
    alias: {
```

-
- `{"`, `content: ...` }` для вставки в `src/data/content.`
3. ****Строгая структура контента:****
- Заголовок с цветным бейджем (например, ``bg-indigo-100``)
 - Определение/правило в центре (``border-blue-500`` или ``border-blue-500``)
 - Обязательная сетка аналогий (2 колонки: неформальный и формальный)
 - "Душные" детали (код, формулы, сложные доказательства)
4. ****Не ломай верстку:**** Не придумывай свои CSS-классы и не используй классы в файлах ``src/data/content.ts``. Не используй чистый `markdown`.
-

ФАЙЛ 4/86: `index.html`

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 14 | РАЗМЕР: 610 байт

```
<!doctype html>
<html lang="ru" class="dark bg-slate-950 text-slate-100">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <link rel="icon" type="image/svg+xml" href="%BASE_URL%/icon.svg">
    <title> Дискретка для тупых – Интерактивный гид по дискретной математике
  </head>
  <body class="bg-slate-950 text-slate-100 min-h-screen">
    <div id="root"></div>
    <script type="module" src="/src/main.tsx"></script>
  </body>
</html>
```

ФАЙЛ 5/86: `metadata.json`

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 8 | РАЗМЕР: 310 байт

```
{
  "name": "Remix Remix: ExamPrep Reader",
  "description": "Удобная web-читалка для подготовки к экзаменам по дискретной математике"
```

Универсальное пособие • полный исходный код

```
-----  
    "@tailwindcss/vite": "4.1.17",  
    "@types/node": "22.19.17",  
    "@types/pdfkit": "^0.17.6",  
    "@types/react": "19.2.7",  
    "@types/react-dom": "19.2.3",  
    "@vitejs/plugin-react": "5.1.1",  
    "esbuild": "^0.28.2",  
    "tailwindcss": "4.1.17",  
    "typescript": "5.9.3",  
    "vite": "7.3.2",  
    "vite-plugin-singlefile": "2.3.0"  
  },  
  "description": "Интерактивное пособие по алгоритмам и  
}
```

ФАЙЛ 7/86: README.md

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 115 | РАЗМЕР: 1000

algv0 – Универсальное пособие по алгоритмам и структурам

Интерактивная web-читалка (React + Vite + Tailwind) с синтаксисом
и автоматическим экспортом всего исходного кода в PDF.

Быстрый старт

```
```bash  
npm install
npm run dev # предпросмотр на http://0.0.0.0:5174
npm run build # сборка в dist/ (single-file)
```
```

Экспорт кода: код → txt → png → pdf

Пайплайн собирает ****весь**** исходный код репозитория в o

Универсальное пособие • полный исходный код

- * ****Содержание**** – на десктопе боковая панель с поиском на телефоне – липкая строка «Содержание · N из M» и в
- * ****Переходы между темами**** – кнопки «Назад / Вперёд» в «Предыдущая / Следующая тема» под текстом; на клавиатуре
- * ****Всплывающие подсказки по терминам**** – текст главы и известные термины (BFS, куча, бор, суффиксная ссылка, подчёркиваются, а по наведению/тапу показывается карта мини-анимацией и кнопкой «Показать симулятор» (симуля Словарь – `src/data/glossary.ts`, мини-анимации – `src
- * ****Реестр интерактивов**** – `src/components/vizRegistry` и для панели под главой, и для модалки из подсказки.

Структура

...

src/

- | | |
|--------------------|----------------------------------|
| App.tsx | – оболочка, привязка глав к визу |
| components/ | – интерактивные симуляторы (Дейк |
| data/ | – контент пособия (главы/билеты |
| scripts/export/ | – пайплайн код → txt → png → pdf |
| public/export/ | – сгенерированные TXT и PDF (обн |
| .github/workflows/ | – автоматизация |
- ...

Структура приложения и парсинг элементов

Если вы хотите парсить HTML конспект внешними инструмен
азуемые теги и классы.

Заголовки

- * Заголовки секций: Используют стандартный тег `

` и
- * Подзаголовки: Используют тег `

`.

Текст и параграфы

- * Обычный текст содержится в тегах `

`.
- * Блоки "Шиза" (Аналогии) и другая текстовая инфографика

Универсальное пособие • полный исходный код

> * ****2D**** — к главе привязан интерактивный визуализатор
>
> Всего глав в пособии: ****25****.

Уровень 1. Нет вообще ничего: ни аналогии, ни 2D

| |
|---|
| Глава id Что делать |
| --- --- --- |
| Алгоритмы на карте `alg-map` Страница-заглушка (8 |

Уровень 2. Темы, которых в пособии НЕТ ВООБЩЕ (дыры)

Это самая большая проблема: визуализаторы для них написаны, но соответствующих глав в `src/data/content.ts` нет — и

| |
|--|
| Билет Тема Статус |
| --- --- --- |
| **1** Дерево отрезков (Segment Tree) Главы нет. Контент в холостую. |
| **7** Планарность и раскраска графов Номерной главы нет; аналогии ** . |
| **11** Мосты в графах Номерной главы нет; есть без аналогии. |
| **13** Эйлеровы пути и циклы Номерной главы нет; есть без аналогии. |

Дополнительно — «осиротевшие» визуализаторы без глав (контент в холостую)

- * `stack-dfs` → `StackViz.tsx` — ****Стек и DFS****
- * `queue-bfs` → `QueueViz.tsx` + `BfsViz.tsx` — ****Очередь и BFS****
- * `heap-beam-search` → `HeapViz.tsx` — ****Куча и лучевой поиск****
- * `segment-trees` → `SegmentTreeVisualizer.tsx` — ****Дерево отрезков****
- * `dynamic-programming` → `DPVisualizer.tsx` — ****Динамическое программирование****

| | | | | | | | | | |
|----|---|---|--|--|---|---|---|--|--|
| 10 | 10. Графы. Компоненты сильной связности (SCC) | | | | | | | | |
| 12 | 12. Графы. Точки сочленения | 1 | | | - | - | | | |
| 14 | 14. Графы. Кратчайший путь. Дейкстра | 3 | | | | | - | | |
| 15 | 15. Графы. Кратчайший путь. Форд–Беллман | 1 | | | | | | | |
| 16 | 16. Графы. Кратчайший путь. Флойд | - | | | - | - | | | |
| 17 | 17. Графы. Алгоритм Джонсона | - | | | - | - | | | |
| 18 | 18. Графы. Остовное дерево. Краскал | 1 | | | | | - | | |
| 19 | 19. Графы. Остовное дерево. Прима | 1 | | | | | - | | |
| 20 | 20. Графы. Остовное дерево. Борувка | 1 | | | | | - | | |
| 21 | 21. Префикс-функция (π), КМП | 4 | | | - | - | | | |
| 22 | 22. Z-функция | 4 | | | - | - | | | |
| 23 | 23. Алгоритм Ахо–Корасик | 2 | | | - | - | | | |
| 24 | 24. Классы сложности, сведение задач | 4 | | | - | - | | | |
| - | Обзор: Кратчайшие пути и Графы | - | | | - | - | | | |
| - | Алгоритмы на карте | - | | | - | - | | | |
| - | Эйлер: Путь ≠ Цикл | - | | | - | - | | | |
| - | Мосты в графах: код + визуализация | 1 | | | | | - | | |

Рекомендуемый порядок работ

1. ****Вернуть потерянные билеты 1, 7, 11, 13**** и подключить ``segment-trees``, ``stack-dfs``, ``queue-bfs``, ``heap-be``.
это 5 готовых компонентов, которые сейчас просто не
2. ****Дописать аналогии**** в 5 главах из «Уровня 3» (по ш
3. ****Добавить 2D**** к 4 главам «Уровня 4» – минимум `inli`
4. ****Решить судьбу `alg-map`**** – раскрыть или удалить.

ФАЙЛ 9/86: `tsconfig.json`

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 32 | РАЗМЕР: 6

```
{
  "compilerOptions": {
    "target": "ES2020",
```



```
import react from "@vitejs/plugin-react";
import { defineConfig } from "vite";
import { viteSingleFile } from "vite-plugin-singlefile"

const __filename = fileURLToPath(import.meta.url);
const __dirname = path.dirname(__filename);

// https://vite.dev/config/
export default defineConfig({
  // На GitHub Pages приложение живёт в /algv0/, локально
  // VITE_BASE можно переопределить, если репозиторий б
  base: process.env.VITE_BASE || "/",
  plugins: [react(), tailwindcss(), viteSingleFile()],
  resolve: {
    alias: {
      "@": path.resolve(__dirname, "src"),
    },
  },
  server: {
    host: "0.0.0.0",
    port: 5173,
    strictPort: true,
    // предпросмотр может открываться через внешний про
    allowedHosts: true,
  },
  preview: {
    host: "0.0.0.0",
    port: 4173,
    strictPort: true,
    allowedHosts: true,
  },
});
```

```

    setCompilerOpen(true);
    requestAnimationFrame(() => {
      requestAnimationFrame(() => {
        document.getElementById("chapter-viz)?.scrollIntoView();
      });
    });
  }, []));

// Стрелки ← → листают темы (как на обучающих сайтах)
useEffect(() => {
  const onKey = (e: KeyboardEvent) => {
    const target = e.target as HTMLInputElement | null;
    if (target && /^(INPUT|TEXTAREA|SELECT)$/.test(target?.tagName)) return;
    if (e.metaKey || e.ctrlKey || e.altKey) return;
    if (e.key === "ArrowRight" && next) goTo(next.id);
    if (e.key === "ArrowLeft" && prev) goTo(prev.id);
  };
  window.addEventListener("keydown", onKey);
  return () => window.removeEventListener("keydown", onKey);
}, [goTo, next, prev]);

const progress = useMemo(() => ((index + 1) / chapterList.length) * 100, [index, chapterList]);

return (
  <div className="min-h-screen bg-slate-950 text-slate-50">
    <Navbar
      activeTab={activeTab}
      setActiveTab={setActiveTab}
      compilerOpen={compilerOpen}
      onToggleCompiler={() => setCompilerOpen((o) => !o)}
    />

    <div className={compilerOpen ? "max-lg:pb-[58dvh]" : "max-lg:pb-[58dvh]"}>
      {activeTab === "guide" ? (
        <div>
          <MobileToc
            selectedChapterId={activeChapterId}

```

```

        index={index}
        total={chapters.length}
        onGo={goTo}
        onOpenSimulator={setModalVizId}
        onOpenCompiler={openCompilerWithViz}
      />
    </main>
  </div>
</>
) : (
  <main className="px-4 sm:px-6 lg:px-10 py-6 lg:py-8">
    <SimulatorHub
      onOpen={setModalVizId}
      onGoChapter={(id) => {
        setActiveTab("guide");
        goTo(id);
      }}
    />
  </main>
)}

<footer className="p-8 text-center text-slate-600">
  © 2026 Universal Educational Guide. Интерактивное руководство
</footer>
</div>

<SimulatorModal vizId={modalVizId} onClose={() => setModalVizId(null)}>
  {compilerOpen && (
    <PythonCompiler
      chapterId={activeChapter.id}
      chapterTitle={activeChapter.title}
      onOpenGuide={() => setActiveTab("guide")}
      onClose={() => setCompilerOpen(false)}
    />
  )}
</div>
);

```

```
* повесить всплывающую карточку (как превью ссылок в Википедии)
*/
export function annotateTerms(root: HTMLElement): void {
  if (!cachedRegex) cachedRegex = buildRegex();
  const re = cachedRegex;

  const perTerm = new Map<string, number>();
  const perSurface = new Map<string, number>();
  const walker = document.createTreeWalker(root, NodeFilter.FILTER_ACCEPT, {
    acceptNode(node) {
      const parent = (node as Text).parentElement;
      if (!parent) return NodeFilter.FILTER_REJECT;
      if (parent.closest(SKIP_SELECTOR)) return NodeFilter.FILTER_REJECT;
      if (!node.nodeValue || node.nodeValue.trim().length === 0) return NodeFilter.FILTER_REJECT;
      return NodeFilter.FILTER_ACCEPT;
    },
  });

  const targets: Text[] = [];
  let current = walker.nextNode();
  while (current) {
    targets.push(current as Text);
    current = walker.nextNode();
  }

  for (const textNode of targets) {
    const text = textNode.nodeValue ?? "";
    re.lastIndex = 0;
    if (!re.test(text)) continue;
    re.lastIndex = 0;

    const frag = document.createDocumentFragment();
    let last = 0;
    let m: RegExpExecArray | null;

    while ((m = re.exec(text)) !== null) {
      const surface = m[1].toLowerCase();
      const id = labelToId.get(surface);
```

```
        annotateTerms(el);
    } catch {
        /* если браузер не умеет lookbehind – просто оста
    }
};

// при смене главы
// eslint-disable-next-line react-hooks/exhaustive-dep
useEffect(run, deps);

// страховка: контент перерисовали, а подсказок в нём
useEffect(() => {
    const el = ref.current;
    if (!el) return;
    if (!el.querySelector(".term-hint")) run();
});
}
```

ФАЙЛ 13/86: src/index.css

КАТЕГОРИЯ: Ядро приложения | СТРОК: 175 | РАЗМЕР: 3.3 K

```
@import "tailwindcss";

@layer utilities {
    .neon-glow-blue {
        box-shadow: 0 0 15px rgba(59, 130, 246, 0.5);
    }
    .neon-glow-emerald {
        box-shadow: 0 0 15px rgba(16, 185, 129, 0.5);
    }
    .neon-glow-rose {
        box-shadow: 0 0 15px rgba(244, 63, 94, 0.5);
    }
    .neon-glow-yellow {
        box-shadow: 0 0 20px rgba(234, 179, 8, 0.8);
    }
}
```

```

        opacity: 0;
        transform: translateY(4px);
    }
    to {
        opacity: 1;
        transform: translateY(0);
    }
}

.term-popover {
    animation: hintIn 0.14s ease-out;
}

/* Интерактивные термины в тексте главы (превью как в В
.term-hint {
    cursor: help;
    border-bottom: 1px dashed rgba(129, 140, 248, 0.75);
    color: #c7d2fe;
    border-radius: 3px;
    padding: 0 1px;
    transition:
        background-color 0.15s ease,
        color 0.15s ease;
}
.term-hint:hover,
.term-hint:focus-visible {
    background: rgba(99, 102, 241, 0.22);
    color: #fff;
    outline: none;
}
@media (hover: none) {
    .term-hint {
        border-bottom-style: solid;
        background: rgba(99, 102, 241, 0.12);
    }
}

/* Контент главы приходит готовым HTML — чиним его под

```

```
    font-size: 1.25rem !important;
    line-height: 1.3 !important;
}
.chapter-body :where(h3) {
    font-size: 1.05rem !important;
    line-height: 1.35 !important;
}
.chapter-body :where(.p-6, .p-8) {
    padding: 0.9rem !important;
}
.chapter-body :where(.text-3xl) {
    font-size: 1.4rem !important;
}
.chapter-body :where(.text-2xl) {
    font-size: 1.2rem !important;
}
.chapter-body :where(.text-xl) {
    font-size: 1.05rem !important;
}
}
```

```
/* Custom scrollbar styling for dark theme */
::-webkit-scrollbar {
    width: 8px;
    height: 8px;
}
::-webkit-scrollbar-track {
    background: #0f172a;
}
::-webkit-scrollbar-thumb {
    background: #334155;
    border-radius: 4px;
}
::-webkit-scrollbar-thumb:hover {
    background: #475569;
}
```

```
/* Индикатор длительности кадра в пошаговых визуализаци
```

```
    category?: string;  
}
```

ФАЙЛ 16/86: src/vite-env.d.ts

КАТЕГОРИЯ: Ядро приложения | СТРОК: 19 | РАЗМЕР: 324 В

```
/// <reference types="vite/client" />
```

```
declare module '*.jpg' {  
    const src: string;  
    export default src;  
}  
declare module '*.jpg?url' {  
    const src: string;  
    export default src;  
}  
declare module '*.jpeg' {  
    const src: string;  
    export default src;  
}  
declare module '*.png' {  
    const src: string;  
    export default src;  
}
```

РАЗДЕЛ: КОНТЕНТ И ДАННЫЕ ПОСОБИЯ

ФАЙЛ 17/86: src/data/additionalTickets.ts

КАТЕГОРИЯ: Контент и данные пособия | СТРОК: 934 | РАЗМЕР: 1000 В

```

        </div>
        <p class="text-slate-300 text-sm mb-4">
            нужно покрыть отрезок, мы берем две самые бо
        </div>
    </div>
    <details class="bg-slate-900/40 rounded-lg border-1
        <summary class="p-4 font-bold text-slate-300
            -b border-slate-700/50">
                Код (Скрыто)
        </summary>
        <div class="p-5 text-sm text-slate-300">
            <pre class="bg-slate-950 p-4 rounded">
int kx = log2(R2 - R1 + 1);
int ky = log2(C2 - C1 + 1);
int ans1 = min(st[R1][C1][kx][ky], st[R1][C2 - (1 << ky) + 1][C1][kx][ky]);
int ans2 = min(st[R2 - (1 << kx) + 1][C1][kx][ky], st[R2][C1][kx][ky]);
int minimum = min(ans1, ans2);</pre>
        </div>
    </details>
</div>
</div>
</section>`,
    },
    {
        id: "treap",
        title: "3. Декартово дерево (Treap)",
        type: "html",
        description: "Дерево поиска по ключу и Куча по приоритету",
        category: "Продвинутые структуры",
        content: `
<section id="treap" class="mb-12 scroll-mt-10">
    <div class="flex items-center mb-6 flex-wrap gap-3">
        <span class="bg-indigo-600 text-white px-4 py-1 rounded">Тема
        <h2 class="text-2xl sm:text-3xl font-bold text-slate-300">Декартово дерево
    </div>
    <div class="space-y-8">
        <div class="bg-slate-700/50 p-6 rounded-xl border-1 border-slate-600">
            <h3 class="text-xl font-bold text-blue-400">Описание

```

```

        auto [L, R] = split(t->left, x);
        t->left = R;
        return {L, t};
    }
}
</pre>
</div>
</details>
</div>
</div>
</section>`,
},
{
    id: "splay-tree",
    title: "4. Splay-дерево",
    type: "html",
    description: "Дерево поиска, которое чинит само себя",
    category: "Продвинутые структуры",
    content: `
<section id="splay-tree" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-indigo-600 text-white px-4 py-1">
    <h2 class="text-2xl sm:text-3xl font-bold text-violet-600">
  </div>

  <div class="space-y-8">

    <!-- 0. Определение -->
    <div class="bg-slate-700/50 p-6 rounded-xl border">
      <h3 class="text-xl font-bold text-blue-400">
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-blue-300 italic">
        <p class="text-lg sm:text-xl font-mono">
        поиска <span class="text-slate-500">без</span>
        : серия поворотов, которая поднимает v в корень
        <p class="text-sm text-slate-400 text-cyan-300">
        т – поворот.</p>
      </div>
    </div>
  </div>
    `
  }
}

```

```
        <li><span class="text-white font-bold">
каждой вставки чинят дерево, чтобы оно <em>
/li>
        <li><span class="text-white font-bold">
менно быть кривым. Но каждый раз, когда мы
ровнее становится.</li>
</ul>
</div>
```

```
<!-- 2. Аналогии -->
```

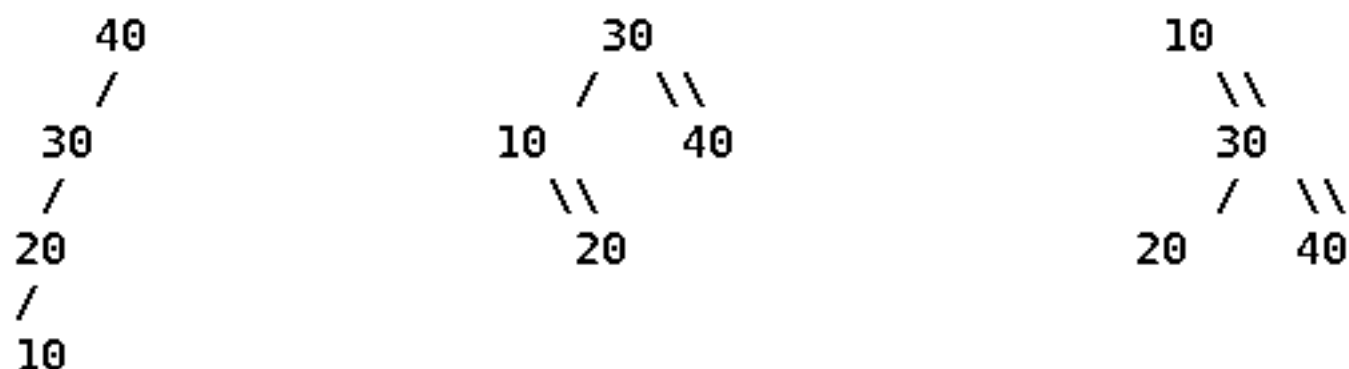
```
<div class="grid grid-cols-1 xl:grid-cols-2 gap-4">
  <div class="bg-slate-800 p-6 rounded-lg border border-emerald-500 shadow-md">
    Аналогия 1: стопка бумаг на столе
  </div>
  <p class="text-slate-300 text-sm">У тебя
скиваешь его и, поработав, кладёшь <span class="text-emerald-400">
неделю самые нужные бумаги сами собой оказыва
ет ровно это: «взял → положил наверх».</p>
</div>
  <div class="bg-slate-900 p-6 rounded-lg border border-rose-500 shadow-md">
    Аналогия 2: тропинка через газон
  </div>
  <p class="text-slate-300 text-sm">Ты идёшь, и
ода дорожка сама укорачивалась вдвое: чем чаще
оплачивает все следующие. Пойдёшь один раз
</div>
</div>
```

```
<!-- 3. Поворот -->
```

```
<div class="bg-slate-800/60 p-6 rounded-xl border border-emerald-500">
  <h3 class="text-lg font-bold text-white mb-4">Поворот
  <p class="text-slate-300 text-sm mb-4">Поворот — это
движение, три перевешенные ссылки</span> и
  <pre class="bg-slate-950 p-4 rounded-lg overflow-hidden">
```

[illegible]

3 шага, а потом делаем `Splay(10)`.
<pre class="bg-slate-950 p-4 rounded-lg overflow-hidden leading-relaxed">старт после Zi
 глубина 3 → 1 10 в корне



итог: было 4 узла в линию (высота 4) → стало высота 3,
а сама десятка теперь в корне: следующий запрос к

<p class="text-slate-300 text-sm mt-4">Обра
инили дерево по пути. Все узлы, мимо
</div>

<!-- 7. Амортизация -->

<div class="bg-slate-800/60 p-6 rounded-xl border-1 border-slate-700">
 <h3 class="text-lg font-bold text-white mb-3">Стро
 <p class="text-slate-300 text-sm mb-3">Стро
 оддерева) по всем узлам, и Zig-Zig/Zig-Zag :
 ах идея такая:</p>

<div class="grid grid-cols-1 md:grid-cols-3 gap-4">
 <div class="bg-slate-900 rounded-lg p-4">
 <p class="text-white font-bold mb-1">
 <p class="text-slate-400 text-xs">c
 </div>

<div class="bg-slate-900 rounded-lg p-4">
 <p class="text-white font-bold mb-1">
 <p class="text-slate-400 text-xs">H
</div>

<div class="bg-slate-900 rounded-lg p-4">
 <p class="text-white font-bold mb-1">
 <p class="text-slate-400 text-xs">д
</div>

```

function rotate(x) {
    const p = x.parent, g = p.parent;

    if (p.left === x) {                // правый поворот
        p.left = x.right;
        if (x.right) x.right.parent = p;
        x.right = p;
    } else {                          // левый поворот (зеркало)
        p.right = x.left;
        if (x.left) x.left.parent = p;
        x.left = p;
    }

    p.parent = x;                      // родитель уехал вниз
    x.parent = g;                      // x занял его место
    if (g) { if (g.left === p) g.left = x; else g.right = x; }
}

// поднимаем x в корень
function splay(x) {
    while (x.parent) {
        const p = x.parent, g = p.parent;

        if (!g) {                     // ZIG: деда не нашли
            rotate(x);
        } else if ((g.left === p) === (p.left === x)) {
            rotate(p);                 // ZIG-ZIG: сносим
            rotate(x);
        } else {
            rotate(x);                 // ZIG-ZAG: сносим
            rotate(x);
        }
    }
    return x;                          // теперь x — корень
}

// поиск: спустились и подняли найденное наверх
function find(root, key) {

```

```

<summary class="p-4 font-bold text-slate-400"
open:border-b border-slate-700/50">
    Вопросы, которые задают на экзамене (
</summary>
<div class="p-5 text-sm text-slate-300 space-y-4">
    <div>
        <strong class="text-white block mb-2">Вопрос</strong>
        <p class="text-slate-400">Последний
        енный или преемник). Splay делают всегда, да
        бы, и амортизация сломалась бы.</p>
    </div>
    <div>
        <strong class="text-white block mb-2">Почему?</strong>
        <p class="text-slate-400">Потому что
        еворачивают бамбук в другую сторону (см. бл
    </div>
    <div>
        <strong class="text-white block mb-2">Аналогия</strong>
        <p class="text-slate-400">Чередовани
        вает один из них в корень, превращая дерево
        ступает только для <em>одной</em> конкретной
    </div>
    <div>
        <strong class="text-white block mb-2">Как работает?</strong>
        <p class="text-slate-400">Треар дер
        play не хранит вообще ничего и держит балан
    </div>
    </div>
</div>
</section>`,
    },
    {
        id: "prefix-sums-2d",
        title: "5. Двумерная матрица префиксных сумм",
        type: "html",
    },

```

```

<section id="graph-planar-colors" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 rounded">
    <h2 class="text-2xl sm:text-3xl font-bold text-blue-400">
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border">
      <h3 class="text-xl font-bold text-blue-400">
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-blue-300 italic">
        <p class="text-xl font-mono text-white">
        ый граф можно раскрасить 4 цветами.</p>
      </div>
      <div class="grid grid-cols-1 gap-6">
        <div class="bg-slate-800 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
            Аналогия 1: Карта мира
          </div>
          <p class="text-slate-300 text-sm mb-4">
            . Границы не пересекаются в одной точке по-
            ы не сливались цветами, Всегда можно всего
          </div>
        </div>
      </div>
    </div>
  </div>
</section>`,
  },
  {
    id: "top-sort",
    title: "9. Графы. Топологическая сортировка",
    type: "html",
    description: "Разложение задач по порядку выполнения",
    category: "Графы",
    content: `
<section id="top-sort" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 rounded">

```



```

        category: "Графы. Продвинутое",
        content: `
<section id="scc-kosaraju" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 rounded">
    <h2 class="text-2xl sm:text-3xl font-bold text-white">
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border">
      <h3 class="text-xl font-bold text-emerald-400">
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-emerald-300 italic">
        <p class="text-xl font-mono text-white">
        нвертированному в порядке top-sort.</p>
      </div>
      <div class="grid grid-cols-1 gap-6">
        <div class="bg-slate-800 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 border
            order border-emerald-500 shadow-md">
            Аналогия 1: Улицы с односторонним движением.
          </div>
          <p class="text-slate-300 text-sm mb-4">
            В реальных городах существуют улицы, которые имеют
            БЫХ направлениях по односторонним улицам. Как это
            у на карте.</p>
          </div>
        </div>
      </div>
    </div>
  </div>
</section>`,
      },
      {
        id: "graph-bridges",
        title: "11. Графы. Мосты",
        type: "html",
        description: "Рёбра, удаление которых расхреначивает граф",
        category: "Графы. Продвинутое",
        content: `
<section id="graph-bridges" class="mb-12 scroll-mt-10">

```

```

<div class="space-y-8">
  <div class="bg-slate-700/50 p-6 rounded-xl border"
    <h3 class="text-xl font-bold text-rose-400"
    <div class="bg-slate-900 p-4 rounded-lg mb-1"
      <p class="text-lg text-rose-300 italic"
      <p class="text-xl font-mono text-white"
тей > 1).</p>
    </div>
    <div class="grid grid-cols-1 gap-6">
      <div class="bg-slate-800 p-6 rounded-lg"
        <div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">
          Аналогия 1: Центральный роутер
        </div>
        <p class="text-slate-300 text-sm mb
правильный роутер - и два сегмента офиса б
        </div>
      </div>
    </div>
  </div>
</section>`,
  },
  {
    id: "graph-euler",
    title: "13. Графы. Эйлеров цикл",
    type: "html",
    description: "Пройти по всем ребрам ровно 1 раз.",
    category: "Графы",
    content: `
<section id="graph-euler" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 r
    <h2 class="text-2xl sm:text-3xl font-bold text-r
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border"
      <h3 class="text-xl font-bold text-indigo-400"
      <div class="bg-slate-900 p-4 rounded-lg mb-1

```

```

<div class="grid grid-cols-1 gap-6">
  <div class="bg-slate-800 p-6 rounded-lg">
    <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
      Аналогия: Копаем туннель под
    </div>
    <p class="text-slate-300 text-sm mb-6">
      ней земли. Но если начать копать одновременно
      в 2 РАЗА меньше работы (геометрически: площ
    </div>
  </div>
</div>
</div>
</section>`,
  },
  {
    id: "mst-kruskal",
    title: "18. Графы. Остовное дерево. Краскал",
    type: "html",
    description: "",
    category: "Графы. Остовные",
    content: `
<section id="mst-kruskal" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 rounded">
    <h2 class="text-2xl sm:text-3xl font-bold text-white">
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-600">
      <h3 class="text-xl font-bold text-emerald-400">
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-emerald-300 italic">
        <p class="text-xl font-mono text-white">
          (проверяем через DSU) - берем в ответ.</p>
      </div>
      <div class="grid grid-cols-1 gap-6">
        <div class="bg-slate-800 p-6 rounded-lg">
          <div class="absolute -top-3 left-4

```

```

<p class="text-slate-300 text-sm mb-6">
    ли вирус). Мы стартуем из одного города, и
    Сеть растет только из одного связного куска
  </div>
</div>
</div>
</div>
</section>`,
  },
  {
    id: "mst-boruvka",
    title: "20. Графы. Остовное дерево. Борувка",
    type: "html",
    description: "",
    category: "Графы. Остовные",
    content: `
<section id="mst-boruvka" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 rounded">
    <h2 class="text-2xl sm:text-3xl font-bold text-white">
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border">
      <h3 class="text-xl font-bold text-emerald-400">
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-emerald-300 italic">
        <p class="text-xl font-mono text-white">
        ается с соседом.</p>
      </div>
      <div class="grid grid-cols-1 gap-6">
        <div class="bg-slate-800 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 border
            order border-emerald-500 shadow-md">
            Аналогия: Племена
          </div>
          <p class="text-slate-300 text-sm mb-6">
            изкому племени для объединения. К вечеру пл
            ства шлют гонцов к ближайшему чужому царств

```

```

        </div>
        <details class="bg-slate-900/40 rounded-lg border
            <summary class="p-4 font-bold text-slate-300
                -b border-slate-700/50">
                    Код (Скрыто)
            </summary>
            <div class="p-5 text-sm text-slate-300
                <pre class="bg-slate-950 p-4 rounded
vector<int> pi(s.length());
for (int i = 1; i < s.length(); i++) {
    int j = pi[i-1];
    while (j > 0 && s[i] != s[j]) j = pi[j-1];
    if (s[i] == s[j]) j++;
    pi[i] = j;
}</pre>
            </div>
        </details>
    </div>
</div>
</section>`,
    },
    {
        id: "string-z-func",
        title: "22. Z-функция",
        type: "html",
        description: "",
        category: "Строки",
        content: `
<section id="string-z-func" class="mb-12 scroll-mt-10">
    <div class="flex items-center mb-6 flex-wrap gap-3">
        <span class="bg-indigo-600 text-white px-4 py-1">
        <h2 class="text-2xl sm:text-3xl font-bold text-violet-400">
    </div>
    <div class="space-y-8">
        <div class="bg-slate-700/50 p-6 rounded-xl border
            <h3 class="text-xl font-bold text-blue-400">
            <div class="bg-slate-900 p-4 rounded-lg mb-4">
                <p class="text-lg text-blue-300 italic">

```

```
category: "Теория сложности",
content: `
<section id="complexity-classes" class="mb-12 scroll-mt-12">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-purple-600 text-white px-4 py-1 rounded-md">Сложность
    <h2 class="text-2xl sm:text-3xl font-bold text-white">Сложность
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-600">
      <h3 class="text-xl font-bold text-purple-400">Сведение
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-purple-300 italic">Сведение (Reduction)
        <p class="text-xl font-mono text-white">Сведение (Reduction) A->B: Если я умею решать A, то я умею решать B.
      </div>
      <div class="grid grid-cols-1 xl:grid-cols-2 gap-4">
        <!-- Аналогия 1 -->
        <div class="bg-slate-800 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 right-4 bottom-4 border border-emerald-500 shadow-md">
            Аналогия 1: Судoku (P vs NP)
          </div>
          <p class="text-slate-300 text-sm mb-4">Пример сортировка чисел). Класс <b>NP</b> – это класс задач, для которых существует полиномиальная проверка (ответ), он БЫСТРО (за класс P) проверяется.
        </div>
        <!-- Аналогия 2 -->
        <div class="bg-slate-900 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 right-4 bottom-4 border border-purple-500 shadow-md">
            Аналогия 2: Машина-переводчик
          </div>
          <p class="text-slate-300 text-sm mb-4">Если у нас есть отличный переводчик на английский (Сведение A->B), то B – это <b>NP-Полная</b> (сосредоточена на самых сложных задачах).
        </div>
      </div>
    </div>
  </div>
`
```

```

        <p><strong class="text-white">Эйлер
        <div class="p-3 bg-slate-950 rounded
            <p class="text-sm text-blue-300
            <p><strong class="text-white">П
            <p><strong class="text-white">Ц
        </div>
    </div>
</div>

<p class="text-slate-300 text-sm">
    <strong>Билет 13 (Различие пути и цикла
    <br>• <span class="text-green-400 font-
шел-вошёл-вышел).
    <br>• <span class="text-yellow-400 font
финиш).
</p>
</div>

<div class="grid grid-cols-1 xl:grid-cols-2 gap
    <div class="bg-slate-800 p-6 rounded-lg bor
        <div class="absolute -top-3 left-4 bg-s
order-green-500 shadow-md">
            Путь: Нормальная прогулка
        </div>
        <p class="text-slate-300 text-sm mb-4">
            Ты вышел из дома <strong>(нечётная)
            талый пришёл в бар <strong>(вторая нечётная
            Домой вернуться не смог, потому что
        </p>
    </div>

    <div class="bg-slate-900 p-6 rounded-lg bor
        <div class="absolute -top-3 left-4 bg-r
der-rose-500 shadow-md bg-slate-950">
            Цикл: Гамильтонов синдром (болезнь
        </div>
        <p class="text-slate-300 text-sm mb-4">
            <strong>Гамильтон — это болезнь.</s

```

```

    <span class="bg-emerald-600 text-white px-4 py-4">
    <h2 class="text-3xl font-bold text-white">Мосты
  </div>

```

```

<div class="space-y-8">
  <div class="bg-slate-700/50 p-6 rounded-xl border-emerald-500">
    <h3 class="text-xl font-bold text-emerald-400">Мост: Единственная веревка

    <div class="bg-slate-900 p-4 rounded-lg mb-4">
      <p class="text-lg text-emerald-300 italic">Представь, что ты альпинист. Ты спустился в пещеру.
      <div class="text-left font-mono text-sm">
        <p><strong class="text-white">tin[v]
        <p><strong class="text-white">low[v]
        <div class="p-3 bg-slate-950 rounded">
          <span class="text-rose-400 font-mono text-sm">
            <p class="text-xl font-bold text-white">Мост: Единственная веревка
            <p class="text-xs text-slate-400">
          </div>
        </div>
      </div>
    </div>
  </div>

```

```

    <p class="text-slate-300 text-sm">
      Если из сына u и всех его потомков <strong>
      Единственная ниточка. Порвётся – граф развалится.
    </p>
  </div>

```

```

<div class="grid grid-cols-1 xl:grid-cols-2 gap-4">
  <div class="bg-slate-800 p-6 rounded-lg border-emerald-500">
    <div class="absolute -top-3 left-4 bg-slate-900 border-emerald-500 shadow-md">
      Мост: Единственная веревка
    </div>
    <p class="text-slate-300 text-sm mb-4">
      Представь, что ты альпинист. Ты спустился в пещеру.
      Из пещеры и нет других выходов – только веревка.
      Если верёвка (ребро v-u) оборвётся – граф развалится.
      Если бы из пещеры и был чёрный ход и

```



```
void dfs(int v, int p = -1) {
    visited[v] = true;
    tin[v] = low[v] = timer++;           // устанавливаем

    for (int to : adj[v]) {
        if (to == p) continue;         // не идём назад

        if (visited[to]) {
            // обратное ребро: обновляем low
            low[v] = min(low[v], tin[to]);
        } else {
            // ребро дерева DFS
            dfs(to, v);                 // рекурсивный вызов
            low[v] = min(low[v], low[to]);

            if (low[to] > tin[v]) {
                // ЭТО МОСТ!
                // bridge_list.push_back({v, to});
            }
        }
    }
}
```

```
void find_bridges() {
    timer = 0;
    visited.assign(n, false);
    tin.assign(n, -1);
    low.assign(n, -1);

    for (int i = 0; i < n; ++i) {
        if (!visited[i]) dfs(i);
    }
}
```

</div>

<p class="text-xs text-slate-400">Занес

</div>

</details>

K5: Полный граф на 5 вершинах

</div>

<p class="text-slate-300 text-sm mb-4">

У K5: `class="text-white">V=5, -white">10 > 9</code> – БАХ, непланарен!`

Все 5 вершин соединены со всеми. На

Теорема Куратовского: K5 – эталон не

Если внутри графа спрятан K5 – забудь,

</p>

</div>

<div class="bg-slate-900 p-6 rounded-lg border">

<div class="absolute -top-3 left-4 bg-rose-500 shadow-md">

K3,3: 3 дома и 3 колодца

</div>

<p class="text-slate-300 text-sm mb-4">

Три дома хотят соединиться с тремя

Нарисовать без пересечений невозможно

У него `class="text-white">V=6 xt-white">9 ≤ 12</code> – проходит, но он не`

Почему? Потому что у двудольного графа

Отсюда более строгое ограничение: `<`

`class="text-white">9 > 8</code>`

</p>

</div>

</div>

<details class="bg-slate-900/40 rounded-lg border">

<summary class="p-4 font-bold text-slate-400 order-slate-700/50">

Доказательство непланарности K5 (Скрыто)

</summary>

<div class="p-5 text-sm text-slate-300 space-y-2">

<p class="text-blue-400">Доказательство

<p>

Пусть K5 планарен. $V = 5$, $E = 10$.

 $F = E - V + 2 = 10 - 5 + 2 = 7$

Важно: Это работает только в тех случаях, когда мы переходим к точкам сочленения. То есть, **вершина** - это тупик со степенью 1).

```
<div class="grid grid-cols-1 xl:grid-cols-2 gap-4">
  <div class="bg-slate-800 p-6 rounded-lg border border-rose-500 shadow-md">
```

Аналогия: Главный перекресток

```
<p class="text-slate-300 text-sm mb-4">
```

Представь город, где два района соединены перекрестком. Если перекресток выйдет из строя, её перекроют (удалят вершину) – районы будут изолированы. **Хрупкость** системы.

```
<div class="bg-slate-900 p-6 rounded-lg border border-emerald-500 shadow-md">
```

Телеком: Центральный свитч

```
<p class="text-slate-300 text-sm mb-4">
```

У тебя несколько компьютеров в одной сети, соединенных в звезду (мостом). Сами свитчи – это точки сочленения.

```
<details class="bg-slate-900/40 rounded-lg border border-slate-700/50">
  <summary class="p-4 font-bold text-slate-400">
```

Душный мод: Код и корень DFS (Скрыто)

```
<div class="p-5 text-sm text-slate-300 space-y-2">
```

```
<p class="text-blue-400 font-bold">Обсуждение
```

```
ong> и показывают физическую уязвимость свя
    <li><strong class="text-emerald-400">SCC
ависимые "конгломераты".</li>
    <li><strong>Как точка сочленения рушит
Косарайю, свернув каждую SCC в одну супер-в
(сделаем неориентированным), то любая <strong
о и есть критическая SCC! Если удалить эту
. Одно слабое звено изолирует целые касты S
</ul>
    </div>
  </div>
</section>`,
  },
];
```

ФАЙЛ 19/86: src/data/content.ts

КАТЕГОРИЯ: Контент и данные пособия | СТРОК: 51 | РАЗМЕР: 1.5 КБ

```
import { Chapter } from "../types";
import { graphChapters } from "./graphContent";
import { additionalTickets } from "./additionalTickets";
import { tickets14to20 } from "./tickets/ticket14_20";
import { tickets21to24 } from "./tickets/ticket21_24";
import { tickets6to13 } from "./tickets/ticket6_13";
import { chapters as newChapters } from "./content_temp";

// We want to combine all of them, but overwrite 7, 11,
const merged = [
  ...graphChapters,
  ...additionalTickets,
  ...tickets6to13,
  ...tickets14to20,
  ...tickets21to24
].filter(c => c && c.id);
```

```
/**
 * Пошаговый дебаггер поверх Pyodide: CPython выполняет
 * под sys.settrace и на каждой строке записывает фрейм
 * как панель Variables в обычном отладчике.
 */
```

```
export interface DebugStep {
  /** Номер строки в коде пользователя (1-based). */
  line: number;
  /** Имя функции-фрейма ("<module>" – верхний уровень)
  func: string;
  /** Локальные переменные: имя → герг (усечённый). */
  locals: Record<string, string>;
}
```

```
export interface DebugResult {
  steps: DebugStep[];
  /** Текст последней необработанной ошибки (или пустая
  error: string;
  /** true, если уперлись в лимит шагов. */
  truncated: boolean;
}
```

```
/** Лимит шагов трассы – защита от бесконечных циклов и
export const DEBUG_STEP_CAP = 500;
```

```
/**
 * Строит самодостаточный python-скрипт: выполняет userSrc
 * и возвращает JSON с шагами (строка + func + locals)
 * последнего выражения (его вернёт runPythonAsync).
 */
```

```
export function buildDebugRunner(userSrc: string): string {
  const srcLiteral = JSON.stringify(userSrc); // JSON-сериализация
  return `import sys as __sys, json as __json

  src = ${srcLiteral}
```

```

}

/**
 * Какие переменные изменились на этом шаге относительно
 * они подсвечиваются (как amber-подсветка изменений в
 * Новые переменные тоже считаются изменёнными.
 */
export function changedVars(
  prev: Record<string, string> | null | undefined,
  cur: Record<string, string>
): Record<string, boolean> {
  const out: Record<string, boolean> = {};
  for (const k of Object.keys(cur)) {
    out[k] = !prev || !(k in prev) || prev[k] !== cur[k];
  }
  return out;
}

/** Базовое имя из карточки переменной: "tin[v]" → "tin"
export const baseVarName = (name: string): string => name.slice(0, -1);

/**
 * Порядок показа в панели переменных: сначала переменные
 * страницы (те, что подсвечивает визуализация), остальные
 */
export function orderVars(
  cur: Record<string, string>,
  syncNames: string[]
): [string, string][] {
  const inSync = new Set(syncNames);
  const syncEntries = Object.entries(cur).filter(([k]) => inSync.has(k));
  const rest = Object.entries(cur)
    .filter(([k]) => !inSync.has(k))
    .sort(([a], [b]) => a.localeCompare(b));
  return [...syncEntries, ...rest];
}

```

```

        "Очередь (FIFO). Кладём старт, достаём вершину —
        у рёбер.",
        complexity: "O(V + E)",
        demo: "bfs",
        simulator: "queue-bfs",
    },
    {
        id: "dfs",
        labels: ["DFS", "обход в глубину", "поиск в глубину"],
        title: "DFS — обход в глубину",
        short:
            "Прёшь вперёд до упора, упёрся — откатываешься на
        formal:
            "Стек (или рекурсия). Из DFS растут: времена вход
        complexity: "O(V + E)",
        demo: "dfs",
        simulator: "stack-dfs",
    },
    {
        id: "heap",
        labels: ["куча", "кучу", "кучи", "куче", "кучей", "куча"],
        title: "Куча (priority queue)",
        short:
            "Не отсортированный список, а «мешок, из которого
        formal:
            "Полное бинарное дерево в массиве: дети узла i ле
        complexity: "вставка/извлечение O(log n), «посмотре
        demo: "heap",
        simulator: "heap-beam-search",
    },
    {
        id: "stack",
        labels: ["стек", "стека", "стеке", "стеком", "LIFO"],
        title: "Стек (LIFO)",
        short: "Стопка тарелок: кладёшь сверху и берёшь све
        formal: "push / pop / top за O(1). На стеке живёт р
        demo: "stack",
        simulator: "stack-dfs",
    },

```

```

        "обход в ширину по бору",
        "какой-то обход",
        "обходом дерева",
        "конечный автомат",
        "конечного автомата",
        "автомат",
        "автомата",
    ],
    title: "Обход бора в ширину (тот самый «какой-то обход»)",
    short:
        "В Ахо–Корасике бор обходят именно BFS: суффиксную",
        "а есть строго по уровням.",
    formal:
        "Кладём в очередь детей корня (их ссылка = корень",
        "и кладём с в очередь.",
    complexity: "O(суммарной длины словаря × алфавит)",
    demo: "trie-bfs",
    simulator: "aho-corasick",
},
{
    id: "suffix-link",
    labels: ["суффиксная ссылка", "суффиксные ссылки",
    title: "Суффиксная ссылка",
    short:
        "«Куда откатиться, если следующая буква не подошла",
        "строки.",
    formal: "link(v) = go(link(parent(v)), ch). Считает",
    demo: "suffix-link",
    simulator: "aho-corasick",
},
{
    id: "toposort",
    labels: ["топологическая сортировка", "топсорт", "топ",
    title: "Топологическая сортировка",
    short:
        "Порядок «сначала штаны, потом ботинки»: каждая с",
        "циклов.",
    formal: "DFS + вершины в обратном порядке выхода, л
```



```

    complexity: "препроцессинг  $O(n \log n)$ , запрос  $O(1)$ ",
    demo: "sparse-table",
    simulator: "sparse-table",
},
{
    id: "segment-tree",
    labels: ["дерево отрезков", "дереве отрезков", "дер"],
    title: "Дерево отрезков",
    short:
        "Матрёшка из отрезков: корень отвечает за весь ма
        товых кусков.",
    formal: "Строится за  $O(n)$ , запрос и обновление —  $O(1)$ ",
    demo: "segment-tree",
    simulator: "segment-trees",
},
{
    id: "bridge",
    labels: ["мост", "моста", "мосты", "мостов", "мостов"],
    title: "Мост",
    short:
        "Ребро, после удаления которого граф разваливается.",
    formal: "Ребро  $(u, v)$  — мост  $\iff \text{low}[v] > \text{tin}[u]$ , где  $\text{tin}[u]$  — время, когда зашло ребро в родителя — нет).",
    complexity: " $O(V + E)$  одним DFS",
    demo: "bridge",
    simulator: "bridges-code",
},
{
    id: "articulation",
    labels: ["точка сочленения", "точки сочленения", "точка сочленения"],
    title: "Точка сочленения",
    short:
        "Вершина-незаменимый-сотрудник: уволь её — и коллапс.",
    formal: "Для не-корня:  $v$  — точка сочленения  $\iff \exists \text{ребро } (u, v) \text{ такое, что } \text{low}[v] \geq \text{tin}[u]$ ",
    demo: "articulation",
    simulator: "graph-articulation",
},
{

```

```

    },
    {
      id: "potential",
      labels: ["потенциал", "потенциалы", "потенциалов",
        title: "Потенциалы (перевзвешивание Джонсона)",
        short:
          "Трюк «поднять все дороги на одинаковую высоту»:",
        formal: " $w'(u,v) = w(u,v) + h(u) - h(v)$ , где  $h$  – рас-
          к путей сохраняется.",
        demo: "relax",
        simulator: "johnson-algo",
    },
    {
      id: "scc",
      labels: ["компонента сильной связности", "компоненты",
        title: "Компоненты сильной связности",
        short:
          "Группы вершин, где из каждой можно дойти до каждо-
        formal: "Косарайю: DFS с сохранением порядка выхода",
        complexity: " $O(V + E)$ ",
        demo: "scc",
        simulator: "scc-kosaraju",
    },
    {
      id: "prefix-function",
      labels: [
        "префикс-функция", "префикс-функции", "префикс-функ-
        "префикс", "префикса", "суффиксом", "суффикс", "с-
        "подстроки", "подстроку", "подстрока",
      ],
      title: "Префикс-функция  $\pi$  (КМП)",
      short:
        "Для каждой позиции запоминаем: какой кусок начал-
          от кусок.",
      formal:
        " $\pi[i]$  – длина наибольшего собственного префикса  $s$ 
          возвращая указатель по тексту.",
      complexity: " $O(n + m)$ ",
    }
  ],

```

```

        "Дейкстра — неотрицательные веса,  $O(E \log V)$ . Форд-Беллман,  $O(V^3)$ .",
        demo: "relax",
        simulator: "dijkstra",
    },
    {
        id: "negative-cycle",
        labels: ["отрицательный цикл", "отрицательного цикла"],
        title: "Отрицательный цикл",
        short:
            "Круг, пройдя по которому вы становитесь «богаче»",
        formal:
            "Форд–Беллман: если на  $V$ -й итерации ещё удаётся с...",
        demo: "relax",
        simulator: "bellman-ford",
    },
    {
        id: "greedy",
        labels: [
            "жадный алгоритм", "жадный", "жадно", "жадная",
            "самое дешёвое ребро", "самое дешёвое ребро", "самое дешёвое ребро",
        ],
        title: "Жадный алгоритм",
        short:
            "На каждом шаге хватаем то, что лучше прямо сейчас",
        formal:
            "Корректность обычно доказывается «леммой о разрезе»",
        demo: "mst",
        simulator: "mst-kruskal",
    },
    {
        id: "euler",
        labels: ["эйлеров цикл", "эйлеров путь", "эйлерова"],
        title: "Эйлеров путь и цикл",
        short:
            "Пройти по каждому ребру ровно один раз, не отрываясь",
        formal:
            "Связный граф: эйлеров цикл    все степени чётные;

```

```

    id: "dijkstra",
    labels: ["Дейкстра", "Дейкстры", "Дейкстре", "Дейкс",
    title: "Алгоритм Дейкстры",
    short:
        "Жадный «навигатор»: всегда раскрываем ближайший
        льные.",
    formal:
        "Куча хранит пары (расстояние, вершина). Достали
    complexity: "O(E log V) с бинарной кучей",
    demo: "relax",
    simulator: "dijkstra",
},
{
    id: "bellman-ford",
    labels: ["Форд-Беллман", "Форда-Беллмана", "Беллман",
    title: "Алгоритм Форда-Беллмана",
    short:
        "Тупой, но честный: V-1 раз проходим по ВСЕМ рёбрам",
    formal:
        "После k-й итерации гарантированно верны все кратчайшие
        ось — есть отрицательный цикл.",
    complexity: "O(V · E)",
    demo: "relax",
    simulator: "bellman-ford",
},
{
    id: "floyd",
    labels: [
        "Флойд", "Флойда", "Флойд-Уоршелл", "Флойда-Уоршелла",
        "промежуточная вершина", "промежуточную вершину",
    ],
    title: "Флойд-Уоршелл: разрешаем вершину k",
    short:
        "Внешний цикл — это не «шаг», а «какие пересадки разрешены
        шить пересадку в k?",
    formal:
        "D[i][j] = min(D[i][j], D[i][k] + D[k][j]). Порядок
```

```

    complexity: "O(E log E)",
    demo: "mst",
    simulator: "mst-kruskal",
},
{
    id: "components",
    labels: [
        "компонента связности", "компоненты связности", "
        "компоненте связности", "связности", "связный", "
    ],
    title: "Компоненты связности",
    short:
        "Граф-архипелаг: острова, между которыми нет мост
        бой.",
    formal:
        "Считаются одним проходом: пока есть непосещённая
    complexity: "O(V + E)",
    demo: "components",
    simulator: "graph-dfs-bfs",
},
{
    id: "graph",
    labels: [
        "граф", "графа", "графе", "графы", "графов", "гра
        "вершины", "вершина", "вершин", "ребра с весом",
    ],
    title: "Граф",
    short:
        "Точки (вершины) и связи между ними (рёбра). Горо
    formal:
        "G = (V, E). Ребро бывает ориентированным (улица
    demo: "graph",
    simulator: "graph-dfs-bfs",
},
{
    id: "coord-compress",
    labels: ["сжатие координат", "сжатия координат", "с
    title: "Сжатие координат",

```

```

    ],
    title: "Включения-исключения на префиксных суммах",
    short:
        "Чтобы получить сумму прямоугольника, берём больш
        ый отрезали дважды.",
    formal:
        "Sum(x1..x2, y1..y2) = P[x2][y2] - P[x1-1][y2] - P[x2][y1-1] + P[x1-1][y1-1]",
    complexity: "запрос O(1) после O(nm) предподсчёта",
    demo: "prefix-sum",
    simulator: "prefix-sums-2d",
},
{
    id: "reduction",
    labels: ["сведение", "сведения", "сведении", "сведён
    title: "Сведение задач",
    short:
        "«Я не умею решать A, зато умею B». Если A можно
    formal:
        "A ≤p B: есть полиномиальная функция, переводящая
        NP-полную задачу к новой.",
    demo: "np",
},
{
    id: "range",
    labels: ["отрезок", "отрезка", "отрезке", "отрезки"
    title: "Запрос на отрезке",
    short:
        "Спрашиваем не про один элемент, а про кусок массива
        предподсчёты.",
    formal:
        "Суммы – префиксные суммы O(1). Минимум без изменен
    demo: "prefix-sum",
    simulator: "prefix-sums-2d",
},
{
    id: "string",
    labels: ["строки", "строке", "строку", "строка", "с
    title: "Строка как массив символов",

```

```

    "Дерево вытянулось в «бамбук»: g → p → x идут в о
    м (p, x). Именно это вдвое сокращает глубину вс
    formal:
    "Если крутить наивно (два раза поднимать x), бамб
    demo: "zig-zig",
    simulator: "splay-rotations",
  },
  {
    id: "zig-zag",
    labels: ["Zig-Zag", "зиг-заг", "ZigZag", "зигзаг"],
    title: "Zig-Zag – x и родитель с разных сторон",
    short:
      "Узлы идут «змейкой»: p – левый сын g, а x – прав
      p и g становятся двумя детьми x.",
    formal: "Эквивалентно двойному повороту AVL-дерева
    demo: "zig-zag",
    simulator: "splay-rotations",
  },
];

/** Быстрый доступ по id. */
export const GLOSSARY_BY_ID = new Map(GLOSSARY.map((e)

/** Все метки, отсортированные от длинных к коротким (ч
export const GLOSSARY_LABELS: { label: string; id: string }[] =
  e.labels.map((label) => ({ label, id: e.id }))
).sort((a, b) => b.label.length - a.label.length);

```

ФАЙЛ 22/86: src/data/graphContent.ts

КАТЕГОРИЯ: Контент и данные пособия | СТРОК: 275 | РАЗМ

```
import { Chapter } from "../types";
```

```
export const graphChapters: Chapter[] = [
  {
```

```
<p class="text-xl font-mono text-white">
</div>

<div class="grid grid-cols-1 xl:grid-cols-2"
  <!-- Аналогия 1 -->
  <div class="bg-slate-800 p-6 rounded-lg"
    <div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">
      Аналогия 1: Позитивное плани
    </div>
    <p class="text-slate-300 text-sm mb
(нельзя поехать и заработать). Он всегда в
    <div id="slot-chapter-image-dijkstr
    </div>

  <!-- Аналогия 2 -->
  <div class="bg-slate-900 p-6 rounded-lg"
    <div class="absolute -top-3 left-4 l
border-rose-500 shadow-md">
      Ограничения
    </div>
    <p class="text-slate-300 text-sm mb
охождение улицы), Дейкстра сломается, так к
    </div>
  </div>

<details class="bg-slate-900/40 rounded-lg l
  <summary class="p-4 font-bold text-slate
-b border-slate-700/50">
    Строгое доказательство (Скрыто)
  </summary>
  <div class="p-5 text-sm text-slate-300"
    <p>Алгоритм использует приоритетную
g V). Гарантирует корректность только для н
    </div>
  </details>

<div id="slot-dijkstra-viz" class="mt-8"></div>
```



```

        <div class="absolute -top-3 left-4 border-rose-500 shadow-md">
            Отрицательный цикл
        </div>
        <p class="text-slate-300 text-sm mb-1">
            г станет ЕЩЕ короче – значит мы попали во в
        </div>
    </div>

    <div id="slot-bellman-viz" class="mt-8"></div>
</div>
</section>`
},
{
    id: "floyd",
    title: "Билет 16. Флойд-Уоршелл",
    type: "html",
    category: "Графы",
    content: `<section id="floyd-content" class="mb-12">
        <div class="flex items-center mb-6">
            <span class="bg-blue-600 text-white px-4 py-1 rounded">
                <h2 class="text-3xl font-bold text-white">Флойд
            </div>

            <div class="space-y-8">
                <div class="bg-slate-700/50 p-6 rounded-xl border">
                    <h3 class="text-xl font-bold text-indigo-400">
                        Алгоритм Флойда-Уоршелла
                    </h3>

                    <div class="bg-slate-900 p-4 rounded-lg mb-4">
                        <p class="text-lg text-indigo-300 italic">
                            Алгоритм Флойда-Уоршелла
                        <p class="text-xl font-mono text-white">
                            Алгоритм Флойда-Уоршелла
                        </div>

                        <div class="grid grid-cols-1 xl:grid-cols-2">
                            <!-- Аналогия 1 -->
                            <div class="bg-slate-800 p-6 rounded-lg">
                                <div class="absolute -top-3 left-4

```

```
<div class="bg-slate-900 p-4 rounded-lg mb-6 text-white">
  <p class="text-sm text-blue-300 italic mb-2">0cc
  <p class="text-lg font-mono text-white">Бор (Терминальные)
</div>
<p class="text-slate-300 text-sm">Позволяет найти
  -mono text-amber-300 bg-black/30 px-1 rounded">
  у.</p>
</div>

<!-- Аналогии -->
<div class="grid grid-cols-1 xl:grid-cols-2 gap-6">
  <div class="bg-slate-800 p-6 rounded-lg border border-rose-900/50 shadow-md">
    <div class="absolute -top-3 left-4 bg-slate-700 text-white p-2 rounded">
      Аналогия 1: Паутина (Бор)
    </div>
    <p class="text-slate-300 text-sm mb-4">Представим, что мы движемся по веткам. Если нужного продолжения ("нити страховки") в самый длинный из известных нам путей нет, мы идем по веткам, пока не найдем "нити страховки".
  </div>
  <div class="bg-slate-900 p-6 rounded-lg border border-rose-900/50 shadow-md">
    <div class="absolute -top-3 left-4 bg-rose-900 text-white p-2 rounded">
      Аналогия 2: Матёшка (Терминальные)
    </div>
    <p class="text-slate-300 text-sm mb-4">Представим, что мы нашли и "he", потому что оно спрятано внутри терминальные ссылки</b> (term_link) – прямые ссылки на терминальные ссылки.
  </div>
</div>

<details class="bg-slate-900/40 rounded-lg border border-rose-900/50 shadow-md">
  <summary class="p-4 font-bold text-slate-400 outline-slate-400/50">
    Внутренняя структура автомата и код BFS (Скрытое)
```

```

        q.push(child);
    }
}
    </div>
  </div>
</details>
</div>
</section>`
  },
  {
    id: "alg-map",
    title: "Алгоритмы на карте",
    type: "html",
    category: "Графы",
    content: `<section id="alg-map-content" class="mb-1">
<div class="flex items-center mb-6">
  <span class="bg-purple-600 text-white px-4 py-1">
    <h2 class="text-3xl font-bold text-white">Алгори
  </div>

  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border">
      <h3 class="text-xl font-bold text-purple-400">
        <p class="text-slate-300 text-sm mb-4">Когда
        ут быть большими дробными числами (долгота
        ву от 0 до N, и уже по ним строим графы.</p>
      </div>
    </div>
  </div>
</section>`
  }
];

```

```

    ],
    correctIndex: 2,
    explanation: "Если нечётных вершин больше 2, эйлерово
        ровно 2 нечётные или 0."
    }
],
"bridges-code": [
    {
        question: "После DFS у нас low[u] = 4, а tin[v] = 2",
        options: [
            "Да, так как low[u] (4) > tin[v] (2)",
            "Нет, low[u] должно быть меньше или равно",
            "Только если граф связный и неориентированный"
        ],
        correctIndex: 0,
        explanation: "Условие моста: low[u] > tin[v]. В нашем
            случае low[u] = 4, tin[v] = 2, поэтому условие
            выполняется."
    },
    {
        question: "Для чего нужен массив low[v]?",
        options: [
            "Для того чтобы было больше переменных в коде",
            "Чтобы знать минимальный tin предка, до которого
                нужно идти",
            "Чтобы хранить глубину рекурсии"
        ],
        correctIndex: 1,
        explanation: "low[v] хранит минимальное время входа
            в вершину v во время поиска мостов и точек сочленения."
    },
    {
        question: "Какова временная сложность алгоритма поиска
            мостов?",
        options: [
            "O(V^2) — ищем мосты в квадрате",
            "O(V + E) — каждый элемент один раз",
            "O(2^V) — полный перебор"
        ],
        correctIndex: 1,
        explanation: "DFS обходит каждую вершину и каждое
            ребро ровно один раз."
    }
]

```

ФАЙЛ 24/86: src/data/vizSync.ts

КАТЕГОРИЯ: Контент и данные пособия | СТРОК: 575 | РАЗМ

```
/**
 * Захардкоженная «карта синхронизации» компилятора с ви
 *
 * Для каждой страницы (главы) здесь вручную перечислен
 * по которым идёт пошаговая подсветка в её интерактивн
 * в первую очередь индексы циклов i, j, k и размеры n,
 *
 * Компилятор НЕ показывает код алгоритма – только комм
 * переменными, чтобы студент повторял шаги руками, гля
 */

export interface SyncVariable {
  /** Имя переменной, как в коде визуализации: i, j, k,
   name: string;
  /** Роль в алгоритме (показывается во всплывающей под
   role: string;
  /** Характерный диапазон/значение на демонстрации. */
  range?: string;
}

export interface PageSync {
  /** Заголовок демонстрации на странице (если интеракт
   vizTitle?: string;
  /** Чему соответствует один шаг визуализации (для тул
   stepNote?: string;
  /** Переменные синхронизации. Пусто – на странице нет
   variables: SyncVariable[];
  /**
   * Минимальный исполняемый скелет для редактора: стро
   * шаг-в-шаг с подсветкой визуализации (без словесных
   */
  code?: string;
}
```

```

        { name: "zig / zag", role: "направление поворота:
    ],
},
"splay-rotations": {
    vizTitle: "Splay: Zig, Zig-Zig и Zig-Zag по шагам",
    stepNote: "Шаг = один кадр поворота из трёх классич
    code: `x, p, g = "x", "p", "g"    # узел, родитель,
print("zig:      ", x, "↑", p)
print("zig-zig: ", x, "↑", p, "↑", g)
print("zig-zag: ", x, "↗", g)`
    variables: [
        { name: "x", role: "узел, который поднимаем к корн
        { name: "p", role: "родитель узла x перед поворот
        { name: "g", role: "дедушка узла x – нужен в случ
    ],
},
"sparse-table": {
    vizTitle: "Разреженная таблица (1D и 2D)",
    stepNote: "Шаг построения = заполнение ячейки уровн
    code: `n = 8                                # размер массива
log = 3                                # k ≤ log2(n)

# 1D: ячейка (i, j) – блок [i, i + 2^j)
i = 0
j = 0

# 2D: 4 ключа → одна ячейка
st2 = {}                                # (r, c, kx, ky) → ответ
r, c, kx, ky = 0, 0, 0, 0
st2[(r, c, kx, ky)] = 0

for j in range(1, log + 1):
    for i in range(n - 2 ** j + 1):
        print(f"1D: st[{i}][{j}]")`
    variables: [
        { name: "n", role: "длина массива / сторона квадр
        { name: "k", role: "уровень таблицы: ячейка храни

```

```

vizTitle: "Куча (priority queue)",
stepNote: "Шаг = просеивание: вставленный элемент в
    .",
code: `a = [7, 3, 5, 1] # куча-массивом
n = len(a)
i = n - 1 # всплываемый элемент

parent = (i - 1) // 2 # родитель
kids = (2 * i + 1, 2 * i + 2) # дети
print(f"i={i} parent={parent} kids={kids}")`,
variables: [
    { name: "n", role: "текущее число элементов в куче" },
    { name: "i", role: "индекс элемента, который сейчас просеиваем" },
    { name: "(i-1)//2", role: "индекс родителя вершины" },
    { name: "2*i+1, 2*i+2", role: "индексы левого и правого ребенка" },
],
},
"stack-dfs": {
    vizTitle: "Стек и обход в глубину",
    stepNote: "Шаг = push новой вершины на стек или pop",
    code: `st = [] # стек = рекурсия
st.append("A") # push – зашли в вершину
st.append("B")
top = st.pop() # pop – возврат на уровень
print("pop →", top, "стек:", st)`,
    variables: [
        { name: "top", role: "вершина стека – то, откуда мы вышли" },
        { name: "v", role: "текущая вершина графа, которую мы рассматриваем" },
    ],
},
"queue-bfs": {
    vizTitle: "Очередь и обход в ширину",
    stepNote: "Шаг = dequeue вершины из головы очереди",
    code: `from collections import deque

q = deque(["A"])
q.append("B") # enqueue соседа
v = q.popleft() # шаг: обрабатываем голову

```

```

        { name: "visited", role: "множество уже посещённых",
          { name: "order", role: "порядок, в котором обход",
        ],
    },
    "graph-components": {
        // отдельного виджета нет, тема опирается на DFS/BFS
        variables: [],
    },
    "top-sort": {
        vizTitle: "Топологическая сортировка",
        stepNote: "Шаг = выход рекурсии из вершины v: она д
        code: `adj = {0: [1, 4], 1: [2, 3], 2: [], 3: [2],
used, order = set(), []

def dfs(v):
    used.add(v)
    for to in adj[v]:          # шаг: ребро v → to
        if to not in used:
            dfs(to)
    order.append(v)           # выход из v!

for v in adj:
    if v not in used:
        dfs(v)
print(order[::-1])           # разворот – ответ`,
    variables: [
        { name: "v", role: "текущая вершина DFS", range:
        { name: "to", role: "вершина, куда ведёт ребро из
        { name: "i", role: "перебор стартовых вершин во в
        { name: "order", role: "список выхода из рекурсии
    ],
},
"scc-kosaraju": {
    vizTitle: "Компоненты сильной связности (Косарайю)"
    stepNote: "Шаг = вершина из order (в обратном поряд
    code: `adj = {0: [1], 1: [2], 2: [0, 3], 3: [], 4:
adjT = {v: [] for v in adj}
for v in adj:

```



```

vizTitle: "Мосты: DFS + tin/low",
stepNote: "Шаг = строка псевдокода слева; ребро (v,
code: `tin, low = {"A": 1, "B": 2, "G": 3}, {"A": 1
timer = 3                                # как на демо

v, to = "B", "G"                        # шаг: возврат из G
is_bridge = low[to] > tin[v]             # 3 > 2 → мост!
print(f"ребро {v}-{to}: мост? {is_bridge}")`,
variables: [
    { name: "v", role: "текущая вершина DFS", range:
    { name: "to", role: "сосед; ребро (v, to) проверя
    { name: "tin[v]", role: "время входа в вершину –
    { name: "low[v]", role: "минимум tin, куда можно
    { name: "timer", role: "глобальный счётчик времени
],
},
"euler-path-vs-cycle": {
    vizTitle: "Эйлеров путь и цикл",
    stepNote: "Шаг = один клик по следующей вершине мар
    code: `edges = [(0,1), (0,2), (1,3), (2,4), (1,2),
deg = {}
for u, v in edges:                      # степени вершин
    deg[u] = deg.get(u, 0) + 1
    deg[v] = deg.get(v, 0) + 1

odd = [v for v in deg if deg[v] % 2]
print(deg, "нечётных:", len(odd))
# 0 → цикл • 2 → путь • иначе – нельзя`,
variables: [
    { name: "path", role: "текущий маршрут – последов
    { name: "last", role: "последняя вершина пути – о
    { name: "deg(v)", role: "степень вершины: чётная
],
},
"planarity-euler-formula": {
    vizTitle: "Планарность: K5 и K3,3",
    stepNote: "Шаг = попытка перетащить вершину так, что
    code: `V, E = 5, 10                  # K5      (K3,3: V=

```

```

-----
    if dist.get(u, float("inf")) + w < dist.get(v, float("inf")):
        dist[v] = dist[u] + w    # релаксация
    if i == 1:
        print(f"итерация {i}: {dist}")
        break`
    variables: [
        { name: "n", role: "число вершин графа – ровно n-1"},
        { name: "m", role: "число рёбер, по которым проходит путь"},
        { name: "i", role: "номер итерации (после i итераций)"},
        { name: "u, v, w", role: "текущее ребро: откуда, куда, вес"},
        { name: "dist[v]", role: "расстояние; если обновилось, то новое"},
    ],
},
floyd: {
    vizTitle: "Флойд–Уоршелл",
    stepNote: "Шаг = проверка dist[i][j] > dist[i][k] + dist[k][j]",
    code: `n = 7                                # матрица n×n (длина ребра не больше 10)
dist = [[float("inf")] * n for _ in range(n)]

for k in range(n):                            # промежуточная вершина
    for i in range(n):                        # откуда
        for j in range(n):                  # куда
            pass                             # шаг: dist[i][j] vs dist[i][k] + dist[k][j]
    variables: [
        { name: "n", role: "число вершин – размер квадрата матрицы"},
        { name: "k", role: "промежуточная вершина – ВНЕШНИЙ цикл"},
        { name: "i", role: "начало пути – строка матрицы"},
        { name: "j", role: "конец пути – столбец матрицы"},
        { name: "dist[i][j]", role: "кратчайший путь i → j"},
        { name: "dist[i][k]", role: "кратчайший путь i → k"},
        { name: "dist[k][j]", role: "кратчайший путь k → j"},
    ],
},
},
"johnson-algo": {
    vizTitle: "Алгоритм Джонсона (перевзвешивание)",
    stepNote: "Шаг = Беллман-Форд из фиктивной вершины",
    code: `# потенциалы из Беллмана-Форда от фиктивной вершины
h = {"A": 0, "B": -3, "C": -1, "D": 1, "E": 2, "F": 4}

```

```

    "mst-prima": {
        vizTitle: "Прим",
        stepNote: "Шаг = из кучи достаётся самое лёгкое ребро",
        code: `import heapq

start = "A"
heap = [(0, start, "start")]    # (вес, вершина, откуда)

w, v, parent = heapq.heappop(heap) # шаг: берём минимум
taken = {start}                  # дерево растёт от start
print(f"берём {v} за {w} (из {parent})"),
        variables: [
            { name: "v", role: "вершина, которую только что д"},
            { name: "u", role: "ещё не взятый сосед v — канди"},
            { name: "w", role: "вес ребра (v, u) — ключ в куч"},
            { name: "start", role: "стартовая вершина, из неё"},
        ],
    },
    "mst-boruvka": {
        vizTitle: "Борувка",
        stepNote: "Шаг = каждая компонента одновременно выб",
        code: `comp = {v: v for v in "ABC"}    # корень комп
cheapest = {}                                # comp → самое дешёвое ребро

u, v, w = "A", "B", 1                      # шаг: ребро-кандидат
if comp[u] != comp[v]:
    cheapest[comp[u]] = (w, u, v)
print(cheapest)                             # за фазу компонент станет
        variables: [
            { name: "comp", role: "компонента связности текущ"},
            { name: "cheapest[comp]", role: "самое дешёвое ре"},
            { name: "u, v, w", role: "ребро-кандидат: концы и"},
        ],
    },
    "string-kmp": {
        vizTitle: "Префикс-функция (КМП)",
        stepNote: "Шаг = обработка символа s[i]: откаты по",
        code: `s = "aabaabaaa"

```

```

    { name: "l, r", role: "правый крайний Z-блок: отпр
    { name: "z[i]", role: "длина наибольшего общего п
  ],
},
"aho-corasick": {
  vizTitle: "Ахо-Корасик",
  stepNote: "Шаг = построение бора по образцам, затем
  code: `trie = {"": {}} # бор: вершина →
state, c = "", "a" # шаг: автомат идёт по симво
nxt = trie.get(state, {}).get(c, "") # go(state, c) с
print(f"δ({state!r}, {c!r}) → {nxt!r}")`,
  variables: [
    { name: "state", role: "текущее состояние автомата"
    { name: "c", role: "очередной символ текста – пер
    { name: "go(v, c)", role: "функция перехода автом
  ],
},
"complexity-classes": {
  variables: [],
},
intro: {
  // мнемодкарточки без шагов алгоритма
  variables: [],
},
"alg-map": {
  variables: [],
},
};

```

```

/** Безопасный доступ: страница без записи получает пус
export const getPageSync = (chapterId?: string | null):
  (chapterId && PAGE_SYNC[chapterId]) || { variables: [

```

```

/**

```

```

* Стартовое содержимое редактора для страницы.

```

```

*

```

```

* Формат: короткий комментарий-шапка + МИНИМАЛЬНЫЙ исп

```

```
<h2 class="text-2xl sm:text-3xl font-bold text-slate-800">
</div>
```

```
<div class="space-y-8">
  <div id="intro" class="bg-slate-700/50 p-6 rounded-lg">
    <h3 class="text-xl font-bold text-blue-400">

    <div class="bg-slate-900 p-4 rounded-lg mb-4">
      <p class="text-lg text-blue-300 italic">
      <p class="text-xl font-mono text-white">
omise (lazy).</p>
    </div>
```

```
<div class="grid grid-cols-1 xl:grid-cols-2">
  <div class="bg-slate-800 p-6 rounded-lg">
    <div class="absolute -top-3 left-4 border
rder border-emerald-500 shadow-md">
```

Аналогия 1: Корпоративная презентация

```
</div>
  <p class="text-slate-300 text-sm mb-4">
ы рассылать 10 000 писем, он пишет одно письмо.
счетах сотрудников только тогда, когда сотрудник
женное обновление).</p>
</div>
```

```
<div class="bg-slate-900 p-6 rounded-lg">
  <div class="absolute -top-3 left-4 border
border-rose-500 shadow-md">
```

Аналогия 2: Матрешки-коробки

```
</div>
  <p class="text-slate-300 text-sm mb-4">
ше, и так далее. Если нам нужно покрасить
Внутри всё синее!" на большую коробку. Опустим
</div>
</div>
```

```
<details class="bg-slate-900/40 rounded-lg">
  <summary class="p-4 font-bold text-slate-300">
```

```

        <p class="text-xl font-mono text-white">
        ух отрезков:  $\min(\text{ST}[L][k], \text{ST}[R - 2^k + 1][k])$ 
        </div>
        <div class="grid grid-cols-1 xl:grid-cols-2">
            <div class="bg-slate-800 p-6 rounded-lg">
                <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
                    Аналогия 1: Школьные линейки
                </div>
                <p class="text-slate-300 text-sm mb-2">
                зок длины 7. Ты берешь две заготовленные заготовки
                ь отрезок длины 7 (перекрывая друг друга пополам)
                ничего складывать, просто пересекаем куски.
                </div>
                <div class="bg-slate-900 p-6 rounded-lg">
                    <div class="absolute -top-3 left-4 border border-rose-500 shadow-md">
                        Аналогия 2: Ступени
                    </div>
                    <p class="text-slate-300 text-sm mb-2">
                    ужно покрыть отрезок, мы берем две самые большие
                    </div>
                </div>
            <details class="bg-slate-900/40 rounded-lg">
                <summary class="p-4 font-bold text-slate-300">
                -b border-slate-700/50">
                    Код (Скрыто)
                </summary>
                <div class="p-5 text-sm text-slate-300">
                    <pre class="bg-slate-950 p-4 rounded">
int k = log2(R - L + 1);
int minimum = min(st[L][k], st[R - (1 << k) + 1][k]);</pre>
                </div>
            </details>
        </div>
    </div>
</section>`
},

```

```

        </div>
        <details class="bg-slate-900/40 rounded-lg"
            <summary class="p-4 font-bold text-slate-300"
                -b border-slate-700/50">
                Код (Скрыто)
            </summary>
            <div class="p-5 text-sm text-slate-300"
                <pre class="bg-slate-950 p-4 rounded
pair<Node*, Node*> split(Node* t, int x) {
    if (!t) return {nullptr, nullptr};
    if (t->val <= x) {
        auto [L, R] = split(t->right, x);
        t->right = L;
        return {t, R};
    } else {
        auto [L, R] = split(t->left, x);
        t->left = R;
        return {L, t};
    }
}</pre>
        </div>
    </details>
</div>
</div>
</section>`
    },
    {
        id: "splay-tree",
        title: "4. Splay-дерево",
        type: "html",
        description: "Дерево поиска, которое самобалансируется",
        category: "Продвинутые структуры",
        content: `
<section id="splay-tree" class="mb-12 scroll-mt-10">
    <div class="flex items-center mb-6 flex-wrap gap-3">
        <span class="bg-indigo-600 text-white px-4 py-1">
        <h2 class="text-2xl sm:text-3xl font-bold text-white">
    </div>

```

```

    if (p.left === x) { // Right rotate
      p.left = x.right;
      x.right = p;
    } else { // Left rotate
      p.right = x.left;
      x.left = p;
    }
  }
}

```

```

    </div>
  </details>

```

```

<div class="bg-indigo-950/60 p-6 rounded-xl">
  <h4 class="text-lg font-bold text-indigo-300">

```

```

    <div class="space-y-4 text-sm text-slate-300">
      <div>
        <strong class="text-white block">
          корень?</strong>
        <p>Оно поднимет узел, на котором
          ска. За счет сдвига этого листа в корень, д
        </p>

```

```

      </div>
      <div>
        <strong class="text-white block">
          <p>Если агент постоянно переключ
            находящимися на разных концах дерева, его м
            ащая дерево в вытянутый бамбук для другого.
            . Постоянное свинг-переключение превратит пр

```

```

      <div>
        <strong class="text-white block">
          <p>Хотя один поиск может стоить
            емах обладают прекрасным свойством: они не м
            по пути. "Палочное" дерево прессуется в сб
            осов.</p>

```

```

      </div>

```



```
    </div>
</section>`
  }
};
```

ФАЙЛ 26/86: src/data/tickets/ticket14_20.ts
КАТЕГОРИЯ: Билеты (учебный контент) | СТРӨК: 248 | РАЗМ

```
import { Chapter } from '../..types';

export const tickets14to20: Chapter[] = [
  {
    id: "dijkstra",
    title: "14. Графы. Поиск кратчайшего пути. Дейкстра",
    type: "html",
    category: "Графы. Пути",
    content: `
<section id="dijkstra-content" class="mb-12 scroll-mt-12">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 rounded">14
    <h2 class="text-2xl sm:text-3xl font-bold text-emerald-400">Графы. Пути
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border border-emerald-500">
      <h3 class="text-xl font-bold text-emerald-400">Аналогия 1
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-emerald-300 italic">Аналогия 1: Позитивное планирование
        <p class="text-xl font-mono text-white">Аналогия 1: Позитивное планирование
      </div>
    </div>
    <div class="grid grid-cols-1 xl:grid-cols-2">
      <!-- Аналогия 1 -->
      <div class="bg-slate-800 p-6 rounded-lg">
        <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
          Аналогия 1: Позитивное планирование
        </div>
      </div>
    </div>
  </div>
</section>`
  }
];
```

```

        <div class="bg-slate-800 p-6 rounded-lg" style="border: 1px solid #000; border-radius: 10px; margin-bottom: 10px;">
            <div class="absolute -top-3 left-4 right-4 bottom-4 border border-emerald-500 shadow-md" style="border: 1px solid #000; border-radius: 10px; padding: 5px;">
                ♂ Аналогия 1: Параноик
            </div>
            <p class="text-slate-300 text-sm mb-0">
                берет ВСЕ возможные дороги V-1 раз подряд.
            </p>
        </div>
        <div class="bg-slate-900 p-6 rounded-lg" style="border: 1px solid #000; border-radius: 10px; margin-bottom: 10px;">
            <div class="absolute -top-3 left-4 right-4 bottom-4 border border-rose-500 shadow-md" style="border: 1px solid #000; border-radius: 10px; padding: 5px;">
                Отрицательный цикл
            </div>
            <p class="text-slate-300 text-sm mb-0">
                попали во временную петлю.</p>
        </div>
    </div>
</div>
</section>`
    },
    {
        id: "floyd",
        title: "16. Графы. Поиск кратчайшего пути. Флойд",
        type: "html",
        category: "Графы. Пути",
        content: `
<section id="floyd-content" class="mb-12 scroll-mt-10">
    <div class="flex items-center mb-6 flex-wrap gap-3">
        <span class="bg-blue-600 text-white px-4 py-1 rounded">
            <h2 class="text-2xl sm:text-3xl font-bold text-white">
        </div>
        <div class="space-y-8">
            <div class="bg-slate-700/50 p-6 rounded-xl border" style="border: 1px solid #000; border-radius: 10px; padding: 10px;">
                <h3 class="text-xl font-bold text-indigo-400">
                <div class="bg-slate-900 p-4 rounded-lg mb-4" style="background-color: #000; color: #fff; padding: 10px;">
                    <p class="text-lg text-indigo-300 italic">

```

```

</div>
<div class="space-y-8">
  <div class="bg-slate-700/50 p-6 rounded-xl border-amber border-amber-500 shadow-md">
    <h3 class="text-xl font-bold text-amber-400">
      <div class="bg-slate-900 p-4 rounded-lg mb-6">
        <p class="text-xl font-mono text-white">
          Дейкстра (Билет 14) быстрая, но работает очень медленно. Мы хотим сделать <b>Джонсона</b> пути остались теми же.
        </p>
        <ol class="text-sm text-slate-300 space-y-4">
          <li>Добавляем <b>фиктивную вершину</b>
          <li>Запускаем от неё <b>медленные пути</b> «потенциалы» <code class="text-pink-400">
          <li>Меняем старые веса: новое равенство потенциалов сокращаются).</li>
          <li>Теперь смело запускаем <b>быстрые пути</b>
        </ol>
      </div>
    </div>
    <div id="slot-johnson-viz" class="mt-8"></div>
  </div>
</div>
</section>`
},
{
  id: "mst-kruskal",
  title: "18. Графы. Остовное дерево. Краскал",
  type: "html",

```

```

        <span class="bg-blue-600 text-white px-4 py-1 rounded">
        <h2 class="text-2xl sm:text-3xl font-bold text-emerald-400">
    </div>
    <div class="space-y-8">
        <div class="bg-slate-700/50 p-6 rounded-xl border border-emerald-500 shadow-md">
            <h3 class="text-xl font-bold text-emerald-400">
            <div class="bg-slate-900 p-4 rounded-lg mb-4">
                <p class="text-lg text-emerald-300 italic">
                <p class="text-xl font-mono text-white">
                    которое торчит из посещенных в непосещенных
            </div>
            <div class="grid grid-cols-1 gap-6">
                <div class="bg-slate-800 p-6 rounded-lg">
                    <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
                        ✦ Аналогия: Заражение вирусом
                    </div>
                    <p class="text-slate-300 text-sm mb-4">
                        ли вирус). Мы стартуем из одного города, и
                        Сеть растет только из одного связного куска
                    </div>
                </div>
            </div>
        </div>
    </div>
</section>`
    },
    {
        id: "mst-boruvka",
        title: "20. Графы. Остовное дерево. Борувка",
        type: "html",
        category: "Графы. Остовные",
        content: `
<section id="mst-boruvka" class="mb-12 scroll-mt-10">
    <div class="flex items-center mb-6 flex-wrap gap-3">
        <span class="bg-blue-600 text-white px-4 py-1 rounded">
        <h2 class="text-2xl sm:text-3xl font-bold text-emerald-400">
    </div>
    <div class="space-y-8">

```

```

content: `
<section id="string-kmp" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-indigo-600 text-white px-4 py-1">
    <h2 class="text-2xl sm:text-3xl font-bold text-white">
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border">
      <h3 class="text-xl font-bold text-blue-400">
      <div class="bg-slate-900 p-4 rounded-lg mb-1">
        <p class="text-lg text-blue-300 italic">
        <p class="text-xl font-mono text-white">
        сом, оканчивающимся в позиции i.</p>
      </div>

      <div class="grid grid-cols-1 xl:grid-cols-2">
        <!-- Аналогия 1 -->
        <div class="bg-slate-800 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
            ♂ Аналогия 1: Поиск без отка
          </div>
          <p class="text-slate-300 text-sm mb">
            Мы идём по строке слева направо.
            строки СЕЙЧАС закончился под моим пальцем?»
            Представь, ты ищешь в тексте слова
            х</code>. Тупой алгоритм начнет искать заголовок
            </code>, а это начало нашего слова! Не надо
          </p>
        </div>

        <!-- Аналогия 2 -->
        <div class="bg-slate-900 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 border border-rose-500 shadow-md">
            Аналогия 2: LLM стриминг
          </div>
          <p class="text-slate-300 text-sm mb">

```

```

for (int i = 1; i < s.length(); i++) {
    int j = pi[i-1];
    while (j > 0 && s[i] != s[j]) j = pi[j-1];
    if (s[i] == s[j]) j++;
    pi[i] = j;
}

```

</div>

</details>

</div>

</div>

</section>

},

{

id: "string-z-func",

title: "22. Z-функция – Сдвиг и наложение",

type: "html",

category: "Строки",

content: `

<section id="string-z-func" class="mb-12 scroll-mt-10">

<div class="flex items-center mb-6 flex-wrap gap-3">

<h2 class="text-2xl sm:text-3xl font-bold text-white">

</div>

<div class="space-y-8">

<div class="bg-slate-700/50 p-6 rounded-xl border">

<h3 class="text-xl font-bold text-emerald-400">

<div class="bg-slate-900 p-4 rounded-lg mb-4">

<p class="text-lg text-emerald-300 italic">

<p class="text-xl font-mono text-white">

са 0) и её суффиксом, начинающимся с i.</p>

</div>

<div class="grid grid-cols-1 xl:grid-cols-2">

<!-- Аналогия 1 -->

<div class="bg-slate-800 p-6 rounded-lg">

<div class="absolute -top-3 left-4">

order border-emerald-500 shadow-md">

Аналогия 1: Линейка-шаблон

```
        <li><code>Z[4], Z[5], Z[6]</code>
      </li>
    </ul>
  </div>
</details>
<details class="bg-slate-900/40 rounded-lg border-1
  <summary class="p-4 font-bold text-slate-300
    -b border-slate-700/50">
      Код C++ (Скрыто)
    </summary>
    <div class="p-5 text-sm text-slate-300"
      <pre class="bg-slate-950 p-4 rounded"
int l = 0, r = 0;
for (int i = 1; i < n; i++) {
  if (i <= r) z[i] = min(r - i + 1, z[i - 1]);
  while (i + z[i] < n && s[z[i]] == s[i + z[i]]) z[i]++;
  if (i + z[i] - 1 > r) { l = i; r = i + z[i] - 1; }
}</pre>
    </div>
  </details>
</div>
</div>
</section>`
  },
  {
    id: "aho-corasick",
    title: "23. Алгоритм Ахо-Корасик",
    type: "html",
    category: "Строки",
    content: `
<section id="aho-corasick" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-indigo-600 text-white px-4 py-1 rounded">
    <h2 class="text-2xl sm:text-3xl font-bold text-white">
  </div>

  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border-l">
      <h3 class="text-xl font-bold text-blue-400 mb-4">

```

```

    if (t[v].link == -1) {
        if (v == 0 || t[v].p == 0) t[v].link = 0;
        else t[v].link = go(get_link(t[v].p), t[v].pch);
    }
    return t[v].link;
}

```

```

int go(int v, char c) {
    if (t[v].go[c] == -1) {
        if (t[v].next[c] != -1) t[v].go[c] = t[v].next[c];
        else t[v].go[c] = v == 0 ? 0 : go(get_link(v), c);
    }
    return t[v].go[c];
}

```

</pre>

</div>

</details>

</div>

</section>

},

{

id: "complexity-classes",

title: "24. Классы сложности, сведение задач",

type: "html",

category: "Теория сложности",

content: `

<section id="complexity-classes" class="mb-12 scroll-mt

<div class="flex items-center mb-6 flex-wrap gap-3">

<span class="bg-purple-600 text-white px-4 py-1

<h2 class="text-2xl sm:text-3xl font-bold text-v

</div>

<div class="space-y-8">

<div class="bg-slate-700/50 p-6 rounded-xl bord

<h3 class="text-xl font-bold text-purple-400

<div class="bg-slate-900 p-4 rounded-lg mb-1

<p class="text-lg text-purple-300 italic

<p class="text-xl font-mono text-white">

Сведение (Reduction) A->B: Если я умею реша

</div>


```
id: "graph-dfs-bfs",
title: "6. Графы. Представление, DFS, BFS",
type: "html",
description: "Представление графов и базовые обходы",
category: "Графы. База",
content: `
<section id="graph-dfs-bfs" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 rounded">База
    <h2 class="text-2xl sm:text-3xl font-bold text-slate-300">Графы
  </div>
  <div class="space-y-8">
    {/* База представлений */}
    <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-600">
      <h3 class="text-xl font-bold text-slate-300">Представление
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-slate-300 italic">Матрица
        <p class="text-xl font-mono text-white">
          (V + E).</p>
      </div>
      <div class="grid grid-cols-1 xl:grid-cols-2">
        <div class="bg-slate-800 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
            Аналогия: Матрица = Таблица
          </div>
          <p class="text-slate-300 text-sm mb-4">
            глупо, но проверка "знает ли Вася Петю" мгно
          </div>
          <div class="bg-slate-900 p-6 rounded-lg">
            <div class="absolute -top-3 left-4 border border-rose-500 shadow-md">
              Аналогия: Списки = Контакты
            </div>
            <p class="text-slate-300 text-sm mb-4">
              т. Оптимально для почти всех задач.</p>
          </div>
        </div>
      </div>
    </div>
  </div>
`
```

- Где используется:
- Кратчайший путь в Невзвешен
- Алгоритм Ахо-Корасик (сборка
- Алгоритм Форда-Фалкерсона /
- Двунаправленный поиск для у

</div>

</div>

<details class="bg-slate-900/40 rounded-lg l

<summary class="p-4 font-bold text-slate

-b border-slate-700/50">

Код обходов C++ (Основа)

</summary>

<div class="grid grid-cols-1 md:grid-co

<pre class="bg-slate-950 p-3 rounded

// DFS (Рекурсия)

vector<bool> vis;

vector<vector<int>> adj;

void dfs(int v) {

vis[v] = true;

for (int u : adj[v]) {

if (!vis[u]) dfs(u);

}

</pre>

<pre class="bg-slate-950 p-3 rounded

// BFS (Очередь)

queue<int> q;

q.push(start_node);

vis[start_node] = true;

while(!q.empty()) {

int v = q.front(); q.pop();

for (int u : adj[v]) {

if (!vis[u]) {

vis[u] = true;

q.push(u);

```

        </div>
      </div>
    </div>
  </div>
</section>`,
  },
  {
    id: "graph-components",
    title: "8. Графы. Компоненты связности",
    type: "html",
    description: "Нахождение кусков графа",
    category: "Графы. База",
    content: `
<section id="graph-components" class="mb-12 scroll-mt-12">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 rounded">Графы
    <h2 class="text-2xl sm:text-3xl font-bold text-emerald-400">8. Графы. Компоненты связности
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border border-emerald-500">
      <h3 class="text-xl font-bold text-emerald-400">Нахождение кусков графа
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-emerald-300 italic">Графы — это структуры, состоящие из вершин и ребер. Компонент связности — это максимальная часть графа, в которой любые две вершины соединены путем.
        <p class="text-xl font-mono text-white">Нахождение количества компонент связности.
      </div>
      <div class="grid grid-cols-1 xl:grid-cols-2 gap-4">
        <div class="bg-slate-800 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
            Аналогия 1: Острова
          </div>
          <p class="text-slate-300 text-sm mb-4">Если мы посмотрим на карту — остались ли серые (неисследованные) участки, которые можно пройти через океан — столько и компонент.
        </div>
      </div>
    </div>
  </div>
</div>
`
  }
]

```

```

        <p class="text-slate-300 text-sm mb-4">
            получить опыт, нужна работа". Алгоритм выявления
        </div>
    </div>

    <details class="bg-slate-900/40 rounded-lg border-1
        <summary class="p-4 font-bold text-slate-300
            -b border-slate-700/50">
                Душный мод: Код на C++ (Скрыто)
            </summary>
            <div class="p-5 text-sm text-slate-300">
                <p class="text-indigo-400 font-bold">
                    <div class="bg-slate-950 p-4 rounded">
                        <pre class="text-xs font-mono">
vector<int> adj[MAXN];
bool visited[MAXN];
vector<int> ans;

void dfs(int v) {
    visited[v] = true;
    for (int to : adj[v]) {
        if (!visited[to]) {
            dfs(to);
        }
    }
    ans.push_back(v); // Добавляем в момент ВЫХОДА (pos
}

void topological_sort(int n) {
    for (int i = 0; i < n; ++i) {
        if (!visited[i]) dfs(i);
    }
    reverse(ans.begin(), ans.end()); // Разворачиваем с
}</pre>
            </div>
        </div>
    </details>
</div>

```

```
<div class="grid grid-cols-1 xl:grid-cols-2
  <div class="bg-slate-800 p-6 rounded-lg
    <div class="absolute -top-3 left-4
  <div class="border-emerald-500 shadow-md">
```

Аналогия: Интриги в офисе

```
</div>
```

```
<p class="text-slate-300 text-sm mb
```

Сильная связность — это "клуб" с
ни в одной SCC. Добавить курьера Гошу, кото
йти наружу (к директору), но обратно в клуб

```
</p>
```

```
</div>
```

```
<div class="bg-slate-900 p-6 rounded-lg
  <div class="absolute -top-3 left-4
  <div class="border-rose-500 shadow-md">
```

Связь с Билетом 12

```
</div>
```

```
<p class="text-slate-300 text-sm mb
```

SCC (Билет 10) склеивает

```
</br><br>
```

Точки сочленения (Билет 12)
Точка сочленения — это вершина, удаление которой
разрывает граф в две или более частей.

```
</p>
```

```
</div>
```

```
</div>
```

```
<details class="bg-slate-900/40 rounded-lg
  <summary class="p-4 font-bold text-slate-300
  <div class="border-slate-700/50">
```

Душный мод: Код Алгоритм Косарайю

```
</summary>
```

```
<div class="p-5 text-sm text-slate-300
```

```
<p class="text-emerald-400 font-bol
```

```
<ol class="list-decimal list-inside
```

1. Запускаем DFS на исходном графе

```
<li>
```

2. Разворачиваем этот список (с

```

        </div>
      </div>
    </div>
  </div>
</section>`,
  },
  {
    id: "graph-articulation",
    title: "12. Графы. Точки сочленения",
    type: "html",
    description: "Вершины, удаление которых убивает свя",
    category: "Графы. Продвинутые",
    content: `
<section id="graph-articulation" class="mb-12 scroll-mt
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 r
    <h2 class="text-2xl sm:text-3xl font-bold text-v
  </div>

  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl bord
      <h3 class="text-xl font-bold text-rose-400 m

      <div class="bg-slate-900 p-4 rounded-lg mb-4
        <p class="text-lg text-rose-300 italic m
        <div class="text-left font-mono text-sm
          <p><strong class="text-white">Точка
рой увеличивает число компонент связности.<
        <div class="p-3 bg-slate-950 rounde
          <span class="text-rose-400 font
            <p class="text-xl font-bold tex
            <p class="text-xs text-slate-400
          </div>
        </div>
      </div>
    </div>
  </div>

  <p class="text-slate-300 text-sm">
    <strong>Важно:</strong> Это работает то

```

Для стартовой вершины обхода правил
то выше неё прыгать некуда). Поэтому для неё
у него **больше 1 независимого ребенка**

</p>

<div class="bg-slate-950 p-4 rounded-lg

<pre class="text-xs font-mono text-

```
void dfs(int v, int p = -1) {
    visited[v] = true;
    tin[v] = low[v] = timer++;
    int children = 0;

    for (int to : adj[v]) {
        if (to == p) continue;
        if (visited[to]) {
            low[v] = min(low[v], tin[to]);
        } else {
            dfs(to, v);
            low[v] = min(low[v], low[to]);
            if (low[to] >= tin[v] && p != -1)
                IS_CUTPOINT(v);
            ++children;
        }
    }
    if (p == -1 && children > 1)
        IS_CUTPOINT(v);
}
```

</div>

</div>

</details>

<div class="bg-indigo-950/60 p-6 rounded-xl bor

<h4 class="text-lg font-bold text-indigo-400

<p class="text-slate-300 text-sm mb-4">

Это глубокий дуальный мост между ориент

</p>

<ul class="list-disc list-inside text-sm te

<strong class="text-rose-400">Точки

ong> и показывают физическую уязвимость свя

```
        е сходится нечетное число линий, значит, из
        а везде должно быть четное число (зашел-вышел)
        </div>
    </div>
</div>
</section>`,
    },
];
```

РАЗДЕЛ: ИНТЕРАКТИВНЫЕ КОМПОНЕНТЫ И СИМУЛЯТОРЫ

ФАЙЛ 29/86: src/components/ArticulationPointsViz.tsx
КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```
import React, { useState, useEffect } from "react";
import { Play, Pause, RotateCcw, Info } from "lucide-react";

interface Node {
  id: number;
  x: number;
  y: number;
  label: string;
}

interface Edge {
  u: number;
  v: number;
}

const nodes: Node[] = [
  { id: 0, x: 250, y: 50, label: "0" },
  { id: 1, x: 100, y: 150, label: "1" },
];
```



```
const [steps, setSteps] = useState<StepInfo[]>([]);
const [currentStepIndex, setCurrentStepIndex] = useSt
const [isPlaying, setIsPlaying] = useState(false);
```

```
useEffect(() => {
  const newSteps: StepInfo[] = [];
  let timer = 0;
  const visited = new Set<number>();
  const ap = new Set<number>();
  const currentTin = Array(nodes.length).fill(-1);
  const currentLow = Array(nodes.length).fill(-1);

  const recordStep = (desc: string, u: number | null) => {
    newSteps.push({
      desc,
      visited: new Set(visited),
      ap: new Set(ap),
      currentNode: u,
      tin: [...currentTin],
      low: [...currentLow],
    });
  };
};
```

```
recordStep("Начало DFS (Тарьян) для поиска точек со
```

```
const dfs = (v: number, p: number = -1) => {
  visited.add(v);
  currentTin[v] = currentLow[v] = timer++;
  let children = 0;

  recordStep(`Заходим в вершину ${v}. tin[${v}]=${c

  for (const to of adj[v]) {
    if (to === p) continue;

    if (visited.has(to)) {
      currentLow[v] = Math.min(currentLow[v], curren
      recordStep(
```

```

        if (!visited.has(i)) {
            dfs(i);
        }
    }

    recordStep("Алгоритм завершен. Найдены все точки со
    setSteps(newSteps);
}, []));

const currentStep = steps[currentStepIndex] || {
    desc: "",
    visited: new Set(),
    ap: new Set(),
    currentNode: null,
    tin: [],
    low: [],
};

useEffect(() => {
    let interval: NodeJS.Timeout;
    if (isPlaying && currentStepIndex < steps.length -
        interval = setInterval(() => {
            setCurrentStepIndex((prev) => prev + 1);
        }, 1500);
    } else if (currentStepIndex >= steps.length - 1) {
        setIsPlaying(false);
    }
    return () => clearInterval(interval);
}, [isPlaying, currentStepIndex, steps.length]);

const togglePlay = () => setIsPlaying(!isPlaying);
const reset = () => {
    setIsPlaying(false);
    setCurrentStepIndex(0);
};

return (
    <div className="bg-slate-900 rounded-xl border bord

```

```

const isBridge =
  (edge.u === 3 && edge.v === 4) ||
  (edge.u === 4 && edge.v === 3);

return (
  <line
    key={"edge-" + i}
    x1={u.x}
    y1={u.y}
    x2={v.x}
    y2={v.y}
    stroke={isBridge ? "#f43f5e" : "#475568"}
    strokeWidth={isBridge ? "4" : "2"}
    strokeDasharray={isBridge ? "8,4" : ""}
  />
);
}}}

{/* Nodes */}
{nodes.map((node) => {
  const isCurrent = currentStep.currentNode === node.id;
  const isVisited = currentStep.visited.has(node.id);
  const isAp = currentStep.ap.has(node.id);

  const fill = isCurrent
    ? "#3b82f6"
    : isAp
    ? "#e11d48"
    : isVisited
    ? "#10b981"
    : "#1e293b";
  const stroke = isCurrent
    ? "#60a5fa"
    : isAp
    ? "#f43f5e"
    : isVisited
    ? "#34d399"
    : "#334155";

```

```

        : "-" }
      </text>
      <text
        x={node.x}
        y={node.y + r + 20}
        textAnchor="middle"
        fill="#94a3b8"
        fontSize="10px"
      >
        low:{" "}
        {currentStep.low[node.id] >= 0
          ? currentStep.low[node.id]
          : "-"}
      </text>
    </>
  })
</g>
);
}}
</svg>
</div>

<div className="bg-slate-950 p-6 border-t lg:bo
  <h4 className="text-sm font-bold text-slate-4
    Статус алгоритма
  </h4>

  <div className="bg-slate-900 border border-sl
    <p className="text-white font-medium min-h-
      {currentStep.desc}
    </p>
  </div>

  <div className="space-y-4 flex-1 overflow-y-a
    <div className="mb-4">
      <h5 className="text-xs text-slate-500 fon
      <div className="flex items-center gap-2 t
        <div className="w-3 h-3 rounded-full bg

```

```

        <div
          className="bg-rose-500 h-full transition
          style={{
            width: `${currentStepIndex / Math.ma
          }}
        />
      </div>
    </div>
  </div>
</div>
);
};

```

ФАЙЛ 30/86: src/components/BellmanFordViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```

import { useState, useEffect } from 'react';

interface Node { id: string; x: number; y: number; name: string; }
interface Edge { u: string; v: string; w: number; }
interface Step {
  iteration: number;
  checkingEdge: { u: string; v: string; w: number } | null;
  distances: { [key: string]: number };
  previous: { [key: string]: string | null };
  log: string;
  hasNegativeCycle?: boolean;
  activeCodeLine: number;
}

export default function BellmanFordViz() {

```

```

  const [hasCycle, setHasCycle] = useState(false);
  const [steps, setSteps] = useState<Step[]>([]);
  const [currentStepIndex, setCurrentStepIndex] = useSt

```

```
const adjList = getAdjList(edges);

const codeLines = [
  { line: 1, text: 'function bellman_ford(graph, start' },
  { line: 2, text: '    dist = {v: ∞}; dist[start] = 0' },
  { line: 3, text: '    for i from 1 to V - 1: // |V| - 1' },
  { line: 4, text: '        for (u, v, weight) in E:' },
  { line: 5, text: '            if dist[u] + weight < dist[v]' },
  { line: 6, text: '                dist[v] = dist[u] + weight' },
  { line: 7, text: '    for (u, v, weight) in E: // Проверка' },
  { line: 8, text: '        if dist[u] + weight < dist[v]' },
  { line: 9, text: '            return "ОБНАРУЖЕН ОТРИЦАТЕЛЬНЫЙ ЦИКЛ"' },
  { line: 10, text: '    return dist' },
];
```

```
useEffect(() => {
  const simulationSteps: Step[] = [];
  let dist: { [key: string]: number } = { S: 0, A: 999 };
  let prev: { [key: string]: string | null } = { S: null };

  simulationSteps.push({
    iteration: 0, checkingEdge: null,
    distances: { ...dist }, previous: { ...prev },
    log: '    Фура заведена. Начинаем с Базы (S). Расстояние до А: 999',
    activeCodeLine: 2,
  });
});
```

```
const vCount = nodes.length;
for (let iter = 1; iter < vCount; iter++) {
  let anyUpdate = false;
  for (const edge of edges) {
    const uDist = dist[edge.u];
    const vDist = dist[edge.v];

    let updated = false;
    if (uDist !== 999 && uDist + edge.w < vDist) {
      dist = { ...dist, [edge.v]: uDist + edge.w };
      prev = { ...prev, [edge.v]: edge.u };
      anyUpdate = true;
    }
  }
  if (!anyUpdate) break;
  simulationSteps.push({
    iteration: iter, checkingEdge: null,
    distances: { ...dist }, previous: { ...prev },
    log: `    Итерация ${iter}. Проверка всех ребер. Найден отрицательный цикл?`,
    activeCodeLine: 7,
  });
}
```

```

    if (!cycleFound) {
      simulationSteps.push({
        iteration: vCount, checkingEdge: null,
        distances: { ...dist }, previous: { ...prev }
        log: ' Проверка завершена. Ни одно ребро не
        activeCodeLine: 10,
      });
    }
  }

  setSteps(simulationSteps);
  setCurrentStepIndex(0);
  setIsPlaying(false);
}, [hasCycle]);

useEffect(() => {
  let timer: any = null;
  if (isPlaying && currentStepIndex < steps.length -
    timer = setTimeout(() => setCurrentStepIndex((p)
  } else if (currentStepIndex >= steps.length - 1) {
    setIsPlaying(false);
  }
  return () => clearTimeout(timer);
}, [isPlaying, currentStepIndex, steps.length]);

const currentStep = steps[currentStepIndex] || null;
if (!currentStep) return <div>Загрузка...</div>;

return (
  <div className="bg-slate-800/90 p-6 rounded-2xl bor
    <div className="flex flex-wrap items-center justifi
      <div className="flex items-center gap-3">
        <div className="p-2.5 bg-blue-600 text-white
          <span className="text-2xl"> </span>
        </div>
      <div>
        <h3 className="text-2xl font-bold text-white

```

```

<defs>
  <marker id="arrow-bf-normal" markerWidth=
<path d="M0,0 L6,3 L0,6 z" fill="#64748b" />
  <marker id="arrow-bf-active" markerWidth=
<path d="M0,0 L6,3 L0,6 z" fill="#f59e0b" />
  <marker id="arrow-bf-cycle" markerWidth=
path d="M0,0 L6,3 L0,6 z" fill="#e11d48" />
</defs>

```

```

{edges.map((edge, idx) => {
  const uNode = nodes.find((n) => n.id === u);
  const vNode = nodes.find((n) => n.id === v);
  const isChecking = currentStep.checkingEdges;
  const isNeg = edge.w < 0;
  const isCyclePart = hasCycle && ((edge.u === u && edge.v === 'D'));

```

```

  let strokeColor = '#475569';
  let marker = 'url(#arrow-bf-normal)';
  if (isChecking) { strokeColor = '#f59e0b';
  else if (currentStep.hasNegativeCycle &&

```

```

  return (
    <g key={idx}>
      <line x1={uNode.x} y1={uNode.y} x2={vNode.x} y2={vNode.y} stroke={
        isChecking && isCyclePart ? 4 : 2} strokeWidth={
        isChecking ? '4,4' : 'none'} />
      <circle cx={({uNode.x + vNode.x} / 2) y={({uNode.y + vNode.y} / 2)}
        stroke={isChecking ? '#f59e0b' : isNeg ? '#f43f5e' : '#475569'}
        strokeDash={isChecking ? [4, 4] : []} />
      <text x={({uNode.x + vNode.x} / 2) y={({uNode.y + vNode.y} / 2)}
        text={isChecking ? 'Checking' : isNeg ? 'Neg' : 'Dist'}
        fontColor={isChecking ? 'white' : 'black'}
        fontSize="11" fontWeight="bold" />
    </g>
  );
}}

```

```

{nodes.map((node) => {
  const dist = currentStep.distances[node.id];

```



```

        )
      }}}}
    </div>
  }}}}
</div>
</div>
</div>

<div className="flex flex-col lg:flex-row gap-6">
  {/* Таблица расстояний */}
  <div className="w-full lg:w-1/2 bg-rose-950/10">
    <div>
      <h4 className="text-lg font-bold text-rose-950">
        Таблица расстояний
      </h4>
      <div className="grid grid-cols-4 gap-2 text-rose-950">
        {nodes.map(n => {
          const d = currentStep.distances[n.id];
          const isTgt = currentStep.checkingEdge === n.id;
          return (
            <div key={n.id} className={`p-2 rounded-lg
              ${isTgt ? 'border-slate-700/60 bg-slate-800/40' : ''}`}>
              <div className="text-xs text-slate-400">
                {n.id}
              </div>
              <div className={`font-mono font-bold text-rose-950`}
                >
                {d}
              </div>
            </div>
          );
        })}
      </div>
    </div>
  </div>
</div>

  {/* Код алгоритма */}
  <div className="w-full lg:w-1/2 bg-slate-900/60">
    <div>
      <h4 className="text-lg font-bold text-slate-400">
        Код алгоритма
      </h4>
      <div className="bg-slate-950/80 rounded-lg p-4">
        {codeLines.map(item => (
          <div key={item.line} className={`py-1.5 px-2
            ${item.line === currentStep.start ? 'bg-blue-900/80 text-amber-400' : ''}`}>
            {item.code}
          </div>
        ))}
      </div>
    </div>
  </div>

```

```
0: [1, 2], 1: [3, 4], 2: [5, 6], 3: [], 4: [], 5: [],  
};
```

```
interface Step {  
  activeLine: number;  
  queue: number[];  
  visited: number[];  
  currentNode: number | null;  
  logs: string;  
}
```

```
const generateBfsSteps = (): Step[] => {  
  const steps: Step[] = [];  
  const queue: number[] = [];  
  const visited = new Set<number>();
```

```
  // Init
```

```
  steps.push({  
    activeLine: 1,  
    queue: [],  
    visited: [],  
    currentNode: null,  
    logs: "Инициализация: начинаем с корня (0)."  
  });
```

```
  queue.push(0);
```

```
  visited.add(0);
```

```
  steps.push({  
    activeLine: 2,  
    queue: [...queue],  
    visited: Array.from(visited),  
    currentNode: null,
```

```
    logs: "Добавляем стартовую вершину 0 в очередь и по  
  });
```

```
  while (queue.length > 0) {
```

```
    steps.push({  
      activeLine: 4,
```

```

    }
  } else {
    steps.push({
      activeLine: 6,
      queue: [...queue],
      visited: Array.from(visited),
      currentNode: curr,
      logs: `Соседей нет или все уже посещены.`
    });
  }
}

steps.push({
  activeLine: 12,
  queue: [],
  visited: Array.from(visited),
  currentNode: null,
  logs: "Очередь пуста. Алгоритм завершен!"
});

return steps;
};

const STEPS = generateBfsSteps();

export default function BfsViz() {
  const [stepIdx, setStepIdx] = useState(0);
  const [isPlaying, setIsPlaying] = useState(false);

  const step = STEPS[stepIdx];

  const handleNext = useCallback(() => {
    setStepIdx(prev => Math.min(prev + 1, STEPS.length - 1));
  }, []);

  // @ts-expect-error зарезервировано под кнопку «назад»
  const handlePrev = useCallback(() => {
    setStepIdx(prev => Math.max(prev - 1, 0));
  }, []);
}

```

```

        disabled={stepIdx >= STEPS.length - 1}
        className="p-2 text-emerald-400 hover:text-
ed:hover:bg-transparent"
        title="Шаг вперед"
      >
        <StepForward className="w-5 h-5" />
      </button>
    </div>
  </div>

  <div className="grid grid-cols-1 lg:grid-cols-2 g
    { /* Визуал графа */ }
    <div className="bg-slate-800 border border-slate
      50px]">
      <svg className="w-full h-full min-h-[350px]" *
        { /* Рёбра */ }
        { EDGES.map((e, idx) => {
          const n1 = NODES.find(n => n.id === e.from)
          const n2 = NODES.find(n => n.id === e.to)
          const isVisited = step.visited.includes(n
          return (
            <line
              key={idx}
              x1={n1.x} y1={n1.y} x2={n2.x} y2={n2.y}
              className={`transition-all duration-5
            '}`}
          />
          );
        })}

        { /* Вершины */ }
        { NODES.map(n => {
          const isCurrent = step.currentNode === n
          const isInQueue = step.queue.includes(n.id)
          const isVisited = step.visited.includes(n

          let fill = '#1e293b'; // slate-800
          let stroke = '#475569'; // slate-600

```

```

    });
  }}}
</svg>
</div>

{/* Код и Очередь */}
<div className="flex flex-col gap-4">
  <div className="bg-slate-900 border border-slate-800 p-4">
    <div className="bg-slate-800 text-slate-400 p-4 font-mono font-size tracking-wider">
      <span>bfs(graph, start)</span>
      <span className="text-blue-400">Mar {step}
    </div>
    <div className="p-4">
      {[
        { line: 1, code: 'queue = Queue()' },
        { line: 2, code: 'queue.push(start); visited.add(start)' },
        { line: 3, code: '' },
        { line: 4, code: 'while not queue.isEmpty()' },
        { line: 5, code: '  node = queue.pop()' },
        { line: 6, code: '  for neighbor in graph[node]:' },
        { line: 7, code: '    if neighbor not in visited:' },
        { line: 8, code: '      visited.add(neighbor)' },
        { line: 9, code: '      queue.push(neighbor)' },
      ]}.map((cl) => {
        const isActive = step.activeLine === cl.line
        return (
          <div key={cl.line} className={`px-2 py-2 border border-l-2 border-blue-400 -ml-0.5` : 'text-blue-400'}>
            <span className="opacity-40 w-6 inline-block">  </span>
            <span className="whitespace-pre">{cl.code}</span>
          </div>
        )
      })
    </div>
  </div>
</div>

```

```
-----
const imageMap: Record<string, { src: string; alt: string;
  dijkstra: {
    src: dijkstraImg,
    alt: 'Добрый Дейкстра',
    caption: '    Добрый – только позитив!',
    borderColor: 'border-blue-500/30',
    captionColor: 'text-blue-400',
  },
  'bellman-ford': {
    src: bellmanImg,
    alt: 'Фура по болоту',
    caption: '    Тяжёлая, но ей пофиг на ямы!',
    borderColor: 'border-rose-500/30',
    captionColor: 'text-rose-400',
  },
  floyd: {
    src: floydImg,
    alt: 'Все ко Всем Флойд',
    caption: '    Все связаны со всеми!',
    borderColor: 'border-emerald-500/30',
    captionColor: 'text-emerald-400',
  },
};

export default function ChapterImage({ vizType }: { vizType: string }) {
  const info = imageMap[vizType];
  if (!info) return null;

  return (
    <div>
      <div className={`rounded-2xl overflow-hidden border-2 border-gray-200`} >
        <img src={info.src} alt={info.alt} className="w-full h-full" />
      </div>
      <p className={`text-center text-xs mt-2 font-bold`} >{info.caption}</p>
    </div>
  );
}
```

```

        {index + 1} / {total}
      </span>
      <button
        disabled={!next}
        onClick={() => next && onGo(next.id)}
        title={next ? next.title : "Это последняя тема"}
        className="flex items-center gap-1 px-2.5 py-1.5
          :bg-slate-700 enabled:hover:text-white disabled:opacity-50"
      >
        <span className="hidden sm:inline">Вперёд</span>
        <ChevronRight className="w-4 h-4" />
      </button>
    </div>
  );
}

return (
  <nav className="grid grid-cols-1 sm:grid-cols-2 gap-4" >
    {prev ? (
      <button
        onClick={() => onGo(prev.id)}
        className="group text-left p-4 rounded-xl bg-white
          transition-all"
      >
        <div className="flex items-center gap-1.5 text-slate-700" >
          <ChevronLeft className="w-3.5 h-3.5" /> Предыдущая
        </div>
        <div className="text-sm font-bold text-slate-700">
          {prev.title}
        </div>
      </button>
    ) : (
      <div className="hidden sm:block" />
    )}
    {next && (
      <button
        onClick={() => onGo(next.id)}
        className="group text-right p-4 rounded-xl bg-white
          transition-all"
      >
        <div className="flex items-center gap-1.5 text-slate-700" >
          Следующая <ChevronRight className="w-3.5 h-3.5" />
        </div>
        <div className="text-sm font-bold text-slate-700">
          {next.title}
        </div>
      </button>
    )}
  </nav>
);

```

```

}

const isTouch = () => typeof window !== "undefined" && window.ontouchstart !== undefined;

export const ChapterView: React.FC<Props> = ({ chapter,
  const contentRef = useRef<HTMLDivElement>(null);
  const [anchor, setAnchor] = useState<HintAnchor | null>(null);
  const hideTimer = useRef<number | null>(null);
  const [saving, setSaving] = useState<"idle" | "work">("idle");

  useTermHints(contentRef, [chapter.id]);

  const cancelHide = useCallback(() => {
    if (hideTimer.current) {
      window.clearTimeout(hideTimer.current);
      hideTimer.current = null;
    }
  }, []);

  const scheduleHide = useCallback(() => {
    cancelHide();
    hideTimer.current = window.setTimeout(() => setAnchor(anchor), 2500);
  }, [cancelHide]);

  useEffect(() => {
    setAnchor(null);
    setSaving("idle");
  }, [chapter.id]);

  const saveHtml = useCallback(async () => {
    setSaving("work");
    try {
      await downloadChapterHtml(chapter);
      setSaving("done");
      window.setTimeout(() => setSaving("idle"), 2500);
    } catch {
      setSaving("idle");
    }
  }, []);

```



```

return (
  <>
    <article className="chapter-body">
      {/* панель загрузки: сохранить именно эту страницу */}
      <div className="flex flex-wrap items-center gap-2">
        <button
          onClick={saveHtml}
          disabled={saving === "work"}
          title="Сохранить эту тему одним автономным файлом"
          className="flex items-center gap-1.5 text-x-small border-1px
            border-slate-500 rounded-3px px-3 py-2.5 hover:text-white
            hover:border-emerald-500" />
          >
            {saving === "work" ? (
              <Loader2 className="w-3.5 h-3.5 animate-spin" />
            ) : saving === "done" ? (
              <Check className="w-3.5 h-3.5 text-emerald-500" />
            ) : (
              <Download className="w-3.5 h-3.5" />
            )}
          >
            {saving === "done" ? "Файл сохранён" : "Скачать"}
        </button>
        <button
          onClick={() => window.print()}
          title="Распечатать или сохранить в PDF средствами браузера"
          className="flex items-center gap-1.5 text-x-small border-1px
            border-indigo-500 rounded-3px px-3 py-2.5 hover:text-white
            hover:border-indigo-500" />
          >
            <Printer className="w-3.5 h-3.5" /> Печать
        </button>
        <span className="text-[11px] text-slate-500">
          </div>

      <div
        ref={contentRef}
        className="prose prose-invert max-w-none"
        onMouseOver={handlePointerOver}
        onMouseOut={handlePointerOut}

```

```

        </button>
      }}
    </div>
  </div>
  {viz.hint && <p className="text-xs text-slate-500">
    <viz.Component />
  </section>
  )}

  <ChapterNav prev={prev} next={next} index={index} />
</article>

<TermPopover
  anchor={anchor}
  onClose={() => setAnchor(null)}
  onOpenSimulator={(id) => {
    setAnchor(null);
    onOpenSimulator(id);
  }}
  onCardEnter={cancelHide}
  onCardLeave={scheduleHide}
/>
</>
);
};

```

ФАЙЛ 35/86: src/components/DfsBridgesSimulator.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 10

```

import { useState, useEffect, useRef } from "react";
import { Undo, Play, Pause, SkipForward, RefreshCw } from "lucide-react";

```

```

interface CodeLine {
  line: string;
  highlight: string;
}

```

```

    { id: 0, label: "A", x: 100, y: 120 },
    { id: 1, label: "B", x: 260, y: 120 },
    { id: 2, label: "C", x: 180, y: 200 },
    { id: 3, label: "D", x: 180, y: 300 },
    { id: 4, label: "E", x: 100, y: 370 },
    { id: 5, label: "F", x: 260, y: 370 },
    { id: 6, label: "G", x: 340, y: 200 }
];

const GRAPH_EDGES = [
  { u: 0, v: 1, key: "0-1" },
  { u: 1, v: 2, key: "1-2" },
  { u: 2, v: 0, key: "0-2" },
  { u: 2, v: 3, key: "2-3" },
  { u: 3, v: 4, key: "3-4" },
  { u: 4, v: 5, key: "4-5" },
  { u: 5, v: 3, key: "3-5" },
  { u: 1, v: 6, key: "1-6" }
];

const DFS_STEPS: DfsStep[] = [
  {
    currentNode: 0, visitedIndices: [0],
    tin: {0:1}, low: {0:1}, stack: [0], bridges: [],
    activeEdge: null, backEdge: null,
    log: "Входим в A. tin[A]=1, low[A]=1.",
    activeCodeLine: [0,1,2]
  },
  {
    currentNode: 1, visitedIndices: [0,1],
    tin: {0:1,1:2}, low: {0:1,1:2}, stack: [0,1], bridges: [],
    activeEdge: [0,1], backEdge: null,
    log: "Идём (A,B). B впервые. tin[B]=2, low[B]=2.",
    activeCodeLine: [4,5,7,10,11]
  },
  {
    currentNode: 6, visitedIndices: [0,1,6],
    tin: {0:1,1:2,6:3}, low: {0:1,1:2,6:3}, stack: [0,1,6]
  }
];

```

```

    },
    {
        currentNode: 5, visitedIndices: [0,1,6,2,3,4,5],
        tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:2,6:3,2:4,3:5,4:6,5:7},
        activeEdge: [4,5], backEdge: null,
        log: "E→F. tin[F]=7, low[F]=7.",
        activeCodeLine: [4,5,7,10,11]
    },
    {
        currentNode: 5, visitedIndices: [0,1,6,2,3,4,5],
        tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:2,6:3,2:4,3:5,4:6,5:7},
        activeEdge: null, backEdge: [5,3],
        log: "Из F видим D (посещён). Обратное ребро (F,D).",
        activeCodeLine: [7,8,9]
    },
    {
        currentNode: 4, visitedIndices: [0,1,6,2,3,4,5],
        tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:2,6:3,2:4,3:5,4:6,5:7},
        activeEdge: null, backEdge: null,
        log: "Возврат F→E. low[E]=min(6,5)=5. (E,F) – не мо",
        activeCodeLine: [11,13,16,17]
    },
    {
        currentNode: 3, visitedIndices: [0,1,6,2,3,4,5],
        tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:2,6:3,2:4,3:5,4:6,5:7},
        activeEdge: null, backEdge: null,
        log: "Возврат E→D. low[D]=min(6,5)=5. (D,E) – не мо",
        activeCodeLine: [11,13,16,17]
    },
    {
        currentNode: 2, visitedIndices: [0,1,6,2,3,4,5],
        tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:2,6:3,2:4,3:5,4:6,5:7},
        activeEdge: null, backEdge: null,
        log: "Возврат D→C. low[5] = 5 > tin[2] = 4! (C,D) – не мо",
        activeCodeLine: [13,14,15,16,17]
    },
    {
        currentNode: 1, visitedIndices: [0,1,6,2,3,4,5],

```



```

        <rect x={n.x-18} y={n.y-28} width={
"transition-all duration-300" />
        <text x={n.x} y={n.y-20} textAnchor=
} l:{step.low[n.id]|| "?"}</text>
      </>
    })
  </g>;
})}
</svg>
<div className="flex items-center justify-bet
  <span>||ar {dfsIdx+1}/{DFS_STEPS.length}</sp
  <div className="flex-1 mx-2 bg-slate-950 h-
    <div className="bg-indigo-600 h-full tran
  <div>
  </div>
</div>
</div>

{/* Code panel + stack */}
<div className="md:col-span-2 space-y-3">
  {/* Code panel */}
  <div className="bg-slate-950 rounded-lg borde
    <div className="bg-slate-900 px-3 py-1.5 te
    >
    <div className="p-2 overflow-x-auto">
      <pre className="text-[10px] font-mono lea
        {CODE_LINES.map((cl, i) => {
          const hl = step.activeCodeLine.includ
          return <div key={i}
            className={`px-1 transition-all dur
              hl ? "bg-indigo-600/20 text-indigo
            }`}>
            <span className="text-slate-600 mr-1
              {cl.line || ' '}
            </div>;
          })}
        </pre>

```

```
    });  
  }  
}
```

ФАЙЛ 36/86: src/components/DijkstraViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```
import { useState, useEffect } from 'react';  
  
interface Node { id: string; x: number; y: number; name: string; }  
interface Edge { u: string; v: string; w: number; }  
interface Step {  
  currentNode: string | null;  
  checkingEdge: { u: string; v: string } | null;  
  distances: { [key: string]: number };  
  visited: { [key: string]: boolean };  
  previous: { [key: string]: string | null };  
  log: string;  
  activeCodeLine: number;  
}  
  
export default function DijkstraViz() {  
  const nodes: Node[] = [  
    { id: 'A', x: 60, y: 150, name: 'Пиццерия (A)' },  
    { id: 'B', x: 160, y: 60, name: 'Дом 1 (B)' },  
    { id: 'C', x: 160, y: 240, name: 'Парк (C)' },  
    { id: 'D', x: 340, y: 60, name: 'Офис (D)' },  
    { id: 'E', x: 340, y: 240, name: 'Дом 2 (E)' },  
    { id: 'F', x: 450, y: 150, name: 'ТЦ (F)' },  
    { id: 'G', x: 250, y: 150, name: 'Метро (G)' },  
  ];  
  
  const edges: Edge[] = [  
    { u: 'A', v: 'B', w: 4 },  
    { u: 'A', v: 'C', w: 2 },  
    { u: 'C', v: 'B', w: 1 },  
  ];
```

```

const prev: Record<string, string | null> = { A: null };

simulationSteps.push({
  currentNode: null, checkingEdge: null,
  distances: { ...dist }, visited: { ...visited },
  log: ` Дейкстра готов к доставке. Начинаем из ПИ,
  activeCodeLine: 2,
});

for (let iter = 0; iter < nodes.length; iter++) {
  let minD = 999;
  let u = null;
  for (const n of nodes) {
    if (!visited[n.id] && dist[n.id] < minD) {
      minD = dist[n.id];
      u = n.id;
    }
  }

  if (!u) break;

  visited[u] = true;
  simulationSteps.push({
    currentNode: u, checkingEdge: null,
    distances: { ...dist }, visited: { ...visited },
    log: ` Выбираем вершину ${u} с мин. непосещённой,
    activeCodeLine: 5,
  });

  const neighbors = edges.filter(e => e.u === u);
  for (const edge of neighbors) {
    const v = edge.v;
    if (visited[v]) continue;

    simulationSteps.push({
      currentNode: u, checkingEdge: { u, v },
      distances: { ...dist }, visited: { ...visited },
      log: ` Смотрим соседа ${v}. Текущее расстояние: ${dist[u]},

```



```

const currentStep = steps[currentStepIndex] || null;

if (!currentStep) return <div className="text-white">

return (
  <div className="bg-slate-800/90 p-6 rounded-2xl border
    { /* Шапка симулятора */ }
    <div className="flex flex-wrap items-center justify
      <div className="flex items-center gap-3">
        <div className="p-2.5 bg-indigo-600 text-white
          <span className="text-2xl"> </span>
        </div>
      <div>
        <h3 className="text-2xl font-bold text-white
          <p className="text-sm text-indigo-300">Полн
        </div>
      </div>

    <div className="flex items-center gap-2">
      <button
        onClick={() => { setIsPlaying(false); setCu
        className="flex items-center gap-1 bg-slate
        ion-colors"
      >
        Сброс
      </button>
      <button
        onClick={() => setIsPlaying(!isPlaying)}
        className={`flex items-center gap-1 px-4 py
          isPlaying ? 'bg-amber-600 hover:bg-amber-!
        }`}
      >
        {isPlaying ? ' Пауза' : '▶ Пуск'}
      </button>
      <button
        onClick={() => { setIsPlaying(false); if (c
        disabled={currentStepIndex >= steps.length

```

```

    let marker = 'url(#arrow-dh-normal)';
    if (isChecking) marker = 'url(#arrow-dh-a
    else if (isOptimal) marker = 'url(#arrow-

    return (
      <g key={idx}>
        <line
          x1={uNode.x} y1={uNode.y}
          x2={vNode.x} y2={vNode.y}
          stroke={isChecking ? '#f59e0b' : is
          strokeWidth={isChecking ? 5 : isOpt
          markerEnd={marker}
          className="transition-all duration-
          strokeDasharray={isChecking ? '4,4'
        />
        <circle
          cx={({uNode.x + vNode.x} / 2} cy={({u
          r="12" fill="#1e293b"
          stroke={isChecking ? '#f59e0b' : '#
        />
        <text
          x={({uNode.x + vNode.x} / 2} y={({uNo
          fill={isChecking ? '#f59e0b' : '#f8
          fontSize="11" fontWeight="bold" tex
        >
          {edge.w}
        </text>
      </g>
    );
  }}}

  {/* Вершины */}
  {nodes.map((node) => {
    const isCurrent = currentStep.currentNode
    const isVisited = currentStep.visited[node.i
    const dist = currentStep.distances[node.i

```

```

        isCurrentNode ? 'bg-emerald-900/60' :
        'text-slate-300'
      }`}>
      <div className="flex items-center gap-2">
        <span className={`px-2 py-0.5 rounded-sm text-slate-300`} `}>
          {u}
        </span>
        <span className="text-slate-400">→</span>
      </div>
      <div className="flex flex-wrap gap-2">
        {adjList[u].map((edge, idx) => {
          const isCheckingThis = currentStep === idx
          return (
            <span key={idx} className={`px-2 py-0.5 rounded-sm
              isCheckingThis ? 'bg-amber-900/60 text-slate-300' :
              'text-slate-400'`} `}>
              <span>{edge.v}</span><span className="font-size-0.8">
                <span className={isCheckingThis ? 'font-weight-normal' : 'font-weight-normal'}>
                  {edge.w}
                </span>
              </span>
            </div>
          )
        })}
      </div>
    </div>
  </div>
</div>

<div className="flex flex-col lg:flex-row gap-6">
  {/* Таблица расстояний */}
  <div className="w-full lg:w-1/2 bg-rose-950/10 p-6">
    <div>
      <h4 className="text-lg font-bold text-rose-950">Таблица расстояний
      </h4>
      <div className="overflow-x-auto">

```

```

    { /* Код алгоритма */ }
    <div className="w-full lg:w-1/2 bg-slate-900/60">
      <div>
        <h4 className="text-lg font-bold text-slate">
          <div className="bg-slate-950/80 rounded-lg">
            {codeLines.map((item) => {
              const isActive = currentStep.activeCodeLine === item.line
              return (
                <div key={item.line} className={`py-1 ${
                  isActive ? 'bg-indigo-900/80 text-white' : 'bg-slate-950/80 text-slate-300'
                }`} >
                  <span className="text-slate-600 select-none">{item.line}</span>
                  <span className="whitespace-pre font-family: monospace">{item.code}</span>
                </div>
              )
            })}
          </div>
        </div>
      </div>
    </div>
  </div>
);
}

```

```

    if (n in memoMap) {
      generatedSteps.push({
        id: stepId++,
        n,
        action: 'memo_hit',
        desc: `✂ Кэш-хит! fibМемо(${n}) уже посчитано`,
        codeLine: 2,
        memoState: { ...memoMap }
      });
      return memoMap[n];
    }

    if (n <= 2) {
      generatedSteps.push({
        id: stepId++,
        n,
        action: 'base_case',
        desc: `Базовый случай: n <= 2 (n=${n}). Возвр`,
        codeLine: 3,
        memoState: { ...memoMap }
      });
      return 1;
    }

    generatedSteps.push({
      id: stepId++,
      n,
      action: 'call',
      desc: `Вычисляем рекурсивно: fibМемо(${n - 1})`,
      codeLine: 4,
      memoState: { ...memoMap }
    });

    const res1 = simulateFib(n - 1);
    const res2 = simulateFib(n - 2);
    memoMap[n] = res1 + res2;

    generatedSteps.push({

```

```

const currentStep = steps[currentStepIndex] || null;
const memoState = currentStep?.memoState || {};

return (
  <div className="bg-slate-900 rounded-2xl border border-slate-800" >
    /* Header & Controls */
    <div className="flex flex-col lg:flex-row lg:items-center" >
      <div>
        <div className="flex items-center gap-3 mb-2" >
          <div className="p-2 bg-emerald-500/10 border border-emerald-500" >
            <RefreshCw className="w-6 h-6" />
          </div>
          <h3 className="text-2xl font-extrabold text-white" >
            Задача
          </h3>
          <p className="text-sm text-slate-400" >
            Смотри, как таблица DP кэширует результаты
          </p>
        </div>

        <div className="flex flex-wrap items-center gap-2" >
          <div className="flex items-center gap-2 bg-slate-800 p-2" >
            <span>N:</span>
            <select
              value={targetN}
              onChange={(e) => setTargetN(Number(e.target.value))}
              disabled={isPlaying}
              className="bg-transparent text-white font-mono" >
              <option value={4}>N = 4</option>
              <option value={5}>N = 5</option>
              <option value={6}>N = 6</option>
              <option value={7}>N = 7</option>
            </select>
          </div>

          <div className="flex items-center bg-slate-950 p-2" >
            <button

```

```

<div className="mb-8 bg-slate-950 p-4 md:p-6 rounded"
  <h4 className="text-xs font-bold uppercase tracking-wider"
  <div className="flex flex-wrap items-center gap-2"
    {Array.from({ length: targetN }, (_, i) => i)}
    const isCached = num in memoState || num <= 2
    const val = num <= 2 ? 1 : memoState[num];
    const isCurrent = currentStep?.n === num;

    return (
      <div
        key={num}
        className={`flex flex-col items-center justify-center gap-2
          ${isCurrent
            ? 'bg-emerald-950 border-emerald-500'
            : isCached
            ? 'bg-slate-900 border-emerald-500/50'
            : 'bg-slate-900/50 border-slate-800'}
        `}
      >
        <span className="text-xs text-slate-400">{num}</span>
        <span className={`text-xl md:text-2xl font-medium text-slate-400
          ${isCached ? val : '0'}>{val}</span>
        </span>
        {num <= 2 && <span className="text-[9px] font-mono text-slate-400">{val}</span>
      </div>
    );
  )}
</div>

```

```

{currentStep && (
  <div className="mt-6 pt-4 border-t border-slate-800"
    <div className="text-sm font-medium text-slate-400"
      <span className="inline-block w-2 h-2 rounded-full bg-slate-900" />
      <span>{currentStep.desc}</span>
    </div>
    <div className="text-xs font-mono bg-slate-900 p-2"
      Текущий N: <strong className="text-emerald-400">{currentStep.n}</strong>
    </div>
  )}

```

```

        В классической рекурсии для  $N=\{targetN\}$  по
        вычисленное значение в таблице, мы момента.
    </p>
    <div className="p-4 bg-slate-900 rounded-xl t-slate-300">
        <span>Всего шагов симуляции: {steps.length}</span>
        <span className="text-emerald-400 font-bo
    </div>
</div>
)}

{activeTab === 'code' && (
    <div className="bg-slate-950 rounded-2xl border
    <div className="bg-slate-900 px-6 py-3 border
        <span className="text-xs font-bold font-m
        <span className="text-xs text-emerald-400
    </div>
    <pre className="p-6 font-mono text-sm space
        {DP_CODE_LINES.map((line, idx) => {
            const lineNum = idx + 1;
            const isActive = currentStep?.codeLine :
            return (
                <div
                    key={lineNum}
                    className={`flex items-center px-4
                        isActive
                            ? 'bg-emerald-600/20 border-l-4
                            : 'text-slate-300 hover:bg-slate
                    `}
                >
                    <span className="w-8 text-xs text-s
                    <span>{line}</span>
                </div>
            );
        })}
    </pre>
    <div className="p-6 bg-slate-900/50 border-
        <span className="p-1.5 bg-emerald-500/20

```



```
];
```

```
const C1_EDGES = [
  { u: 0, v: 1 }, { u: 0, v: 2 },
  { u: 1, v: 3 }, { u: 2, v: 4 },
  { u: 1, v: 2 },
  { u: 3, v: 4 }
];
```

```
export function EulerSimulator() {
  const [c1Path, setC1Path] = useState<number[]>([]);
  const [c1VisitedEdges, setC1VisitedEdges] = useState<
  const [c1Msg, setC1Msg] = useState("Кликни на вершину
  const [c1Done, setC1Done] = useState(false);
```

```
const resetC1 = () => {
  setC1Path([]); setC1VisitedEdges([]); setC1Msg("Кли
};
```

```
const clickC1 = (vId: number) => {
  if (c1Done && c1Path.length > 0) return;
  if (c1Path.length === 0) {
    setC1Path([vId]);
    setC1Msg("Старт! Кликай соседние вершины, чтобы п
    return;
  }
```

```
const last = c1Path[c1Path.length - 1];
const edge = C1_EDGES.find(e => (e.u === last && e.v === vId));
if (!edge) {
  setC1Msg("Нет ребра между этими вершинами! Выбери
  return;
}
```

```
const key = `${Math.min(last, vId)}-${Math.max(last, vId)}`;
if (c1VisitedEdges.includes(key)) {
  setC1Msg("Это ребро уже пройдено! Эйлер не прощае
  setC1Done(true); return;
}
```

```
const nextEdges = [...c1VisitedEdges, key];
```

```

        return <line key={i} x1={u.x} y1={u.y} x2={v.x} y2={v.y}
          stroke={used ? "#6366f1" : "#475569"}
          strokeWidth={used ? 4 : 2}
          className="transition-all duration-300" />
      )}
    {C1_NODES.map(n => {
      const isLast = c1Path[c1Path.length-1] === n.id;
      const visited = c1Path.includes(n.id);
      return <g key={n.id} className="cursor-pointer"
        {isLast && <circle cx={n.x} cy={n.y} r={11}
          <circle cx={n.x} cy={n.y} r={11}
            fill={isLast ? "#6366f1" : visited ? "#6366f1" : "#fff"}
            stroke={isLast ? "#fff" : visited ? "#6366f1" : "#6366f1"}
            strokeWidth={2.5}
            className="transition-all duration-200" />
            <text x={n.x} y={n.y+4} textAnchor="middle">{n.name}
            </text>
          </g>
        )}
      </svg>
      <div className="absolute top-2 left-2 bg-slate-500 text-white p-2"
        >Pe6ëp: {c1VisitedEdges.length}/{C1_EDGES.length}</div>
    </div>

    <div className={`p-3 rounded-xl border text-sm ${
      c1Done && c1VisitedEdges.length === C1_EDGES.length
        ? "bg-emerald-950/40 border-emerald-500/30 text-emerald-500"
        : c1Done
        ? "bg-rose-950/40 border-rose-500/30 text-rose-500"
        : "bg-slate-900/50 border-slate-700 text-slate-500"}
    `}>
      {c1Msg}
    </div>
  </div>
);
}

```

```
        закрытые вершины.',
        'Отрицательные ребра создают гравитационный коллапс',
        'Дейкстра просто не любил отрицательные балансы на
    ],
    correctAnswer: 1,
    explanation: 'Именно! Дейкстра уверен: если ты приедешь
        ля чего нужен алгоритм Беллмана-Форда.',
},
{
    id: 3,
    question: 'Какова ключевая фишка алгоритма Борувки',
    options: [
        'Он использует меньше памяти благодаря коммунистиче
        'Он умеет работать с отрицательными весами без ци
        'Он позволяет выполнять шаги слияния колхозов (ко
        'Он был разработан в Токио и поэтому самый быстрый
    ],
    correctAnswer: 2,
    explanation: 'Абсолютно верно! В Борувке каждая ком
        лелить вычисления на GPU или многопроцессорных кл
},
{
    id: 4,
    question: 'В чем главная суть Динамического программи
    options: [
        'Писать код очень быстро, динамично нажимая на кл
        'Запоминать (кешировать) результаты уже решенных
        'Использовать динамическую типизацию как в Python
        'Постоянно менять требования к проекту в процессе
    ],
    correctAnswer: 1,
    explanation: 'Точно! Главный принцип ДП – мемоизаци
},
{
    id: 5,
    question: 'Какая структура данных идеально помогает
    options: [
        'СНМ (Система непересекающихся множеств / Disjoin
```

```

const handleRestart = () => {
  setCurrentQIndex(0);
  setSelectedOption(null);
  setIsAnswered(false);
  setScore(0);
  setShowResults(false);
};

if (showResults) {
  return (
    <div className="bg-slate-900/90 border border-slate-
      my-12">
      <div className="w-20 h-20 bg-gradient-to-tr from-
        -6 shadow-xl shadow-indigo-500/30">
        <Award className="w-10 h-10 text-white" />
      </div>
      <h3 className="text-3xl font-bold text-white mb-
      <p className="text-slate-400 text-sm mb-6">Ты пр

    <div className="bg-slate-950 border border-slate-
      <p className="text-slate-400 text-sm uppercase
      <p className="text-5xl font-black text-indigo
      <p className="text-slate-300 text-sm">
        {score === quizQuestions.length
          ? ' Идеально! Ты настоящий сеньор-раскра
          : score >= 3
          ? ' Отличный результат! Главные концепции
          : ' Стоит повторить главы пособия и еще
        </p>
      </div>

    <button
      onClick={handleRestart}
      className="inline-flex items-center gap-2 px-4
        d rounded-xl shadow-lg shadow-indigo-600/30
    >
      <RotateCcw className="w-5 h-5" />
      <span>Пройти тест заново</span>

```

```

        }
    }

    return {
        <div
            key={idx}
            onClick={() => handleSelectOption(idx)}
            className={"flex items-start gap-4 p-4"}
        >
            <div className="mt-0.5 shrink-0">
                {isAnswered && isCorrect ? (
                    <CheckCircle2 className="w-6 h-6 text-emerald-500" />
                ) : isAnswered && isSelected && !isCorrect ? (
                    <XCircle className="w-6 h-6 text-red-500" />
                ) : (
                    <div className={"w-6 h-6 rounded-full"}
                        order-indigo-500 bg-indigo-500 text-white"
                        {String.fromCharCode(65 + idx)}
                    </div>
                )}
            </div>
            <div className="flex-1 text-sm md:text-base">
                {option}
            </div>
        </div>
    );
}}}
</div>
</div>

{isAnswered && (
    <div className="bg-slate-950 border border-slate-800 p-4">
        <div className="flex items-center gap-2 mb-2">
            <AlertCircle className="w-4 h-4" />
            <span>Объяснение:</span>
        </div>
        <p className="text-slate-300 text-sm leading-relaxed">
            {currentQ.explanation}
        </p>
    </div>
)}

```

```

const [currentStepIndex, setCurrentStepIndex] = useSt
const [isPlaying, setIsPlaying] = useState(false);

const nodeNames = ['А 0', 'Б 1', 'В 2', 'Г 3', 'Д 4',
const nodesSvg = [
  { id: 0, name: 'Аэропорт 0', x: 100, y: 60 },
  { id: 1, name: 'Аэропорт 1', x: 250, y: 60 },
  { id: 2, name: 'Аэропорт 2', x: 400, y: 60 },
  { id: 3, name: 'Аэропорт 3', x: 100, y: 180 },
  { id: 4, name: 'Аэропорт 4', x: 400, y: 180 },
  { id: 5, name: 'Аэропорт 5', x: 175, y: 280 },
  { id: 6, name: 'Аэропорт 6', x: 325, y: 280 },
];

const initialMatrix = [
  [0, 4, 999, 2, 999, 999, 999],
  [999, 0, 3, 999, 999, 3, 999],
  [999, 999, 0, 999, 2, 999, 999],
  [999, 999, 999, 0, 4, 2, 999],
  [999, 999, 999, 999, 0, 999, 1],
  [999, 999, 999, 999, 999, 0, 5],
  [999, 1, 999, 999, 999, 999, 0]
];

const adjList: { [key: number]: { v: number; w: numbe
  0: [{ v: 1, w: 4 }, { v: 3, w: 2 }],
  1: [{ v: 2, w: 3 }, { v: 5, w: 3 }],
  2: [{ v: 4, w: 2 }],
  3: [{ v: 4, w: 4 }, { v: 5, w: 2 }],
  4: [{ v: 6, w: 1 }],
  5: [{ v: 6, w: 5 }],
  6: [{ v: 1, w: 1 }],
];

const codeLines = [
  { line: 1, text: 'function floyd_warshall(matrix, V
  { line: 2, text: '    dist = copy(matrix)' },
  { line: 3, text: '    for k from 0 to V - 1: // Пос

```

```

        });
      }
    }
  }
}

simulationSteps.push({
  k: 7, i: -1, j: -1, matrix: dist.map(r => [...r])
  log: ' Все пары отработаны. Алгоритм Флойда завершён',
  activeCodeLine: 8,
});

setSteps(simulationSteps);
setCurrentStepIndex(0);
setIsPlaying(false);
}, []));

useEffect(() => {
  let timer: any = null;
  if (isPlaying && currentStepIndex < steps.length - 1) {
    timer = setTimeout(() => setCurrentStepIndex((p) => p + 1), 1000);
  } else if (currentStepIndex >= steps.length - 1) {
    setIsPlaying(false);
  }
  return () => clearTimeout(timer);
}, [isPlaying, currentStepIndex, steps.length]);

const currentStep = steps[currentStepIndex] || null;
if (!currentStep) return <div>Загрузка...</div>;

return (
  <div className="bg-slate-800/90 p-6 rounded-2xl border border-slate-700">
    <div className="flex flex-wrap items-center justify-center gap-4">
      <div className="flex items-center gap-3">
        <div className="p-2.5 bg-emerald-600 text-white rounded-md">
          <span className="text-2xl"> </span>
        </div>
        <div>

```

```

const isTarget = currentStep.i === u &&
const isVia = (currentStep.i === u && c
let strokeColor = '#475569'; let marker
if (isTarget) { strokeColor = '#10b981'
else if (isVia) { strokeColor = '#818cf8'
return (
  <g key={`$${u}-${edge.v}`}>
    <line x1={uNode.x} y1={uNode.y} x2=
markerEnd={marker} strokeDasharray={isTarg
    <circle cx={({uNode.x + vNode.x) / 2}
isVia ? '#818cf8' : '#64748b'} strokeWidth=
    <text x={({uNode.x + vNode.x) / 2} y=
8fafc'} fontSize="10" fontWeight="bold" tex
  </g>
);
});
}}}
{nodesSvg.map((node) => {
  const isK = currentStep.k === node.id;
  const isI = currentStep.i === node.id;
  const isJ = currentStep.j === node.id;
  let fill = '#1e293b'; let stroke = '#64748b';
  if (isK) { fill = '#4f46e5'; stroke = '#a8a8a8';
  else if (isI) { fill = '#b45309'; stroke = '#a8a8a8';
  else if (isJ) { fill = '#047857'; stroke = '#a8a8a8';
  return (
    <g key={node.id} className="transition-
    <circle cx={node.x} cy={node.y} r={isI
sJ ? 4 : 2} className="transition-colors du
    <text x={node.x} y={node.y + 5} fill=
    <text x={node.x} y={node.y + 32} fill=
  </text>
  </g>
);
  });
  }}}
</svg>

<div className="w-full bg-slate-950/80 p-4 ro

```



```

<div className="flex flex-col lg:flex-row gap-6">
  {/* Матрица расстояний */}
  <div className="w-full lg:w-1/2 bg-rose-950/10">
    <div className="flex justify-between items-center">
      <h4 className="text-lg font-bold text-rose-950">
        Матрица расстояний
      </h4>
    </div>
    <div className="overflow-x-auto">
      <table className="w-full text-center border-collapse">
        <thead><tr className="bg-rose-950/40 text-rose-950">
          <th className="p-2 border-r border-rose-950">
            Шаг
          </th>
          {nodeNames.map((n, idx) => <th key={idx} className="p-2 border-r border-rose-950 font-bold" : ''>`>{n.split(' ')[0]} {idx + 1}</th>)}
        </tr></thead>
        <tbody className="text-xs font-mono">
          {currentStep.matrix.map((row, i) => (
            <tr key={i} className="border-b border-rose-950">
              <td className={`p-2 bg-rose-950/40 font-bold` : ''}>
                Шаг
              </td>
              {nodeNames[i].split(' ')[0]} {i + 1}
            </td>
            {row.map((val, j) => {
              const isUpd = currentStep.i === i
              const isVia = currentStep.k !== i
              const isEnd = currentStep.j === j);
              let bg = 'bg-slate-900/40 text-slate-950';
              if (isUpd) bg = 'bg-emerald-900/80 text-emerald-950';
              else if (isVia) bg = 'bg-indigo-900/80 text-indigo-950';
              else if (i === j) bg = 'bg-slate-900/40 text-slate-950';
              return (<td key={j} className={`p-2 ${bg} font-mono` : ''}>
                {val}
              </td>)});
            </tr>
          ))}
        </tbody>
      </table>
    </div>
  </div>
</div>

```

```

    { id: 'F', label: 'F', x: 250, y: 200 },
    { id: 'G', label: 'G', x: 350, y: 200 },
];

const edges: Edge[] = [
    { u: 'A', v: 'B' }, { u: 'A', v: 'C' },
    { u: 'B', v: 'D' }, { u: 'B', v: 'E' },
    { u: 'C', v: 'F' }, { u: 'C', v: 'G' },
];

const adj: Record<string, string[]> = {
    'A': ['B', 'C'],
    'B': ['A', 'D', 'E'],
    'C': ['A', 'F', 'G'],
    'D': ['B'],
    'E': ['B'],
    'F': ['C'],
    'G': ['C'],
};

type Action = {
    type: 'visit' | 'process' | 'backtrack';
    node: string;
    desc: string;
    queueOrStack?: string[];
    visitedNodes?: string[];
};

function generateDFSSteps(start: string) {
    const steps: Action[] = [];
    const vis = new Set<string>();
    const stack: string[] = []; // for visualization, j

    function dfs(v: string) {
        vis.add(v);
        stack.push(v);
        steps.push({ type: 'visit', node: v, desc: `Замечание: узел ${v} был посещен`,
            visitedNodes: Array.from(vis) });
    }
}

```

```

        steps.push({ type: 'visit', node: u, desc: `04 ${u} ${vis} ${vis === 1 ? 'backtrack' : 'visit'}`,
                    tack: [...q], visitedNodes: [...Array.from(Array(1000), () => 0)]
                  })
      }
    }

    steps.push({ type: 'backtrack', node: '', desc: `04 ${u} ${vis} ${vis === 1 ? 'backtrack' : 'visit'}`,
                tack: [...q], visitedNodes: [...Array.from(Array(1000), () => 0)] });

    return steps;
  }
}

```

```

export function GraphTraversalViz() {
  const [mode, setMode] = useState<"dfs" | "bfs">("dfs");
  const [autoPlay, setAutoPlay] = useState(false);
  const [stepIdx, setStepIdx] = useState(0);

  const steps = useMemo(() => mode === 'dfs' ? generateDfsSteps(1000) : generateBfsSteps(1000), [mode]);

  useEffect(() => {
    setStepIdx(0);
    setAutoPlay(false);
  }, [mode]);

  useEffect(() => {
    if (autoPlay) {
      const t = setInterval(() => {
        setStepIdx(p => {
          if (p >= steps.length - 1) {
            setAutoPlay(false);
            return p;
          }
          return p + 1;
        });
      }, 1200);
      return () => clearInterval(t);
    }
  }, [autoPlay, steps]);
}

```



```

        {/* Controls */}
        <div className="flex bg-slate-900 border
            <button onClick={() => {setAutoPlay
"p-2 text-slate-400 hover:text-white hover:
            <Undo strokeWidth={3} size={16}
            </button>
            <button onClick={() => setAutoPlay(
justify-center min-w-[80px] ${mode === 'df
>
            {autoPlay ? <><Pause size={16}
            </button>
            <button onClick={() => {setAutoPlay
= steps.length - 1} className="p-2 text-sla
hover:bg-transparent">
            <SkipForward strokeWidth={3} si
            </button>
            <button onClick={() => {setAutoPlay
g-slate-800 rounded-lg ml-2 border-l border
            <RefreshCw strokeWidth={3} size
            </button>
        </div>

        {/* Description Log */}
        <div className="flex-1 bg-slate-900/50
ow-hidden">
            <div className="text-[10px] text-sl
{steps.length}</div>
            <div className="text-sm text-slate-:
                {step.desc}
            </div>
        </div>
    </div>
</div>
);
}

```

```

    if (l < n && a[l].priority > a[largest].priority) l = l;
    if (r < n && a[r].priority > a[largest].priority) l = r;
    if (largest !== idx) {
      [a[largest], a[idx]] = [a[idx], a[largest]];
      idx = largest;
    } else break;
  }
  return a;
}

function heapInsert(arr: Patient[], item: Patient): Patient[] {
  return siftUp([...arr, { ...item }]);
}

// Position in tree layout
function nodePos(index: number): { x: number; y: number } {
  const level = Math.floor(Math.log2(index + 1));
  const posInLevel = index - (Math.pow(2, level) - 1);
  const count = Math.pow(2, level);
  const x = ((posInLevel + 0.5) / count) * 100;
  const y = 10 + level * 24;
  return { x, y };
}

// Priority presets for realistic ER scenarios
const PATIENT_PRESETS = [
  { name: 'Иван (Инфаркт)', symptom: ' Болиит сердце', emoji: '👤' },
  { name: 'Маша (Перелом)', symptom: ' Открытый перелом', emoji: '👤' },
  { name: 'Кот Барсик (Укус)', symptom: ' Укусил шмеля', emoji: '🐈' },
  { name: 'Семён (Температура)', symptom: ' Жар 39.5', emoji: '👤' },
  { name: 'Оля (Порез)', symptom: ' Глубокий порез', emoji: '👤' },
  { name: 'Пётр (Кашель)', symptom: ' Простуда', emoji: '👤' },
];

const INITIAL_PATIENTS: Patient[] = (() => {
  let arr: Patient[] = [];
  const start = [
    { name: 'Иван (Инфаркт)', symptom: ' Болиит сердце', emoji: '👤' },
  ];
  arr.push(...start);
  return arr;
})();

```

```

    const finalSymptom = preset.symptom;

    const newPatient: Patient = {
      id: `p${uid++}`,
      name: finalName,
      emoji: finalEmoji,
      symptom: finalSymptom,
      priority,
      animState: 'in',
    };

    const nextHeap = heapInsert(heap, newPatient).map(x => {
      // Mark specifically this new patient as 'in' in the heap
      const target = nextHeap.find(x => x.priority === priority);
      const marked = nextHeap.map(x => x.id === (target?.id));

      setHeap(marked);
      addLog(` Поступил пациент: ${finalName} с тяжестью ${finalSymptom}`);

      setTimeout(() => {
        setHeap(nextHeap);
        setBusy(false);
      }, 500);
    }, [busy, heap]);

    const handleRandomInsert = () => {
      const preset = PATIENT_PRESETS[Math.floor(Math.random() * PATIENT_PRESETS.length)];
      const randomShift = Math.floor(Math.random() * 10);
      const prio = Math.max(10, Math.min(99, preset.priority + randomShift));
      insertPatient(preset.name, prio);
    };

    const handleCustomInsert = (e: React.FormEvent) => {
      e.preventDefault();
      if (!customName.trim()) return;
      insertPatient(customName, customPriority);
      setCustomName('');
    };

```

```

        setBusy(false);
      }, 600);
    }, 800);

    }, 350);
  }, 650);
}, [busy, heap]);

const reset = () => {
  setHeap(INITIAL_PATIENTS.map(x => ({ ...x, animStat: 0 })));
  setLog([' База данных скорой помощи перезагружена.']);
  setHealing(null);
  setTimeout(() => setHeap(INITIAL_PATIENTS.map(x => ({ ...x, animStat: 0 } ))), 1000);
};

// --- Render binary tree lines ---
const edges = heap.flatMap((_, idx) => {
  const res: React.ReactElement[] = [];
  const p = nodePos(idx);
  const l = 2 * idx + 1;
  const r = 2 * idx + 2;
  if (l < heap.length) {
    const lp = nodePos(l);
    res.push(
      <line key={`el${idx}`}
        x1={` ${p.x}%`} y1={` ${p.y + 4}%`}
        x2={` ${lp.x}%`} y2={` ${lp.y - 4}%`}
        stroke="#475569" strokeWidth="2"
      />
    );
  }
  if (r < heap.length) {
    const rp = nodePos(r);
    res.push(
      <line key={`er${idx}`}
        x1={` ${p.x}%`} y1={` ${p.y + 4}%`}
        x2={` ${rp.x}%`} y2={` ${rp.y - 4}%`}
        stroke="#475569" strokeWidth="2"
      />
    );
  }
});

```



```

<form onSubmit={handleCustomInsert} className="
  <input
    type="text"
    required
    value={customName}
    onChange={e => setCustomName(e.target.value)}
    placeholder="Имя пациента"
    className="flex-1 bg-slate-800 border border-emerald-500 transition-colors placeholder:
  />
  <div className="flex flex-col items-center min-w-[300px]">
    <span className="text-[9px] font-mono text-white">
      <input
        type="range"
        min="10"
        max="99"
        value={customPriority}
        onChange={e => setCustomPriority(Number(e.target.value))}
        className="w-20 accent-emerald-500 cursor-pointer"
      />
    </div>
    <button
      type="submit"
      disabled={busy || !customName.trim() || heap.length === 0}
      className="bg-teal-600 hover:bg-teal-500 disabled:bg-slate-800 text-white px-4 rounded-xl shadow-lg active:scale-95 transition-all"
    >
      Вести
    </button>
  </form>

  {/* Забрать самого тяжелого */}
  <button
    onClick={extractMax}
    disabled={busy || heap.length === 0}
    className="flex-1 min-w-[170px] bg-gradient-to-r from-teal-600 to-teal-500 disabled:cursor-not-allowed text-white font-mono transition-all cursor-pointer flex items-center justify-center"
  >
    Вывести
  </button>

```

```

        ${patient.animState === 'winner' ?
        ${patient.animState === 'out' ? 'an
        ${isRoot ? 'bg-rose-950/80 border-r
    `}
    style={{
      left: `${pos.x}%`,
      top: `${pos.y}%`,
      width: isRoot ? 60 : 52,
      height: isRoot ? 60 : 52,
    }}
  >
    <span className="text-2xl leading-non
    <span className="text-[10px] font-mon
      {patient.priority}%
    </span>

    {/* Tooltip */}
    <div className="absolute bottom-full
      <div className="bg-slate-950 border
0 shadow-xl">
      <div className="font-bold text-wh
      <div className="text-emerald-400"
      </div>
    </div>
  </div>
    );
  }}}
</div>

  <div className="w-full h-3 bg-gradient-to-r f
</div>

{/* ПРАВАЯ КОЛОНКА: ОПЕРАЦИОННЫЙ СТОЛ */}
<div className="lg:col-span-4 bg-slate-900 roun
  <div className="absolute top-4 left-4 text-[1
    Операционная койка
  </div>

```

```
      {log.map((entry, idx) => (  
        <div  
          key={idx}  
          className={`text-xs font-mono flex items-center justify-between gap-2`}  
        >  
          {idx === 0 ? <span className="text-emerald-500 font-weight-bold">1</span> :  
            <span>{entry}</span>  
        </div>  
      )  
    )  
  </div>  
</div>  
);  
};
```

ФАЙЛ 43/86: src/components/hints/demoKit.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```
import React, { createContext, useContext, useEffect, useLayoutEffect, useState } from 'react';
```

```
/** Общие примитивы для мини-визуализаций подсказок. */
```

```
export const W = 260;
```

```
export const H = 130;
```

```
/**
```

```
 * Общий множитель скорости. Кадры были слишком короткими, поэтому решил
```

```
 * заметить, что именно переехало, и анимация читалась
```

```
 */
```

```
export const SPEED = 2.1;
```

```
/** Длительность переезда узлов. Должна быть заметно меньше, чем длительность
```

```
export const MOVE_MS = 800;
```

```
export const EASE = "cubic-bezier(.34,.01,.2,1)";
```

```
export const MOVE = `all ${MOVE_MS}ms ${EASE}`;
```

```
/** Наведение на карточку ставит анимацию на паузу — motion-анимация
```

```

        </div>
      ))
      <div className="text-[9px] text-slate-600 text-center"
        {paused ? "пауза – курсор на карточке" : "наведи"}>
      </div>
    </div>
  );
};

```

```

export const Node: React.FC<{
  x: number;
  y: number;
  r?: number;
  fill: string;
  label?: string | number;
  stroke?: string;
}> = ({ x, y, r = 13, fill, label, stroke }) => (
  <g style={{ transition: MOVE }}>
    <circle cx={x} cy={y} r={r} fill={fill} stroke={stroke} />
    {label !== undefined && (
      <text x={x} y={y} textAnchor="middle" dominantBaseline="middle">
        {label}
      </text>
    )}
  </g>
);

```

```

export const Edge: React.FC<{
  a: [number, number];
  b: [number, number];
  color?: string;
  width?: number;
  dashed?: boolean;
}> = ({ a, b, color = C.edge, width = 2, dashed }) => (
  <line
    x1={a[0]}
    y1={a[1]}
    x2={b[0]}
    y2={b[1]}
    stroke={color}
    stroke-width={width}
    stroke-dasharray={dashed ? [4, 4]} : []
  />
);

```

```

    <text x={130} y={112} textAnchor="middle" fontSize=
      9
    </text>
    <text x={72} y={54} textAnchor="middle" fontSize=
      3
    </text>
    <text x={188} y={54} textAnchor="middle" fontSize=
      4
    </text>

    <Node x={pos.i[0]} y={pos.i[1]} label="i" fill={C
    <Node x={pos.k[0]} y={pos.k[1]} label="k" fill={v
    <Node x={pos.j[0]} y={pos.j[1]} label="j" fill={r
  </Svg>
);
};

```

/** Компоненты связности: несколько «островов», которые

```

export const ComponentsDemo: React.FC = () => {
  const comps: [string, string][][] = [
    ["A", "B"], ["B", "C"]],
    ["D", "E"]],
    ["F", "G"], ["G", "H"], ["F", "H"]],
  ];
  const pos: Record<string, [number, number]> = {
    A: [26, 40], B: [62, 92], C: [26, 92],
    D: [120, 40], E: [120, 92],
    F: [190, 34], G: [232, 78], H: [178, 96],
  };
  const colors = [C.active, C.warn, C.done];
  const step = useLoop(comps.length + 1, 900);
  const compOf = (v: string) => comps.findIndex((e) =>

  return (
    <Svg caption={step === 0 ? "Граф распался на острова"
      {comps.flatMap((edges, ci) =>
        edges.map(([u, v], i) => {
          <Edge key={` ${ci} - ${i}`} a={pos[u]} b={pos[v]}

```

```

        fill={C.warn}
      >
        {w}
      </text>
    })
  </g>
);
}}
{Object.entries(pos).map(([k, [x, y]]) => {
  <Node key={k} x={x} y={y} label={k} fill={C.idle}
  >>
})}
</Svg>
);
};

```

/** Сжатие координат: разрежённые числа → плотные индексы

```

export const CoordCompressDemo: React.FC = () => {
  const raw = [1000, 5, 74000, 5, 300];
  const sorted = [5, 300, 1000, 74000];
  const step = useLoop(3, 1200);
  const boxW = 46;
  const startX = (260 - raw.length * (boxW + 4)) / 2;

  return (
    <Svg
      caption={
        step === 0
          ? "Исходные координаты – огромные и с дырами"
          : step === 1
            ? "Сортируем уникальные значения"
            : "Каждое число заменяем его номером → плотные индексы"
      }
    >
      {raw.map((v, i) => {
        <g key={i}>
          <rect x={startX + i * (boxW + 4)} y={20} width={boxW} height={20} />
          <text x={startX + i * (boxW + 4) + boxW / 2} y={30}>
            {v}
          </text>
        </g>
      })}
    </Svg>
  );
};

```

```

const color = (id: string) =>
  id === "x" ? "#6366f1" : id === "p" ? "#0ea5e9" : id

return (
  <Svg caption={captions[step]}>
    {f.edges.map(([a, b], i) => (
      <line
        key={i}
        x1={f.pos[a][0]}
        y1={f.pos[a][1]}
        x2={f.pos[b][0]}
        y2={f.pos[b][1]}
        stroke={C.edge}
        strokeWidth={1.8}
        style={{ transition: MOVE }}
      />
    ))}
    {Object.entries(f.pos).map(([id, [x, y]]) => (
      <g key={id} style={{ transform: `translate(${x}, ${y})` }}>
        <circle r={13} fill={color(id)} stroke={C.idle} />
        <text textAnchor="middle" dominantBaseline="bottom">
          {id}
        </text>
      </g>
    ))}
  </Svg>
);
};

```

/** Zig – один поворот, когда родитель уже корень. */

```

export const ZigDemo: React.FC = () => (
  <SplayFrames
    captions={["p – корень, x под ним", "Один поворот: "]}
    frames={[
      { pos: { p: [130, 32], x: [78, 92] }, edges: [{"p": "x", "x": "p"}]},
      { pos: { x: [130, 32], p: [182, 92] }, edges: [{"p": "x", "x": "p"}]}
    ]}
  />
);

```



```

    });
};

export const RelaxDemo: React.FC = () => {
  const step = useLoop(3, 1100);
  const dV = [12, 12, 7][step];
  const improved = step === 2;
  return (
    <Svg caption={improved ? "12 > 3 + 4 → пишем 7. Путь" : ""}>
      <Edge a={[60, 70]} b={[200, 70]} color={improved ? "red" : "black"} />
      <text x={130} y={54} textAnchor="middle" fontSize={14} />
      <Node x={60} y={70} r={20} fill={C.active} label="3" />
      <Node x={200} y={70} r={20} fill={improved ? C.done : C.active} label="4" />
      <text x={60} y={106} textAnchor="middle" fontSize={14} />
      <text x={200} y={106} textAnchor="middle" fontSize={14} />
      <text x={130} y={22} textAnchor="middle" fontSize={14} />
    </Svg>
  );
};

export const BridgeDemo: React.FC = () => {
  const step = useLoop(2, 1300);
  const pos: Record<string, [number, number]> = {
    A: [40, 40], B: [40, 95], C: [95, 68], D: [170, 68]
  };
  const edges: [string, string][] = [
    ["A", "B"], ["A", "C"], ["B", "C"], ["C", "D"], ["D", "A"]
  ];
  const cut = step === 1;
  return (
    <Svg caption={cut ? "Убрали C-D → граф развалился на 2 компонента" : ""}>
      {edges.map(([u, v], i) => {
        const isBridge = u === "C" && v === "D";
        if (isBridge && cut) return null;
        return <Edge key={i} a={pos[u]} b={pos[v]} color={isBridge && cut ? "red" : "black"} />
      })}
      {Object.entries(pos).map(([k, [x, y]]) => (
        <Node key={k} x={x} y={y} label={k} fill={cut ? C.done : C.active} />
      ))}
    </Svg>
  );
};

```

```

return (
  <Svg caption={`Жадно берём самые лёгкие рёбра без ц
    {edges.map((e, i) => {
      const on = taken.has(`${e.u}-${e.v}`);
      return (
        <g key={i}>
          <Edge a={pos[e.u]} b={pos[e.v]} color={on ?
          <text x={({pos[e.u][0] + pos[e.v][0]} / 2} y=
            textAnchor="middle" fontSize={9} fontWeigh
          </g>
        );
      });
    })}
    {Object.entries(pos).map(([k, [x, y]]) => (<Node
  </Svg>
);
};

export const SccDemo: React.FC = () => {
  const pos: Record<string, [number, number]> = {
    A: [45, 35], B: [110, 35], C: [77, 95], D: [185, 45]
  };
  const edges: [string, string][] = [["A", "B"], ["B",
  const step = useLoop(2, 1400);
  const scc = ["A", "B", "C"];
  const inScc = (u: string, v: string) => step === 1 &&
  return (
    <Svg caption={step === 1 ? "{A,B,C} – цикл: из кажд
      {edges.map(([u, v], i) => {
        <Edge key={i} a={pos[u]} b={pos[v]} color={inSc
      })}
      {Object.entries(pos).map(([k, [x, y]]) => {
        <Node key={k} x={x} y={y} label={k} fill={step
      })}
    </Svg>
  );
};

```

```

        </g>
      )})
      <text x={130} y={16} textAnchor="middle" fontSize=
    </Svg>
  );
};

export const QueueDemo: React.FC = () => {
  const states = [["A", "B"], ["A", "B", "C"], ["B", "C"], ["C"]];
  const step = useLoop(states.length, 800);
  const items = states[step];
  return (
    <Svg caption="pop_front слева, push_back справа – к
      <defs>
        <marker id="mdArrow" markerWidth="6" markerHeight="6"
          <path d="M0,0 L6,3 L0,6 Z" fill={C.done} />
        </marker>
      </defs>
      <text x={12} y={44} fontSize={9} fill={C.dim}>ВЫХОД</text>
      <text x={206} y={44} fontSize={9} fill={C.dim}>ВХОД</text>
      {items.map((v, i) => (
        <g key={v} style={{ transition: "all .3s" }}>
          <rect x={30 + i * 54} y={56} width={46} height={20} fill={C.dim} />
          <text x={53 + i * 54} y={71} textAnchor="middle" />
        </g>
      ))}
      <line x1={24} y1={71} x2={10} y2={71} stroke={C.dim} />
    </Svg>
  );
};
};

export const DsuDemo: React.FC = () => {
  const parents = [[0, 1, 2, 3], [0, 0, 2, 3], [0, 0, 2, 3], [0, 0, 2, 3]];
  const step = useLoop(parents.length, 950);
  const p = parents[step];
  const pos: [number, number][] = [[50, 95], [105, 95], [160, 95], [215, 95]];
  const rootColor = ["#6366f1", "#10b981", "#f59e0b", "#f59e0b"];

```



```

    {a.map((v, i) => (
      <g key={i}>
        <rect x={20 + i * 39} y={24} width={34} height={12}
          fill={show && i >= l && i <= r ? C.active : C.dim} />
        <text x={37 + i * 39} y={36} textAnchor="middle">{v}</text>
      </g>
    ))}
    <text x={4} y={72} fontSize={8} fill={C.dim}>P</text>
    {pref.map((v, i) => (
      <g key={i}>
        <rect x={20 + i * 33} y={78} width={29} height={12}
          fill={show && i === l ? C.bad : show && i === r ? C.active : C.dim}
          stroke={C.idleStroke} style={{ transition: "stroke 0.2s" }} />
        <text x={34 + i * 33} y={90} textAnchor="middle">{v}</text>
      </g>
    ))}
  </Svg>
);
};

```

```

export const SparseTableDemo: React.FC = () => {
  const step = useLoop(3, 1000);
  const len = [1, 2, 4][step];
  const count = 8 - len + 1;
  return (
    <Svg caption={`Уровень j=${step}: готовые ответы для j`}>
      {Array.from({ length: 8 }).map((_, i) => (
        <g key={i}>
          <rect x={12 + i * 30} y={18} width={26} height={12}
            fill={C.dim} />
          <text x={25 + i * 30} y={29} textAnchor="middle">{i}</text>
        </g>
      ))}
      {Array.from({ length: count }).map((_, i) => (
        <rect key={i} x={12 + i * 30} y={52 + (i % 4) * 12}
          fill={C.active} opacity={i === 0 ? 1 : 0.4} stroke={C.dim} />
      ))}
    </Svg>
  );
};

```

```

const s = "abacaba";
const pi = [0, 0, 1, 0, 1, 2, 3];
const step = useLoop(s.length + 1, 700);
const i = Math.max(0, step - 1);
const len = step > 0 ? pi[i] : 0;
return (
  <Svg caption={step > 0 ? `π[${i}] = ${len}: префикс`
    {s.split("").map((ch, idx) => (
      <g key={idx}>
        <rect x={18 + idx * 33} y={26} width={28} height={20}
          fill={step > 0 && (idx < len || (idx > i - 1)) ? C.fill : C.idle}
          style={{ transition: "fill .25s" }} />
        <text x={32 + idx * 33} y={40} textAnchor="middle">{ch}</text>
        <text x={32 + idx * 33} y={70} textAnchor="middle">{idx}</text>
        <text x={32 + idx * 33} y={70} textAnchor="middle">{len}</text>
        <text x={32 + idx * 33} y={70} textAnchor="middle">{step}</text>
      </g>
    ))}
    <text x={4} y={40} fontSize={8} fill={C.dim}>s</text>
    <text x={4} y={70} fontSize={8} fill={C.dim}>π</text>
    <text x={130} y={106} textAnchor="middle" fontSize={12}>
      π[i] — длина самого длинного «префикс = суффикс»
    </text>
  </Svg>
);
};

```

```

export const DpDemo: React.FC = () => {
  const rows = 3, cols = 5;
  const step = useLoop(rows * cols + 1, 320);
  return (
    <Svg caption={`Заполняем таблицу подзадач: готово ${step}`}
      {Array.from({ length: rows }).map((_, r) =>
        Array.from({ length: cols }).map((_, c) => {
          const idx = r * cols + c;
          const filled = idx < step;
          return (
            <g key={idx}>

```

```
| "segment-tree" | "bridge" | "articulation" | "mst"  
| "prefix-function" | "dp" | "floyd" | "components" |  
| "zig" | "zig-zig" | "zig-zag";
```

```
const REGISTRY: Record<DemoKind, React.FC> = {  
  bfs: BfsDemo,  
  dfs: DfsDemo,  
  heap: HeapDemo,  
  stack: StackDemo,  
  queue: QueueDemo,  
  dsu: DsuDemo,  
  trie: TrieDemo,  
  "trie-bfs": TrieBfsDemo,  
  "suffix-link": SuffixLinkDemo,  
  toposort: ToposortDemo,  
  relax: RelaxDemo,  
  "prefix-sum": PrefixSumDemo,  
  "sparse-table": SparseTableDemo,  
  "segment-tree": SegmentTreeDemo,  
  bridge: BridgeDemo,  
  articulation: ArticulationDemo,  
  mst: MstDemo,  
  amortized: AmortizedDemo,  
  np: NpDemo,  
  scc: SccDemo,  
  "prefix-function": PrefixFunctionDemo,  
  dp: DpDemo,  
  floyd: FloydDemo,  
  components: ComponentsDemo,  
  graph: GraphBasicsDemo,  
  "coord-compress": CoordCompressDemo,  
  zig: ZigDemo,  
  "zig-zig": ZigZigDemo,  
  "zig-zag": ZigZagDemo,  
};
```

```
export const MiniDemo: React.FC<{ kind: DemoKind }> = (  
  const [paused, setPaused] = useState(false);
```

```

    window.addEventListener("keydown", onKey);
    document.body.style.overflow = "hidden";
    return () => {
      window.removeEventListener("keydown", onKey);
      document.body.style.overflow = "";
    };
  }, [vizId, onClose]);

const viz = getViz(vizId ?? undefined);
if (!vizId || !viz) return null;
const Component = viz.Component;

return createPortal(
  <div className="fixed inset-0 z-[110] flex items-end">
    <div className="absolute inset-0 bg-slate-950/80">
      <div className="relative w-full sm:max-w-5xl max-h-1000 shadow-2xl flex flex-col">
        <div className="flex items-center gap-3 px-4 py-4">
          <div className="min-w-0 flex-1">
            <h3 className="font-bold text-white text-sm">
              {viz.hint} && <p className="text-[11px] text-white">
                {viz.description}
              </p>
            </div>
            <button
              onClick={onClose}
              className="p-2 rounded-lg bg-slate-800 hover:bg-slate-700"
              aria-label="Заккрыть симулятор"
            >
              <X className="w-5 h-5" />
            </button>
          </div>
          <div className="overflow-y-auto overscroll-contain">
            <Component />
          </div>
        </div>
      </div>
    </div>,
    document.body
  );
};

```



```

    }
    const vw = window.innerWidth;
    const vh = window.innerHeight;
    const width = Math.min(CARD_W, vw - 16);
    const height = cardRef.current?.offsetHeight ?? 300

    let left = anchor.rect.left + anchor.rect.width / 2
    left = Math.max(8, Math.min(left, vw - width - 8));

    const below = anchor.rect.top < height + GAP + 8;
    const top = below ? anchor.rect.bottom + GAP : anchor

    setPos({ top: Math.max(8, Math.min(top, vh - height
    }, [anchor]));

useEffect(() => {
  if (!anchor) return;
  const onKey = (e: KeyboardEvent) => {
    if (e.key === "Escape") onClose();
  };
  window.addEventListener("keydown", onKey);
  window.addEventListener("scroll", onClose, true);
  return () => {
    window.removeEventListener("keydown", onKey);
    window.removeEventListener("scroll", onClose, true);
  };
}, [anchor, onClose]);

if (!anchor) return null;
const entry = GLOSSARY_BY_ID.get(anchor.termId);
if (!entry) return null;
const viz = getViz(entry.simulator);
const simulatorId = entry.simulator;

return createPortal(
  <div
    ref={cardRef}
    onMouseEnter={onCardEnter}

```

```
        {viz && simulatorId && (
          <button
            onClick={() => onOpenSimulator(simulatorId)}
            className="w-full flex items-center justify-center text-sm font-medium rounded-lg py-2 transition-colors"
          >
            <Play className="w-3.5 h-3.5" /> Показать
          </button>
        )}
      </div>
    </div>,
    document.body
  );
};
```

ФАЙЛ 50/86: src/components/JohnsonViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```
import { useState } from 'react';
import { RotateCcw, SkipForward, Undo } from 'lucide-react';

export function JohnsonViz() {
  const [step, setStep] = useState(0);
  const MAX_STEP = 7;

  const nodes = [
    { id: 'S', x: 200, y: 150, label: 'S' },
    { id: 'A', x: 200, y: 50, label: 'A' },
    { id: 'B', x: 320, y: 220, label: 'B' },
    { id: 'C', x: 80, y: 220, label: 'C' },
  ];

  const edges = [
```

```

    }
    if (e.id === 'CA') {
      if (step === 6) return "stroke-amber-400 stroke-width-2px";
      if (step > 6) return "stroke-emerald-500";
    }

    if (step >= 4) return "stroke-slate-600"; // otherwise
    return "stroke-slate-500";
  };

const getMarkerUrl = (e: any) => {
  if (e.u === 'S') {
    if (step === 0 || step === 7) return "";
    if (step === 1) return "url(#arrow-amber)";
    if (step === 2) return "url(#arrow-pink)";
    return "url(#arrow-slate-dim)";
  }
  if (e.id === 'AB' && step === 4) return "url(#arrow-amber)";
  if (e.id === 'AB' && step > 4) return "url(#arrow-slate-dim)";
  if (e.id === 'BC' && step === 5) return "url(#arrow-pink)";
  if (e.id === 'BC' && step > 5) return "url(#arrow-slate-dim)";
  if (e.id === 'CA' && step === 6) return "url(#arrow-amber)";
  if (e.id === 'CA' && step > 6) return "url(#arrow-emerald-500)";

  if (step >= 4) return "url(#arrow-slate-dim)";
  return "url(#arrow-slate)";
};

const getEdgeContent = (e: any) => {
  if (e.u === 'S') return e.weight;

  if (step < 4) return e.weight;

  if (e.id === 'AB') {
    if (step === 4) return `${e.weight} + (${0})`;
    return e.newWeight;
  }
  if (e.id === 'BC') {

```

```

ми</span>.
    Это наша база для расчета потенциалов
</div>
);
case 2: return (
    <div>
        <b>Шаг 2: Ищем потенциалы  $h(v)$ .</b>
        Запускаем медленный, но мощный алгоритм
        для поиска минимальных вершин:
        <span className="text-pink-300 ml-1">
            ...
        </span>
    </div>
);
case 3: return (
    <div>
        <b>Шаг 3: Магическая формула.</b><br>
        Всем ребрам графа назначаем новый вес:
        <div className="text-center font-monospace">
            ...
        </div>
        Это повышает или понижает вес ребра
    </div>
);
case 4: return (
    <div>
        <b>Шаг 3.1: Пересчет ребра  $A \rightarrow B$ .</b>
        Старый вес: <span className="text-pink-300">
            ...
        </span>
        Вычисляем:  $-2 + 0 - (-2) =$  <span className="text-pink-300">
            ...
        </span>
    </div>
);
case 5: return (
    <div>
        <b>Шаг 3.2: Пересчет ребра  $B \rightarrow C$ .</b>
        Старый вес: 1. <br>
        Вычисляем:  $1 + (-2) - (-1) =$  <span className="text-pink-300">
            ...
        </span>
    </div>
);
case 6: return (
    <div>
        <b>Шаг 3.3: Пересчет ребра  $C \rightarrow A$ .</b>
        Старый вес: 2. <br>
        Вычисляем:  $2 + (-2) - (-1) =$  <span className="text-pink-300">
            ...
        </span>
    </div>
);

```

```

        </marker>
        <marker id="arrow-pink" viewBox="0 0 10 5" style="fill:#f06292">
tart-reverse">
            <path d="M 0 0 L 10 5 L 0 5" style="stroke:#f06292,stroke-width:2px;stroke-linecap:round;stroke-linejoin:round;fill:none;fill-rule:evenodd;"/>
        </marker>
        <marker id="arrow-emerald" viewBox="0 0 10 5" style="fill:#2e7d32">
o-start-reverse">
            <path d="M 0 0 L 10 5 L 0 5" style="stroke:#2e7d32,stroke-width:2px;stroke-linecap:round;stroke-linejoin:round;fill:none;fill-rule:evenodd;"/>
        </marker>
    </defs>

```

```

    { /* Edges */ }
    { edges.map((e, i) => {
        const u = nodes.find(n => n.id === e.u);
        const v = nodes.find(n => n.id === e.v);

        const isS = e.u === 'S';
        if (isS && (step === 0 || step === 1)) return null;

        const midX = (u.x + v.x) / 2;
        const midY = (u.y + v.y) / 2;

        const edgeColorClass = getEdgeColorClass(e);
        const textHigh = isEdgeTextHigh(e);
        const textEmer = isEdgeTextEmer(e);

        return (
            <g key={i}>
                <line
                    x1={u.x} y1={u.y}
                    x2={v.x} y2={v.y}
                    className={`stroke-${edgeColorClass}`}
                    markerEnd={getMarkerEnd(e)}
                />

                <text
                    x={midX} y={midY}
                    textAnchor="middle"
                    dominantBaseline="middle"
                    style={textStyle}
                />
            </g>
        );
    }) }

```

```

                                </text>
                                </g>
                                }}
                                </g>
                                )
                                }}}
                                </svg>
                                </div>

                                <div className="w-full md:w-[320px] flex
                                  { /* Log Box */
                                    <div className="bg-[#0f172a] p-5 rounded
l justify-center leading-relaxed text-sm text-white
                                  {getDesc()}
                                  </div>

                                  { /* Controls */
                                    <div className="flex bg-slate-900 border
                                      <button onClick={() => setStep(1)}
                                        className="p-3 bg-slate-800 border
-slate-400 transition-colors" title="Сбросить"
                                        <RotateCcw strokeWidth={3}
                                      </button>
                                      <button onClick={() => setStep(1)}
                                        className="p-3 bg-slate-800 border
-slate-400 transition-colors" title="Вар на
                                        <Undo strokeWidth={3} size={24}
                                      </button>
                                      <button onClick={() => setStep(
                                        className={`flex-1 font-bold text-white
                                          ${step === MAX_STEP
                                            ? 'bg-emerald-600/50'
                                            : 'bg-indigo-600 hover:bg-indigo-500'}
                                          {step === MAX_STEP ? "Готово" : "В процессе"}
                                          {step !== MAX_STEP && <Skip
                                        </button>
                                    </div>
                                  }
                                }
                                </div>

```

```
const initialNodes: GNode[] = [
  { id: 0, x: 150, y: 100, label: "0" },
  { id: 1, x: 300, y: 100, label: "1" },
  { id: 2, x: 225, y: 220, label: "2" }, // 0->1->2->0
  { id: 3, x: 450, y: 160, label: "3" },
  { id: 4, x: 600, y: 160, label: "4" }, // 3 <-> 4 (Se
];
```

```
const initialEdges: GEdge[] = [
  { u: 0, v: 1 },
  { u: 1, v: 2 },
  { u: 2, v: 0 },
  { u: 2, v: 3 }, // Connection edge
  { u: 3, v: 4 },
  { u: 4, v: 3 },
];
```

```
interface AlgoStep {
  phase: "init" | "dfs1" | "transpose" | "dfs2" | "fini
  desc: string;
  visited: Set<number>;
  currentNode: number | null;
  order: number[];
  transposed: boolean;
  sccMap: Record<number, number>; // node -> sccId
  currentSccId: number;
  activeEdges: GEdge[];
}
```

```
export const KosarajuViz: React.FC = () => {
  const [steps, setSteps] = useState<AlgoStep[]>([]);
  const [currentStepIdx, setCurrentStepIdx] = useState(0);
  const [isPlaying, setIsPlaying] = useState(false);
```

```
  useEffect(() => {
    // Generate step-by-step Kosaraju steps
    const generatedSteps: AlgoStep[] = [];
```

```
// DFS 1: post-order list
const dfs1 = (u: number) => {
  visited.add(u);
  recordStep(
    "dfs1",
    `DFS 1: зашли в вершину ${u}, помечаем посещение`,
    u,
    false,
    -1,
  );

  for (const to of adj[u]) {
    if (!visited.has(to)) {
      dfs1(to);
    }
  }

  order.push(u);
  recordStep(
    "dfs1",
    `DFS 1: вершина ${u} полностью обработана. Записываем в стек`,
    u,
    false,
    -1,
  );
};

// Run DFS1 on all unvisited
for (let i = 0; i < 5; i++) {
  if (!visited.has(i)) {
    dfs1(i);
  }
}

const topoOrder = [...order].reverse();
recordStep(
  "transpose",
  `Первый проход завершен! Стек выхода в топологический порядок`,
);
```



```

        "dfs2",
        `DFS 2: берем следующую непосещенную из стека
        u,
        true,
        sccId,
    );
    dfs2(u, sccId);
} else {
    recordStep(
        "dfs2",
        `DFS 2: вершина ${u} из стека уже покрашена в
        u,
        true,
        sccId,
    );
}
}

recordStep(
    "finished",
    `Алгоритм успешно завершен! Найдено ${sccId} компонент
    null,
    true,
    sccId,
);

setSteps(generatedSteps);
setCurrentStepIdx(0);
}, []));

const step = steps[currentStepIdx] || {
    phase: "init",
    desc: "Загрузка...",
    visited: new Set(),
    currentNode: null,
    order: [],
    transposed: false,
    sccMap: {},

```

```

        onClick={togglePlay}
        className="flex items-center gap-2 px-3 py-
d-500 text-white"
    >
        {isPlaying ? {
            <Pause className="w-4 h-4 ml-1" />
        } : {
            <Play className="w-4 h-4 ml-1" />
        }}
        {isPlaying ? "Пауза " : "Старт "}
    </button>

    <button
        onClick={reset}
        className="flex items-center justify-center
ors"
    >
        <RotateCcw className="w-4 h-4" />
    </button>
</div>
</div>

<div className="grid grid-cols-1 lg:grid-cols-12
    {/* Playfield SVG */}
    <div className="lg:col-span-8 bg-[#0B1120] rela
        <svg className="w-full h-full max-w-[650px]"
            {/* Markers for directed arrows */}
            <defs>
                <marker
                    id="arrow"
                    viewBox="0 0 10 10"
                    refX="22"
                    refY="5"
                    markerWidth="6"
                    markerHeight="6"
                    orient="auto-start-reverse"
                >
                    <path d="M 0 1 L 10 5 L 0 9 z" fill="#6

```

```

        step.sccMap[edge.v] &&
        step.sccMap[edge.u] === step.sccMap[edge.v]
    let stroke = "#475569";
    let markerId = "arrow";

    if (step.transposed) {
        stroke = isSccEdge ? "#818cf8" : "#4f468c";
        markerId = "arrow-transposed";
    } else {
        if {
            step.currentNode === edge.u ||
            step.currentNode === edge.v
        } {
            stroke = "#10b981";
            markerId = "arrow-active";
        }
    }

    // Adjust slightly for bidirectional link
    const isBidi =
        (edge.u === 3 && edge.v === 4) ||
        (edge.u === 4 && edge.v === 3);
    const dy = isBidi ? (edge.u === 3 ? -10 : 10) : 0;

    return (
        <line
            key={`edge-${idx}`}
            x1={x1}
            y1={y1 + dy}
            x2={x2}
            y2={y2 + dy}
            stroke={stroke}
            strokeWidth="2.5"
            markerEnd={`url(#${markerId})`}
            className="transition-all duration-300"
        />
    );
})}

```

```

        </text>

        {sccId && {
            <text
                x={node.x}
                y={node.y - 28}
                textAnchor="middle"
                fill="#10b981"
                fontSize="10px"
                fontWeight="bold"
            >
                SCC #{sccId}
            </text>
        }}
    </g>
);
}}
</svg>

{step.transposed && {
    <div className="absolute top-4 left-4 bg-in
    1 rounded">
        Граф транспонирован (Ребра обратные)
    </div>
}}
</div>

{/* Logs & Stacks */}
<div className="lg:col-span-4 bg-slate-950 p-5
00 overflow-y-auto">
    <h4 className="text-xs font-bold text-slate-4
        Порядок обхода и стек
    </h4>

    <div className="bg-slate-900 border border-sl
        <p className="text-xs text-white leading-re
            {step.desc}
        </p>

```

```

        className={`text-[11px] p-1.5 rounded`
      }
    >
      {s.desc}
    </div>
  )))
</div>
</div>
</div>

<div className="mt-4 pt-3 border-t border-slate-200">
  <span>
    Шаг {currentStepIdx + 1} из {steps.length}
  </span>
  <div className="flex gap-1.5">
    <button
      disabled={currentStepIdx === 0}
      onClick={() => setCurrentStepIdx((prev) => prev - 1)}
      className="bg-slate-900 border border-slate-200 px-2 py-1 rounded"
    >
      Назад
    </button>
    <button
      disabled={currentStepIdx >= steps.length - 1}
      onClick={() => setCurrentStepIdx((prev) => prev + 1)}
      className="bg-slate-900 border border-slate-200 px-2 py-1 rounded"
    >
      Вперед
    </button>
  </div>
</div>
</div>
</div>
</div>
);
};

```

```

-----
    { id: 5, label: 'F', x: 82, y: 50, color: 'none' },
    { id: 6, label: 'G', x: 20, y: 80, color: 'none' },
    { id: 7, label: 'H', x: 50, y: 87, color: 'none' },
    { id: 8, label: 'I', x: 80, y: 80, color: 'none' },
  ];

// 12 Edges connecting the 9 vertices
const initialEdges: Edge[] = [
  { id: '0-1', u: 0, v: 1, weight: 2, status: 'pending' },
  { id: '3-4', u: 3, v: 4, weight: 3, status: 'pending' },
  { id: '1-2', u: 1, v: 2, weight: 4, status: 'pending' },
  { id: '0-3', u: 0, v: 3, weight: 5, status: 'pending' },
  { id: '1-4', u: 1, v: 4, weight: 6, status: 'pending' },
  { id: '4-5', u: 4, v: 5, weight: 7, status: 'pending' },
  { id: '3-6', u: 3, v: 6, weight: 8, status: 'pending' },
  { id: '2-5', u: 2, v: 5, weight: 9, status: 'pending' },
  { id: '6-7', u: 6, v: 7, weight: 10, status: 'pending' },
  { id: '4-7', u: 4, v: 7, weight: 11, status: 'pending' },
  { id: '7-8', u: 7, v: 8, weight: 12, status: 'pending' },
  { id: '5-8', u: 5, v: 8, weight: 13, status: 'pending' },
];

export const KruskalSimulator: React.FC = () => {
  const [activeAlgo, setActiveAlgo] = useState<'kruskal'>('kruskal');
  const [vertices, setVertices] = useState<Vertex[]>([
    // Initial vertices
  ]);
  const [edges, setEdges] = useState<Edge[]>(initialEdges);
  const [stepIndex, setStepIndex] = useState<number>(0);
  const [heapQueue, setHeapQueue] = useState<string[]>([]);
  const [activeCheatTab, setActiveCheatTab] = useState<'none'>('none');

  const [logs, setLogs] = useState<StepLog[]>([
    {
      title: 'Введение в раскраску (Краскал)',
      description: '«Краскал – это раскраска?». Представим, что мы начинаем!',
      type: 'info',
    },
  ]);
};

```

```

        type: 'success'
    }, ...prev]);
},
// Step 5: Inspect B-C (4)
() => {
    setEdges(prev => prev.map(e => e.id === '1-2' ? {
    setLogs(prev => [{
        title: 'Шаг 3: Берем ребро B-C (вес 4)',
        description: 'Оно соединяет красную вершину B и
        type: 'info'
    }, ...prev]));
},
// Step 6: Include B-C
() => {
    setVertices(prev => prev.map(v => v.id === 2 ? {
    setEdges(prev => prev.map(e => e.id === '1-2' ? {
    setLogs(prev => [{
        title: 'Успех: Ребро B-C в осто́ве',
        description: 'Вершина C перекрашена в красный ц
        type: 'success'
    }, ...prev]));
},
// Step 7: Inspect A-D (5)
() => {
    setEdges(prev => prev.map(e => e.id === '0-3' ? {
    setLogs(prev => [{
        title: 'Шаг 4: Берем ребро A-D (вес 5)',
        description: 'ВНИМАНИЕ! Ребро соединяет КРАСНУЮ
        ю – мы перекрашиваем всё в один цвет!"',
        type: 'warning'
    }, ...prev]));
},
// Step 8: Include A-D (Merge to purple)
() => {
    setVertices(prev => prev.map(v => v.color === 'bl
    setEdges(prev => prev.map(e => e.status === 'incl
    setLogs(prev => [{
        title: 'Слияние кланов: Ребро A-D добавлено',

```

```

        type: 'success'
    }, ...prev]);
},
// Step 13: Inspect D-G (8)
() => {
    setEdges(prev => prev.map(e => e.id === '3-6' ? {
    setLogs(prev => [{
        title: 'Шаг 7: Берем ребро D-G (вес 8)',
        description: 'Оно соединяет фиолетовую вершину D и G',
        type: 'info'
    }, ...prev]));
},
// Step 14: Include D-G
() => {
    setVertices(prev => prev.map(v => v.id === 6 ? {
    setEdges(prev => prev.map(e => e.id === '3-6' ? {
    setLogs(prev => [{
        title: 'Успех: Ребро D-G в осто́ве',
        description: 'Вершина G окрашена в фиолетовый.',
        type: 'success'
    }, ...prev]));
},
// Step 15: Inspect C-F (9) - Cycle!
() => {
    setEdges(prev => prev.map(e => e.id === '2-5' ? {
    setLogs(prev => [{
        title: 'Шаг 8: Берем ребро C-F (вес 9)',
        description: 'Оно соединяет вершины C и F, которые уже соединены.',
        type: 'warning'
    }, ...prev]));
},
// Step 16: Reject C-F
() => {
    setEdges(prev => prev.map(e => e.id === '2-5' ? {
    setLogs(prev => [{
        title: 'Отказ: Цикл обнаружен!',
        description: 'Выбрасываем ребро C-F.',
        type: 'error'
    }, ...prev]));
},

```



```

    },
    // Step 21: Inspect H-I (12)
    () => {
        setEdges(prev => prev.map(e => e.id === '7-8' ? {
            setLogs(prev => [{
                title: 'Шаг 11: Берем ребро H-I (вес 12)',
                description: 'Соединяет фиолетовую H и последнюю I',
                type: 'info'
            }, ...prev]));
    },
    // Step 22: Include H-I
    () => {
        setVertices(prev => prev.map(v => v.id === 8 ? {
            setEdges(prev => prev.map(e => e.id === '7-8' ? {
            setLogs(prev => [{
                title: 'Успех: MST построено Краскалом!',
                description: 'Все 9 вершин окрашены в единый фиолетовый цвет. Суммарный вес: 3 + 4 + 5 + 7 + 8 + 10 + 12 = 51.',
                type: 'success'
            }, ...prev]));
    },
    ];

// --- PRIM STEPS ---
const primSteps = [
    // Step 1: Start at E
    () => {
        setVertices(prev => prev.map(v => v.id === 4 ? {
            setHeapQueue(['D-E (3)', 'B-E (6)', 'E-F (7)', 'E-G (5)'],
            setEdges(prev => prev.map(e => ['3-4', '1-4', '4-5', '4-6', '4-7', '4-8', '4-9']),
            setLogs(prev => [{
                title: 'Шаг 1: Плесень зарождается в Токио (Вершина E)',
                description: 'Мы выбрали вершину E как стартовую вершину. Добавили ее в приоритетную очередь (Heap): D-E(3), B-E(6), E-F(7), E-G(5).',
                type: 'info'
            }, ...prev]));
    },
    // Step 2: Pop D-E (3)

```

```

        type: 'success'
    }, ...prev]);
},
// Step 6: Pop A-B (2)
() => {
    setEdges(prev => prev.map(e => e.id === '0-1' ? {
    setLogs(prev => [{
        title: 'Шаг 4: Куча выдает ребро A-B (вес 2)',
        description: 'Плесень стремительно бежит к вершине A.',
        type: 'info'
    }, ...prev]);
},
// Step 7: Include A-B, add B's edges
() => {
    setVertices(prev => prev.map(v => v.id === 1 ? {
    setEdges(prev => prev.map(e => e.id === '0-1' ? {
        eap' } : e));
    setHeapQueue(['B-C (4)', 'B-E (6 - цикл)', 'E-F (4)']);
    setLogs(prev => [{
        title: 'Успех: Вершина B захвачена плесенью',
        description: 'В кучу добавлено B-C(4). Ребро B-A (вес 2) включено.',
        type: 'success'
    }, ...prev]);
},
// Step 8: Pop B-C (4)
() => {
    setEdges(prev => prev.map(e => e.id === '1-2' ? {
    setLogs(prev => [{
        title: 'Шаг 5: Куча выдает ребро B-C (вес 4)',
        description: 'Плесень тянется к вершине C.',
        type: 'info'
    }, ...prev]);
},
// Step 9: Include B-C, add C's edges
() => {
    setVertices(prev => prev.map(v => v.id === 2 ? {
    setEdges(prev => prev.map(e => e.id === '1-2' ? {
        eap' } : e));

```

```

    setEdges(prev => prev.map(e => e.id === '4-5' ? {
      eap' } : e));
    setHeapQueue(['D-G (8)', 'C-F (9 - цикл)', 'E-H (
    setLogs(prev => [{
      title: 'Успех: Вершина F захвачена',
      description: 'Ребро F-I(13) добавлено в кучу. Р
      type: 'success'
    }, ...prev]));
  },
// Step 14: Pop D-G (8)
() => {
  setEdges(prev => prev.map(e => e.id === '3-6' ? {
  setLogs(prev => [{
    title: 'Шаг 8: Куча выдает ребро D-G (вес 8)',
    description: 'Плесень расширяется вниз к G.',
    type: 'info'
  }, ...prev]));
},
// Step 15: Include D-G, add G's edges
() => {
  setVertices(prev => prev.map(v => v.id === 6 ? {
  setEdges(prev => prev.map(e => e.id === '3-6' ? {
    eap' } : e));
  setHeapQueue(['C-F (9 - цикл)', 'G-H (10)', 'E-H
  setLogs(prev => [{
    title: 'Успех: Вершина G захвачена',
    description: 'В кучу добавлено G-H(10).',
    type: 'success'
  }, ...prev]));
},
// Step 16: Pop C-F (9) - Cycle!
() => {
  setEdges(prev => prev.map(e => e.id === '2-5' ? {
  setLogs(prev => [{
    title: 'Шаг 9: Куча выдает ребро C-F (вес 9)',
    description: 'Вершины C и F уже заражены плесенью
    type: 'warning'
  }, ...prev]));

```

```

        type: 'warning'
      }, ...prev]));
    },
    // Step 21: Reject E-H
    () => {
      setEdges(prev => prev.map(e => e.id === '4-7' ? {
        setHeapQueue(['H-I (12)', 'F-I (13)']));
      setLogs(prev => [{
        title: 'Отказ: Игнорируем E-H',
        description: 'Выбрасываем из кучи.',
        type: 'error'
      }, ...prev]));
    },
    // Step 22: Pop H-I (12)
    () => {
      setEdges(prev => prev.map(e => e.id === '7-8' ? {
        setLogs(prev => [{
          title: 'Шаг 12: Куча выдает H-I (вес 12)',
          description: 'Плесень тянется к последней чистой',
          type: 'info'
        }, ...prev]));
    },
    // Step 23: Include H-I
    () => {
      setVertices(prev => prev.map(v => v.id === 8 ? {
        setEdges(prev => prev.map(e => e.id === '7-8' ? {
        setHeapQueue(['F-I (13 - цикл)']));
      setLogs(prev => [{
        title: 'Успех: Плесень захватила весь граф',
        description: 'Все 9 вершин поглощены. Итоговый граф',
        description: 'иной – мы разрастались из одной точки с помощью',
        type: 'success'
      }, ...prev]));
    },
  ];

  // --- BORUVKA STEPS ---
  const boruvkaSteps = [

```

```

        return e;
    }));
    setLogs(prev => [{
        title: 'Слияние: Деревни объединились в колхозы',
        description: 'Все выбранные дороги построены ра
            оз {D, E, F, G, H, I}.',
        type: 'success'
    }, ...prev]);
},
// Step 3: Five-Year Plan 2 (Inspecting bridge betw
()) => {
    // Now both kolkhozes look at all outgoing roads
    // Outgoing roads between {A, B, C} and {D, E, F,
    // A-D (5), B-E (6), C-F (9).
    // Cheapest is A-D (5).
    setEdges(prev => prev.map(e => e.id === '0-3' ? {
    setLogs(prev => [{
        title: 'Вторая пятилетка: Колхозы выбирают мост
        description: 'Теперь Красный и Синий колхозы од
            -D с весом 5. Остальные мосты B-E(6) и C-F(9)
        type: 'warning'
    }, ...prev]);
},
// Step 4: Concurrently merge into single Kolkhoz
()) => {
    setVertices(prev => prev.map(v => ({ ...v, color:
    setEdges(prev => prev.map(e =>
        e.status === 'included' || e.id === '0-3' ? { .
    ));
    setLogs(prev => [{
        title: 'Объединение завершено: Один гигантский
        description: 'Оба колхоза объединились по мосту
        type: 'success'
    }, ...prev]);
},
// Step 5: Check remaining edges & mark as rejected
()) => {
    setEdges(prev => prev.map(e => e.status === 'pend

```

```

        title: 'Введение в Плесень (Прима)',
        description: 'Логика «Плесени»: выбираем стар
            едь (Heap) и захватываем соседей!',
        type: 'info',
    },
  ]});
} else {
  setLogs([
    {
      title: 'Введение в Коллективизацию (Борувка)',
      description: 'Логика «Борувки»: Каждая из 9 д
        чтобы объединиться в колхоз.',
      type: 'info',
    },
  ]});
}
};

const getColorClass = (color: string, id: number) =>
  switch (color) {
    case 'red': return 'bg-red-500 border-red-300 tex
    case 'blue': return 'bg-blue-500 border-blue-300
    case 'purple': return 'bg-purple-600 border-purple
    case 'mold': return 'bg-emerald-500 border-emerald
    default:
      if (activeAlgo === 'prim' && id === 4 && stepIn
        return 'bg-slate-700 border-emerald-500 text-
      }
      return 'bg-slate-700 border-slate-500 text-slate
  }
};

const getEdgeStroke = (status: string, color: string)
  if (status === 'inspecting') return '#f59e0b';
  if (status === 'rejected') return '#ef4444';
  if (status === 'in_heap') return '#0284c7';
  if (status === 'included') {
    if (color === 'red') return '#ef4444';

```

```

        >
          <GitGraph className="w-3.5 h-3.5" />
          <span>Краскал (Билет 18)</span>
        </button>
        <button
          onClick={() => handleReset('prim')}
          className={
            "flex items-center gap-1.5 px-3 py-2
+
            (activeAlgo === 'prim'
              ? 'bg-emerald-600 text-white shadow
              : 'text-slate-400 hover:text-slate-:
            )
          >
          <Layers className="w-3.5 h-3.5" />
          <span>Прима (Билет 19)</span>
        </button>
        <button
          onClick={() => handleReset('boruvka')}
          className={
            "flex items-center gap-1.5 px-3 py-2
+
            (activeAlgo === 'boruvka'
              ? 'bg-rose-600 text-white shadow-md
              : 'text-slate-400 hover:text-slate-:
            )
          >
          <Users className="w-3.5 h-3.5" />
          <span>Борувка (Билет 20)</span>
        </button>
      </div>

    <button
      onClick={() => handleReset(activeAlgo)}
      className="flex items-center justify-cent
-200 font-semibold text-xs rounded-xl borde
    >
    <RotateCcw className="w-3.5 h-3.5" />

```

```

        <div className="w-3 h-3 rounded-full" />
      </div>
      <div className="flex items-center gap-2" />
        <div className="w-3 h-3 rounded-full" />
      </div>
      <div className="flex items-center gap-2" />
        <div className="w-3 h-3 rounded-full" />
      </div>
    </>
  ) : activeAlgo === 'prim' ? (
    <>
      <div className="flex items-center gap-2" />
        <div className="w-3 h-1 bg-sky-600" />
      </div>
      <div className="flex items-center gap-2" />
        <div className="w-3 h-3 rounded-full" />
      </div>
    </>
  ) : (
    <>
      <div className="flex items-center gap-2" />
        <div className="w-3 h-3 rounded-full" />
      </div>
      <div className="flex items-center gap-2" />
        <div className="w-3 h-3 rounded-full" />
      </div>
      <div className="flex items-center gap-2" />
        <div className="w-3 h-3 rounded-full" />
      </div>
    </>
  )
}
</div>

<div className="absolute top-4 right-4 bg-slate-300 ext-slate-300">
  {Math.floor(stepIndex / currentSteps.length)}
</div>

```



```

        </g>
    });
    }}}
</svg>

{/* HTML Overlay for Vertices */}
<div className="absolute inset-0 w-full h-full" >
    {vertices.map(vertex => {
        <div
            key={vertex.id}
            style={{ top: vertex.y + "%", left: vertex.x + "%" }}
            className={"absolute -translate-x-1/2 -translate-y-1/2"}
            font-bold text-base border-2 transition-all duration-300
            id={vertex.id}
        >
            <span>{vertex.label}</span>
            {vertex.id === 4 && activeAlgo === 'Prim' ? (
                <span className="text-[8px] block -mt-10px">
                    MST построено с помощью {activeAlgo}
                </span>
            ) : null}
        </div>
    })}
</div>

{stepIndex >= currentSteps.length && (
    <div className="absolute inset-x-6 bottom-6 shadow-xl z-20">
        <p className={"font-bold text-base " + "text-rose-300"}>
            MST построено с помощью {activeAlgo}
        </p>
        <p className="text-slate-300 text-xs mt-2">
            Общий вес минимального остова: 51. Всего ребер: 5
        </p>
    </div>
)}
</div>

{/* Logs / Heap */}

```



```

<div className="lg:col-span-2">
  <div className="bg-slate-950 p-4 rounded-1">
    <p className="text-slate-400 text-xs font-mono">
      <Code className="w-3.5 h-3.5 text-indigo-400">
    </p>
    <pre className="text-xs text-emerald-400 font-mono">
      80 flex-1">
{`# 1. Инициализируем DSU
def find(i):
    if parent[i] == i: return i
    parent[i] = find(parent[i])
    return parent[i]

def union(i, j):
    r_i, r_j = find(i), find(j)
    if r_i != r_j:
        parent[r_i] = r_j
        return True
    return False

# 2. Жадный выбор ребер
edges.sort() # O(E log E)
mst = []
for w, u, v in edges:
    if union(u, v):
        mst.append((u, v, w))`}
    </pre>
  </div>
</div>
</div>
)}

{activeCheatsTab === 'prim' && {
  <div className="grid grid-cols-1 lg:grid-cols-2">
    <div className="lg:col-span-1 space-y-4">
      <div className="bg-slate-950 p-4 rounded-1">
        <p className="text-slate-400 text-xs font-mono">
          <Clock className="w-3.5 h-3.5 text-emerald-400">

```

```

        for next_w, v in graph[u]:
            if not visited[v]:
                heapq.heappush(heap, (next_w, v, u))
    return mst` }
    </pre>
</div>
</div>
}}

{activeCheatTab === 'boruvka' && {
  <div className="grid grid-cols-1 lg:grid-cols-2" >
    <div className="lg:col-span-1 space-y-4">
      <div className="bg-slate-950 p-4 rounded-3xl">
        <p className="text-slate-400 text-xs font-medium">
          <Clock className="w-3.5 h-3.5 text-rose-500" />
        </p>
        <p className="text-2xl font-black text-white">Борувка</p>
        <p className="text-slate-400 text-xs font-medium">
          Алгоритм Борувки — это алгоритм нахождения минимального остовного
          дерева графа. Он работает за O(E log V) времени.
        </p>
      </div>
      <div className="bg-slate-950 p-4 rounded-3xl">
        <p className="text-slate-400 text-xs font-medium">
          <Award className="w-3.5 h-3.5 text-rose-500" />
        </p>
        <p className="text-2xl font-black text-white">Алгоритм Крускала</p>
        <p className="text-slate-400 text-xs font-medium">
          Алгоритм Крускала — это алгоритм нахождения минимального остовного
          дерева графа. Он работает за O(E^2 log V) времени.
        </p>
      </div>
      <div className="bg-slate-950 p-4 rounded-3xl">
        <p className="text-slate-400 text-xs font-medium">
          Алгоритм Прима — это алгоритм нахождения минимального остовного
          дерева графа. Он работает за O(V^2 log V) времени.
        </p>
        <p className="text-xs text-slate-300 leading-relaxed">
          , организуя массовое распараллеленное слияние.
        </p>
      </div>
    </div>
    <div className="lg:col-span-2">
      <div className="bg-slate-950 p-4 rounded-3xl">
        <p className="text-slate-400 text-xs font-medium">
          Алгоритм Борувки — это алгоритм нахождения минимального остовного
          дерева графа. Он работает за O(E log V) времени.
        </p>
      </div>
    </div>
  </div>
}

```

```
-----  
import floydImg from '../assets/floyd-network.jpg';  
  
const cards = [  
  {  
    img: dijkstraImg,  
    alt: 'Добрый робот Дейкстра',  
    title: '    Добрый (Дейкстра)',  
    desc: 'Быстрый, жадный, но работает только с',  
    highlight: 'положительными',  
    highlightColor: 'text-emerald-400',  
    rest: 'весами. Подсунь отрицательное ребро – сломает',  
    complexity: ' $O((V+E) \log V)$ ',  
    border: 'border-blue-500/30 hover:border-blue-400/60',  
    titleColor: 'text-blue-400',  
    badge: 'text-blue-400/70 bg-blue-950/40',  
  },  
  {  
    img: bellmanImg,  
    alt: 'Фура по болоту Форд-Беллман',  
    title: '    Фура по Болоту (Ф-Б)',  
    desc: 'Медленный, тяжёлый – зато тащит',  
    highlight: 'отрицательные',  
    highlightColor: 'text-rose-400',  
    rest: 'веса и детектит отрицательные циклы.',  
    complexity: ' $O(V \cdot E)$ ',  
    border: 'border-rose-500/30 hover:border-rose-400/60',  
    titleColor: 'text-rose-400',  
    badge: 'text-rose-400/70 bg-rose-950/40',  
  },  
  {  
    img: floydImg,  
    alt: 'Все-ко-Все Флойд',  
    title: '    Все ко Все (Флойд)',  
    desc: 'Матрица + три цикла (',  
    highlight: 'k, i, j',  
    highlightColor: 'text-emerald-400',  
    rest: ')). Ищет пути от всех ко всем.',  
    complexity: ' $O(V^3)$ ',
```

```

interface Props {
  selectedChapterId: string;
  setSelectedChapterId: (id: string) => void;
  /** Заголовок текущей темы — показываем в кнопке, что */
  currentTitle: string;
  index: number;
  total: number;
}

/**
 * Мобильное содержание: липкая панель снизу-сверху + в
 * На десктопе не рендерится (там обычный сайдбар).
 */
export const MobileToc: React.FC<Props> = ({ selectedCh
  const [open, setOpen] = useState(false);

  useEffect(() => {
    document.body.style.overflow = open ? "hidden" : ""
    return () => {
      document.body.style.overflow = "";
    };
  }, [open]);

  useEffect(() => {
    const onKey = (e: KeyboardEvent) => e.key === "Escap
    window.addEventListener("keydown", onKey);
    return () => window.removeEventListener("keydown",
  }, []);

  return (
    <>
      <div className="lg:hidden sticky top-[57px] z-40
        <button
          onClick={() => setOpen(true)}
          className="w-full flex items-center gap-2.5 p
          aria-label="Открыть содержание"
        >
          <span className="p-1.5 rounded-lg bg-indigo-6

```

```
};
```

ФАЙЛ 55/86: src/components/Navbar.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```
import React, { useState } from 'react';
import { BookOpen, Cpu, Sparkles, FileDown, FileText, Cpu } from 'lucide-react';
import { chapters } from '../data/content';
import { downloadBookHtml } from '../utils/exportHtml';
```

```
interface NavbarProps {
  activeTab: 'guide' | 'simulator';
  setActiveTab: (tab: 'guide' | 'simulator') => void;
  /** Открыта ли боковая панель компилятора. */
  compilerOpen: boolean;
  onToggleCompiler: () => void;
}
```

```
export const Navbar: React.FC<NavbarProps> = ({ activeTab, compilerOpen, onToggleCompiler }) => {
  const [busy, setBusy] = useState(false);
```

```
  const saveBook = async () => {
    setBusy(true);
    try {
      await downloadBookHtml(chapters);
    } finally {
      setBusy(false);
    }
  };
};
```

```
  return (
    <header className="bg-slate-900 border-b border-slate-800">
      <div className="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8">
        <div className="flex items-center gap-3 w-full">
```

```

        <Cpu className="w-4 h-4" /> Тренажёры
    </button>
    <button
        id="tab-compiler-btn"
        onClick={onToggleCompiler}
        aria-pressed={compilerOpen}
        title="Боковая панель Python-компилятора (на
            страницах)"
        className={`flex-1 lg:flex-none flex items-
            uration-200 whitespace-nowrap ${
                compilerOpen
                    ? 'bg-emerald-600 text-white shadow-lg
                    : 'text-slate-400 hover:text-white'
            }`}
    >
        <Terminal className="w-4 h-4" /> Python
    </button>
    <a
        id="download-pdf-btn"
        href={` ${import.meta.env.BASE_URL}export/full
        download="full_code_all_pages.pdf"
        target="_blank"
        rel="noreferrer"
        className="flex items-center justify-center
        over:bg-emerald-600/20 border border-emerald
        title="Скачать полный PDF сборник всех страниц"
    >
        <FileDown className="w-4 h-4" /> Скачать PDF
    </a>
    <a
        id="download-txt-btn"
        href={` ${import.meta.env.BASE_URL}export/full
        download="full_code_all_pages.txt"
        target="_blank"
        rel="noreferrer"
        className="flex items-center justify-center
        :bg-sky-600/20 border border-sky-500/30 tra
        title="Скачать полный TXT файл со всем кодом"
    >

```



```

const pts = [
  {id:0,x:160,y:40},
  {id:1,x:270,y:100},
  {id:2,x:230,y:230},
  {id:3,x:90,y:230},
  {id:4,x:50,y:100}
];
const edges = [
  [0,1],[0,2],[0,3],[0,4],
  [1,2],[1,3],[1,4],
  [2,3],[2,4],
  [3,4]
];
function findPt(id:number) { return pts.f
function intersect(a:number,b:number,c:nu
  if (a===c||a===d||b===c||b===d) return
  const p1=findPt(a),p2=findPt(b),p3=find
  function ccw(ax:number,ay:number,bx:num
    return (cy-ay)*(bx-ax)>(by-ay)*(cx-ax
  }
  return ccw(p1.x,p1.y,p3.x,p3.y,p4.x,p4.y)
    && ccw(p1.x,p1.y,p2.x,p2.y,p3.x,p3.y)
}
const crosses: number[] = [];
for(let i=0;i<edges.length;i++) {
  for(let j=i+1;j<edges.length;j++) {
    if (intersect(edges[i][0],edges[i][1]
      if (!crosses.includes(i)) crosses.p
      if (!crosses.includes(j)) crosses.p
    }
  }
}
const lines = []; const circles = [];
for(let i=0;i<edges.length;i++) {
  const p1=findPt(edges[i][0]),p2=findPt(
  const xing = crosses.includes(i);
  lines.push(<line key={`l${i}`} x1={p1.x
    stroke={xing ? "#f43f5e" : "#4f46e5"}

```

```

        return (cy-ay)*(bx-ax)>(by-ay)*(cx-ax)
    }
    return ccw(p1.x,p1.y,p3.x,p3.y,p4.x,p4.y)
    && ccw(p1.x,p1.y,p2.x,p2.y,p3.x,p3.y)
}
const crosses:number[]=[];
for(let i=0;i<edges.length;i++) for(let j=i+1;j<edges.length;j++)
    if (intersect(edges[i][0],edges[i][1],edges[j][0],edges[j][1]))
        if (!crosses.includes(i)) crosses.push(i)
        if (!crosses.includes(j)) crosses.push(j)
}
const lines = []; const circles = [];
for(let i=0;i<edges.length;i++) {
    const p1=findPt(edges[i][0]),p2=findPt(edges[i][1]);
    const xing=crosses.includes(i);
    lines.push(<line key={`l${i}`} x1={p1.x} x2={p2.x} y1={p1.y} y2={p2.y}
        stroke={xing?"#f43f5e":"#4f46e5"} stroke-width={2}/>);
}
const labels = ["\u04141","\u04142","\u04143"];
for(const p of pts) {
    circles.push(<g key={`c${p.id}`}>
        <circle cx={p.x} cy={p.y} r={8} fill="#4f46e5" stroke="#4f46e5" stroke-width={2}/>
        <text x={p.x} y={p.y+3} textAnchor="middle">{labels[p.id]}</text>
    </g>);
}
return <>{lines}{circles}</>;
})()
</svg>
<div className="absolute bottom-2 left-2 right-2 top-20">
    V=6, E=9 {'>'} 2V-4 = 8 (двудольный) {'-->'}
</div>
</div>
</div>

```

<div className="bg-amber-950/40 border border-amber-950/40">

▲ Запомни: K5 и K3,3 – два кита непланарности.

```

chapterId: string;
chapterTitle: string;
/** Перейти к странице пособия (посмотреть визуализацию) */
onOpenGuide: () => void;
/** Скрыть панель. */
onClose: () => void;
}

type PyodideRuntime = {
  runPythonAsync: (source: string) => Promise<unknown>;
  setStdout: (options: { batched: (message: string) => void }) => void;
  setStderr: (options: { batched: (message: string) => void }) => void;
  setStdin: (options: { stdin: () => string | number | null }) => void;
};

type PyodideModule = {
  loadPyodide: (options: { indexURL: string }) => Promise<PyodideRuntime>;
};

let runtimePromise: Promise<PyodideRuntime> | null = null;

function getPythonRuntime() {
  if (!runtimePromise) {
    runtimePromise = import(/* @vite-ignore */ PYODIDE_URL)
      .then((module as PyodideModule).loadPyodide({ indexURL:
    }
  );
  return runtimePromise;
}

function errorText(error: unknown) {
  if (error instanceof Error) return error.message;
  return String(error);
}

/* -----
/*  Шаблон синхронизации: короткая шапка + минимальный
/*  скелет, шаг-в-шаг с подсветкой визуализации (см. da

```

```

    },
    {
        label: "while ...",
        tip: "Классический while-счётчик — как n-1 итерация",
        code: "i = 1\nwhile i < 5:\n    print(f\"итерация {i}\")\n",
    },
    {
        label: "for i, x in enumerate(...)",
        tip: "Индекс и элемент одновременно: i — позиция, x — элемент",
        code: "for i, x in enumerate(['a', 'b', 'c']):\n    print(f\"{i}: {x}\")\n",
    },
],
},
{
    group: "Структуры",
    items: [
        {
            label: "Стек (список)",
            tip: "LIFO, как рекурсия DFS: append = push, pop = pull",
            code: "st = []\nst.append('A')    # push\nst.pop()    # pull\n",
        },
        {
            label: "Очередь (deque)",
            tip: "FIFO для BFS: popleft() за O(1). list.pop(0) за O(n)",
            code: "from collections import deque\nq = deque()\nq.append(1)\nq.popleft()\n",
        },
        {
            label: "Куча (heapq)",
            tip: "Мин-куча: (вес, вершина) — как очередь приоритетов",
            code: "import heapq\nh = []\nheapq.heappush(h, (1, 'a'))\nheapq.heappush(h, (2, 'b'))\nheapq.heappop(h)\n",
        },
        {
            label: "Матрица n × m",
            tip: "Таблица расстояний: dist[i][j], бесконечность — float('inf')",
            code: "n, m = 3, 4\ndist = [[float('inf')] * m for _ in range(n)]\ndist[0][0] = 0\n",
        },
    ],
},

```

```

const [outputOpen, setOutputOpen] = useState(true);
const editorRef = useRef<HTMLTextAreaElement>(null);

const [desyncedFrom, setDesyncedFrom] = useState<string | null>(
  savedEditor &&
  savedEditor.chapterId !== chapterId &&
  savedEditor.code !== savedEditor.template &&
  savedEditor.code.trim() !== "" &&
  savedEditor.code !== template
    ? chapterId
    : null
);
const prevTemplateRef = useRef(template);

useEffect(() => {
  savedEditor = { code, template, chapterId };
}, [code, template, chapterId]);

// Смена страницы: нетронутый редактор получает свежий
// свой код не затираем – предлагаем синхронизировать
useEffect(() => {
  if (template === prevTemplateRef.current) return;
  const prevTemplate = prevTemplateRef.current;
  prevTemplateRef.current = template;
  if (code === prevTemplate || code.trim() === "") {
    setCode(template);
    setDesyncedFrom(null);
  } else {
    setDesyncedFrom(chapterId);
  }
  // eslint-disable-next-line react-hooks/exhaustive-deps
}, [template]);

const resync = useCallback(() => {
  prevTemplateRef.current = template;
  setCode(template);
  setDesyncedFrom(null);
  setStripTab("vars");

```

```

        setCopied(true);
        setTimeout(() => setCopied(false), 1600);
    } catch {
        setRuntimeMessage("Не получилось скопировать – бу
    }
}, [code]);

```

```

/** Вставка сниппета из шпаргалки в позицию курсора.
const insertSnippet = useCallback(({snippet: string}) => {
    const ta = editorRef.current;
    setCode((current) => {
        if (!ta) return current + (current.endsWith("\n")
        const start = ta.selectionStart ?? current.length
        const end = ta.selectionEnd ?? start;
        const padded = (start > 0 && !current.slice(0, st
        const next = current.slice(0, start) + padded + c
        requestAnimationFrame(() => {
            ta.focus();
            ta.selectionStart = ta.selectionEnd = start + p
        });
        return next;
    });
}, []);

```

```

const hasViz = sync.variables.length > 0;

```

```

return (
    <aside
        aria-label="Python-компилятор"
        className="fixed z-40 inset-x-0 bottom-0 h-[58dvh
            px] flex flex-col bg-slate-900 border-t lg:bord
    >
        {/** — Заголовок панели —————
        <div className="shrink-0 flex items-center gap-2 p
            <span className={`w-2 h-2 rounded-full shrink-0
            <Terminal className="w-4 h-4 text-emerald-400 s
            <span className="text-[13px] font-bold text-whi
            <span className="hidden sm:inline text-[10px] t

```

```

        -60 text-white text-xs font-bold transition
      >
        {running ? <Loader2 className="w-3.5 h-3.5" />
        {running ? "Запуск..." : "Запустить"}}
      </button>
    </Tooltip>
  </div>
</div>

{/* — Мини-вкладки: переменные страницы / шпаргалка */}
<div className="shrink-0 border-b border-slate-800" />
  <div className="flex items-center gap-1 px-2 pt-2" />
    <button
      type="button"
      onClick={() => setStripTab("vars")}
      className={`inline-flex items-center gap-1.5
        stripTab === "vars" ? "bg-slate-800 text-white" : ""
      }`>
      >
        <Variable className="w-3.5 h-3.5" /> Переменные
    </button>
    <button
      type="button"
      onClick={() => setStripTab("hints")}
      className={`inline-flex items-center gap-1.5
        stripTab === "hints" ? "bg-slate-800 text-white" : ""
      }`>
      >
        <HelpCircle className="w-3.5 h-3.5" /> Подсказки
    </button>
    <div className="ml-auto pr-1">
      <Tooltip
        side="bottom"
        content={
          <div>
            <div>hasViz</div>
            <div>
              ? "Редактор и визуализация смотрят на..."
              : "На выбранной странице нет интерактивных элементов"
            </div>
          </div>
        </div>
      </div>
    </div>
  </div>

```

```

0/40 text-amber-400 hover:bg-amber-500/20 t
>
  <Unlink className="w-3.5 h-3.5 shrink
  Страница сменилась – подставить её пе
  </button>
</Tooltip>
)}
{hasViz && {
  <div className="flex flex-wrap items-cent
  {sync.variables.map((v) => {
    <Tooltip
      key={v.name}
      side="bottom"
      content={
        <span>
          <span className="block font-bol
            <code className="font-mono te
          </span>
          <span className="block">{v.role
            {v.range && <span className="bl
          </span>
        }
      }
    >
      <code
        tabIndex={0}
        className="cursor-help rounded-md
font-bold text-emerald-300 hover:bg-emeral
      >
        {v.name}
      </code>
    </Tooltip>
  )}}
  {sync.stepNote && {
    <Tooltip side="bottom" content={`War
    <span
      tabIndex={0}
      className="cursor-help inline-fle
text-[10px] text-slate-400 hover:text-white

```



```

<div className="flex-1 min-h-0 flex flex-col">
  <textarea
    ref={editorRef}
    value={code}
    onChange={(event) => setCode(event.target.value)}
    onKeyDown={(event) => {
      if ((event.ctrlKey || event.metaKey) && event.defaultPrevented) {
        event.preventDefault();
        void runCode();
      }
      event.stopPropagation();
    }}
    spellCheck={false}
    aria-label="Код Python"
    className="flex-1 min-h-[90px] w-full resize-none focus:ring-2 focus:ring-inset focus:outline-none"
    placeholder="Напишите Python-код..."
  />
  <div className="shrink-0 border-t border-slate-500">
    <label htmlFor="python-stdin" className="text-slate-500">
      stdin для input()
    </label>
    <input
      id="python-stdin"
      value={stdin}
      onChange={(event) => setStdin(event.target.value)}
      spellCheck={false}
      placeholder="строки ввода через ↵"
      className="w-full bg-transparent font-mono"
    />
  </div>
</div>

```

```

{/* — Вывод ————— */}
<div className="shrink-0 border-t border-slate-800">
  <button
    type="button"
    onClick={() => setOutputOpen((o) => !o)}
  />

```

```

    color: string;
    bubbleText: string;
    animState: 'in' | 'idle' | 'eating' | 'out';
  }

const PRESETS = [
  { name: 'Студент Вася', emoji: '🍷', color: 'from-blue',
    bubbleText: 'голода после матана!' },
  { name: 'Кодер Семён', emoji: '🍲', color: 'from-violet',
    bubbleText: 'ищу ИИ за порцию борща!' },
  { name: 'Кот Борис', emoji: '🐈', color: 'from-amber',
    bubbleText: 'есть сметаны, плиз.' },
  { name: 'Бабушка Люба', emoji: '👵', color: 'from-rose',
    bubbleText: 'есть сметаны, плиз.' },
  { name: 'Инопланетянин', emoji: '👽', color: 'from-emerald',
    bubbleText: 'есть сметаны, плиз.' },
  { name: 'Бизнесмен', emoji: '💼', color: 'from-slate',
    bubbleText: 'есть сметаны, плиз.' },
];

let counter = 3;

export const QueueViz: React.FC = () => {
  const [queue, setQueue] = useState<Customer[]>([
    { id: 'q1', name: 'Студент Вася', emoji: '🍷', color: 'from-blue',
      bubbleText: 'голода после матана!', animState: 'idle' },
    { id: 'q2', name: 'Кодер Семён', emoji: '🍲', color: 'from-violet',
      bubbleText: 'ищу ИИ за порцию борща!', animState: 'idle' },
    { id: 'q3', name: 'Кот Борис', emoji: '🐈', color: 'from-amber',
      bubbleText: 'есть сметаны, плиз.', animState: 'idle' },
  ]);

  const [servedHistory, setServedHistory] = useState<string[]>([]);
  const [isServing, setIsServing] = useState(false);
  const [shakeEmpty, setShake] = useState(false);
  const [log, setLog] = useState<string[]>([]);

  const addLog = (msg: string) => setLog(prev => [...prev, msg]);

```

```

    setIsServing(true);
    const headGuest = queue[0];

    addLog(`  DEQUEUE: Повар наливает суп первому – ${headGuest.name}!`);

    // Step 1: Head guest gets eating animation
    setQueue(prev => prev.map((c, idx) => idx === 0 ? {
      ...c, animState: 'eating'
    } : c));

    setTimeout(() => {
      // Step 2: Guest leaves with happy face, queue sh
      setQueue(prev => prev.map((c, idx) => idx === 0 ? {
        ...c, animState: 'happy'
      } : c));

      setServedHistory(prev => [`${headGuest.emoji} ${headGuest.name} съел суп`]);

      setTimeout(() => {
        setQueue(prev => prev.slice(1));
        setIsServing(false);
        addLog(`  ${headGuest.name} поел и ушел сытым.`,
          { delay: 400 });
      }, 900);
    }, [queue, isServing]);

    const reset = () => {
      counter = 3;
      setQueue([
        { id: 'r1', name: 'Студент Вася', emoji: '👨🎓', color: 'red', col: 1,
          text: 'У меня нет голода после матана!', animState: 'in' },
        { id: 'r2', name: 'Кодер Семён', emoji: '👨💻', color: 'blue', col: 2,
          text: 'Пожалуйста, мне нужно написать ИИ за порцию борща!', animState: 'in' },
        { id: 'r3', name: 'Кот Борис', emoji: '🐱', color: 'green', col: 3,
          text: 'Мне нужен суп без сметаны, плиз.', animState: 'in' },
      ]);
      setServedHistory([]);
      setIsServing(false);
      setLog(['  Столовая сброшена. Снова голодная толпа!']);
      setTimeout(() => {
        setQueue(prev => prev.map(c => ({ ...c, animState: 'in' })));
      }, 500);
    };
  
```

```

    <button
      onClick={reset}
      className="px-5 py-3.5 rounded-2xl bg-slate-800
        r-pointer border border-slate-700"
    >
      Сброс
    </button>
  </div>

  { /* ИНТЕРАКТИВНАЯ КУХНЯ */ }
  <div className="grid grid-cols-1 md:grid-cols-12" >

    { /* ЛЕВАЯ КОЛОНКА: ОЧЕРЕДЬ ЗА СУПОМ */ }
    <div className={`md:col-span-9 bg-slate-900 rounded-2xl
      -[300px] overflow-hidden ${shakeEmpty ? 'animate-shake' : ''}` >
      <div className="absolute top-4 left-4 text-[1.2em] font-mono" >
        Очередь голодных гостей (Буфер FIFO / BFS)
      </div>

      <div className="flex-1 flex flex-col justify-between" >
        <div className="flex items-end justify-between" >

          { /* ПОВАРИХА И КОТЕЛ С СУПОМ */ }
          <div className="flex flex-col items-center" >
            <div className="relative" >
              <div className="absolute -top-10 left-1/2 -translate-x-1/2" >
                <div className="w-1.5 h-6 bg-slate-800 rounded" >
                  <div className="w-2 h-4 bg-slate-400 rounded" >
                </div>
                <span className="text-5xl block animate-pulse" >
                </div>
              <div className="text-center" >
                <span className="text-3xl block" >
                <span className="text-[10px] font-bold font-mono tracking-wider" >
                  ПОВАР
                </span>
              </div>
            </div>
          </div>
        </div>
      </div>
    </div>
  </div>

```

```

        {customer.bubbleText}
        <div className="absolute -bottom-2 text-3xl font-medium
it rotate-45"></div>
    </div>

    {/* Персонаж */}
    <div className={`
        w-14 h-14 rounded-2xl border-2
relative
        ${customer.color}
        ${isHead ? 'ring-4 ring-amber-500' : ''}
    `}>
        <span className="text-3xl">{customer.name}</span>

        {/* Индексы HEAD/TAIL */}
        {(isHead || isTail) && (
            <span className={`
                absolute -bottom-2 text-3xl font-medium
                ${isHead ? 'bg-amber-500' : ''}
            `}>
                {isHead && isTail ? 'h+t' : ''}
            </span>
        )}
    </div>
</div>
);
    })
  })
</div>

</div>
</div>

{/* Пол кухни */}
<div className="w-full h-3 bg-gradient-to-r from-amber-500 to-amber-200">
</div>

{/* ПРАВАЯ КОЛОНКА: СЫТЫЕ И ДОВОЛЬНЫЕ */}

```

```
    {/* ЛОГ ДЕЙСТВИЙ */}
    <div className="bg-slate-900 border border-slate-
      <div className="text-[10px] font-mono text-slate
        {log.map((entry, idx) => (
          <div
            key={idx}
            className={`text-xs font-mono flex items-ce
          >
            {idx === 0 ? <span className="text-indigo-5
            <span>{entry}</span>
          </div>
        )}}
      </div>
    </div>
  );
};
```

ФАЙЛ 59/86: src/components/SalmonAutomatonWidget.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРО

```
import React, { useState, useEffect } from 'react';
import { Play, RotateCcw, Plus, Volume2, Info } from 'lucide-react';
```

```
interface TrieNode {
  id: number;
  char: string;
  parent: number;
  children: { [key: string]: number };
  fail: number;
  term_link: number;
  is_terminal: boolean;
  words: string[];
  depth: number;
  x?: number;
  y?: number;
```

```

wordList.forEach((word) => {
  let curr = 0;
  for (let i = 0; i < word.length; i++) {
    const c = word[i].toUpperCase();
    if (newNodes[curr].children[c] === undefined) {
      count++;
      newNodes[curr].children[c] = count;
      newNodes[count] = {
        id: count,
        char: c,
        parent: curr,
        children: {},
        fail: 0,
        term_link: 0,
        is_terminal: false,
        words: [],
        depth: newNodes[curr].depth + 1
      };
    }
    curr = newNodes[curr].children[c];
  }
  if (!newNodes[curr].words.includes(word)) {
    newNodes[curr].words.push(word);
    newNodes[curr].is_terminal = true;
  }
});

let currentX = 0;
const paddingX = 70;
const paddingY = 80;

const layoutDFS = (u: number, depth: number) => {
  const childKeys = Object.values(newNodes[u].children);
  newNodes[u].y = 40 + depth * paddingY;

  if (childKeys.length === 0) {
    newNodes[u].x = 40 + currentX * paddingX;
  }

```

```

    const updated = [...words, cleanWord];
    setWords(updated);
    setNewWord('');
    buildInitialTrie(updated);
  };

const removeWord = (wordToRemove: string) => {
  if (words.length <= 1) {
    setSpeechText('Оставь хотя бы одно слово, иначе мы не сможем продолжить');
    return;
  }
  const updated = words.filter((w) => w !== wordToRemove);
  setWords(updated);
  buildInitialTrie(updated);
};

// Запуск BFS
const startBFS = () => {
  const rootChildren = Object.values(nodes[0].children);
  const queue = [...rootChildren];
  const updatedNodes = { ...nodes };
  rootChildren.forEach((childId) => {
    updatedNodes[childId].fail = 0;
  });

  setNodes(updatedNodes);
  setBfsQueue(queue);
  setSimulationStep(1);
  setCurrentNode(null);
  setSpeechText(
    'Начинаем обход в ширину (BFS)! Все вершины на главном уровне  

    уже суффикса просто нет. Жми «Следующий шаг»!'
  );
};

// Один шаг BFS
const stepBFS = () => {
  if (bfsQueue.length === 0) {

```



```

    }

    setNodes(updatedNodes);
    setBfsQueue(newQueue);
    setCurrentNode(r);
    setSpeechText(logs);
  };

  return (
    <div className="bg-slate-800/90 rounded-2xl p-6 border" >
      {/* Шапка виджета */}
      <div className="flex flex-wrap items-center justify-between" >
        <div className="flex items-center space-x-3" >
          <span className="text-3xl" > </span>
          <div>
            <h4 className="text-2xl font-bold text-white" >
              Интерактивный конструктор: Говорящий Эхо-
            </h4>
            <p className="text-sm text-slate-400" >
              Построение дерева и расчет суффиксных ссы
            </p>
          </div>
        </div>
        <div className="flex items-center gap-2" >
          <button
            onClick={() => buildInitialTrie(words)}
            className="flex items-center gap-1 bg-slate-700 text-white
            transition-colors"
            title="Сбросить автомат"
          >
            <RotateCcw className="w-4 h-4" /> Сбросить
          </button>
        </div>
      </div>

      {/* Верхняя панель: Говорящий лосось и диалоговое
      <div className="grid grid-cols-1 md:grid-cols-3 gap-4" >
        {/* Аватар Лосося (Сеня) с анимацией моргания и

```

```

        </>
    }}

    {/* Рот (анимация разговора) */}
    {isTalking ? (
        <path d="М 165 105 Q 155 120 165 125 Q
    ) : (
        <path d="М 165 110 Q 155 115 168 118" s
    )}

    {/* Улыбка / жабры */}
    <path d="М 130 80 Q 125 100 130 120" stro
    <path d="М 120 85 Q 115 100 120 115" stro
</svg>

    {/* Декоративные пузыри */}
    <div className="absolute -top-1 right-4 tex
    <div className="absolute top-8 right-1 text
</div>
<span className="mt-3 bg-rose-600/30 border b
    Эхо-Лосось Сеня
</span>
</div>

{/* Пузырь с текстом (Баббл) */}
<div className="md:col-span-2 bg-slate-800 bord
    tween min-h-[140px]">
    {/* Треугольник баббла */}
    <div className="absolute -left-3 top-1/2 -tran
        slate-800 border-b-[12px] border-b-transpar

    <div className="flex items-center space-x-2 t
        <Volume2 className={`w-4 h-4 ${isTalking ?
        <span>Прямое вещание из аквариума:</span>
    </div>

    <p className="text-slate-200 text-base leadin
        {speechText}

```

```

        </div>
      </div>
    </div>
  </div>

  { /* SVG Canvas for Automaton Tree */
  <div className="mb-8 bg-slate-900 border border-slate-800" >
    <h5 className="text-white font-bold mb-3 flex items-center" >
      <span> </span> Дерево и Суффиксные (fail) ссылки
    </h5>

    {(() => {
      let maxX = 0;
      let maxY = 0;
      Object.values(nodes).forEach(n => {
        if (n.x && n.x > maxX) maxX = n.x;
        if (n.y && n.y > maxY) maxY = n.y;
      });
      const svgWidth = Math.max(maxX + 80, 400);
      const svgHeight = Math.max(maxY + 60, 200);

      return (
        <svg viewBox={`0 0 ${svgWidth} ${svgHeight}`} >
          <defs>
            <marker id="arrowhead" markerWidth="6" markerHeight="6" >
              <polygon points="0 0, 6 3, 0 6" fill="white" />
            </marker>
            <marker id="fail-arrow" markerWidth="6" markerHeight="6" >
              <polygon points="0 0, 6 3, 0 6" fill="white" />
            </marker>
          </defs>

          { /* Draw Edges */
          Object.values(nodes).map(node => {
            if (node.id === 0) return null;
            const parent = nodes[node.parent];
            if (!parent.x || !parent.y || !node.x || !node.y) return null;

```

```

        return (
          <path
            key={`fail-${node.id}`}
            d={`M ${node.x} ${node.y - 10} Q ${node.x +
- 16}`}
            fill="none"
            stroke={isCurrent ? '#f43f5e' : 'rgb(0,0,0)'}
            strokeWidth={isCurrent ? 2 : 1.5}
            strokeDasharray="4 4"
            markerEnd="url(#fail-arrow)"
          />
        );
      })}

    {/* Draw Nodes */}
    {Object.values(nodes).map(node => {
      if (!node.x || !node.y) return null;
      const isRoot = node.id === 0;
      const isCurrent = currentNode === node.id;
      const isInQueue = bfsQueue.includes(node.id);

      let fill = '#1e293b';
      let stroke = '#475569';

      if (isCurrent) {
        fill = '#2563eb';
        stroke = '#60a5fa';
      } else if (isInQueue) {
        fill = '#b45309';
        stroke = '#fbbf24';
      } else if (node.is_terminal) {
        fill = '#059669';
        stroke = '#34d399';
      }

      return (
        <g key={`node-${node.id}`}>

```

```

<div className="bg-slate-900/50 p-5 rounded-xl" style={{width: 100%;}}
  <div>
    <h5 className="text-white font-bold mb-3 flex items-center">
      <span> </span> Словарь (Паттерны)
    </h5>
    <div className="flex flex-wrap gap-2 mb-4">
      {words.map((w) => {
        <span
          key={w}
          className="bg-slate-800 border border-slate-700 border-radius-4px p-2 px-4 py-1 gap-1 group"
        >
          {w}
          <button
            onClick={() => removeWord(w)}
            className="text-slate-500 hover:text-white"
            title="Удалить"
          >
            ×
          </button>
        </span>
      )})}
    </div>
  </div>

```

```

<form onSubmit={handleAddWord} className="flex flex-wrap gap-2">
  <input
    type="text"
    value={newWord}
    onChange={(e) => setNewWord(e.target.value)}
    placeholder="НОВОЕ СЛОВО..."
    maxLength={8}
    className="bg-slate-800 border border-slate-700 border-radius-4px p-2 px-4 py-1 outline-none focus:border-blue-500"
  />
  <button
    type="submit"
    className="bg-slate-700 hover:bg-slate-600"
  >
    Добавить
  </button>
</form>

```



```
    });  
  };
```

ФАЙЛ 60/86: src/components/SegmentTreeVisualizer.tsx
КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```
import React, { useState, useEffect, useCallback, useMemo } from 'react';  
import { Play, Pause, RotateCcw, ChevronLeft, ChevronRight } from 'lucide-react';
```

```
// ===== TYPES =====
```

```
interface Step {  
  id: number;  
  desc: string;  
  codeLine: number;  
  treeState: Record<number, number>; // node index  
  highlightNodes: number[]; // current highlighted nodes  
  mergeChildren?: [number, number]; // pair being merged  
  arrayHighlight?: [number, number]; // [L, R] range  
  result?: number; // partial result  
}
```

```
// ===== PURE LOGIC: build tree =====
```

```
function buildTreeFull(arr: number[]): Record<number, number> {  
  const n = arr.length;  
  const tree: Record<number, number> = {};  
  function go(v: number, l: number, r: number) {  
    if (l === r) { tree[v] = arr[l]; return; }  
    const m = (l + r) >> 1;  
    go(2 * v, l, m);  
    go(2 * v + 1, m + 1, r);  
    tree[v] = (tree[2 * v] ?? 0) + (tree[2 * v + 1] ?? 0);  
  }  
  go(1, 0, n - 1);  
  return tree;  
}
```

```

        go(2 * v + 1, m + 1, r);
        tree[v] = (tree[2 * v] ?? 0) + (tree[2 * v + 1] ?? 0);
        steps.push({ id: id++, desc: `Объединяем: tree[${v}]`,
            state: { ...tree }, highlightNodes: [v], mergeChildren: true });
    }
    go(1, 0, n - 1);
    steps.push({ id: id++, desc: `Дерево построено! Корень: tree[1]`,
        Highlight: [0, n - 1] });
    return steps;
}

```

// 2. Top-Down Query steps

```

function genQueryTopDownSteps(arr: number[], ql: number, qr: number) {
    const n = arr.length;
    const tree = buildTreeFull(arr);
    const steps: Step[] = [];
    let id = 0;

    function go(v: number, l: number, r: number, ql: number, qr: number) {
        if (ql > r || qr < l) {
            steps.push({ id: id++, desc: `query(${v}, [${l}..${r}]`,
                codeLine: 2, treeState: tree, highlightNodes: [v], mergeChildren: true });
            return 0;
        }
        if (ql <= l && r <= qr) {
            steps.push({ id: id++, desc: `query(${v}, [${l}..${r}]`,
                value: tree[v], codeLine: 4, treeState: tree, highlightNodes: [v], mergeChildren: true });
            return tree[v];
        }
        const m = (l + r) >> 1;
        steps.push({ id: id++, desc: `query(${v}, [${l}..${r}]`,
            codeLine: 6, treeState: tree, highlightNodes: [v], mergeChildren: true });
        const left = go(2 * v, l, m, ql, qr);
        const right = go(2 * v + 1, m + 1, r, ql, qr);
        const res = left + right;
        steps.push({ id: id++, desc: `Возврат в узел ${v}: ${res}`,
            mergeChildren: [2 * v, 2 * v + 1], arrayHighlight: [left, right], codeLine: 8 });
        return res;
    }
}

```



```

        highlights.push(r);
    }
    l >>= 1;
    r >>= 1;
    if (l < r) {
        steps.push({ id: id++, desc: `Поднимаемся: l = ${l}, r = ${r}, arrayHighlight: [qL, qR], result: res });
    }
}
steps.push({ id: id++, desc: `Запрос sum([${qL}..${qR}], ${qL}, ${qR})`, arrayHighlightNodes: [1], arrayHighlight: [qL, qR], result: res });
return steps;
}

```

// ===== TREE RENDERING HELPER =====

```
interface VNode { v: number; l: number; r: number; val: number; left: VNode; right: VNode; }
```

```

function buildVisualTree(treeState: Record<number, number>) {
    if (l === r) return { v, l, r, val: treeState[v] ?? null, left: null, right: null };
    const m = (l + r) >> 1;
    return { v, l, r, val: treeState[v] ?? null, left: buildVisualTree(treeState, v + 1, m + 1, r), right: buildVisualTree(treeState, m + 1, m + 1, r) };
}

```

// ===== CODE SNIPPETS =====

```

const BUILD_CODE = [
    "build(v, l, r) {",
    "    if (l === r) { tree[v] = A[l]; return; }",
    "    // -----",
    "    // -----",
    "    mid = (l + r) >> 1;",
    "    build(2*v, l, mid);",
    "    build(2*v+1, mid+1, r);",
    "    tree[v] = tree[2*v] + tree[2*v+1];",
    "}"
];

const QUERY_TD_CODE = [
    "query(v, l, r, qL, qR) {",

```

```

const clr = { indigo: { btn: 'bg-indigo-600 hover:bg-
border-indigo-500 bg-indigo-600', childRing: 'ring-
-900' }, emerald: { btn: 'bg-emerald-600 hover:bg-e
30 border-emerald-500 bg-emerald-600', childRing: '
-slate-900' }, amber: { btn: 'bg-amber-600 hover:bg
der-amber-500 bg-amber-600', childRing: 'ring-rose-!
[color];

return { step, idx, setIdx, playing, setPlaying, speed
}

// ===== SIMULATION PANEL =====
function SimPanel({ title, icon, steps, code, color, arr
| 'emerald' | 'amber'; arr: number[] }) {
  const p = Player({ steps, color });
  if (!p) return null;
  const { step, idx, setIdx, playing, setPlaying, speed
  const n = arr.length;

  const vTree = useMemo(() => buildVisualTree(step.tree

  const renderNode = useCallback((nd: VNode): React.Rea
    const isHigh = step.highlightNodes.includes(nd.v);
    const isChild = step.mergeChildren?.includes(nd.v);
    const has = nd.val !== null;

    let cls = 'bg-slate-800 border-slate-700 text-slate
    if (isHigh) cls = `${clr.ring} text-white ring-2 sc
    else if (isChild) cls = `${clr.childRing} text-ambe
    else if (has) cls = `${clr.filled} text-slate-200`;

    return (
      <div key={nd.v} className="flex flex-col items-ce
        <div className={`flex flex-col items-center jus
          ${cls}`}>
          <span className="text-[7px] opacity-60 font-m
          <span className="text-[9px] font-bold">[{nd.l

```



```

        />
      </div>
      <div className="flex items-center gap-1">
        <span className="text-xs text-slate-400">
          <input
            type="number"
            min={0}
            max={arr.length - 1}
            value={qR}
            onChange={e => setQR(Math.min(Number(e.target.value), arr.length - 1))}
            className="w-14 bg-slate-900 border border-indigo-500 focus:border-indigo-500"
          />
        </div>
      </div>
      <div className="text-[10px] text-slate-500">
        Запрос:  $\text{sum}(A[\{qL\}..\{qR\}]) = \{arr.slice(qL, qR + 1).reduce((acc, val) => acc + val, 0))\}$ 
      </div>
    </div>
  </div>
</div>

{/* ---- TABS ---- */}
<div className="flex flex-wrap items-center gap-1">
  <button onClick={() => setView('build')} className="text-sm font-medium border border-indigo-500 p-2 rounded-md transition-colors duration-200" > Построить
  </button>
  <button onClick={() => setView('queries')} className="text-sm font-medium border border-emerald-500 p-2 rounded-md transition-colors duration-200" > Запрос: св
  </button>
  <button onClick={() => setView('theory')} className="text-sm font-medium border border-blue-500 p-2 rounded-md transition-colors duration-200" > Почему / Спра
  </button>
  <button onClick={() => setView('info')} className="text-sm font-medium border border-slate-500 p-2 rounded-md transition-colors duration-200" > Справка
  </button>
</div>

```

```

{view === 'theory' && {
  <div className="space-y-5">
    {/* Why it splits this way */}
    <div className="bg-slate-950 p-5 rounded-xl b
      <h4 className="text-base font-bold text-whi
        во разбилось именно так для N={arr.length}?
        <p className="text-sm text-slate-300 leadin
          Каждый узел берёт <code className="bg-sla
          к получает <code className="bg-slate-900 px
          -slate-900 px-1 rounded font-mono text-emerg
          евая часть забирает на 1 больше (округление
        </p>
        <div className="bg-slate-900 p-4 rounded-lg
          <div className="text-slate-400">Корень: L
          <div className="text-slate-300">mid = {(0
          <div className="text-indigo-300">→ Левый:
          <div className="text-emerald-300">→ Правы
          h - 1) >> 1)} эл.)</div>
          <div className="text-slate-500 mt-2">Всег
          } внутренних.</div>
        </div>
      </div>

    {/* Comparison cards */}
    <div className="grid grid-cols-1 xl:grid-cols
      <div className="bg-slate-950 p-5 rounded-xl
        <div className="absolute -top-3 left-4 bg
        e border border-emerald-500 shadow-md">
          ↳ Запрос Сверху-вниз (Рекурсия)
        </div>
        <p className="text-xs text-slate-300 mb-3
          Начинаем с корня. Если отрезок узла полн
          наче спускаемся в обоих детей.
        </p>
        <div className="space-y-1.5 text-xs">
          <div className="flex justify-between bo
          span className="text-rose-400 font-mono">0(

```

```
    });  
  }  
}
```

ФАЙЛ 61/86: src/components/Sidebar.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```
import React, { useMemo, useState } from "react";  
import { ChevronRight, Compass, Search, Sparkles, X } from "lucide-react";  
import { chapters } from "../data/content";  
import { VIZ_REGISTRY } from "../vizRegistry";
```

```
interface SidebarProps {  
  selectedChapterId: string;  
  setSelectedChapterId: (id: string) => void;  
  /** Мобильный режим: список живёт в выдвижной панели */  
  onNavigate?: () => void;  
}
```

```
export const Sidebar: React.FC<SidebarProps> = ({ selectedChapterId, setSelectedChapterId, onNavigate }) => {  
  const [query, setQuery] = useState("");
```

```
  const groups = useMemo(() => {  
    const q = query.trim().toLowerCase();  
    const filtered = q ? chapters.filter((c) => c.title.toLowerCase().includes(q)) : chapters;  
    const map = new Map<string, typeof chapters>();  
    for (const c of filtered) {  
      const key = c.category?.trim() || "Прочие темы";  
      if (!map.has(key)) map.set(key, []);  
      (map.get(key) as typeof chapters).push(c);  
    }  
    return Array.from(map.entries());  
  }, [query]);
```

```
  return (  
    <aside className="w-full bg-slate-900/80 lg:bg-transparent" data-bbox="99 951 1000 1000">
```

```

        onClick={() => {
          setSelectedChapterId(chapter.id);
          onNavigate?.();
          window.scrollTo({ top: 0, behavior: 'smooth' });
        }}
        className={`w-full text-left px-3 py-2 border rounded-md
          ${isSelected
            ? "bg-indigo-600/15 border-indigo-600/20"
            : "bg-slate-800/40 border-transparent"}
        `}
      >
        {hasViz && (
          <span
            className="w-1.5 h-1.5 rounded-full bg-indigo-600/80"
            title="Есть интерактивная визуализация"
          />
        )}
        <span className="font-semibold text-white">
          <ChevronRight
            className={`w-4 h-4 shrink-0 mt-0.5
              ${isSelected ? "text-indigo-400" : "text-white"}
            `}
          />
        </button>
      </div>
    </div>
  </div>
</div>

```

```

<div className="shrink-0 m-3 mt-0 bg-gradient-to-r from-indigo-600/20 to-indigo-600/0
  ] text-slate-300 space-y-1.5 hidden lg:block">
  <div className="font-bold text-indigo-400 flex items-center">
    <Sparkles className="w-3.5 h-3.5" /> Как пользоваться?
  </div>
  <p className="leading-relaxed">
    Подчёркнутые термины в тексте раскрываются по
  </p>
</div>

```



```
// Stack state
const [stack, setStack] = useState<Plate[]>([
  { id: '1', name: 'Тарелка из-под пасты', icon: ' ' },
  { id: '2', name: 'Тарелка из-под пиццы', icon: ' ' },
  { id: '3', name: 'Блюдец от торта', icon: ' ' },
]);
const [popMessage, setPopMessage] = useState<string | null>('');

// Queue state
const [queue, setQueue] = useState<Person[]>([
  { id: '1', name: 'Студент Вася', icon: ' ' },
  { id: '2', name: 'Голодный кодер', icon: ' ' },
  { id: '3', name: 'Кот Борис', icon: ' ' },
]);
const [dequeueMessage, setDequeueMessage] = useState<string | null>('');

// Heap state
const [heap, setHeap] = useState<Hypothesis[]>([
  { id: '1', text: 'Кандидат: "Привет, я искусственный интеллект"', icon: ' ' },
  { id: '2', text: 'Кандидат: "Привет, я испек вкусный пирог"', icon: ' ' },
  { id: '3', text: 'Кандидат: "Привет, я испанский интеллект"', icon: ' ' },
]);
const [newCustomText, setNewCustomText] = useState('');
const [newCustomProb, setNewCustomProb] = useState(0.5);
const [heapMessage, setHeapMessage] = useState<string | null>('');

// Stack handlers
const addPlate = () => {
  const presets = [
    { name: 'Сковорода от омлета', icon: ' ' },
    { name: 'Кастрюля от супа', icon: ' ' },
    { name: 'Чашка от кофе', icon: '☹' },
    { name: 'Миска с остатками салата', icon: ' ' },
    { name: 'Тарелка от тако', icon: ' ' },
  ];
  const randomPreset = presets[Math.floor(Math.random() * presets.length)];
  const newPlate = {
    id: Date.now().toString(),
    name: randomPreset.name,
    icon: randomPreset.icon,
  };
  setStack([newPlate, ...stack]);
};

const popPlate = () => {
  if (stack.length > 0) {
    setStack(stack.slice(1));
    setPopMessage('Поплавок удален');
  }
};

const enqueuePerson = () => {
  const newPerson = {
    id: Date.now().toString(),
    name: 'Новый человек',
    icon: ' '
  };
  setQueue([newPerson, ...queue]);
};

const dequeuePerson = () => {
  if (queue.length > 0) {
    setQueue(queue.slice(1));
    setDequeueMessage('Человек удален');
  }
};

const addHypothesis = () => {
  const newHypothesis = {
    id: Date.now().toString(),
    text: 'Новая гипотеза',
    icon: ' '
  };
  setHeap([newHypothesis, ...heap]);
};

const removeHypothesis = () => {
  if (heap.length > 0) {
    setHeap(heap.slice(1));
    setHeapMessage('Гипотеза удалена');
  }
};

const newCustomHypothesis = () => {
  if (newCustomText.trim() > 0) {
    const newHypothesis = {
      id: Date.now().toString(),
      text: newCustomText,
      icon: ' '
    };
    setHeap([newHypothesis, ...heap]);
    setNewCustomText('');
    setNewCustomProb(0.5);
  }
};

const resetCustomHypothesis = () => {
  setNewCustomText('');
  setNewCustomProb(0.5);
};
```

```

        icon: randomPreset.icon
    };
    setQueue(prev => [...prev, newPerson]);
    setDequeueMessage(` Встал в конец очереди: ${newPe
};

const servePerson = () => {
    if (queue.length === 0) {
        setDequeueMessage("⚠ Очередь пуста! Суп ждет нов
        return;
    }
    const firstPerson = queue[0];
    setQueue(prev => prev.slice(1));
    setDequeueMessage(` Выдан суп первому в очереди (F
};

const resetQueue = () => {
    setQueue([
        { id: '1', name: 'Студент Вася', icon: ' ' },
        { id: '2', name: 'Голодный кодер', icon: ' ' },
        { id: '3', name: 'Кот Борис', icon: ' ' },
    ]);
    setDequeueMessage(" Очередь восстановлена.");
};

// Heap handlers
const addHypothesis = (e: React.FormEvent) => {
    e.preventDefault();
    if (!newCustomText.trim()) return;
    const item: Hypothesis = {
        id: Date.now().toString(),
        text: `Кандидат: "${newCustomText}"`,
        probability: Number(newCustomProb)
    };
    setHeap(prev => [...prev, item]);
    setNewCustomText('');
    setHeapMessage(` Добавлена гипотеза с вероятностью
};

```

```

    Проверь работу принципов LIFO, FIFO и Приоритет
  </p>
</div>

```

```

{/* Tabs */}
<div className="grid grid-cols-1 md:grid-cols-3 gap-3">
  <button
    onClick={() => setActiveTab('stack')}
    className={`flex items-center justify-between gap-2
      ${activeTab === 'stack'
        ? 'bg-blue-600/20 border-blue-500 text-blue-600'
        : 'bg-slate-800/60 border-slate-700/60 text-slate-400'}
    `}
  >
    <div className="flex items-center gap-3">
      <Layers className={`w-5 h-5 ${activeTab === 'stack' ? 'text-blue-600' : 'text-slate-400'}
        <div className="text-left">
          <div className="font-bold text-sm text-white">Stack
          <div className="text-xs opacity-80">LIFO
        </div>
      </div>
    </div>
    <span className="text-xs bg-blue-500/20 text-blue-600">
      LIFO
    </span>
  </button>

  <button
    onClick={() => setActiveTab('queue')}
    className={`flex items-center justify-between gap-2
      ${activeTab === 'queue'
        ? 'bg-indigo-600/20 border-indigo-500 text-indigo-600'
        : 'bg-slate-800/60 border-slate-700/60 text-slate-400'}
    `}
  >
    <div className="flex items-center gap-3">
      <AlignLeft className={`w-5 h-5 ${activeTab === 'queue' ? 'text-indigo-600' : 'text-slate-400'}
        <div className="text-left">
          <div className="font-bold text-sm text-white">Queue
          <div className="text-xs opacity-80">FIFO
        </div>
      </div>
    </div>
    <span className="text-xs bg-indigo-500/20 text-indigo-600">
      FIFO
    </span>
  </button>

```

```

    </h3>
    <p className="text-slate-400 text-xs mt-1">
      Новые тарелки ставятся наверх. Мыть можн
    </p>
  </div>
  <div className="flex flex-wrap items-center">
    <button
      onClick={addPlate}
      className="flex-1 md:flex-none flex item
      rounded-xl text-sm font-bold shadow-lg tra
    >
      <Plus className="w-4 h-4" /> Поставить
    </button>
    <button
      onClick={popPlate}
      className="flex-1 md:flex-none flex item
      border-blue-500/40 px-4 py-2.5 rounded-xl
    >
      Помыть верхнюю
    </button>
    <button
      onClick={resetStack}
      title="Сбросить"
      className="p-2.5 bg-slate-800 hover:bg-
      tion-colors cursor-pointer"
    >
      <RotateCcw className="w-5 h-5" />
    </button>
  </div>
</div>

{popMessage && {
  <div className="bg-blue-950/60 border borde
  imate-fade-in">
    <span className="inline-block w-2 h-2 roun
    {popMessage}
  </div>
}}

```

```

        isTop ? 'bg-blue-500 text-white
    }`}>
        {isTop ? 'Готово к помывке' : `
    </span>
  </div>
  );
  }}}}
</div>
})
<div className="mt-6 text-xs text-slate-500
  ПОЛОЖИЛ ПОСЛЕДНЕЙ → ПОМЫЛ ПЕРВОЙ (LIFO) •
</div>
</div>
</div>
)}

{/* QUEUE TAB */}
{activeTab === 'queue' && {
  <div className="space-y-6">
    <div className="bg-slate-800/80 p-5 rounded-x
      tify-between gap-4">
      <div>
        <h3 className="text-lg font-bold text-whi
          <span> Симуляция очереди за супом</span>
          <span className="text-xs font-mono bg-il
        /span>
        </h3>
        <p className="text-slate-400 text-xs mt-1
          Кто первым пришел, тот первым получил с
        </p>
        </div>
        <div className="flex flex-wrap items-center
          <button
            onClick={addPerson}
            className="flex-1 md:flex-none flex ite
          -2.5 rounded-xl text-sm font-bold shadow-lg
          >
            <Plus className="w-4 h-4" /> Встать в о

```

```

<div className="flex flex-col items-center justify-center gap-300">
  <span className="text-3xl"> </span>
  <span className="text-xs font-bold font-mono">
    </div>

  <div className="text-indigo-500 flex items-center justify-center gap-300">
    <ArrowRight className="w-6 h-6 animate-pulse">
    </div>

  {queue.map((person, index) => {
    const isFirst = index === 0;
    return (
      <div
        key={person.id}
        className={`flex flex-col items-center justify-center gap-300
          ${isFirst
            ? 'bg-gradient-to-b from-indigo-500 to-white'
            : 'bg-slate-900/90 border-slate-500'}
          relative`
        >
        {isFirst && (
          <span className="absolute -top-100 left-0 right-0
            se tracking-wider shadow">
              Первый (Head)
            </span>
          )}
        <span className="text-4xl mb-2">{person.name}</span>
        <span className={`font-bold text-2xl font-mono`}
          >{person.name}</span>
        <span className={`text-[10px] font-mono`}
          >{isFirst ? 'Получает сун' : 'В'}</span>
      </div>
    )
  })}

```

```

        Взять кандидата с макс. вероятностью
    </button>
    <button
      onClick={resetHeap}
      title="Сбросить"
      className="p-2.5 bg-slate-800 hover:bg-
tion-colors cursor-pointer"
    >
      <RotateCcw className="w-5 h-5" />
    </button>
  </div>
</div>

{/* Add Form */}
<form onSubmit={addHypothesis} className="bg-
s-12 gap-4 items-end">
  <div className="md:col-span-6">
    <label className="block text-xs font-bold">
      <input
        type="text"
        value={newCustomText}
        onChange={e => setNewCustomText(e.target
placeholder='например: "Привет, я лучший"'
        className="w-full bg-slate-900 border b
rder-emerald-500 transition-colors"
      />
    </div>
    <div className="md:col-span-3">
      <label className="block text-xs font-bold">
        Вероятность: <span className="text-emera
      </label>
      <input
        type="range"
        min="0.01"
        max="0.99"
        step="0.01"
        value={newCustomProb}
        onChange={e => setNewCustomProb(Number(e

```

```

    </div>

    {sortedHeap.map(({item, index}) => {
      const isTop = index === 0;
      const percent = (item.probability * 100).toFixed(
      return {
        <div
          key={item.id}
          className={`flex items-center justify-between
            isTop
              ? 'bg-gradient-to-r from-emerald-500 to-slate-900/10 scale-[1.01]'
              : 'bg-slate-900/80 border-slate-800'
            }`}
        >
          <div className="flex items-center justify-between" >
            <span className={`flex items-center justify-between
              isTop ? 'bg-emerald-500 text-white' : 'bg-slate-900/80 text-slate-400'
            }`>
              #{index + 1}
            </span>
            <div>
              <div className={`font-bold text-white`} >
                {item.text}
              </div>
              {isTop && (
                <span className="text-[10px] text-white" >
                  Вершина Кучи (Root) • Будущее
                </span>
              )}
            </div>
          </div>
        </div>

        <div className="flex items-center justify-between" >
          {/* Custom progress bar */}
          <div className="hidden sm:block" >
            <div>

```



```

onOpen: (vizId: string) => void;
/** Перейти к теме, к которой привязан тренажёр. */
onGoChapter: (chapterId: string) => void;
}

/**
 * Каталог всех интерактивных тренажёров.
 *
 * Раньше вкладка «Симулятор» показывала один случайный
 * теперь это витрина: видно всё, что вообще можно поты
 */
export const SimulatorHub: React.FC<Props> = ({ onOpen,
  const [query, setQuery] = useState("");

  const items = useMemo(() => {
    const titleById = new Map(chapters.map((c) => [c.id, c.title]));
    const all = Object.entries(VIZ_REGISTRY).map(([id, entry],
      id,
      entry,
      chapterTitle: titleById.get(id),
    ]));
    const q = query.trim().toLowerCase();
    if (!q) return all;
    return all.filter(
      (i) =>
        i.entry.title.toLowerCase().includes(q) ||
        (i.entry.hint ?? "").toLowerCase().includes(q) ||
        (i.chapterTitle ?? "").toLowerCase().includes(q)
    );
  }, [query]);

  return (
    <div>
      <div className="mb-6">
        <h2 className="text-xl sm:text-2xl font-extrabold">Симулятор
        <p className="text-sm text-slate-400 leading-relaxed">
          Все интерактивные демонстрации в одном месте.
          Здесь их можно открыть напрямую.

```

```

        className="flex items-center gap-1.5 text-white
transition-colors"
      >
        <Play className="w-3.5 h-3.5" /> Запустить
      </button>
      {chapterTitle && (
        <button
          onClick={() => onGoChapter(id)}
          title={chapterTitle}
          className="flex items-center gap-1.5 text-white
0 hover:text-white transition-colors min-w-[100px]"
        >
          <BookOpen className="w-3.5 h-3.5 shrink-0" />
          <span className="truncate max-w-[90px]">{chapterTitle}
        </button>
      )}
    </div>
  </div>
)
</div>
</div>
);
};

```

ФАЙЛ 64/86: src/components/SplayRotationSandbox.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 10

```

import React, { useCallback, useMemo, useState } from "react";
import { CheckCircle2, Dices, Eye, EyeOff, RotateCcw, Undo } from "lucide-react";

```

```
/**
```

```
 * Песочница поворотов.
```

```
 *
```

```
 * Здесь ничего не подсказывается: дано настоящее дерево.
```

```
 * поднять отмеченный узел в корень. Клик по узлу = один шаг.
```

```

    },
    {
      name: "Один уровень",
      hintCase: "Zig",
      target: 2,
      build: () => ({ key: 4, left: { key: 2, left: leaf(
    },
    {
      name: "Смешанный",
      hintCase: "Zig-Zag, затем Zig-Zig",
      target: 5,
      build: () => {
        const n5 = leaf(5);
        const n6: TNode = { key: 6, left: n5, right: leaf(
        const n4: TNode = { key: 4, left: leaf(3), right:
        return { key: 8, left: n4, right: leaf(9) };
      },
    },
  ];

/* ----- операции над деревом -----

export const findPath = (root: TNode | null, key: number): TNode[] => {
  const path: TNode[] = [];
  let cur = root;
  while (cur) {
    path.push(cur);
    if (key === cur.key) return path;
    cur = key < cur.key ? cur.left : cur.right;
  }
  return [];
};

/** Поднять узел key на один уровень (поворот с родителем)
export function rotateUp(root: TNode, key: number): TNode {
  const path = findPath(root, key);
  if (path.length < 2) return root;

```

```

    x: number;
    y: number;
    depth: number;
}

function layout(root: TNode | null): { nodes: Placed[];
  const nodes: Placed[] = [];
  const edges: [number, number][] = [];
  let order = 0;

  const walk = (n: TNode | null, depth: number) => {
    if (!n) return;
    walk(n.left, depth + 1);
    nodes.push({ key: n.key, x: order++, y: depth, depth });
    if (n.left) edges.push([n.key, n.left.key]);
    if (n.right) edges.push([n.key, n.right.key]);
    walk(n.right, depth + 1);
  };
  walk(root, 0);

  const cols = Math.max(1, order);
  const rows = Math.max(1, Math.max(...nodes.map((n) => n.depth)));
  const colW = 46;
  const rowH = 62;
  return {
    nodes: nodes.map((n) => ({ ...n, x: n.x * colW + colW / 2, y: n.y * rowH })),
    edges,
    width: cols * colW,
    height: rows * rowH + 20,
  };
}

/* ----- КОМПОНЕНТ -----

export const SplayRotationSandbox: React.FC = () => {
  const [presetIdx, setPresetIdx] = useState(0);
  const preset = PRESETS[presetIdx];

```

```

    setMoves((m) => m.slice(0, -1));
  };

const wrong = moves.filter((m) => !m.correct).length;
const minimal = useMemo(() => findPath(preset.build()))

return (
  <div className="w-full bg-slate-950 rounded-2xl border-2 border-slate-800">
    <div className="flex items-start justify-between p-4">
      <div>
        <h4 className="text-sm sm:text-base font-bold text-slate-400">
          <p className="text-[12px] text-slate-400 leading-4">
            Клик по узлу = один поворот с его родителем
            <span className="font-mono font-bold text-indigo-400">
              поворотов выбираешь сам, разбор покажем в к
            </p>
          </div>
          <button
            onClick={() => reset((presetIdx + 1) % PRESETS.length)}
            className="flex items-center gap-1.5 px-3 py-1.5 text-sm font-bold transition-colors shrink-0"
          >
            <Dices className="w-3.5 h-3.5" /> Другое дерево
          </button>
        </div>
        <div className="flex gap-2 mb-3 overflow-x-auto no-scrollbar">
          {PRESETS.map((p, i) => (
            <button
              key={p.name}
              onClick={() => reset(i)}
              className={`px-3 py-1.5 rounded-lg text-[11px] font-medium
                ${i === presetIdx ? "bg-indigo-600 text-white" : "bg-slate-800 text-slate-400"}
              `}
            >
              {p.name}
            </button>
          ))}
        </div>
      </div>
    </div>
  )
)

```

```

        cursor: clickable ? "pointer" : "default"
      }}
    >
      {isTarget && {
        <circle r={22} fill="none" stroke="#6366f1" stroke-width={2} />
      }}
      <circle
        r={16}
        fill={isTarget ? "#6366f1" : path.hasStroke ? "#a5b4fc" : "#475568"}
        stroke={isTarget ? "#a5b4fc" : "#475568"}
        strokeWidth={2}
      />
      <text textAnchor="middle" dominantBaseline="middle" />
      {n.key}
    </text>
  </g>
);
}}
</svg>
</div>

```

```

<div className="flex flex-wrap items-center gap-2" />
  <button
    onClick={undo}
    disabled={history.length === 0}
    className="flex items-center gap-1.5 px-3 py-1.5 text-sm font-medium text-white disabled:opacity-40 transition-colors" />
    <Undo2 className="w-3.5 h-3.5" /> ОТМЕНИТЬ
  </button>
  <button
    onClick={() => reset()}
    className="flex items-center gap-1.5 px-3 py-1.5 text-sm font-medium text-white disabled:opacity-40 transition-colors" />
    <RotateCcw className="w-3.5 h-3.5" /> ЗАНОВО
  </button>
  <button
    onClick={() => redo}
    disabled={history.length === 0}
    className="flex items-center gap-1.5 px-3 py-1.5 text-sm font-medium text-white disabled:opacity-40 transition-colors" />
    <RotateCcw className="w-3.5 h-3.5" /> ОТМЕНИТЬ
  </button>
  <button
    onClick={() => redo}
    disabled={history.length === 0}
    className="flex items-center gap-1.5 px-3 py-1.5 text-sm font-medium text-white disabled:opacity-40 transition-colors" />
    <RotateCcw className="w-3.5 h-3.5" /> ЗАНОВО
  </button>

```

```

        <ul className="list-disc pl-5 space-y-0.5">
          {moves.map(
            (m, i) =>
              !m.correct && (
                <li key={i}>
                  ход {i + 1}: крутили <span className="text-red-500">
                    {m.expected}
                  </li>
                )
              )
          )}
        </ul>
      )}
      {wrong === 0 && moves.length > minimal && (
        <div className="opacity-80">Порядок верный,
      )}
    </div>
  )}
</div>
);
};

```

ФАЙЛ 65/86: src/components/SplayRotationsViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```

import React, { useEffect, useMemo, useState } from "react";
import { Gauge, Pause, Play, RotateCcw } from "lucide-react";

```

```
/**
```

```
* Разбор трёх случаев операции Splay: Zig, Zig-Zig, Zig-Zag
```

```
*
```

```
* Что сделано ради понятности:
```

```
* • у узлов настоящие числа (20 / 40 / 60), а не абстрактные значения
```

```
* сразу видно, что дерево остаётся деревом поиска;
```

```
* • под деревом печатается обход слева направо: он ОДНОУРОВНЕВНЫЙ
```

```
* это и есть доказательство, что поворот ничего не добавляет
```

```

-----
const AIM_MS = 2600;
const RESULT_MS = 3400;
const MOVE = "transform 1200ms cubic-bezier(.34,.01,.2,

/* ----- Zig -----
const ZIG_BEFORE = {
  pos: { p: [195, 50], x: [105, 130], C: [280, 130], A:
  edges: [["p", "x"], ["p", "C"], ["x", "A"], ["x", "B"]
} as unknown as Pick<Frame, "pos" | "edges">;

const ZIG: Case = {
  key: "zig",
  label: "Zig",
  when: "родитель x — уже корень, деда нет",
  rotations: "1 поворот",
  keys: { x: 20, p: 40 },
  inorder: ["A", "20", "B", "40", "C"],
  ranges: "A < 20 · В между 20 и 40 · C > 40",
  frames: [
    {
      ...ZIG_BEFORE,
      kind: "start",
      title: "Было: 20 висит под корнем 40",
      text: "Нам нужен ключ 20, а он на глубине 1. Деду
      ms: RESULT_MS,
    },
    {
      ...ZIG_BEFORE,
      kind: "aim",
      title: "Прицел: крутим ребро 40 — 20",
      text: "Ничего пока не двигается. Смотри на подсве
      focus: ["p", "x"],
      ms: AIM_MS,
    },
    {
      pos: { x: [160, 50], A: [80, 130], p: [245, 130],
      edges: [["x", "A"], ["x", "p"], ["p", "B"], ["p",
      kind: "result",

```



```

        ...ZZ_S0,
        kind: "aim",
        title: "Прицел 1: ВЕРХНЕЕ ребро 60 – 40",
        text: "Главная хитрость этого случая: первым крути
        focus: ["g", "p"],
        ms: AIM_MS,
    },
    {
        ...ZZ_S1,
        kind: "result",
        title: "Поворот 1: 40 поднялось над 60",
        text: "40 встало в корень, 60 опустилось к нему п
            убине 1, а не 2.",
        moved: ["p", "g", "C"],
        ms: RESULT_MS,
    },
    {
        ...ZZ_S1,
        kind: "aim",
        title: "Прицел 2: ребро 40 – 20",
        text: "А теперь обычный одиночный поворот – тот ж
        focus: ["p", "x"],
        ms: AIM_MS,
    },
    {
        pos: { x: [100, 45], A: [45, 118], p: [188, 118],
        edges: [{"x", "A"}, {"x", "p"}, {"p", "B"}, {"p",
        kind: "result",
        title: "Стало: 20 в корне, бамбук стал кустом",
        text: "Сравни с первым кадром: А и В были на глуб
            дерево, а не только достаёт ключ.",
        moved: ["x", "p", "A", "B"],
        ms: RESULT_MS,
    },
    ],
};

```

```

        title: "Поворот 1: змейка выпрямилась",
        text: "40 заняло место 20 и забрало его к себе ле
        moved: ["x", "p", "C"],
        ms: RESULT_MS,
    },
    {
        ...ZG_S1,
        kind: "aim",
        title: "Прицел 2: ребро 60 – 40",
        text: "Остался одиночный поворот вокруг деда – и
        focus: ["g", "x"],
        ms: AIM_MS,
    },
    {
        pos: { x: [180, 50], p: [90, 130], g: [275, 130],
        edges: [["x", "p"], ["x", "g"], ["p", "A"], ["p",
        kind: "result",
        title: "Стало: 20 и 60 – два сына сорока",
        text: "Дерево стало идеально симметричным: все че
            очку». ",
        moved: ["x", "p", "g", "C"],
        ms: RESULT_MS,
    },
    ],
};

```

```
const CASES: Case[] = [ZIG, ZIGZIG, ZIGZAG];
```

```

const COLOR: Record<string, { fill: string; stroke: str
    x: { fill: "#6366f1", stroke: "#a5b4fc" },
    p: { fill: "#0ea5e9", stroke: "#7dd3fc" },
    g: { fill: "#f59e0b", stroke: "#fcd34d" },
    sub: { fill: "#1e293b", stroke: "#475569" },
};

```

```

const ROLE_RU: Partial<Record<NodeId, string>> = {
    x: "x – ищем его",
    p: "p – родитель",

```

```

        strokeDasharray="4 3"
        markerEnd="url(#splay-arrow)"
    />
  })
</g>
);
}}
}}}

{edges.map(([a, b], i) => {
  const pa = pos[a];
  const pb = pos[b];
  if (!pa || !pb) return null;
  const hot = Boolean(focus && focus.includes(a));
  return (
    <line
      key={` ${a} - ${b} - ${i} `}
      x1={pa[0]}
      y1={pa[1]}
      x2={pb[0]}
      y2={pb[1]}
      stroke={hot ? "#fbbf24" : "#475569"}
      strokeWidth={hot ? 5 : 2}
      opacity={focus && !hot ? 0.25 : 1}
      style={{ transition: "all 1200ms cubic-bezier(0.4, 0, 0.2, 1)" }}
    />
  );
}}

{(Object.keys(pos) as NodeId[]).map((id) => {
  const p = pos[id];
  if (!p) return null;
  const sub = isSubtree(id);
  const c = COLOR[sub ? "sub" : id];
  const r = sub ? 15 : 20;
  const hot = Boolean(focus && focus.includes(id));
  return (
    <g
      key={id}

```

```

const current = active.frames[idx];
const prevPos = active.frames[Math.max(idx - 1, 0)].pos;
const duration = useMemo(() => Math.round(current.ms - prevPos));
const last = idx === total - 1;

useEffect(() => {
  setFrame(0);
}, [caseIdx]);

useEffect(() => {
  if (!playing) return;
  const t = setTimeout(() => setFrame((f) => (f + 1) % total), duration);
  return () => clearTimeout(t);
}, [playing, frame, duration, total]);

return (
  <div className="w-full bg-slate-950 rounded-2xl border border-slate-800">
    <div className="flex gap-2 mb-4 overflow-x-auto no-scrollbar">
      {CASES.map((c, i) => (
        <button
          key={c.key}
          onClick={() => setCaseIdx(i)}
          className={`px-3 sm:px-4 py-2 rounded-lg text-slate-300
            ${i === caseIdx ? "bg-indigo-600 text-white"}
          `}
        >
          {c.label}
        </button>
      ))}
    </div>

    <div className="mb-3 text-xs sm:text-[13px] text-slate-400">
      <span className="font-bold text-indigo-300">Корреляция</span>
      <span className="text-slate-500">({active.rotations})</span>
    </div>

    {/* шаг + пояснение стоят НАД картинкой: сначала шаг, потом пояснение */}
    <div className="mb-3 rounded-xl border border-slate-800">

```

```

        <p className="text-[11px] text-slate-500 mt-1"
      </div>
    </div>

    {/* индикатор автопрокрутки */}
    <div className="h-1 w-full bg-slate-800 rounded-f
      <div
        key={` ${caseIdx}- ${frame}- ${playing}- ${slow}`
        className="h-full bg-indigo-500"
        style={
          playing
            ? { animation: `demoProgress ${duration}m
              : { width: "100%", background: "#334155"
            }
          />
        </div>

    <div className="flex flex-wrap items-center gap-2"
      <button
        onClick={() => {
          setPlaying(false);
          setFrame((f) => (f - 1 + total) % total);
        }}
        disabled={idx === 0}
        className="px-3 py-2 rounded-lg bg-slate-800
          d:hover:text-slate-300 text-xs font-bold tr
      >
        ← Назад
      </button>
      <button
        onClick={() => {
          setPlaying(false);
          setFrame((f) => (f + 1) % total);
        }}
        className="px-4 py-2 rounded-lg bg-indigo-600
          w-indigo-900/40"
      >
        {last ? "в Сначала" : "Дальше →"}

```

```

        }}
      </div>
    </div>

    <p className="mt-2 text-[11px] text-slate-500">
      Листай кнопкой «Дальше» в своём темпе – кадры н
    </p>
  </div>
);
};

```

ФАЙЛ 66/86: src/components/SplayTreeViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```

import React, { useState } from "react";
import {
  RotateCcw,
  HelpCircle,
  BookOpen,
} from "lucide-react";

interface SplayNode {
  key: number;
  left: SplayNode | null;
  right: SplayNode | null;
  x?: number;
  y?: number;
}

// Simple BST insertion to build initial tree
const insertBST = (root: SplayNode | null, key: number) => {
  if (!root) return { key, left: null, right: null };
  if (key < root.key) {
    root.left = insertBST(root.left, key);
  } else if (key > root.key) {

```

```
const createInitialTree = (): SplayNode => {
  let r: SplayNode | null = null;
  const initialKeys = [40, 20, 50, 10, 30, 60];
  initialKeys.forEach((key) => {
    r = insertBST(r, key);
  });
  return r!;
};

const [tree, setTree] = useState<SplayNode>(createInitialTree);
const [inputValue, setInputValue] = useState<string>('');
const [log, setLog] = useState<string[]>([
  "Дерево инициализировано. Попробуйте кликнуть на узел."
]);
const [highlightedNode, setHighlightedNode] = useState<SplayNode | null>(null);

// Splay operation that tracks steps!
const splayStepByStep = (
  root: SplayNode | null,
  key: number,
): { newRoot: SplayNode | null; steps: string[] } => {
  const steps: string[] = [];
  if (!root) return { newRoot: null, steps: ["Дерево пустое."] };

  // We implement the classic top-down or bottom-up splay operation.
  // For visualization logs, we do a bottom-up explanation.
  let path: { node: SplayNode; dir: "left" | "right" | null }[] = [];
  let current: SplayNode | null = root;

  // Find the element or the element where search failed.
  let lastNode: SplayNode = root;
  while (current) {
    lastNode = current;
    if (key === current.key) {
      path.push({ node: current, dir: null });
      break;
    } else if (key < current.key) {
      path.push({ node: current, dir: "left" });
    } else {
      path.push({ node: current, dir: "right" });
    }
    current = current[dir];
  }

  // Now we perform the splay operation using rotations.
  // The path contains the nodes visited during the search, from root to the target node.
  // We will perform rotations to move the target node to the root.
  // The rotations are:
  // 1. Zig-Zag: If the target node is the left child of its parent and the parent is the right child of its grandparent, perform a left rotation on the parent and a right rotation on the grandparent.
  // 2. Zig-Zig: If the target node is the left child of its parent and the parent is the left child of its grandparent, perform a right rotation on the grandparent.
  // 3. Zag-Zag: If the target node is the right child of its parent and the parent is the left child of its grandparent, perform a left rotation on the grandparent and a right rotation on the parent.
  // 4. Zag-Zig: If the target node is the right child of its parent and the parent is the right child of its grandparent, perform a left rotation on the grandparent.
  // The rotations are performed from the bottom of the path to the top.
  // The final root is the node at the top of the path.
  // The steps array contains the sequence of rotations performed.
  // The newRoot is the root of the tree after the splay operation.
  // The steps array is returned for visualization logs.
  // The newRoot and steps are returned.
  return { newRoot: lastNode, steps };
};
```

```

    } else if (k > node.left.key) {
        // Zig-Zag (Left-Right)
        node.left.right = splayAction(node.left.right);
        if (node.left.right) {
            node.left = leftRotate(node.left);
            steps.push(`Компонент Zig-Zag: левый поворот`);
        }
    }

    if (!node.left) return node;
    steps.push(
        `Финальный поворот Zig (Правое вращение корня)`
    );
    return rightRotate(node);
} else {
    if (!node.right) return node;

    if (k < node.right.key) {
        // Zag-Zig (Right-Left)
        node.right.left = splayAction(node.right.left);
        if (node.right.left) {
            node.right = rightRotate(node.right);
            steps.push(`Компонент Zig-Zag: правый поворот`);
        }
    } else if (k > node.right.key) {
        // Zag-Zag (Right-Right)
        node.right.right = splayAction(node.right.right);
        node = leftRotate(node);
        steps.push(
            `Вращение Zag-Zag (Левый поворот родителя)`
        );
    }

    if (!node.right) return node;
    steps.push(
        `Финальный поворот Zag (Левое вращение корня)`
    );
    return leftRotate(node);
}

```



```

    handleSplay(val);
    setInputValue("");
};

const resetTree = () => {
  setTree(createInitialTree());
  setHighlightedNode(null);
  setLog(["Дерево возвращено в исходное состояние."]);
};

// Assign coordinate positions using a simple recursion
const computeCoordinates = (
  node: SplayNode | null,
  x: number,
  y: number,
  levelWidth: number,
): void => {
  if (!node) return;
  node.x = x;
  node.y = y;
  if (node.left) {
    computeCoordinates(node.left, x - levelWidth, y + 1);
  }
  if (node.right) {
    computeCoordinates(node.right, x + levelWidth, y + 1);
  }
};

const drawTree = cloneTree(tree);
computeCoordinates(drawTree, 300, 40, 140);

// Render lines and nodes
const renderLines = (node: SplayNode | null): React.ReactNode[] => {
  if (!node) return [];
  let lines: React.ReactNode[] = [];
  if (node.left && node.left.x && node.left.y) {
    lines.push(
      <line
    
```

```

    data-title={`Кликните, чтобы выполнить Splay(${
>
    <circle
      cx={node.x}
      cy={node.y}
      r="18"
      fill={isHighlighted ? "#4f46e5" : "#1e293b"}
      stroke={isHighlighted ? "#818cf8" : "#64748b"}
      strokeWidth={isHighlighted ? "3" : "2"}
      className="transition-all duration-300 group-
    />
    <text
      x={node.x}
      y={node.y}
      dy=".3em"
      textAnchor="middle"
      fill="white"
      fontSize="12px"
      fontWeight="bold"
      className="select-none pointer-events-none"
    >
      {node.key}
    </text>
  </g>,
);
circles = circles.concat(renderNodes(node.left));
circles = circles.concat(renderNodes(node.right));
return circles;
};

return (
  <div className="bg-slate-900 rounded-xl border bord
    {/* Header */}
    <div className="bg-slate-800 p-4 border-b border-
      <h3 className="font-bold text-white flex items-
        <BookOpen className="w-5 h-5 text-indigo-400"
        Интерактивное Splay-дерево
      </h3>

```

```

        e.preventDefault();
        const val = parseInt(inputValue);
        if (!isNaN(val)) {
            let newTree = cloneTree(tree);
            newTree = insertBST(newTree, val);
            const { newRoot, steps } = splayStep(newTree);
            if (newRoot) setTree(newRoot);
            setLog([`Вставили [${val}].`, ...steps]);
            setInputValue("");
            setHighlightedNode(val);
        }
    }}
    className="bg-indigo-950 hover:bg-slate-900"
  >
    Вставить
  </button>
</form>

<button
  onClick={resetTree}
  className="flex items-center gap-1.5 px-3"
  >
    <RotateCcw className="w-3.5 h-3.5" /> Сбросить
  </button>
</div>
</div>

{/* Logs */}
<div className="lg:col-span-5 bg-slate-950 border-1px border-slate-800
  to overflow-y-auto custom-scrollbar">
  <h4 className="text-xs font-bold text-slate-400">
    Ход вращения (Splay-логи)
  </h4>
  <div className="flex-1 space-y-2 mb-4 overflow-hidden">
    {log.map((step, idx) => (
      <div
        key={idx}

```

```

    <p className="font-bold text-slate-300 mb-1">
      1. Отсутствующий элемент
    </p>
    <p className="text-slate-400 leading-relaxed">
      Допустим, мы ищем{" "}
      <code className="text-indigo-400 font-sans">его нет. Алгоритм спустится к листу, на
      неудачей {например,{" "}}
      <code className="text-indigo-400">[20]</code>
      <code className="text-indigo-400">[30]</code>
      поднимет в корень{" "}
      <strong className="text-white">
        последний успешно просмотренный узел
      </strong>
      , на котором он споткнулся! Это оставля
      сверху.
    </p>
  </div>
  <div className="mt-3 text-[10px] text-indigo-400">
    Поиск настраивает локальный кэш под реали
  </div>
</div>

<div className="bg-slate-900/60 p-4 rounded-l" style="border: 1px solid #334d5b; margin-top: 10px;">
  <div>
    <p className="font-bold text-slate-300 mb-1">
      2. Эффект качелей (Swing)
    </p>
    <p className="text-slate-400 leading-relaxed">
      Если агент запрашивает по очереди{" "}
      <code className="text-rose-400">[10]</code>
      <code className="text-rose-400">[60]</code>
      углы), дерево будет превращаться в палку
      сторону. Первая операция Splay(60) стои
      <strong className="text-white">0(n)</strong>
      глубину n. Вторая Splay(10) заставляет
      Постоянный свинг убивает производительн
      сложности <strong className="text-white">

```

```
import React, { useState, useCallback } from 'react';
import { ArrowUp, Sparkles } from 'lucide-react';

interface Plate {
  id: string;
  name: string;
  emoji: string;
  color: string;
  isDirty: boolean;
  animState: 'in' | 'idle' | 'washing' | 'clean';
}

const PLATE_PRESETS = [
  { name: 'Тарелка из-под спагетти', emoji: ' ', color: 'from-pa' },
  { name: 'Тарелка из-под пиццы', emoji: ' ', color: 'from-pa' },
  { name: 'Блюдец от торта', emoji: ' ', color: 'from-pa' },
  { name: 'Тарелка от тако', emoji: ' ', color: 'from-pa' },
  { name: 'Тарелка из-под суши', emoji: ' ', color: 'from-pa' },
  { name: 'Сковорода от глазуньи', emoji: ' ', color: 'from-pa' },
];

let counter = 0;

export const StackViz: React.FC = () => {
  const [dirtyStack, setDirtyStack] = useState<Plate[]>(
    [
      { id: 'p1', name: 'Тарелка из-под спагетти', emoji: ' ', animState: 'idle' },
      { id: 'p2', name: 'Тарелка из-под пиццы', emoji: ' ', animState: 'idle' },
      { id: 'p3', name: 'Блюдец от торта', emoji: ' ', animState: 'idle' },
    ]
  );

  const [cleanRack, setCleanRack] = useState<Plate[]>(
    []
  );
  const [isWashing, setIsWashing] = useState(false);
  const [showSoap, setShowSoap] = useState(false);
  const [shakeEmpty, setShake] = useState(false);
```

```

        setTimeout(() => setShake(false), 400);
        return;
    }

    setIsWashing(true);
    const topPlate = dirtyStack[dirtyStack.length - 1];

    addLog(`  POP: Начинаем мыть верхнюю тарелку с ${topPlate.name} 🍴`);

    // Step 1: Start washing animation (sponge and soap)
    setDirtyStack(prev => prev.map(p => p.id === topPlate.id ? { ...p, isDirty: false } : p));
    setShowSoap(true);

    setTimeout(() => {
        // Step 2: Wash complete. Plate becomes clean and is moved to clean rack
        const cleanPlate: Plate = {
            ...topPlate,
            isDirty: false,
            animState: 'clean',
        };

        setCleanRack(prev => [cleanPlate, ...prev].slice(0, cleanRack.length));
        setDirtyStack(prev => prev.slice(0, prev.length - 1));
        setShowSoap(false);
        setIsWashing(false);
        addLog(`  Готово! Чистая тарелка ${topPlate.emoji} 🍴`);
    }, 850);
}, [dirtyStack, isWashing]);

const reset = () => {
    counter = 0;
    setDirtyStack([
        { id: 'r1', name: 'Тарелка из-под спагетти', emoji: '🍝', mState: 'in' },
        { id: 'r2', name: 'Тарелка из-под пиццы', emoji: '🍕', imState: 'in' },
        { id: 'r3', name: 'Блюде от торта', emoji: '🍰', mState: 'in' },
    ]);
    setCleanRack([]);
    setShowSoap(false);
    setIsWashing(false);
    addLog('🧼 Сброс анимации');
};

```

```

disabled={isWashing || dirtyStack.length === 0}
className="flex-1 min-w-[150px] bg-gradient-to-r from-slate-800 to-slate-900
opacity-40 disabled:cursor-not-allowed text-white font-mono text-sm
border border-slate-700 border-radius-10px"
>
  <span> Помыть верхнюю (POP)</span>
</button>

<button
  onClick={reset}
  className="px-5 py-3.5 rounded-2xl bg-slate-800 text-white font-mono text-sm
  cursor-pointer border border-slate-700"
>
  Сброс
</button>
</div>

{/* ИНТЕРАКТИВНАЯ КУХНЯ */}
<div className="grid grid-cols-1 md:grid-cols-12 gap-4">

  {/* ЛЕВАЯ КОЛОНКА: СТОПКА ГРЯЗНЫХ ТАРЕЛОК */}
  <div className="md:col-span-7 bg-slate-900 rounded-2xl p-4
  min-h-[380px]">
    <div className="absolute top-4 left-4 text-[10px] font-mono">
      Грязная стопка (Стек вызовов / LIFO)
    </div>

    {/* Визуализация раковины */}
    <div className="absolute top-4 right-4 flex items-center justify-center
    h-12 w-12 bg-slate-800/80 text-[10px] font-mono">
      <span className="w-1.5 h-1.5 rounded-full bg-white border border-slate-700
      border-radius-50% display-block margin: 0 auto 0 auto 2px">
        <span>РАКОВИНА </span>
      </span>
    </div>

    {/* Мыльные пузыри при мытье */}
    {showSoap && (
      <div className="absolute inset-0 pointer-events-none">
        <div className="absolute w-6 h-6 rounded-2xl bg-white opacity-10
        border border-slate-700 border-radius-50% display-block margin: 0 auto 0 auto 2px">

```

```

        className={`
          w-full max-w-[280px] h-10 rounded
          shadow-md select-none transition-all duration
            ${plate.color}
            ${plate.animState === 'in' ? 'animate-in' : ''}
            ${plate.animState === 'washing' ? 'animate-washing' : ''}
            ${isTop ? 'ring-4 ring-blue-500/50' : ''}
        `}
      >
        <span className="text-lg">{plate.emblem}</span>
        <span className="text-slate-800 text-sm">
          {isTop && (
            <span className="text-[9px] bg-blue-500/50 rounded
              LIFO
            </span>
          )}
        </div>
      </div>
    );
  }}}
</div>
)}

{/* Столешница раковины */}
<div className="w-full h-4 bg-gradient-to-r from-slate-900 to-slate-800"
  </div>

{/* ПРАВАЯ КОЛОНКА: СУШИЛКА ДЛЯ ЧИСТЫХ ТАРЕЛОК */}
<div className="md:col-span-5 bg-slate-900 rounded-2xl p-4"
  <div className="absolute top-4 left-4 text-[10px] font-mono"
    Сушилка для чистой посуды
  </div>

  <div className="absolute top-4 right-4 flex items-center justify-between"
    order-emerald-800/80 text-[9px] font-mono">
    <Sparkles className="w-3.5 h-3.5 animate-pulse" />
    <span>ЧИСТО </span>
  </div>

```



```
        </div>
      )})
    </div>
  </div>
);
};
```

ФАЙЛ 68/86: src/components/StringAlgorithmsViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```
import { useState, useEffect, useRef, useMemo } from "react";
import { Play, Pause, SkipForward, Undo, RefreshCw } from "lucide-react";
```

```
function generateKmpSteps(pattern: string, text: string): any[] {
  if (!pattern) pattern = " ";
  const s = pattern + "#" + text;
  const n = s.length;
  const pi = new Array(n).fill(0);
  const steps: any[] = [];

  steps.push({ i: 0, j: 0, pi: [...pi], desc: `Начало` });

  for (let i = 1; i < n; i++) {
    let j = pi[i - 1];

    steps.push({ i, j, pi: [...pi], highlight: [i, j] });

    while (j > 0 && s[i] !== s[j]) {
      steps.push({ i, j, pi: [...pi], highlight: [i, j] });
      // ид назад: откат j = pi[j - 1] = pi[j - 1]
      j = pi[j - 1];
    }

    if (s[i] === s[j]) {

```

```

        steps.push({ i, l, r, zTarget: z[i], zSource: i
while (i + z[i] < n && s[z[i]] === s[i + z[i]])
    steps.push({ i, l, r, zTarget: z[i], zSource: i
    падают! Окно растёт.` }));
    z[i]++;
}

if (i + z[i] < n) {
    steps.push({ i, l, r, zTarget: z[i], zSource: i
    [i]]}' разные. Стоп.` }));
} else {
    steps.push({ i, l, r, z: [...z], desc: `Упёр
}

steps.push({ i, l, r, z: [...z], desc: `Фиксиру

if (i + z[i] - 1 > r) {
    l = i;
    r = i + z[i] - 1;
    steps.push({ i, l, r, z: [...z], desc: ` 0
}

if (z[i] === pattern.length) {
    steps.push({ i, l, r, z: [...z], found: true
}
}
return { s, steps, patternLength: pattern.length };
}

```

```

export function StringAlgorithmsViz({ defaultMode = "kmp"
const [mode, setMode] = useState<"kmp" | "z">(defaultMode);
const [pattern, setPattern] = useState("aba");
const [text, setText] = useState("abacaba");

```

```

// Ensure inputs are valid dynamically
const safePattern = pattern || " ";
const safeText = text || " ";

```

```

<div className="flex items-center justify-b
  <div className="flex bg-slate-900 borde
    <button onClick={() => setMode("kmp
      className={`px-3 py-1.5 text-xs
e" : "text-slate-400 hover:text-slate-300"}
      КМП (π-ФУНКЦИЯ)
    </button>
    <button onClick={() => setMode("z")
      className={`px-3 py-1.5 text-xs
" : "text-slate-400 hover:text-slate-300"}`
      Z-ФУНКЦИЯ
    </button>
  </div>

  <div className="flex items-center gap-2
    <div className="bg-slate-800 p-1 fl
      <span className="text-[10px] te
      <input value={pattern} onChange
xt-white text-xs font-bold font-mono px-2 p
    </div>
    <div className="bg-slate-800 p-1 fl
      <span className="text-[10px] te
      <input value={text} onChange={e
te text-xs font-bold font-mono px-2 py-1 ou
    </div>
  </div>
</div>

<div className="bg-slate-950 rounded-xl bor
px] flex items-center justify-start">
  <div className="flex gap-1.5 px-4 h-ful
    {s.split("").map((char, index) => {
      const isPattern = index < patte
      const isSeparator = index === p

      let borderColor = "border-slate
      let bgColor = "bg-slate-900";
      let textColor = isSeparator ? "

```



```

        </button>
        <button onClick={reset} className="border border-slate-800">
            <RefreshCw strokeWidth={3} size={24} />
        </button>
    </div>

    {/* Description Log */}
    <div className="flex-1 bg-slate-900/50 overflow-hidden">
        <div className="absolute top-0 right-0">
            <span className="font-mono text-sm">
                {steps.length}
            </span>
        </div>
        <div className="text-[10px] text-slate-400">
            {steps.map((step, index) => (
                <div className="text-sm text-slate-400">
                    {step.desc}
                </div>
            ))}
        </div>
    </div>
</div>

{/* Why this matters card */}
<div className="bg-slate-800 p-4 rounded-xl">
    <span className="text-2xl mt-1"> </span>
    <p className="text-xs text-slate-400">
        Обрати внимание на хитрость с решёткой.
        Мы пишем <b>Шаблон + # + Текст</b>.
        Когда значени {mode === 'kmp' ? "проблема была найдена!" : ""}
        Пройди шаги вручную, посмотри, как работает алгоритм для пропуска работы (в Z).
    </p>
</div>

{/* Code Sneak Peek */}
<div className="bg-slate-900 border border-slate-800">
    <div className="bg-slate-800 px-4 py-2 border border-slate-400 uppercase">

```

```

    side?: "top" | "bottom";
    className?: string;
  }

/**
 * Всплывающая подсказка (как title="", только красивая
 * Появляется по наведению курсора и по фокусу с клавиатуры
 */
export const Tooltip: React.FC<TooltipProps> = ({ content, ...props }) => {
  const [open, setOpen] = useState(false);

  const hiddenShift = side === "top" ? "translate-y-1" : "translate-y-0";

  return (
    <span
      className={`relative inline-flex ${className ?? ""}
        onMouseEnter={() => setOpen(true)}
        onMouseLeave={() => setOpen(false)}
        onFocus={() => setOpen(true)}
        onBlur={() => setOpen(false)}
      >
      {children}
      <span
        role="tooltip"
        className={`pointer-events-none absolute left-1/2 top-1/2
          bg-slate-900 px-3 py-2 text-left text-[11px]
          duration-150 ${
            side === "top" ? "bottom-full mb-2" : "top-full mt-2"
          }
          ${open ? "visible translate-y-0 opacity-100" : "invisible"}
        >
        {content}
        <span
          className={`absolute left-1/2 h-2 w-2 -translate-x-1/2
            side === "top" ? "-bottom-[5px] border-b border-slate-500" :
            "top-[5px] border-t border-slate-500"
          }`
        />
      </span>
    </span>
  );
};

```

```
const dagEdges: TEdge[] = [
  { u: 0, v: 1 },
  { u: 0, v: 2 },
  { u: 1, v: 3 },
  { u: 2, v: 3 },
  { u: 3, v: 4 },
];

interface TStep {
  desc: string;
  visited: Set<number>;
  fullyProcessed: Set<number>;
  currentNode: number | null;
  topoList: number[];
  dfsStack: number[];
}

export const TopologicalSortViz: React.FC = () => {
  const [steps, setSteps] = useState<TStep[]>([]);
  const [currentStepIdx, setCurrentStepIdx] = useState(0);
  const [isPlaying, setIsPlaying] = useState(false);

  useEffect(() => {
    const listSteps: TStep[] = [];
    const visited = new Set<number>();
    const fullyProcessed = new Set<number>();
    const topoList: number[] = [];
    const dfsStack: number[] = [];

    const recordStep = (desc: string, currentNode: number) => {
      listSteps.push({
        desc,
        visited: new Set(visited),
        fullyProcessed: new Set(fullyProcessed),
        currentNode,
        topoList: [...topoList],
        dfsStack: [...dfsStack],
      });
    };
  });
};
```

```

    });
  };

  // Run DFS for all unvisited
  for (let i = 0; i < 5; i++) {
    if (!visited.has(i)) {
      recordStep(
        `Запускаем новый обход DFS от нетронутой верши`,
        null,
      );
      dfs(i);
    }
  }

  recordStep(
    "Топологическая сортировка завершена! Полученный",
    null,
  );
  setSteps(listSteps);
  setCurrentStepIdx(0);
}, []);

const step = steps[currentStepIdx] || {
  desc: "Запуск...",
  visited: new Set(),
  fullyProcessed: new Set(),
  currentNode: null,
  topoList: [],
  dfsStack: [],
};

useEffect(() => {
  let timer: NodeJS.Timeout;
  if (isPlaying && currentStepIdx < steps.length - 1) {
    timer = setInterval(() => {
      setCurrentStepIdx((prev) => prev + 1);
    }, 1600);
  } else {

```



```

        </button>

        <button
            onClick={reset}
            className="flex items-center justify-center
            ors"
        >
            <RotateCcw className="w-4 h-4" />
        </button>
    </div>
</div>

<div className="grid grid-cols-1 lg:grid-cols-12 "
    {/* Playfield SVG */}
    <div className="lg:col-span-8 bg-[#0B1120] rela
        <svg className="w-full h-full max-w-[650px]"
            <defs>
                <marker
                    id="dag-arrow"
                    viewBox="0 0 10 10"
                    refX="24"
                    refY="5"
                    markerWidth="6"
                    markerHeight="6"
                    orient="auto-start-reverse"
                >
                    <path d="M 0 1 L 10 5 L 0 9 z" fill="#4
                </marker>
                <marker
                    id="dag-arrow-active"
                    viewBox="0 0 10 10"
                    refX="24"
                    refY="5"
                    markerWidth="6"
                    markerHeight="6"
                    orient="auto-start-reverse"
                >
                    <path d="M 0 1 L 10 5 L 0 9 z" fill="#8

```

```

    if (isCurrent) {
      fill = "#312e81";
      stroke = "#818cf8";
      textColor = "text-white font-bold";
    } else if (isProcessed) {
      fill = "#022c22";
      stroke = "#059669";
      textColor = "text-emerald-300";
    } else if (isVisited) {
      fill = "#3730a3";
      stroke = "#6366f1";
      textColor = "text-indigo-200";
    }

    return (
      <g key={node.id}>
        <rect
          x={node.x - 55}
          y={node.y - 22}
          width="110"
          height="44"
          rx="8"
          fill={fill}
          stroke={stroke}
          strokeWidth="2"
          className="transition-all duration-500"
        />
        <text
          x={node.x}
          y={node.y - 4}
          textAnchor="middle"
          fill="white"
          fontSize="11px"
          fontWeight="bold"
          className="pointer-events-none"
        >
          {{node.label}} {node.name.split(" "
        </text>
      )
    )
  }
}

```

```

        <span className="text-slate-600 text-
            ожидаем обработку узлов...
        </span>
    ) : (
        getTopoListString()
    })
</div>
</div>

{/* Color key */}
<div className="space-y-1">
    <span className="text-[10px] text-slate-5
        ВЕРШИНЫ В DFS:
    </span>
    <div className="flex items-center gap-2 t
        <span className="w-3.5 h-3.5 rounded bg
        <span className="text-slate-400 text-[1
            Серые (зашли)
        </span>
    </div>
    <div className="flex items-center gap-2 t
        <span className="w-3.5 h-3.5 rounded bg
        <span className="text-slate-400 text-[1
            Черные (готово)
        </span>
    </div>
</div>
</div>

<div className="mt-4 pt-3 border-t border-sla
    <span>
        Шаг {currentStepIdx + 1} из {steps.length}
    </span>
    <div className="flex gap-1">
        <button
            disabled={currentStepIdx === 0}
            onClick={() => setCurrentStepIdx((prev)
            className="bg-slate-900 border border-s

```

```
-----
import { JohnsonViz } from "./JohnsonViz";
import { KosarajuViz } from "./KosarajuViz";
import { KruskalSimulator } from "./KruskalSimulator";
import MnemonicCards from "./MnemonicCards";
import { PlanarityDemo } from "./PlanarityDemo";
import { QueueViz } from "./QueueViz";
import { SalmonAutomatonWidget } from "./SalmonAutomaton";
import { SegmentTreeVisualizer } from "./SegmentTreeViz";
import { SplayTreeViz } from "./SplayTreeViz";
import { SplayRotationsViz } from "./SplayRotationsViz";
import { SplayRotationSandbox } from "./SplayRotationSandbox";
import { StackViz } from "./StackViz";
import { StringAlgorithmsViz } from "./StringAlgorithmsViz";
import { TopologicalSortViz } from "./TopologicalSortViz";
import { WaterfallAnimationWidget } from "./WaterfallAnimation";
import ChapterImage from "./ChapterImage";
import { Simulator } from "./Simulator";
import { PrefixSumViz } from "./visualizers/PrefixSumViz";
import { SparseTableViz } from "./visualizers/SparseTableViz";
```

```
/** Разбор поворота и песочница всегда идут парой: посмотри
**
```

```
 * Анимация и песочница идут друг под другом, а не в две
 * колонке дерево сжималось до нечитаемого размера, а по
 * компактности. Между ними – подпись, объясняющая перемену
 */
```

```
const SplayRotationsPair: React.FC = () => (
  <div className="space-y-4">
    <SplayRotationsViz />
    <p className="flex items-start gap-2 text-[12px] leading-3 py-2">
      <span aria-hidden="true"> </span>
      <span>
        Разобрался – проверь себя: ниже то же самое дерево
        не решишь.
      </span>
    </p>
    <SplayRotationSandbox />
  </div>
);
```

```

    title: "Дерево отрезков",
    hint: "Запросы и обновления на отрезке за  $O(\log n)$ .
    Component: SegmentTreeVisualizer,
},
"sparse-table": {
    title: "Разреженная таблица (1D и 2D)",
    hint: "Наведите курсор на любую ячейку – подсветится.
    Component: SparseTableViz,
},
"prefix-sums-2d": {
    title: "Префиксные суммы (1D и 2D)",
    hint: "Наведите на число – увидите, из чего оно сложилось.
    Component: PrefixSumViz,
},
"dynamic-programming": {
    title: "Динамическое программирование",
    hint: "Таблица подзадач заполняется на глазах.",
    Component: DPVisualizer,
},
"splay-tree": {
    title: "Splay-дерево",
    hint: "Покадровый разбор трёх поворотов, песочница
    Component: () => (
        <div className="space-y-10">
            <SplayRotationsPair />
            <SplayTreeViz />
        </div>
    ),
},
"splay-rotations": {
    title: "Splay: Zig, Zig-Zig и Zig-Zag по шагам",
    hint: "Сверху – покадровый разбор поворота, снизу –
    Component: () => <SplayRotationsPair />,
},
"graph-dfs-bfs": { title: "DFS и BFS на графе", Component:
"top-sort": { title: "Топологическая сортировка", Component:
"scc-kosaraju": { title: "Компоненты сильной связности", Component:
"graph-articulation": { title: "Точки сочленения", Component:

```

```
    title: "Ахо-Корасик",
    hint: "Бор, суффиксные ссылки и BFS-обход по уровням",
    Component: () => (
      <div className="space-y-10">
        <WaterfallAnimationWidget />
        <SalmonAutomatonWidget />
      </div>
    ),
  },
intro: { title: "Мнемокарточки", Component: MnemonicCards },
"everyday-basics": {
  title: "Бытовой тренажёр: стек, очередь, куча",
  hint: "Тарелки, очередь в столовой и мешок гипотез",
  Component: Simulator,
},
};

export const getViz = (id?: string): VizEntry | undefined;
```

ФАЙЛ 72/86: src/components/WaterfallAnimationWidget.tsx
КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 10

```
import React, { useState } from 'react';
import { ArrowRight, CheckCircle2, AlertCircle, Lightbulb } from 'lucide-react';

// Структура заранее подготовленного красивого бора для Waterfall
// Словарь: ["HE", "SHE", "HIS", "HERS"]
interface WNode {
  id: number;
  label: string; // путь от корня
  char: string;
  x: number; // Координаты для SVG водопада
  y: number;
  depth: number;
  fail: number;
```

```

const [activeMode, setActiveMode] = useState<'training' | 'search'>('training');

// === СОСТОЯНИЯ РЕЖИМА ПОИСКА (SEARCH) ===
const [textToScan, setTextToScan] = useState<string>('');
const [currentIndex, setCurrentIndex] = useState<number>(0);
const [currentNode, setCurrentNode] = useState<number>(0);
const [isSearchDone, setIsSearchDone] = useState<boolean>(false);
const [searchLog, setSearchLog] = useState<string>('');
  'Режим поиска: Введите текст и жмите «Следующий шаг»';
);
const [jumpPath, setJumpPath] = useState<{ from: number; to: number }>(null);
const [foundMatches, setFoundMatches] = useState<{ index: number; match: string }>([]);
const [activeSearchCodeLine, setActiveSearchCodeLine] = useState<number>(1);

// === СОСТОЯНИЯ РЕЖИМА ОБУЧЕНИЯ (TRAINING) ===
// Шаги обучения от 0 до 8 (по числу вершин в боре, и т.д.)
const [trainStep, setTrainStep] = useState<number>(0);

const handleTextChange = (e: React.ChangeEvent<HTMLInputElement>) => {
  const clean = e.target.value.toUpperCase().replace(/[^a-zA-Z ]/g, '');
  setTextToScan(clean);
  resetSearchSimulation();
};

const resetSearchSimulation = () => {
  setCurrentIndex(0);
  setCurrentNode(0);
  setIsSearchDone(false);
  setSearchLog('Симуляция сброшена. Лосось вернулся на базу');
  setJumpPath(null);
  setFoundMatches([]);
  setActiveSearchCodeLine(1);
};

const stepSearchSimulation = () => {
  if (currentIndex >= textToScan.length) {
    setIsSearchDone(true);
    setSearchLog('Мы проплыли весь текст! Лосось доволен');
  }
};

```

```

const currentMatches: { index: number; word: string }[] = [];
const matchedWordsForLog: string[] = [];

let temp = curr;
while (temp !== 0) {
    if (WATERFALL_NODES[temp].is_terminal) {
        WATERFALL_NODES[temp].words.forEach((word) => {
            currentMatches.push({ index: currentIndex, word: word });
            matchedWordsForLog.push(word);
        });
    }
    temp = WATERFALL_NODES[temp].term_link;
}

if (matchedWordsForLog.length > 0) {
    logMsg += `    БИНГО! Найдено слово: ${matchedWordsForLog.join(' ')}\n`;
    setActiveSearchCodeLine(4);
}

setCurrentNode(curr);
setCurrentIndex(currentIndex + 1);
setSearchLog(logMsg);
if (currentMatches.length > 0) {
    setFoundMatches((prev) => [...prev, ...currentMatches]);
}
};

// ПОЛНАЯ ПОШАГОВАЯ ИНФОРМАЦИЯ О ПОСТРОЕНИИ ДЕРЕВА (В
const TRAINING_STEPS_INFO = [
    {
        targetNodeId: 0,
        title: 'Шаг 0 (Depth 0): Вершина #0 (ROOT)',
        desc: 'Начало алгоритма BFS. Корень (ROOT) не обрабатывается. Другие вершины сразу инициализируются с fail = ROOT',
        codeLine: 1,
        scoutPos: 0,
        inspectedParentFail: 0,
    },

```



```

    title: 'Шаг 4 (Depth 2): Вершина #5 (HI)',
    desc: 'Обрабатываем #5 (HI) на Depth 2. Родитель :
        H" и "S" не подходят. Ветки нет, fail[HI] = ROOT',
    codeLine: 3,
    scoutPos: 0,
    inspectedParentFail: 0,
    checkTargetChar: 'I',
    checkingBranchesFrom: 0,
},
{
    targetNodeId: 8,
    title: 'Шаг 5 (Depth 2): Вершина #8 (SH) — КАК СТАТЬ
    desc: '    ВАЖНЫЙ ВОПРОС! Обрабатываем #8 (SH) на D
        ). Рыба плавает в ROOT и ищет ветку "H". У корня
        суффикс "H"!',
    codeLine: 4,
    scoutPos: 0,
    inspectedParentFail: 0,
    checkTargetChar: 'H',
    checkingBranchesFrom: 0,
},
{
    targetNodeId: 3,
    title: 'Шаг 6 (Depth 3): Вершина #3 (HER)',
    desc: 'Переходим на Depth 3 к #3 (HER). Родитель :
        е подходят. fail[HER] = ROOT.',
    codeLine: 3,
    scoutPos: 0,
    inspectedParentFail: 0,
    checkTargetChar: 'R',
    checkingBranchesFrom: 0,
},
{
    targetNodeId: 6,
    title: 'Шаг 7 (Depth 3): Вершина #6 (HIS)',
    desc: 'Обрабатываем #6 (HIS) на Depth 3. Родитель :
        но ветка "S" → #7 (S) совпадает! fail[HIS] = #7
    codeLine: 4,

```

```

    <span className="text-3xl"> </span>
  <div>
    <h4 className="text-2xl font-bold text-white">
      Анимация водопада: Полное обучение автомата
    </h4>
    <p className="text-sm text-slate-400">
      Визуализация пошагового построения всех ф
    </p>
  </div>
</div>

{/* Переключатель режимов */}
<div className="bg-slate-900 p-1.5 rounded-xl b
  <button
    onClick={() => setActiveMode('training')}
    className={`flex items-center space-x-1.5 p-
      activeMode === 'training'
        ? 'bg-indigo-600 text-white shadow-lg s
        : 'text-slate-400 hover:text-white'
    `}
  >
    <GitBranch className="w-4 h-4" />
    <span>Режим 1: Полное обучение (BFS от корн
  </button>
  <button
    onClick={() => setActiveMode('search')}
    className={`flex items-center space-x-1.5 p-
      activeMode === 'search'
        ? 'bg-blue-600 text-white shadow-lg sha
        : 'text-slate-400 hover:text-white'
    `}
  >
    <Play className="w-4 h-4" />
    <span>Режим 2: Поиск в тексте</span>
  </button>
</div>
</div>

```

```

ion-all"
  >
    Назад
  </button>
  <button
    onClick={() => setTrainStep((prev) => M
    disabled={trainStep === TRAINING_STEPS_
    className="flex-1 bg-indigo-600 hover:b
  ition-all shadow-lg shadow-indigo-900/50"
  >
    {trainStep === TRAINING_STEPS_INFO.leng
  </button>
</div>
</div>

{/* Блок с кодом построения fail-ссылки и опи
<div className="lg:col-span-2 bg-slate-900/90
  >
    <div>
      <h5 className="text-white font-bold mb-3
        <span className="flex items-center gap-
          <span>❌</span> Код BFS построения fail
        </span>
        <span className="text-xs bg-indigo-950
          Вершина: #{targetTrainingNode.id} ({t
        </span>
      </h5>
      <div className="bg-slate-950 p-4 rounded-
        <div className={`p-1.5 rounded flex ite
        -l-4 border-indigo-500 text-indigo-300 font
          <span>1. let u = trie[r][char]; // Те
            {currentTrainInfo.codeLine === 1 && <
          тут</span>}
        </div>
        <div className={`p-1.5 rounded flex ite
        -l-4 border-indigo-400 text-indigo-200 font
          <span>2. let v = fail[r]; // Подъем к
            ainInfo.inspectedParentFail}.label}}</span>

```

```

<div className="grid grid-cols-1 lg:grid-cols-3"
  <div className="bg-slate-900/60 p-5 rounded-x
    <div>
      <h5 className="text-white font-bold mb-2"
        <span>» </span> Текст для сканирования
      </h5>
      <p className="text-xs text-slate-400 mb-4"
        Введи текст, чтобы лосось просканировал
      </p>
      <input
        type="text"
        value={textToScan}
        onChange={handleTextChange}
        maxLength={15}
        placeholder="ВВЕДИТЕ ТЕКСТ..."
        className="w-full bg-slate-800 border b
        cus:outline-none focus:border-blue-500"
      />
    </div>

    <div>
      <div className="mb-4 flex flex-wrap gap-1"
        <span className="text-xs text-slate-400"
          {textToScan.split('').map((char, idx) =>
            <span
              key={idx}
              className={`w-7 h-8 flex items-cent
                idx === currentIndex
                  ? 'bg-blue-600 text-white shadow
                  : idx < currentIndex
                  ? 'bg-emerald-950/60 text-emera
                  : 'bg-slate-800 text-slate-500'
                }`}
            >
              {char}
            </span>
          )}}
        </div>
      </div>
    </div>
  </div>
  {currentIndex >= textToScan.length && (

```

```

        ружок по fail!</span>}
      </div>
      <div className={`p-1.5 rounded flex item
border-blue-400 text-blue-200 font-bold` :
        <span>3. curr = trie[curr].get(char)
        {activeSearchCodeLine === 3 && <span>
ем по течению</span>}
      </div>
      <div className={`p-1.5 rounded flex item
l-4 border-emerald-500 text-emerald-300 font
        <span>4. if (is_terminal[curr]) report
        {activeSearchCodeLine === 4 && <span>
овпадение!</span>}
      </div>
    </div>
  </div>

  <div>
    <h5 className="text-slate-300 font-bold m
    <AlertCircle className="w-3.5 h-3.5 tex
    </h5>
    <div className="bg-slate-800 p-3.5 rounded
tems-center">
      {searchLog}
    </div>
  </div>
</div>
))

```

```

{ /* SVG Анимация водопада */}
<div className="bg-gradient-to-b from-blue-950 vi
-hidden shadow-2xl">
  { /* Водопадные фоновые эффекты */}
  <div className="absolute inset-0 opacity-10 bg-
o-900 to-transparent pointer-events-none"></d
  <div className="absolute top-0 left-1/2 -transl
    <div className="w-8 bg-blue-400 h-full animat

```

```

        strokeDasharray="4,4"
        opacity="0.4"
      />
      <rect x="25" y={level.y - 12} width="14" height="24" />
    />
    <text x="32" y={level.y + 4} fill="#94a0b4" />
      {level.label}
    </text>
  </g>
)}}
```

```

{/* Рендер прямых переходов (потоков воды) */}
Object.entries(TRIE_TRANSITIONS).map(([fromId, toId]) => {
  const fromNode = WATERFALL_NODES[Number(fromId)];
  return Object.entries(children).map(([className, node]) => {
    const toNode = WATERFALL_NODES[toId];

    // Подсветка в режиме поиска
    let isHighlighted = activeMode === 'search' ?
    type === 'normal';

    // Подсветка в режиме обучения (когда ищем ответ)
    let isTrainingExamined = activeMode === 'train' ?
    let isMatchBranch = isTrainingExamined &amp; toNode.label === node.label;

    return (
      <g key={`edge-${fromNode.id}-${toNode.id}`} />
      <rect key={`edge-${fromNode.id}-${toNode.id}`} />
      <line
        x1={fromNode.x}
        y1={fromNode.y}
        x2={toNode.x}
        y2={toNode.y}
        stroke={isHighlighted ? '#3b82f6' : '#ccc'}
        strokeWidth={isHighlighted || isTrainingExamined ? 2 : 1}
        className={isHighlighted || isTrainingExamined ? 'highlighted' : ''}
      />
      <rect key={`edge-${fromNode.id}-${toNode.id}`} />
    )
  )
})

```

```

        let isHighlighted = activeMode === 'search' ||
e === 'fail';
        let isJustEstablished = activeMode === 'transition' ||
-ссылку

// Вычисляем изогнутую кривую для красоты
const dx = targetNode.x - node.x;
const dy = targetNode.y - node.y;
const cx = node.x + dx / 2 + (node.id % 2 === 0 ? -dx / 4 : dx / 4);
const cy = node.y + dy / 2 - 30;

return (
  <g key={`fail-${node.id}`}>
    <path
      d={`M ${node.x} ${node.y} Q ${cx} ${cy} ${targetNode.x} ${targetNode.y}`}
      fill="none"
      stroke={isHighlighted ? '#f43f5e' : '#000000'}
      strokeWidth={isHighlighted ? 4 : 2}
      strokeDasharray="6,6"
      className={isHighlighted ? 'highlighted' : ''}
      opacity={isHighlighted ? 1 : 0.5}
    />
    {isJustEstablished && (
      <text x={cx} y={cy - 10} fill="#10b98f">
        fail[{node.label}] = {targetNode.label}
      </text>
    )}
  </g>
);
}}
}}

```

```

{/* Рендер вершин (бассейнов водопада) */}
Object.values(WATERFALL_NODES).map((node) => {
  const isCurrent = activeFishPosNode.id === node.id;
  const isTargetChild = activeMode === 'transition' || activeMode === 'fail';
  const hasWords = node.words.length > 0;

```

```

        <text x="0" y="2" fontSize="10" f
            ИЩЕМ FAIL
        </text>
    </g>
}}

{/* Индикатор словарных слов */}
{hasWords && {
    <g transform={`translate(${node.x +
        <circle cx="0" cy="0" r="9" fill=
        <text x="0" y="3" fontSize="10" f
            *
        </text>
    </g>
}}
</g>
});
}}

{/* ОДИН ПЛАВНЫЙ АНИМИРОВАННЫЙ ЭХО-ЛОСОСЬ */}
<g>
    {/* Круг-подложка под рыбу */}
    <circle
        cx={activeFishPosNode.x + 15}
        cy={activeFishPosNode.y - 25}
        r="18"
        fill={activeMode === 'training' ? '#8b5
        stroke="#ffffff"
        strokeWidth="2"
        style={{ transition: 'cx 0.8s cubic-bez
        className="shadow-lg"
    />
    {/* Сама рыба */}
    <text
        x={activeFishPosNode.x + 15}
        y={activeFishPosNode.y - 19}
        fontSize="18"
        textAnchor="middle"

```



```
        </div>
      )}
    </div>
  );
};
```

РАЗДЕЛ: ВИЗУАЛИЗАТОРЫ (ВЛОЖЕННЫЕ)

ФАЙЛ 73/86: src/components/visualizers/PrefixSumViz.tsx
КАТЕГОРИЯ: Визуализаторы (вложенные) | СТРОК: 330 | РАЗМЕР: 10.5 КБ

```
import React, { useMemo, useState } from "react";

/**
 * Префиксные суммы: 1D и 2D.
 *
 * Главная фишка – наведение на число:
 * * в 1D наводим на P[i] – подсвечивается кусок массива
 * * в 1D наводим на a[i] – видно, в какие префиксы он входит
 * * в 2D наводим на ячейку – подсвечивается прямоугольник
 * а при выбранном запросе показывается формула включения-исключения
 */

const A1 = [3, 1, 4, 1, 5, 9, 2, 6];

const A2 = [
  [1, 2, 3, 4],
  [5, 6, 7, 8],
  [9, 1, 2, 3],
  [4, 5, 6, 7],
];

const cellBase =
```

```

const [hover, setHover] = useState<{ kind: "pref" | "a" }>(null)
const [range, setRange] = useState<{ l: number; r: number }>({ l: 0, r: 0 })

const covered =
  hover?.kind === "pref"
    ? { from: 0, to: hover.i - 1 }
    : hover?.kind === "a"
      ? { from: hover.i, to: hover.i }
      : null;

const sum = pref[range.r + 1] - pref[range.l];

return (
  <div className="space-y-5">
    <div className="overflow-x-auto pb-1">
      <div className="inline-block min-w-full">
        {/* индексы */}
        <div className="flex gap-1.5 mb-1 pl-[52px]">
          {A1.map((_, i) => (
            <div key={i} className={`${cellBase} !h-5`}>
              {i}
            </div>
          ))}
        </div>

        {/* массив a */}
        <div className="flex gap-1.5 items-center mb-1">
          <div className="w-[46px] shrink-0 text-right">
            {A1.map((v, i) => {
              const inCover = covered && i >= covered.from && i < covered.to
              const inRange = i >= range.l && i <= range.r
              return (
                <div
                  key={i}
                  onMouseEnter={() => setHover({ kind: "a", i })}
                  onMouseLeave={() => setHover(null)}
                  onClick={() => setRange({r} => (i < r ? i : r))}
                  className={`${cellBase} cursor-pointer`}
                >
                  {v}
                </div>
              )
            })}
          </div>
        </div>
      </div>
    </div>
  )
)

```

```

        </div>
    );
    }}}
</div>
</div>
</div>

{/* Пояснение под курсором */}
<div className="min-h-[62px] bg-slate-900 border
    {hover?.kind === "pref" ? {
        hover.i === 0 ? {
            <p className="text-slate-300">
                <b className="text-emerald-400">P[0] = 0<
                отрезка, начинающегося с нуля.
            </p>
        ) : {
            <p className="text-slate-300">
                <b className="text-emerald-400">
                    P[{hover.i}] = {pref[hover.i]}
                </b>{" "}
                – это сумма всего, что набежало от начала
                <span className="font-mono text-indigo-300">
                    a[0..{hover.i} - 1] = {A1.slice(0, hove
                </span>
            </p>
        )
    ) : hover?.kind === "a" ? {
        <p className="text-slate-300">
            <b className="text-indigo-400">
                a[{hover.i}] = {A1[hover.i]}
            </b>{" "}
            входит во все префиксы, начиная с{" "}
            <span className="font-mono text-emerald-300">
                подвинуть границу запроса.
            </span>
        </p>
    ) : {
        <p className="text-slate-500">
            Наведите курсор на любое число: y <b classN

```

```

const [hover, setHover] = useState<{ i: number; j: number }>({ i: 0, j: 0 });
const [rect, setRect] = useState({ r1: 1, c1: 1, r2: 1, c2: 1 });

const D = P[rect.r1][rect.c1];
const B = P[rect.r1][rect.c2 + 1];
const Cc = P[rect.r2 + 1][rect.c1];
const Aa = P[rect.r2 + 1][rect.c2 + 1];
const total = Aa - B - Cc + D;

const cornerOf = (i: number, j: number) => {
  if (i === rect.r2 + 1 && j === rect.c2 + 1) return "A";
  if (i === rect.r1 && j === rect.c2 + 1) return "B";
  if (i === rect.r2 + 1 && j === rect.c1) return "C";
  if (i === rect.r1 && j === rect.c1) return "D";
  return null;
};

return (
  <div className="space-y-5">
    <div className="grid gap-5 lg:grid-cols-2">
      {/* Исходная матрица */}
      <div className="overflow-x-auto">
        <div className="text-xs text-slate-400 mb-2">Исходная матрица
        <div className="inline-block">
          {A2.map((row, i) => {
            <div key={i} className="flex gap-1.5 mb-1">
              {row.map((v, j) => {
                const inRect = i >= rect.r1 && i <= rect.r2 && j >= rect.c1 && j <= rect.c2;
                return (
                  <div
                    key={j}
                    onClick={() => setRect((r) => {
                      i < r.r1 || j < r.c1 ? { r1: i, c1: j, r2: i, c2: j } : r
                    })}
                    className={` ${cellBase} cursor-pointer ${inRect ? "bg-slate-200" : ""}` />
                )
              })}
            </div>
          })}
        </div>
      </div>
    </div>
  </div>
);

```

```

        {v}
        {corner && !isHover && {
            <span className="absolute -top-
ht">
                {corner}
            </span>
        }}
    </div>
    );
    }}}
    </div>
    )))
    </div>
    </div>
    </div>

<div className="min-h-[54px] bg-slate-900 border
    {hover ? {
        <p className="text-slate-300">
            <b className="text-amber-400">
                 $P[\{hover.i\}][\{hover.j\}] = \{P[hover.i][hov$ 
            </b>{" "}
            – сумма всего прямоугольника от левого верх
        </p>
    } : {
        <p className="text-slate-500">Наведите курсор
    }}
    </div>

<div className="bg-slate-900 border border-indigo
    <p className="font-mono text-sm sm:text-base te
        сумма = <span className="text-emerald-400">A<
        <span className="text-rose-400">C</span> + <s
        <span className="text-indigo-300 font-bold">{
    </p>
    <p className="text-[11px] text-slate-500 mt-1">
        Вычли два «хвоста» и вернули дважды вычитенный
    </p>

```

```

    [67, 19, 44, 55, 33, 21, 65, 87],
    [14, 51, 99, 13, 22, 64, 43, 76],
    [89, 32, 54, 71, 15, 88, 29, 60],
    [25, 41, 16, 92, 9, 17, 56, 31],
    [59, 18, 77, 24, 61, 82, 35, 12],
    [73, 8, 38, 85, 47, 95, 19, 58],
    [39, 81, 62, 28, 51, 42, 85, 14]
];

const ST2D: number[][][] = Array.from({length: 8}, () =>
    Array.from({length: 8}, () =>
        Array.from({length: 4}, () =>
            Array(4).fill(0)
        )
    )
);

for (let r = 0; r < 8; r++) {
    for (let c = 0; c < 8; c++) {
        ST2D[r][c][0][0] = val2D[r][c];
    }
}

for (let kx = 0; kx <= 3; kx++) {
    for (let ky = 0; ky <= 3; ky++) {
        if (kx === 0 && ky === 0) continue;
        for (let r = 0; r + (1 << kx) <= 8; r++) {
            for (let c = 0; c + (1 << ky) <= 8; c++) {
                if (kx > 0) {
                    ST2D[r][c][kx][ky] = Math.min(
                        ST2D[r][c][kx-1][ky],
                        ST2D[r + (1 << (kx-1))][c][kx-1][ky]
                    );
                } else {
                    ST2D[r][c][kx][ky] = Math.min(
                        ST2D[r][c][kx][ky-1],
                        ST2D[r][c + (1 << (ky-1))][kx][ky-1]
                    );
                }
            }
        }
    }
}

```

```

        ) : tab === "2d_build" ? (
          <Viz2DBuild key="2d_build" />
        ) : (
          <Viz2DQuery key="2d_query" />
        )}
      </AnimatePresence>
    </div>
  );
};

const Viz1D: React.FC = () => {
  const [step, setStep] = useState(0);
  const [hover, setHover] = useState<{ i: number; j: number }>({ i: 0, j: 0 });

  // Total steps: we animate computing each element for
  const computeSteps: { i: number; j: number }[] = [];
  for (let j = 1; j < LOG; j++) {
    for (let i = 0; i + (1 << j) <= N; i++) {
      computeSteps.push({ i, j });
    }
  }

  const maxStep = computeSteps.length;
  const covered = hover ? { from: hover.i, to: hover.i + (1 << hover.j) } : null;

  return (
    <div className="flex flex-col gap-5">
      <div className="flex items-center justify-between">
        <div>
          <h3 className="text-lg font-bold text-indigo-600">
            Построение Sparse Table (Min)
          </h3>
          <p className="text-sm text-slate-400">
            Формула:  $ST[i][j] = \min(ST[i][j-1], ST[i + (1 \ll (j-1))][j-1])$ 
          </p>
        </div>
        <div className="flex gap-2">
          <button

```

```

        </div>
    );
    }}}
</div>
<div className="mt-2 text-xs min-h-[36px]">
    {hover && covered ? (
        <p className="text-slate-300">
            <b className="text-sky-400">
                ST[{hover.i}][{hover.j}] = {stID[hover.i]}
            </b>{" "}
            – это минимум на отрезке{" "}
            <span className="font-mono text-sky-300">
                [{covered.from}, {covered.to}]
            </span>{" "}
            длины  $2^{\{hover.j\}}$  = {1 << hover.j}:{" "}
            <span className="font-mono text-slate-400">
                min({arrID.slice(covered.from, covered.to)}
            </span>
        </p>
    ) : (
        <p className="text-slate-500">
            Наведите курсор (или тапните) на любое чи
        </p>
    )}
</div>
</div>

<div className="overflow-x-auto pb-2">
    <div className="inline-block min-w-full bg-slate-500">
        <div className="grid grid-cols-[auto_repeat(4,1fr)]">
            {/* Headers */}
            <div className="text-slate-500 font-mono text-sm">
                i \ j
            </div>
            <div className="text-center font-bold text-slate-500">
                2^0 (L=1)
            </div>
        </div>
    </div>
</div>

```



```

        if (
            currentStep.j === j + 1 &&
            currentStep.i === i - (1 << (currentStep.j - j))
        ) {
            isSrc2 = true; // wait, no. Src2 is the source of the next step
            // Let's re-eval exact source:
            // We are asking if THIS cell [i][j] is the source of the next step
            if (currentStep.j - 1 === j) {
                if (currentStep.i === i) isSrc1 = true;
                if (currentStep.i + (1 << (currentStep.j - j)) === i)
                    isSrc2 = true;
            }
        }
    }

    return (
        <div key={j} className="relative flex flex-col items-center justify-center"
        {!isValid ? (
            <div className="w-[clamp(30px,9vw,17px)] h-[clamp(11px,3vw,17px)]"
        ) : (
            <motion.div
                onMouseEnter={() => isVisible = true}
                onMouseLeave={() => setHover(false)}
                onClick={() => isVisible && setHover(true)}
                initial={{ opacity: 0, scale: 0.8 }}
                animate={{
                    opacity: isVisible ? 1 : 0,
                    scale: isVisible ? 1 : 0.8,
                }}
                className={`w-[clamp(30px,9vw,17px)] h-[clamp(11px,3vw,17px)] font-bold transition-colors duration-200
                    hover && hover.i === i && hover.s === s
                    ? "bg-sky-500 text-white"
                    : isActive
                    ? "bg-indigo-500 text-white"
                    : isSrc1
                    ? "bg-emerald-500 text-white"
                    : isSrc2
                    ? "bg-amber-500 text-white"
                `}
            />
        )}
    )

```

```
</div>
```

```
{/* Formula helper text */}
```

```
<div className="bg-slate-900 border border-slate-100">
```

```
  {step > 0 && step <= maxStep ? (
```

```
    (() => {
```

```
      const c = computeSteps[step - 1];
```

```
      const half = 1 << (c.j - 1);
```

```
      return (
```

```
        <p className="text-lg text-slate-300">
```

```
          <span className="text-white font-bold">
```

```
            ST[{c.i}][{c.j}]
```

```
          </span>{" "}
```

```
          = min(
```

```
            <span className="text-emerald-400 font-l">
```

```
              ST[{c.i}][{c.j - 1}]
```

```
            </span>
```

```
          ,
```

```
            <span className="text-amber-400 font-bo">
```

```
              ST[{c.i + half}][{c.j - 1}]
```

```
            </span>
```

```
          ) &rarr; min(
```

```
            <span className="text-emerald-400">{st1D
```

```
            <span className="text-amber-400">
```

```
              {st1D[c.i + half][c.j - 1]}
```

```
            </span>
```

```
          ) ={" "}
```

```
            <span className="text-indigo-400 font-b">
```

```
              {st1D[c.i][c.j]}
```

```
            </span>
```

```
          </p>
```

```
        );
```

```
      ))()
```

```
    ) : (
```

```
      <p className="text-slate-500 italic">
```

```
        Нажмите далее, чтобы увидеть, как блоки сво
```

```
      </p>
```

```
    ))
```

```

rollbar">
{[...Array(k + 1)].map((_, i) => k - i).map((
  const stepL = 1 << step;
  const stepSize = 8 - stepL + 1;
  const isCurr = step === k;

  return (
    <React.Fragment key={step}>
      {idx > 0 && <div className="text-slate
      <div className={`flex flex-col items-c
      <h4 className={`font-bold mb-3 flex
    }>
      <span>Матрица блоков {stepL}x{step
      <span className={`text-[10px] font
rder-indigo-800/40 text-indigo-300' : 'bg-s
      </h4>
      <div className="max-w-full overflow-
      <div className={`grid gap-[2px] p-
go-900/40 shadow-[0_0_25px_rgba(99,102,241,
eColumns: `repeat(${stepSize}, max-content)
      {Array.from({length: stepSize * st
        const r = Math.floor(idx / step
        const c = idx % stepSize;

        let highlightClass = isCurr ? 'b
/50 cursor-pointer' : 'bg-slate-800/50 text
        let zIndex = 'z-0';

        const isActive = !!activeCell &
ll.c + L) && ((r - activeCell.r) % (1 << st

        if (isActive) {
          if (isCurr) {
            highlightClass = `bg-indigo
{locked && locked.r === r && locked.c === c
            zIndex = 'z-10';
          } else {
            const prevL = 1 << (k - 1);

```



```

                <span className="text
tiveCell.r][activeCell.c + prevL][k-1][k-1]
            </div>
            <div className="flex ite
                <span className="flex
-500 shadow-[0_0_8px_#10b981]"></div> ST[{a
                <span className="text
>{ST2D[activeCell.r + prevL][activeCell.c][
            </div>
            <div className="flex ite
                <span className="flex
shadow-[0_0_8px_#f59e0b]"></div> ST[{active
                <span className="text
[activeCell.r + prevL][activeCell.c + prevL
            </div>
        </>
    )
    }}({})
</div>

    <div className="pt-3 mt-3 border-t
    ) = <span className="text-white
0">{ST2D[activeCell.r][activeCell.c][k][k]}
    </div>
    <p className="mt-4 text-[12px] font
список матриц (слева) вниз, чтобы увидеть,
    </div>
    ) : {
        <div className="bg-slate-950/40 border
mt-2 flex items-center justify-center anim
            Наведите курсор на матрицу {L}x{L
        </div>
    }}
</div>
    )}
</div>
</div>
</div>

```

```

                {val2D[r][c]}
            </div>
        )
    }}}

    <div
        className="absolute border-[3px
ow-[0_0_15px_rgba(244,63,94,0.3)] border-da
        style={{
            top: `${(r1 / 8) * 100}%`,
            left: `${(c1 / 8) * 100}%`,
            marginTop: '8px',
            marginLeft: '8px',
            height: 'calc(' + (H/8*100) +
            width: 'calc(' + (W/8*100) +
        }}
    />

    /* Показываем 4 угла в самой матрице
    <div className="absolute pointer-e
        shadow-[0_0_10px_#f43f5e]" style={{ top: `
        lc(`${(Ly / 8) * 100}% - 16px)`, height: `ca
        <div className="absolute pointer-e
        shadow-[0_0_10px_#3b82f6]" style={{ top: `
        width: `calc(`${(Ly / 8) * 100}% - 16px)`,
        <div className="absolute pointer-e
        ld-400 shadow-[0_0_10px_#10b981]" style={{
        8px)`, width: `calc(`${(Ly / 8) * 100}% - 16
        <div className="absolute pointer-e
        00 shadow-[0_0_10px_#f59e0b]" style={{ top:
        00}% + 8px)`, width: `calc(`${(Ly / 8) * 100
        </div>

    <div className="bg-slate-950 p-4 round
er-slate-800 shadow-[inset_0_2px_10px_rgba(
        <label className="flex items-cen
            <span className="text-slate-
                <div className="flex gap-2">

```

```

        <span className="text-purple-400">int<
ext-amber-300">1</span>); <span className="
        <span className="text-purple-400">int<
"text-slate-500">// Выс: {Lx}</span><br/>
        <span className="text-purple-400">int<
"text-slate-500">// Шир: {Ly}</span><br/><b

        <span className="text-slate-500">// 2.
        <span className="text-indigo-400">int<
        &nbsp;&nbsp;&nbsp;<span className="text-slate-500">
В запроса)</span><br/>
        &nbsp;&nbsp;&nbsp;<span className="text-rose-400">
r1</span>][<span className="text-white bg-r

        &nbsp;&nbsp;&nbsp;<span className="text-slate-500">
во, чтобы правый край уперся в c2)</span><b
        &nbsp;&nbsp;&nbsp;<span className="text-blue-400">
r1</span>][<span className="text-white bg-b

        &nbsp;&nbsp;&nbsp;<span className="text-slate-500">
px, чтобы нижний край уперся в r2)</span><b
        &nbsp;&nbsp;&nbsp;<span className="text-emerald-400">
nded">r2 - Lx + 1</span>][<span className="

        &nbsp;&nbsp;&nbsp;<span className="text-slate-500">
и влево от c2)</span><br/>
        &nbsp;&nbsp;&nbsp;<span className="text-amber-400">
">r2 - Lx + 1</span>][<span className="text
        {'}}'}});
    </div>

    <div className="w-full flex justify-centr
    <div className="flex flex-col items-cent
    <h4 className="text-slate-400 font-bo
        Откуда берутся эти 4 числа: матриц
border-slate-700 shadow-sm">ST[][][{{kx}}][{{k
    </h4>
    <div className="grid gap-[2px] p-2.5

```

```
        wrap bg-[#0a0f1e] py-3 px-2 rounded-lg border
            <span className="text-slate-400 font
                <span className="text-rose-400">
                <span className="text-blue-400 m
            n>
                <span className="text-emerald-400
            span>
                <span className="text-amber-400 m
                &nbsp;<span className="text-slate-400
                <span className="ml-3 text-emerald-
+ 1][c1][kx][ky], ST2D[r2 - Lx + 1][c2 - Ly
            </div>
        </div>
    </div>
</div>
    )
}
```

РАЗДЕЛ: HTML-РАЗМЕТКА (LAYOUT)

ФАЙЛ 75/86: src/layout/footer.html

КАТЕГОРИЯ: HTML-разметка (layout) | СТРОК: 137 | РАЗМЕР

```
        </div>
    </main>
</div>

<script>
    function setMode(mode) {
        document.getElementById('btn-mode-normal').
            ? 'px-3 py-1 rounded text-sm font-bold
```



```

    });

    if (theme === 'dark') document.getElementById('dark-mode')?.classList.add('dark');
    if (theme === 'light') document.getElementById('dark-mode')?.classList.remove('dark');
    if (theme === 'terminal') document.getElementById('dark-mode')?.classList.add('terminal');
  });

  document.getElementById('btn-theme-dark').addEventListener('click', () => {
    document.getElementById('dark-mode')?.classList.add('dark');
    document.getElementById('dark-mode')?.classList.remove('light');
    document.getElementById('dark-mode')?.classList.remove('terminal');
    document.getElementById('dark-mode')?.classList.add('transition-colors');
  });

  document.getElementById('btn-theme-light').addEventListener('click', () => {
    document.getElementById('dark-mode')?.classList.remove('dark');
    document.getElementById('dark-mode')?.classList.add('light');
    document.getElementById('dark-mode')?.classList.remove('terminal');
    document.getElementById('dark-mode')?.classList.add('transition-colors');
  });

  document.getElementById('btn-theme-notebook').addEventListener('click', () => {
    document.getElementById('dark-mode')?.classList.remove('dark');
    document.getElementById('dark-mode')?.classList.remove('light');
    document.getElementById('dark-mode')?.classList.add('terminal');
    document.getElementById('dark-mode')?.classList.add('transition-colors');
  });
}

// Initialize theme and mode
setTheme('dark');

// Setup intersection observer for TOC highlighting
document.addEventListener('DOMContentLoaded', () => {
  const mobileMenuBtn = document.getElementById('mobile-menu');
  const mobileDrawer = document.getElementById('mobile-drawer');
  const closeDrawerBtn = document.getElementById('close-drawer');
  const drawerOverlay = document.getElementById('drawer-overlay');

  function toggleDrawer() {
    const isClosed = mobileDrawer.classList.contains('closed');
    if (isClosed) {
      mobileDrawer.classList.remove('closed');
      if (drawerOverlay) {
        drawerOverlay.classList.remove('hidden');
        // Timeout allows display:block to take effect
        setTimeout(() => drawerOverlay.classList.remove('hidden'), 100);
      }
    } else {
      mobileDrawer.classList.add('closed');
      drawerOverlay?.classList.add('hidden');
    }
  }

  mobileMenuBtn.addEventListener('click', toggleDrawer);
  closeDrawerBtn.addEventListener('click', toggleDrawer);
  drawerOverlay?.addEventListener('click', toggleDrawer);
});

```

```
        // Highlight active TOC link lo
        const id = entry.target.id;
        document.querySelectorAll('aside
            if (link.getAttribute('href
                link.classList.add('bg-
            } else {
                link.classList.remove('l
            }
        });
    });
}, observerOptions);

document.querySelectorAll('section[id^="que
    observer.observe(section);
});
});
</script>
</body>
</html>
```

ФАЙЛ 76/86: src/layout/header.html

КАТЕГОРИЯ: HTML-разметка (layout) | СТРОК: 1169 | РАЗМЕР: 1000

```
<!DOCTYPE html>
<html lang="ru" class="scroll-smooth">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width,
  <title>Пособие по алгебре: От пиццы к предикатам</t
  <script src="https://cdn.tailwindcss.com"></script>
  <script src="https://polyfill.io/v3/polyfill.min.js
  <script id="MathJax-script" async src="https://cdn.
  <script>
    window.MathJax = {
```



```
        color: #3b2818 !important; /* brown ink */
        background-image: repeating-linear-gradient(
        background-attachment: local !important;
        font-family: "Comic Sans MS", "Caveat", "Serif";
    }
body.theme-notebook .bg-slate-900, body.theme-notebook .bg-slate-700 {
    background-color: rgba(255, 255, 255, 0.7);
    border-color: #cbd5e1 !important;
    box-shadow: 2px 2px 5px rgba(0,0,0,0.05);
}
body.theme-notebook .text-slate-200, body.theme-notebook .text-slate-400 { color: #cbd5e1 !important; }
body.theme-notebook .text-slate-400 { color: #546e7a !important; }
body.theme-notebook .border-slate-600, body.theme-notebook .border-slate-800 { border-color: #94a3b8 !important; }
body.theme-notebook .font-mono { font-family: "Monospace"; }
body.theme-notebook #top-controls { background-color: #f1f3f4; }

/* CREEPY BLINKING */
@keyframes creepy-blink {
    0%, 95%, 98%, 100% { transform: scaleY(1); }
    96.5% { transform: scaleY(0.1); }
}
.creepy-eye {
    transform-origin: center;
    transform-box: fill-box;
    animation: creepy-blink 4s infinite;
}
.creepy-eye-alt {
    transform-origin: center;
    transform-box: fill-box;
    animation: creepy-blink 5.2s infinite;
    animation-delay: 1.5s;
}
.creepy-emoji {
    display: inline-block;
```

```

        <path d="M0,0 L6,3 L0,6 Z" fill="#f97311" stroke="black" stroke-width="1" />
    </marker>
    <marker id="arrow-pink" markerWidth="6" markerHeight="6" orient="auto" />
        <path d="M0,0 L6,3 L0,6 Z" fill="#f472b1" stroke="black" stroke-width="1" />
    </marker>
    <marker id="arrow-cyan" markerWidth="6" markerHeight="6" orient="auto" />
        <path d="M0,0 L6,3 L0,6 Z" fill="#2dd4b1" stroke="black" stroke-width="1" />
    </marker>
    <marker id="arrow-indigo" markerWidth="6" markerHeight="6" orient="auto" />
        <path d="M0,0 L6,3 L0,6 Z" fill="#818cf8" stroke="black" stroke-width="1" />
    </marker>
    <marker id="arrow-lime" markerWidth="6" markerHeight="6" orient="auto" />
        <path d="M0,0 L6,3 L0,6 Z" fill="#a3e63d" stroke="black" stroke-width="1" />
    </marker>
    <marker id="arrow-green" markerWidth="6" markerHeight="6" orient="auto" />
        <path d="M0,0 L6,3 L0,6 Z" fill="#4ade80" stroke="black" stroke-width="1" />
    </marker>

    <!-- ЛЕГО БЛОКИ -->
    <g id="lego-yellow">
        <rect x="0" y="0" width="40" height="20" fill="yellow" />
        <rect x="6" y="-5" width="8" height="6" fill="yellow" />
        <rect x="26" y="-5" width="8" height="6" fill="yellow" />
        <line x1="2" y1="2" x2="38" y2="2" stroke="black" stroke-width="1" />
    </g>
    <g id="lego-red">
        <rect x="0" y="0" width="40" height="20" fill="red" />
        <rect x="6" y="-5" width="8" height="6" fill="red" />
        <rect x="26" y="-5" width="8" height="6" fill="red" />
        <line x1="2" y1="2" x2="38" y2="2" stroke="black" stroke-width="1" />
    </g>
    <g id="lego-rem">
        <rect x="0" y="0" width="40" height="10" fill="red" />
        <rect x="6" y="-5" width="8" height="6" fill="red" />
        <rect x="26" y="-5" width="8" height="6" fill="red" />
        <line x1="2" y1="2" x2="38" y2="2" stroke="black" stroke-width="1" />
    </g>
    <!-- МЯСОРУБКА -->

```

```

<aside id="mobile-drawer" class="fixed inset-y-0
  nslate-x-full transition-transform duration-300
  w-none lg:block lg:w-72 lg:shrink-0 lg:z-auto
  <div class="h-full flex flex-col p-6 overflow-hidden
    lg:rounded-2xl lg:border lg:border-slate-700
      <div class="flex justify-between items-center
        <h3 class="font-black text-transparent bg-clip-text
          ider text-sm">
            Содержание
          </h3>
          <button id="close-drawer-btn" class="p-2
            <svg xmlns="http://www.w3.org/2000/svg"
              <path stroke-linecap="round" stroke-width="2"
            </svg>
          </button>
        </div>
      <nav class="space-y-3 text-sm font-medium">
        <details class="group" open>
          <summary class="list-none cursor-pointer text-blue-500
            <a href="#question-1" class="text-blue-500 pl-3 font-bold text-blue-300">
              1. Алгебраические структуры
            </a>
          </summary>
          <div class="pl-6 space-y-2 text-sm">
            <a href="#section-1-1" class="text-blue-500">1.1
            <a href="#section-1-2" class="text-blue-500">1.2
            <a href="#section-1-3" class="text-blue-500">1.3
            <a href="#section-1-4" class="text-blue-500">1.4
          </div>
        </details>
        <details class="group">
          <summary class="list-none cursor-pointer text-indigo-500
            <a href="#question-2" class="text-indigo-500 pl-3 font-bold text-indigo-300">
              2. Комплексные числа
            </a>
          </summary>
          <div class="pl-6 space-y-2 text-sm">
            <a href="#section-2-1" class="text-indigo-500">2.1
            <a href="#section-2-2" class="text-indigo-500">2.2
            <a href="#section-2-3" class="text-indigo-500">2.3
            <a href="#section-2-4" class="text-indigo-500">2.4
          </div>
        </details>
      </nav>
    </div>
  </div>

```

```
<details class="group">
  <summary class="list-none cursor-pointer-fuchsia-500 pl-3 font-bold text-fuchsia-500">
    5. Группа корней из единицы
  </a>
  </summary>
  <div class="pl-6 space-y-2 text-fuchsia-500">
    <a href="#q5-def" class="block hover-fuchsia-500">
    <a href="#q5-group" class="block hover-fuchsia-500">
    <a href="#q5-prim" class="block hover-fuchsia-500">
    <a href="#q5-crit" class="block hover-fuchsia-500">
    <a href="#q5-cyclic" class="block hover-fuchsia-500">
  </div>
</details>
```

```
<details class="group">
  <summary class="list-none cursor-pointer-lime-500 pl-3 font-bold text-lime-300">
    6. Делимость, НОД и Евклид
  </a>
  </summary>
  <div class="pl-6 space-y-2 text-lime-500">
    <a href="#q6-1" class="block hover-lime-500">
    <a href="#q6-2" class="block hover-lime-500">
    <a href="#q6-3" class="block hover-lime-500">
    <a href="#q6-4" class="block hover-lime-500">
    <a href="#q6-5" class="block hover-lime-500">
    <a href="#q6-6" class="block hover-lime-500">
  </div>
</details>
```

```
<details class="group">
  <summary class="list-none cursor-pointer-lime-500 pl-3 font-bold text-lime-300">
    7. Делимость, НОД и Евклид
  </a>
  </summary>
  <div class="pl-6 space-y-2 text-lime-500">
    <a href="#q7-1" class="block hover-lime-500">
    <a href="#q7-2" class="block hover-lime-500">
    <a href="#q7-3" class="block hover-lime-500">
    <a href="#q7-4" class="block hover-lime-500">
    <a href="#q7-5" class="block hover-lime-500">
    <a href="#q7-6" class="block hover-lime-500">
  </div>
</details>
```

```

<div class="pl-6 space-y-2 text
  <a href="#q9-1" class="bloc
  <a href="#q9-2" class="bloc
  <a href="#q9-3" class="bloc
  <a href="#q9-4" class="bloc
</div>
</details>

<details class="group">
  <summary class="list-none curso
    <a href="#question-10" clas
der-rose-500 pl-3 font-bold text-rose-400">
      10. Делимость и Схема Г
    </a>
  </summary>
  <div class="pl-6 space-y-2 text
    <a href="#q10-1" class="blo
    <a href="#q10-2" class="blo
    <a href="#q10-3" class="blo
a>
      <a href="#q10-4" class="blo
      <a href="#q10-5" class="blo
    </div>
  </details>

  <details class="group">
    <summary class="list-none curso
      <a href="#question-11" clas
der-cyan-500 pl-3 font-bold text-cyan-400">
        11. НОД и НОК многочлен
      </a>
    </summary>
    <div class="pl-6 space-y-2 text
      <a href="#q11-1" class="blo
      <a href="#q11-2" class="blo
нственности НОД</a>
      <a href="#q11-3" class="blo
ПИРАЛЬ)</a>

```



```
14. Кратность корня. Ко  
    </a>  
  </summary>  
  <div class="pl-6 mt-2 space-y-2"  
    <a href="#q14-1" class="blo  
    <a href="#q14-2" class="blo  
    <a href="#q14-3" class="blo  
deg)</a>  
    <a href="#q14-4" class="blo  
ition-colors border border-rose-500/30"> K  
  </div>  
</details>  
  
<details class="group">  
  <summary class="list-none curso  
    <a href="#question-15" clas  
der-cyan-500 pl-3 font-bold text-cyan-400">  
    15. Производная. Критер  
    </a>  
  </summary>  
  <div class="pl-6 mt-2 space-y-2"  
    <a href="#q15-1" class="blo  
    <a href="#q15-2" class="blo  
    <a href="#q15-3" class="blo  
    <a href="#q15-4" class="blo  
  </div>  
</details>  
  
<details class="group">  
  <summary class="list-none curso  
    <a href="#question-16" clas  
der-red-500 pl-3 font-bold text-red-400">  
    16. Неприводимые многоч  
    </a>  
  </summary>  
  <div class="pl-6 mt-2 space-y-2"  
    <a href="#q16-1" class="blo  
    <a href="#q16-2" class="blo
```

```

        detail.addEventListener('toggle
            if (detail.open) {
                document.querySelectorA
                    if (other !== detail
                        other.removeAtt
                    }
            }
        });
    });
});

// Scroll spy logic
document.addEventListener('DOMConten
    const sections = document.query
    ="q5-"], div[id^="q6-"], div[id^="q7-"], di
    v[id^="q13-"], div[id^="q14-"], div[id^="q1
    let isScrolling = false;

    document.querySelectorAll('.gro
        link.addEventListener('clie
            isScrolling = true;
            setTimeout(() => isScro
        });
    });

    const observer = new Intersection
        if (isScrolling) return;

        let visibleId = null;
        entries.forEach(entry => {
            if (entry.isIntersectin
                visibleId = entry.t

                // Highlight active
                document.querySelec
                    a.style.fontWeig
                    if (a.href.ends
                        a.style.fon

```

```
modal.style.display = 'flex';
modal.style.flexDirection = 'column';
modal.style.alignItems = 'center';
modal.style.justifyContent = 'center';

const box = document.createElement('div');
box.style.width = '80%';
box.style.height = '80%';
box.style.backgroundColor = '#1a1a1a';
box.style.padding = '20px';
box.style.borderRadius = '10px';
box.style.display = 'flex';
box.style.flexDirection = 'column';
box.style.color = '#fff';

const header = document.createElement('div');
header.innerText = 'Экспорт в PDF';
header.style.marginBottom = '10px';

const subHeader = document.createElement('div');
subHeader.innerText = 'Из-за особенностей браузера  
все веб-студии в новой вкладке (через меню)';
subHeader.style.marginBottom = '10px';
subHeader.style.color = '#f87171';
subHeader.style.fontSize = '0.9em';

const textarea = document.createElement('div');
textarea.value = contentRaw;
textarea.style.flex = '1';
textarea.style.width = '100%';
textarea.style.color = '#000';
textarea.style.padding = '10px';
textarea.style.fontFamily = 'monospace';

const btnContainer = document.createElement('div');
btnContainer.style.display = 'flex';
btnContainer.style.gap = '10px';
btnContainer.style.marginTop = '10px';
```

```

        box.appendChild(btnContainer);
        modal.appendChild(box);
        document.body.appendChild(modal);
    }

```

```

function downloadMD() {
    let clone = document.querySelector('#clone');

    // Smart markdown parsing based on marked.js
    // Smart markdown parsing based on marked.js
    clone.querySelectorAll('h2').forEach(el => {
    clone.querySelectorAll('h3').forEach(el => {
    clone.querySelectorAll('.analog').forEach(el => {
    clone.querySelectorAll('li').forEach(el => {
    clone.querySelectorAll('svg').forEach(el => {
        let caption = '';
        if (el.nextElementSibling && el.nextElementSibling.tagName === 'CAPTION') {
            caption = el.nextElementSibling.textContent;
        }
        el.textContent = '\n\n[ Изображение: ' + caption + ' ]';
    });
    clone.querySelectorAll('.img-caption').forEach(el => {
        let text = clone.innerHTML;
        text = text.replace(/\n{3,}/g, '\n\n');

        if (window !== window.top) {
            fallbackModal(text, 'Markdown');
            return;
        }
        const blob = new Blob([text], { type: 'text/markdown' });
        const url = URL.createObjectURL(blob);
        const link = document.createElement('a');
        link.href = url;
        link.download = 'vyisshaya_algebra.md';
        document.body.appendChild(link);
        link.click();
    });
}

```

```

nt; }

body.bg-white [class*="]

nt; }

body.bg-white [class*="]
body.bg-white .analogy-
body.bg-white .img-plac
body.bg-white .info-blo

body.bg-white .analogy-
or: #94a3b8 !important; }
body.bg-white .img-capt

/* Tweak card blocks */
body.bg-white .card-red

body.bg-white aside { b
#334155 !important; }
body.bg-white .text-whi

/* Ensure active links
body.bg-white aside a.t
body.bg-white aside a.t
body.bg-white aside a.t
body.bg-white aside a.t
body.bg-white aside a.t
body.bg-white aside a.t
body.bg-white aside a:h

body.bg-white table tr.
body.bg-white table tr.
body.bg-white .text-tea
body.bg-white .text-ros
body.bg-white .text-eme
body.bg-white .text-cya
body.bg-white .text-amb
body.bg-white .text-fuc
body.bg-white .text-lim
body.bg-white .text-blur

```

```
                <span> </span> Word
            </button>
            <a href="algebra.pdf" target="_blank">
- sm font-bold flex items-center justify-center
                <span> </span> PDF
            </a>
            <button onclick="downloadMD()">
- bold flex items-center justify-center gap-2
                <span> </span> Markdown
            </button>
            <button onclick="downloadHTML()">
t-sm font-bold flex items-center justify-center
                <span> </span> HTML
            </button>
        </div>

        <!-- Смена темы -->
        <div>
            <div class="text-xs text-slate-500">
                <div class="flex items-center gap-2">
                    <button id="theme-dark" onclick="toggleTheme('dark')">
justify-center text-amber-300 hover:text-white
line-amber-500" title="Темная тема"> </button>
                    <button id="theme-light" onclick="toggleTheme('light')">
r justify-center text-amber-50 hover:text-white
">* </button>
                    <button id="theme-terminal" onclick="toggleTheme('terminal')">
- center justify-center text-green-400 hover:text-white
терминала"> </button>
                </div>
            </div>
        </div>
    </div>
</aside>

<!-- ОСНОВНОЙ КОНТЕНТ -->
<main class="flex-1 bg-slate-800 p-4 sm:p-8 md:p-12">
```

```

<!-- ===== WTF CONTAINER =====
<div id="wtf-container" class="hidden">
  <h2 class="text-3xl font-black text-cen

    <div class="overflow-x-auto w-full pb-4"
      <div class="grid grid-cols-3 gap-4"
        <!-- Column headers -->
        <div class="bg-yellow-900/40 bo
оля и кольца, целые числа</div>
        <div class="bg-orange-900/40 bo
ольцо полиномов</div>
        <div class="bg-cyan-900/40 bord
сные числа</div>

        <!-- Row 1: База -->
        <a href="#question-1" onclick="setM
-4 border-yellow-500 hover:bg-slate-700 tra
        <div class="text-xs text-yellow
        <div class="font-bold text-slate
        </a>
        <a href="#question-9" onclick="setM
-4 border-orange-500 hover:bg-slate-700 tra
        <div class="text-xs text-orange
        <div class="font-bold text-slate
        </a>
        <a href="#question-2" onclick="setM
-4 border-cyan-500 hover:bg-slate-700 trans
        <div class="text-xs text-cyan-5
        <div class="font-bold text-slate
        </a>

        <!-- Row 2: Базовые операции / Евклид
        <a href="#question-6" onclick="setM
-4 border-yellow-500 hover:bg-slate-700 tra
        <div class="text-xs text-yellow
        <div class="font-bold text-slate
        </a>
        <a href="#question-10" onclick="setM

```

```

        <div class="font-bold text-slate-500">
    </a>

    <!-- Row 5: Корни / Уравнения -->
    <div class="hidden md:block"></div>
    <a href="#question-13" onclick="setMainQuestion(13)">
- l-4 border-orange-500 hover:bg-slate-700 transition-colors
        <div class="text-xs text-orange-500">
            <div class="font-bold text-slate-500">
    </a>
    <a href="#question-4" onclick="setMainQuestion(4)">
- l-4 border-cyan-500 hover:bg-slate-700 transition-colors
        <div class="text-xs text-cyan-500">
            <div class="font-bold text-slate-500">
    </a>

    <!-- Row 6: Кратность -->
    <div class="hidden md:block"></div>
    <a href="#question-14" onclick="setMainQuestion(14)">
- l-4 border-orange-500 hover:bg-slate-700 transition-colors
        <div class="text-xs text-orange-500">
            <div class="font-bold text-slate-500">
    </a>
    <div class="hidden md:block"></div>

    <!-- Row 7: Производная -->
    <div class="hidden md:block"></div>
    <a href="#question-15" onclick="setMainQuestion(15)">
- l-4 border-orange-500 hover:bg-slate-700 transition-colors
        <div class="text-xs text-orange-500">
            <div class="font-bold text-slate-500">
    </a>
    <div class="hidden md:block"></div>

    <!-- КЛАСТЕР 3: НЕПРИВОДИМОСТЬ -->
    <div class="col-span-1 md:col-span-1">
b-2 border-b border-emerald-500/30 pb-2">Кл

```


Просто перепиши формулу с буквой k .

k , замени его на жареного карася из

Двойная делимость

Δr_i

вверх по уравнениям).

Δr_i

рху вниз, показывая, что Δr_i

d больше.

оритмы экзекуции)

со следующим. Если в конце 0 – пациент мертв

c . Пока получается 0 – корень жив. Как то

корня минус один.

```
-----
                                <h4 class="text-cyan">
                                <p class="text-sm t
х двух, потом последних. <br>
                                <code class="text-x
                                </div>
                                </li>

                                <li class="bg-slate-800/50 p
                                <span class="text-2xl">
                                <div>
                                <h4 class="text-cyan">
                                <p class="text-sm t
есть сложение – есть ноль. Если есть умноже
                                <code class="text-x
                                </div>
                                </li>

                                <li class="bg-slate-800/50 p
                                <span class="text-2xl">
                                <div>
                                <h4 class="text-cyan">
                                <p class="text-sm t
авь их драться. Умножение всегда врывается
                                <code class="text-x
                                </div>
                                </li>
                                </ul>
                                </div>
                                </section>
                                </div>

                                <div id="questions-container">
                                <!-- ===== ВОПРОС 1 =====
```

```

-----
for (const sheet of Array.from(document.styleSheets))
  try {
    const rules = (sheet as CSSStyleSheet).cssRules;
    parts.push(Array.from(rules).map((r) => r.cssText));
  } catch {
    const href = (sheet as CSSStyleSheet).href;
    if (!href) continue;
    try {
      const res = await fetch(href);
      if (res.ok) parts.push(await res.text());
    } catch {
      /* чужой домен – пропускаем */
    }
  }
}
return parts.join("\n");
}

const EXTRA_CSS = `
/* – правки для автономного файла – */
html { color-scheme: dark; }
body { background: #020617; color: #e2e8f0; font-family:
a.term-hint { text-decoration: none; }
.export-shell { max-width: 62rem; margin: 0 auto; padding:
.export-head { border-bottom: 1px solid #1e293b; padding:
.export-note { background: #0f172a; border: 1px solid #
  border-radius: .5rem; padding: .85rem 1rem; font-size:
.export-note a { color: #a5b4fc; }
.export-gloss { margin-top: 3rem; border-top: 1px solid
.export-gloss h2 { color: #fff; font-size: 1.25rem; font
.export-term { background: #0f172a; border: 1px solid #
.export-term h3 { color: #c7d2fe; font-size: .95rem; font
.export-term p { margin: 0 0 .3rem; font-size: .85rem;
.export-term .formal { color: #94a3b8; font-size: .8rem;
.export-term .cx { color: #6ee7b7; font-size: .78rem; font
.export-foot { margin-top: 3rem; border-top: 1px solid #
@media print {
  body { background: #fff; color: #0f172a; }

```

```

    host.querySelectorAll("details").forEach((d) => d.setOpen(true));

    return { body: host.innerHTML, termIds: ids };
}

```

```

function glossarySection(termIds: string[]): string {
  if (!termIds.length) return "";
  const items = termIds
    .map((id) => {
      const e = GLOSSARY_BY_ID.get(id);
      if (!e) return "";
      return `<div class="export-term" id="term-${escapeHtml(id)}">
        <h3>${escapeHtml(e.title)}</h3>
        <p>${escapeHtml(e.short)}</p>
        ${e.formal ? `<p class="formal">${escapeHtml(e.formal)}</p>` : ""}
        ${e.complexity ? `<p class="cx">Сложность: ${escapeHtml(e.complexity)}</p>` : ""}
      </div>`;
    })
    .join("\n");

  return `<section class="export-gloss">
    <h2>Словарик терминов из этой темы</h2>
    <p style="font-size:.8rem;color:#94a3b8;margin:0 0 1rem 0">
      Перейдите по ссылкам, чтобы узнать больше о терминах, упомянутых
      здесь.</p>
    ${items}
  </section>`;
}

```

```

function wrap(opts: { title: string; subtitle: string; css: string; extraCss: string }): string {
  const stamp = new Date().toLocaleString("ru-RU");
  return `<!doctype html>
<html lang="ru">
<head>
<meta charset="utf-8">
<meta name="viewport" content="width=device-width, initial-scale=1">
<title>${escapeHtml(opts.title)}</title>
<style>${opts.css}</style>
<style>${EXTRA_CSS}</style>

```

```
/** Имя файла: «04-splay-derevo.html» – без кириллицы и
function safeName(title: string, id: string): string {
  const num = title.match(/^\\s*(\\d+)/)?.[1];
  const base = id.replace(/^[^a-z0-9-]+/gi, "-").toLowerCase();
  return `${num ? String(num).padStart(2, "0") + "-" : ""}${base}`;
}
```

```
/** Выгрузить одну главу. */
export async function downloadChapterHtml(chapter: Chapter) {
  const css = await collectCss();
  const { body, termIds } = prepareBody(chapter.content);
  const html = wrap({
    title: chapter.title,
    subtitle: chapter.category || "Универсальное пособие",
    css,
    body,
    gloss: glossarySection(termIds),
    origin: window.location.origin,
  });
  triggerDownload(html, safeName(chapter.title, chapter.id));
}
```

```
/** Выгрузить всё пособие одним файлом со сквозным оглавлением
export async function downloadBookHtml(chapters: Chapter[]) {
  const css = await collectCss();
  const allTerms: string[] = [];
  const bodies: string[] = [];
```

```
const toc = chapters
  .map((c) => `<li style="margin:.2rem 0"><a href="#${c.id}>${c.title}</a>`
  .join("\\n");
```

```
chapters.forEach((c) => {
  const { body, termIds } = prepareBody(c.content);
  termIds.forEach((t) => {
    if (!allTerms.includes(t)) allTerms.push(t);
  });
});
```

```

*/
import fs from "node:fs";
import path from "node:path";
import { fileURLToPath } from "node:url";
import { build } from "esbuild";

const ROOT = path.resolve(path.dirname(fileURLToPath(import.meta.url)), "..");
const TMP = path.join(ROOT, "tmp", "audit");

fs.mkdirSync(TMP, { recursive: true });
const bundle = path.join(TMP, "content.bundle.mjs");

await build({
  entryPoints: [path.join(ROOT, "src/data/content.ts")],
  bundle: true,
  format: "esm",
  platform: "node",
  outfile: bundle,
  logLevel: "error",
});

const { chapters } = await import(`file://${bundle}`);

// Список интерактивов берём из реестра визуализаторов
const registry = fs.readFileSync(path.join(ROOT, "src/registry.json"), "utf-8");
const registryBody = registry.slice(registry.indexOf("VIZUALIZATORY"), registry.length);
const vizIds = new Set([...registryBody.matchAll(/^\s{2}[a-zA-Z0-9_-]+/g)].map(m => m[0]));

const rows = chapters.map((c) => {
  const html = c.content ?? "";
  const analogies = (html.match(/Аналогия/gi) ?? []).length;
  return {
    id: c.id,
    title: c.title,
    num: Number.parseInt(c.title.match(/^(\\d+)\\.\\/)?.[1]),
    analogies,
    hasAnalogy: analogies > 0,
    inlineSvg: /<svg[\\s>]/i.test(html),
  };
});

```

}

ФАЙЛ 80/86: scripts/audit-terms.mjs

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 68 | РАЗМ

/**

* Проверка покрытия глав всплывающими подсказками:
 * в каждой ли теме находятся термины из глоссария.
 *

* node scripts/audit-terms.mjs

*/

import { build } from "esbuild";

import path from "node:path";

const ROOT = process.cwd();

await build({

entryPoints: ["src/data/content.ts", "src/data/glossa

bundle: true, format: "esm", platform: "node",

outdir: "tmp/audit-terms", outExtension: { ".js": ".m

external: ["react", "react-dom", "motion", "lucide-re

});

const { chapters } = await import(path.join(ROOT, "tmp/

const { GLOSSARY_LABELS, GLOSSARY } = await import(path

const esc = (s) => s.replace(/[\.*\+\?\^\\$\{\}\(\)|[\]\[\]]/g, "\\

const re = new RegExp(

`(?!<![\\p{L}\\p{N}_-])(\${GLOSSARY_LABELS.map((l) => e

"giu"

);

const map = new Map(GLOSSARY_LABELS.map((l) => [l.label

// те же лимиты, что и в src/hooks/useTermHints.ts

const MAX_PER_SURFACE = 2;

const MAX_PER_TERM = 8;

ФАЙЛ 81/86: scripts/export/build-pdf.mjs

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 64 | РАЗМ

```
/**
 * ШАГ 3: PNG -> PDF
 *
 * Склеивает отрендеренные страницы tmp/export-pages/pages/
 * в единый документ public/export/full_code_all_pages.pdf
 */
import fs from "node:fs";
import path from "node:path";
import PDFDocument from "pdfkit";
import { ROOT, OUT_DIR, PAGES_DIR, PDF_PATH, ensureDir, }

// A4 в пунктах PDF
const A4 = { width: 595.28, height: 841.89 };

async function main() {
  if (!fs.existsSync(PAGES_DIR)) {
    throw new Error(`Нет ${path.relative(ROOT, PAGES_DIR)}`);
  }

  const pages = fs
    .readdirSync(PAGES_DIR)
    .filter((f) => f.endsWith(".png"))
    .sort();

  if (pages.length === 0) throw new Error("Не найдено ни одной страницы");

  ensureDir(OUT_DIR);

  const doc = new PDFDocument({
    size: [A4.width, A4.height],
    margin: 0,
    autoFirstPage: false,
```

```
/**
 * ШАГ 1: КОД -> TXT
 *
 * Собирает ВСЕ исходный код репозитория в один текстовый файл
 * public/export/full_code_all_pages.txt
 *
 * Список файлов берётся из git (чтобы ничего не забыть
 * с фоллбэком на обход файловой системы, если git недоступен)
 */
import fs from "node:fs";
import path from "node:path";
import { execFileSync } from "node:child_process";
import { ROOT, OUT_DIR, TXT_PATH, HR, SUBHR, ensureDir, }

/** Расширения, которые считаем «кодом» и включаем в документацию */
const CODE_EXT = new Set([
  ".ts", ".tsx", ".js", ".jsx", ".mjs", ".cjs",
  ".css", ".scss", ".html", ".json", ".md", ".yaml", ".yml",
]);

/** Явные исключения: генерируемое, бинарное и просто служебные файлы */
const EXCLUDE_FILES = new Set(["package-lock.json", "build", "dist"]);
const EXCLUDE_DIRS = ["node_modules", "dist", "build", "public"];

/** Человекочитаемые категории для оглавления. */
const CATEGORIES = [
  [/^src\/data\/tickets\/$/, "Билеты (учебный контент)"],
  [/^src\/data\/$/, "Контент и данные пособия"],
  [/^src\/components\/visualizers\/$/, "Визуализаторы (вспомогательные)"],
  [/^src\/components\/$/, "Интерактивные компоненты и сервисы"],
  [/^src\/layout\/$/, "HTML-разметка (layout)"],
  [/^src\/utils\/$/, "Утилиты"],
  [/^src\/$/, "Ядро приложения"],
  [/^scripts\/$/, "Скрипты сборки и экспорта"],
  [/^\.github\/$/, "CI / автоматизация"],
  [/^[^\/]+\$/ , "Конфигурация проекта"],
];
```

```

    }

    return files
      .filter((rel) => !EXCLUDE_DIRS.some((d) => rel.startsWith(d)))
      .filter((rel) => !EXCLUDE_FILES.has(path.basename(rel)))
      .filter((rel) => CODE_EXT.has(path.extname(rel)))
      .filter((rel) => fs.existsSync(path.join(ROOT, rel)))
      .sort((a, b) => {
        const ia = ORDER.indexOf(categoryOf(a));
        const ib = ORDER.indexOf(categoryOf(b));
        return ia !== ib ? ia - ib : a.localeCompare(b);
      });
  }

  /** Достаём заголовки глав из данных пособия (без запуска)
  function extractChapters(files) {
    const chapters = [];
    const seen = new Set();
    for (const rel of files.filter((f) => f.startsWith("src/"))) {
      const src = fs.readFileSync(path.join(ROOT, rel), "utf8");
      const re = /id:\s*"([^"]+)"\s*,\s*\n?\s*title:\s*"([^"]+)"/g;
      let m;
      while ((m = re.exec(src)) !== null) {
        const [, id, title] = m;
        if (seen.has(id)) continue;
        seen.add(id);
        chapters.push({ id, title, file: rel });
      }
    }
    return chapters.sort((a, b) => {
      const na = parseInt(a.title.match(/(\d+)/)?.[0] ?? 0);
      const nb = parseInt(b.title.match(/(\d+)/)?.[0] ?? 0);
      return na - nb;
    });
  }

  function main() {
    const files = listFiles();

```

```

    }
    push(`  ${String(i + 1).padStart(3, " ")} . ${rel}`)
  });
  push();
  push();

  current = null;
  files.forEach((rel, i) => {
    const cat = categoryOf(rel);
    if (cat !== current) {
      current = cat;
      push(HR);
      push(`РАЗДЕЛ: ${cat.toUpperCase()}`);
      push(HR);
      push();
    }
    const content = fs.readFileSync(path.join(ROOT, rel), "utf8");
    const lines = content.split("\n");
    push(SUBHR);
    push(`ФАЙЛ ${i + 1}/${files.length}: ${rel}`);
    push(`КАТЕГОРИЯ: ${cat} | СТРОК: ${lines.length} |`);
    push(SUBHR);
    push();
    push(content.replace(/\n+$/, ""));
    push();
    push();
  });

  push(HR);
  push(`КОНЕЦ ДОКУМЕНТА • ${files.length} файлов • сгенерировано`);
  push(HR);

  const txt = out.join("\n") + "\n";
  fs.writeFileSync(TXT_PATH, txt, "utf8");

  console.log("[1/3] код -> txt");
  console.log(`файлов:  ${files.length}`);
  console.log(`глав:      ${chapters.length}`);

```

```
/** 1-bit ч/б страницы весят в 2-4 раза меньше. EXPORT
monochrome: process.env.EXPORT_GRAYSCALE !== "1",
size: "A4",
font: "DejaVu-Sans-Mono",
fontFile: "/usr/share/fonts/truetype/dejavu/DejaVuSan
pointsize: 8,
/** Максимум символов в строке (далее – жёсткий пере
cols: 138,
/** Строк содержимого на странице (+2 служебные: шапка
rows: 76,
};

export const HR = "=".repeat(PAGE.cols);
export const SUBHR = "-".repeat(PAGE.cols);

export function ensureDir(dir) {
  fs.mkdirSync(dir, { recursive: true });
}

export function humanSize(bytes) {
  if (bytes < 1024) return `${bytes} B`;
  if (bytes < 1024 * 1024) return `${(bytes / 1024).toFixed(2)} KB`;
  return `${(bytes / 1024 / 1024).toFixed(2)} MB`;
}

/** ImageMagick 7 (`magick`) или 6 (`convert`). */
export function imageMagickCmd() {
  for (const cmd of ["magick", "convert"]) {
    try {
      execFileSync(cmd, ["-version"], { stdio: "ignore" });
      return cmd;
    } catch {
      /** пробуем следующий */
    }
  }
  throw new Error(
    "ImageMagick не найден. Установите его: sudo apt-get install imagemagick"
  );
}
```

```

    rest = rest.slice(width);
    first = false;
  }
  return parts;
}

function paginate(txt) {
  const source = txt.replace(/\r\n/g, "\n").split("\n");
  const pages = [];
  for (let i = 0; i < source.length; i += PAGE.rows) {
    pages.push(source.slice(i, i + PAGE.rows));
  }
  return pages;
}

async function main() {
  if (!fs.existsSync(TXT_PATH)) {
    throw new Error(`Нет ${path.relative(ROOT, TXT_PATH)}`);
  }

  const im = imageMagickCmd();
  const pages = paginate(fs.readFileSync(TXT_PATH, "utf8"));

  fs.rmSync(PAGES_DIR, { recursive: true, force: true });
  ensureDir(PAGES_DIR);

  const total = pages.length;
  const jobs = pages.map((lines, idx) => {
    const num = String(idx + 1).padStart(4, "0");
    const header = `Универсальное пособие • полный исходный код
    Math.max(1, PAGE.cols - 44 - `стр. ${idx + 1} / ${total}`);
    const body = [header, "-".repeat(PAGE.cols), ...lines];
    const txtFile = path.join(PAGES_DIR, `page-${num}.txt`);
    const pngFile = path.join(PAGES_DIR, `page-${num}.png`);
    fs.writeFileSync(txtFile, body + "\n", "utf8");
    return { txtFile, pngFile, num };
  });
});

```

```
-----
const bytes = jobs.reduce((s, j) => s + fs.statSync(j)
console.log("[2/3] txt -> png");
console.log(`      страниц: ${total}`);
console.log(`      выход:   ${path.relative(ROOT, PAGE)}
}

main().catch((err) => {
  console.error("Ошибка рендера страниц:", err.message);
  process.exit(1);
});
```

=====

РАЗДЕЛ: CI / АВТОМАТИЗАЦИЯ

=====

ФАЙЛ 85/86: .github/workflows/deploy-pages.yml

КАТЕГОРИЯ: CI / автоматизация | СТРОК: 83 | РАЗМЕР: 2.9

name: Deploy app to GitHub Pages

```
# ДВЕ настройки репозитория нужно сделать один раз ВРУЧНУЮ
# (у бота-интеграции нет admin-прав на них – API отвечает)
#
# 1. Settings → Pages → Build and deployment → Source
#    Если там «Deploy from a branch» (legacy) – шаг D
# 2. Settings → Environments → github-pages → Deployment
#    добавьте паттерн «arena/**» и ветку «main» (или
#    иначе деплой из новой рабочей ветки отклоняется .
```

on:

push:

branches:

- main

Позволяет проверить Pages прямо из рабочей ветки

- "arena/**"

```
    with:
      path: ./dist

deploy:
  name: Publish to GitHub Pages
  needs: build
  runs-on: ubuntu-latest
  environment:
    name: github-pages
    url: ${ steps.deployment.outputs.page_url }
  steps:
    # Ранняя и понятная ошибка вместо невнятного «Fail»
    # когда в репозитории Pages не переключён на сбор
    - name: Preflight – Pages собирается через GitHub
      env:
        GH_TOKEN: ${ secrets.GITHUB_TOKEN }
      run: |
        bt=$(gh api "repos/${ github.repository }/pages" |
        echo "Pages build_type: ${bt}")
        if [ "${bt}" != "workflow" ]; then
          echo "::error title=Pages не настроен::Settings this
          те этот запуск. См. комментарий в начале деплоя
          exit 1
        fi

    - name: Deploy
      id: deployment
      uses: actions/deploy-pages@v5
```

ФАЙЛ 86/86: .github/workflows/rebuild-pdf.yml

КАТЕГОРИЯ: CI / автоматизация | СТРОК: 128 | РАЗМЕР: 4.0 КБ

name: Rebuild code PDF

Пайплайн пересборки PDF при каждом коммите:

steps:

- name: Checkout
uses: actions/checkout@v6
with:
 fetch-depth: 0
 persist-credentials: true
- name: Setup Node.js
uses: actions/setup-node@v7
with:
 node-version: "24"
 cache: npm
- name: Install ImageMagick + DejaVu fonts
run: |
 sudo apt-get update -qq
 sudo apt-get install -y --no-install-recommends
 # На Ubuntu политика ImageMagick иногда запрещает
 # разрешаем то, что нужно пайплайну (text -> pdf)
 for policy in /etc/ImageMagick-6/policy.xml /etc/ImageMagick-6/policy.xml; do
 if [-f "\$policy"]; then
 sudo sed -i 's/rights="none" pattern="\{PATTERN\}" /rights="write" pattern="\{PATTERN\}" /'
 fi
 done
 convert -version
- name: Install dependencies
run: npm ci --no-audit --no-fund
- name: Step 1 – код → txt
run: npm run export:txt
- name: Step 2 – txt → png
env:
 EXPORT_DENSITY: \${GITHUB_EVENT_INPUTS.DENSITY:-100}
run: npm run export:png

```
run: |
  git config user.name "github-actions[bot]"
  git config user.email "41898282+github-action@users.noreply.github.com"
  git add -- public/export/full_code_all_pages.*
  if git diff --cached --quiet; then
    echo "Экспорт не изменился – коммит не нужен"
    exit 0
  fi
  git commit -m "chore(export): пересборка full_code_all_pages.*"
  git push origin "HEAD:${GITHUB_REF_NAME}"
```